

Software testing

Contents

1	Introduction	1
1.1	Software testing	1
1.1.1	Overview	1
1.1.2	History	2
1.1.3	Testing methods	3
1.1.4	Testing levels	5
1.1.5	Testing Types	6
1.1.6	Testing process	9
1.1.7	Automated testing	10
1.1.8	Testing artifacts	11
1.1.9	Certifications	11
1.1.10	Controversy	12
1.1.11	Related processes	12
1.1.12	See also	13
1.1.13	References	13
1.1.14	Further reading	15
1.1.15	External links	15
2	Black-box testing	16
2.1	Black-box testing	16
2.1.1	Test procedures	16
2.1.2	Hacking	16
2.1.3	See also	16
2.1.4	References	17
2.1.5	External links	17
2.2	Exploratory testing	17
2.2.1	History	17
2.2.2	Description	17
2.2.3	Benefits and drawbacks	17
2.2.4	Usage	18
2.2.5	See also	18
2.2.6	References	18
2.2.7	External links	18

2.3	Session-based testing	18
2.3.1	Elements of session-based testing	18
2.3.2	Planning	19
2.3.3	See also	19
2.3.4	References	19
2.3.5	External links	19
2.4	Scenario testing	19
2.4.1	History	20
2.4.2	Methods	20
2.4.3	See also	20
2.4.4	References	20
2.5	Equivalence partitioning	20
2.5.1	Further reading	21
2.5.2	References	21
2.6	Boundary-value analysis	21
2.6.1	Formal Definition	22
2.6.2	Application	22
2.6.3	References	23
2.7	All-pairs testing	23
2.7.1	Rationale	23
2.7.2	N-wise testing	23
2.7.3	Example	23
2.7.4	Notes	23
2.7.5	See also	24
2.7.6	External links	24
2.8	Fuzz testing	24
2.8.1	History	24
2.8.2	Uses	24
2.8.3	Techniques	25
2.8.4	Reproduction and isolation	25
2.8.5	Advantages and disadvantages	25
2.8.6	See also	26
2.8.7	References	26
2.8.8	Further reading	26
2.8.9	External links	26
2.9	Cause-effect graph	26
2.9.1	See also	27
2.9.2	Further reading	27
2.10	Model-based testing	27
2.10.1	Models	27
2.10.2	Deploying model-based testing	28

2.10.3	Deriving tests algorithmically	28
2.10.4	Solutions	29
2.10.5	See also	29
2.10.6	References	29
2.10.7	Further reading	30
2.11	Web testing	30
2.11.1	Web application performance tool	30
2.11.2	Web security testing	31
2.11.3	Testing the user interface of web applications	31
2.11.4	Open Source web testing tools	31
2.11.5	Windows-based web testing tools	31
2.11.6	Cloud-based testing tools	31
2.11.7	See also	31
2.11.8	References	31
2.11.9	Further reading	32
2.12	Installation testing	32
3	White-box testing	33
3.1	White-box testing	33
3.1.1	Overview	33
3.1.2	Levels	33
3.1.3	Basic procedure	34
3.1.4	Advantages	34
3.1.5	Disadvantages	34
3.1.6	Modern view	34
3.1.7	Hacking	34
3.1.8	See also	34
3.1.9	References	34
3.1.10	External links	35
3.2	Code coverage	35
3.2.1	Coverage criteria	35
3.2.2	In practice	37
3.2.3	Usage in industry	37
3.2.4	See also	37
3.2.5	References	38
3.3	Modified Condition/Decision Coverage	38
3.3.1	Definitions	38
3.3.2	Criticism	39
3.3.3	References	39
3.3.4	External links	39
3.4	Fault injection	39
3.4.1	History	39

3.4.2	Software Implemented fault injection	39
3.4.3	Fault injection tools	40
3.4.4	Fault Injection in Functional Properties or Test Cases	42
3.4.5	Application of fault injection	42
3.4.6	See also	42
3.4.7	References	42
3.4.8	External links	43
3.5	Bebugging	43
3.5.1	See also	43
3.5.2	References	43
3.6	Mutation testing	43
3.6.1	Goal	44
3.6.2	Historical overview	44
3.6.3	Mutation testing overview	44
3.6.4	Mutation operators	45
3.6.5	See also	45
3.6.6	References	45
3.6.7	Further reading	46
3.6.8	External links	46
4	Testing of non functional software aspects	47
4.1	Non-functional testing	47
4.2	Software performance testing	47
4.2.1	Testing types	47
4.2.2	Setting performance goals	48
4.2.3	Prerequisites for Performance Testing	49
4.2.4	Tools	49
4.2.5	Technology	50
4.2.6	Tasks to undertake	50
4.2.7	Methodology	50
4.2.8	See also	51
4.2.9	External links	51
4.3	Stress testing	51
4.3.1	Field experience	51
4.3.2	Rationale	51
4.3.3	Relationship to branch coverage	52
4.3.4	Examples	52
4.3.5	Load test vs. stress test	52
4.3.6	See also	52
4.3.7	References	53
4.4	Load testing	53
4.4.1	Software load testing	53

4.4.2	Physical load testing	54
4.4.3	Car charging system	54
4.4.4	See also	54
4.4.5	References	54
4.4.6	External links	55
4.5	Volume testing	55
4.6	Scalability testing	55
4.6.1	External links	55
4.7	Compatibility testing	55
4.8	Portability testing	56
4.8.1	Use cases	56
4.8.2	Attributes	56
4.8.3	See also	57
4.8.4	References	57
4.9	Security testing	57
4.9.1	Confidentiality	57
4.9.2	Integrity	57
4.9.3	Authentication	58
4.9.4	Authorization	58
4.9.5	Availability	58
4.9.6	Non-repudiation	58
4.9.7	Security Testing Taxonomy	58
4.9.8	See also	58
4.10	Attack patterns	59
4.10.1	Categories	59
4.10.2	Structure	59
4.10.3	Further reading	61
4.10.4	References	61
4.11	Pseudolocalization	61
4.11.1	Localization process	62
4.11.2	Pseudolocalization in Microsoft Windows	62
4.11.3	Pseudolocalization process at Microsoft	62
4.11.4	Pseudolocalization tools for other platforms	63
4.11.5	See also	63
4.11.6	External links	63
4.11.7	References	63
4.12	Recovery testing	63
4.12.1	See also	63
4.13	Soak testing	63
4.13.1	See also	64
4.14	Characterization test	64

4.14.1	References	64
4.14.2	External links	64
5	Unit testing	65
5.1	Unit testing	65
5.1.1	Benefits	65
5.1.2	Separation of interface from implementation	66
5.1.3	Parameterized unit testing	66
5.1.4	Unit testing limitations	67
5.1.5	Applications	67
5.1.6	See also	69
5.1.7	Notes	69
5.1.8	External links	69
5.2	Self-testing code	69
5.2.1	See also	69
5.2.2	Further reading	70
5.3	Test fixture	70
5.3.1	Electronics	70
5.3.2	Software	70
5.3.3	Physical testing	71
5.3.4	See also	71
5.3.5	References	71
5.4	Method stub	71
5.4.1	See also	72
5.4.2	References	72
5.4.3	External links	72
5.5	Mock object	72
5.5.1	Reasons for use	72
5.5.2	Technical details	72
5.5.3	Use in test-driven development	73
5.5.4	Limitations	73
5.5.5	See also	73
5.5.6	References	74
5.5.7	External links	74
5.6	Lazy systematic unit testing	74
5.6.1	Lazy Specification	74
5.6.2	Systematic Testing	74
5.6.3	References	74
5.7	Test Anything Protocol	75
5.7.1	History	75
5.7.2	Specification	75
5.7.3	Usage examples	75

5.7.4	References	75
5.7.5	External links	75
5.8	xUnit	75
5.8.1	xUnit architecture	75
5.8.2	xUnit frameworks	76
5.8.3	See also	76
5.8.4	References	76
5.8.5	External links	76
5.9	List of unit testing frameworks	76
5.9.1	Columns (Classification)	76
5.9.2	Languages	77
5.9.3	See also	78
5.9.4	References	79
5.9.5	External links	87
5.10	SUnit	87
5.10.1	History	87
5.10.2	External links	87
5.11	JUnit	87
5.11.1	Example of JUnit test fixture	88
5.11.2	Ports	88
5.11.3	See also	88
5.11.4	References	88
5.11.5	External links	88
5.12	CppUnit	89
5.12.1	See also	89
5.12.2	Further reading	89
5.12.3	References	89
5.12.4	External links	89
5.13	Test::More	89
5.13.1	External links	89
5.14	NUnit	89
5.14.1	Features	89
5.14.2	Runners	89
5.14.3	Assertions	90
5.14.4	Example	90
5.14.5	Extensions	90
5.14.6	See also	91
5.14.7	References	91
5.14.8	Further reading	91
5.14.9	External links	91
5.15	NUnitAsp	91

5.15.1	How It Works	91
5.15.2	Credits & History	91
5.15.3	See also	91
5.15.4	External links	92
5.16	csUnit	92
5.16.1	Special features	92
5.16.2	See also	92
5.16.3	External links	92
5.17	HtmlUnit	92
5.17.1	See also	92
5.17.2	References	93
5.17.3	External links	93
6	Test automation	94
6.1	Test automation framework	94
6.1.1	Overview	94
6.1.2	Code-driven testing	94
6.1.3	Graphical User Interface (GUI) testing	95
6.1.4	API driven testing	95
6.1.5	What to test	95
6.1.6	Framework approach in automation	95
6.1.7	Defining boundaries between automation framework and a testing tool	96
6.1.8	See also	97
6.1.9	References	97
6.1.10	External links	97
6.2	Test bench	97
6.2.1	Components of a test bench	97
6.2.2	Kinds of test benches	98
6.2.3	An example of a software test bench	98
6.2.4	References	98
6.3	Test execution engine	98
6.3.1	Concept	98
6.3.2	Functions	99
6.3.3	Operations types	99
6.4	Test stubs	99
6.4.1	Example	99
6.4.2	See also	99
6.4.3	References	99
6.4.4	External links	99
6.5	Testware	99
6.5.1	References	100
6.5.2	See also	100

6.6	Test automation framework	100
6.6.1	Overview	100
6.6.2	Code-driven testing	101
6.6.3	Graphical User Interface (GUI) testing	101
6.6.4	API driven testing	101
6.6.5	What to test	101
6.6.6	Framework approach in automation	102
6.6.7	Defining boundaries between automation framework and a testing tool	102
6.6.8	See also	103
6.6.9	References	103
6.6.10	External links	103
6.7	Data-driven testing	103
6.7.1	Introduction	104
6.7.2	Methodology Overview	104
6.7.3	Data Driven	104
6.7.4	See also	104
6.7.5	References	104
6.8	Modularity-driven testing	104
6.8.1	Test Script Modularity Framework	104
6.8.2	References	104
6.9	Keyword-driven testing	104
6.9.1	Overview	105
6.9.2	Advantages	105
6.9.3	Methodology	105
6.9.4	Definition	105
6.9.5	See also	106
6.9.6	References	106
6.9.7	External links	106
6.10	Hybrid testing	106
6.10.1	Pattern	106
6.10.2	See also	106
6.10.3	References	107
6.11	Lightweight software test automation	107
6.11.1	References	107
6.11.2	See also	107
7	Testing process	108
7.1	Software testing controversies	108
7.1.1	Agile vs. traditional	108
7.1.2	Exploratory vs. scripted	108
7.1.3	Manual vs. automated	109
7.1.4	Software design vs. software implementation	109

7.1.5	Who watches the watchmen?	109
7.1.6	References	109
7.2	Test-driven development	109
7.2.1	Test-driven development cycle	110
7.2.2	Development style	111
7.2.3	Best practices	111
7.2.4	Benefits	112
7.2.5	Limitations	113
7.2.6	Test-driven work	113
7.2.7	TDD and ATDD	113
7.2.8	TDD and BDD	113
7.2.9	Code visibility	114
7.2.10	Software for TDD	114
7.2.11	Fakes, mocks and integration tests	114
7.2.12	TDD for complex systems	115
7.2.13	See also	116
7.2.14	References	116
7.2.15	External links	117
7.3	Agile testing	117
7.3.1	Overview	117
7.3.2	Further reading	117
7.3.3	References	117
7.4	Bug bash	117
7.4.1	See also	118
7.4.2	References	118
7.5	Pair Testing	118
7.5.1	Description	118
7.5.2	Benefits and drawbacks	118
7.5.3	Usage	118
7.5.4	See also	118
7.6	Manual testing	118
7.6.1	Overview	118
7.6.2	Stages	119
7.6.3	Comparison to Automated Testing	120
7.6.4	References	120
7.6.5	See also	120
7.7	Regression testing	120
7.7.1	Background	120
7.7.2	Uses	121
7.7.3	See also	121
7.7.4	References	121

7.7.5	External links	121
7.8	Ad hoc testing	121
7.8.1	See also	122
7.8.2	References	122
7.9	Sanity testing	122
7.9.1	Mathematical	122
7.9.2	Software development	122
7.9.3	See also	123
7.9.4	References	123
7.10	Integration testing	123
7.10.1	Purpose	123
7.10.2	Limitations	124
7.10.3	References	124
7.10.4	See also	124
7.11	System testing	124
7.11.1	Testing the whole system	124
7.11.2	Types of tests to include in system testing	124
7.11.3	See also	125
7.11.4	References	125
7.12	System integration testing	125
7.12.1	Introduction	125
7.12.2	Data driven method	125
7.12.3	References	126
7.12.4	See also	126
7.13	Acceptance testing	126
7.13.1	Overview	127
7.13.2	Process	127
7.13.3	User acceptance testing	127
7.13.4	Operational acceptance testing	128
7.13.5	Acceptance testing in extreme programming	128
7.13.6	Types of acceptance testing	128
7.13.7	List of acceptance-testing frameworks	129
7.13.8	See also	129
7.13.9	References	129
7.13.10	Further reading	129
7.13.11	External links	129
7.14	Risk-based testing	130
7.14.1	Assessing risks	130
7.14.2	Types of Risks	130
7.14.3	References	130
7.15	Software testing outsourcing	130

7.15.1	Top established global outsourcing cities	131
7.15.2	Top Emerging Global Outsourcing Cities	131
7.15.3	Vietnam outsourcing	131
7.15.4	Argentina outsourcing	131
7.15.5	References	131
7.16	Tester driven development	131
7.17	Test effort	132
7.17.1	Methods for estimation of the test effort	132
7.17.2	Test efforts from literature	132
7.17.3	References	132
7.17.4	External links	132
8	Testing artefacts	133
8.1	IEEE 829	133
8.1.1	Use of IEEE 829	134
8.1.2	External links	134
8.2	Test strategy	134
8.2.1	Test Levels	134
8.2.2	Roles and Responsibilities	134
8.2.3	Environment Requirements	134
8.2.4	Testing Tools	134
8.2.5	Risks and Mitigation	134
8.2.6	Test Schedule	134
8.2.7	Regression test approach	135
8.2.8	Test Groups	135
8.2.9	Test Priorities	135
8.2.10	Test Status Collections and Reporting	135
8.2.11	Test Records Maintenance	135
8.2.12	Requirements traceability matrix	135
8.2.13	Test Summary	135
8.2.14	See also	135
8.2.15	References	136
8.3	Test plan	136
8.3.1	Test plans	136
8.3.2	IEEE 829 test plan structure	136
8.3.3	See also	137
8.3.4	References	137
8.3.5	External links	137
8.4	Traceability matrix	138
8.4.1	Sample traceability matrix	138
8.4.2	See also	138
8.4.3	References	138

8.4.4	External links	138
8.5	Test case	138
8.5.1	Formal test cases	138
8.5.2	Informal test cases	138
8.5.3	Typical written test case format	139
8.5.4	See also	139
8.5.5	References	139
8.5.6	External links	139
8.6	Test data	139
8.6.1	Limitations	139
8.6.2	Domain testing	140
8.6.3	Test data generation	140
8.6.4	See also	140
8.6.5	References	140
8.7	Test suite	140
8.7.1	Types	140
8.7.2	See also	140
8.7.3	References	140
8.8	Test script	140
8.8.1	See also	141
8.9	Test harness	141
8.9.1	Notes	142
9	Static testing	143
9.1	Static code analysis	143
9.1.1	Rationale	143
9.1.2	Tool types	143
9.1.3	Formal methods	144
9.1.4	See also	144
9.1.5	References	144
9.1.6	Bibliography	145
9.1.7	Sources	145
9.1.8	External links	145
9.2	Software review	145
9.2.1	Varieties of software review	145
9.2.2	Different types of Peer reviews	146
9.2.3	Formal versus informal reviews	146
9.2.4	IEEE 1028 generic process for formal reviews	146
9.2.5	Value of reviews	146
9.2.6	See also	147
9.2.7	References	147
9.3	Software peer review	147

9.3.1	Purpose	147
9.3.2	Distinction from other types of software review	147
9.3.3	Review processes	147
9.3.4	“Open source” reviews	148
9.3.5	References	148
9.4	Software audit review	148
9.4.1	Objectives and participants	148
9.4.2	Tools	148
9.4.3	References	148
9.5	Software technical review	149
9.5.1	Objectives and participants	149
9.5.2	Process	149
9.5.3	References	149
9.6	Management review	149
9.7	Software inspection	149
9.7.1	Introduction	149
9.7.2	The Inspection process	150
9.7.3	Inspection roles	150
9.7.4	Related inspection types	150
9.7.5	See also	151
9.7.6	References	151
9.7.7	External links	151
9.8	Fagan inspection	151
9.8.1	Examples	151
9.8.2	Usage	151
9.8.3	Roles	152
9.8.4	Benefits and results	152
9.8.5	Improvements	152
9.8.6	Example	153
9.8.7	References	153
9.9	Software walkthrough	153
9.9.1	Process	153
9.9.2	Objectives and participants	153
9.9.3	See also	154
9.9.4	References	154
9.10	Code review	154
9.10.1	Introduction	154
9.10.2	Types	154
9.10.3	Criticism	155
9.10.4	See also	155
9.10.5	References	155

9.10.6	Further reading	155
9.10.7	External links	155
9.11	Automated code review	155
9.11.1	Automated code review tools	156
9.11.2	See also	156
9.11.3	References	156
9.12	Code reviewing software	156
9.13	Static code analysis	156
9.13.1	Rationale	156
9.13.2	Tool types	157
9.13.3	Formal methods	157
9.13.4	See also	157
9.13.5	References	157
9.13.6	Bibliography	158
9.13.7	Sources	158
9.13.8	External links	158
9.14	List of tools for static code analysis	158
9.14.1	By language	159
9.14.2	Formal methods tools	162
9.14.3	See also	163
9.14.4	References	163
9.14.5	External links	163
10	GUI testing and review	164
10.1	GUI software testing	164
10.1.1	Test Case Generation	164
10.1.2	Planning and artificial intelligence	164
10.1.3	Running the test cases	165
10.1.4	See also	166
10.1.5	References	166
10.2	Usability testing	166
10.2.1	What usability testing is not	166
10.2.2	Methods	166
10.2.3	How many users to test?	168
10.2.4	Example	169
10.2.5	Usability Testing Education	169
10.2.6	See also	169
10.2.7	References	169
10.2.8	External links	170
10.3	Think aloud protocol	170
10.3.1	See also	170
10.3.2	References	170

10.4	Usability inspection	171
10.4.1	References	171
10.4.2	External links	171
10.4.3	See also	171
10.5	Cognitive walkthrough	171
10.5.1	Introduction	171
10.5.2	Walking through the tasks	171
10.5.3	Common mistakes	172
10.5.4	History	172
10.5.5	References	172
10.5.6	Further reading	172
10.5.7	External links	173
10.5.8	See also	173
10.6	Heuristic evaluation	173
10.6.1	Introduction	173
10.6.2	Nielsen's heuristics	173
10.6.3	Gerhardt-Powals' cognitive engineering principles	174
10.6.4	Weinschenk and Barker classification	175
10.6.5	See also	175
10.6.6	References	175
10.6.7	Further reading	175
10.6.8	External links	176
10.7	Pluralistic walkthrough	176
10.7.1	Procedure	176
10.7.2	Characteristics of Pluralistic Walkthrough	177
10.7.3	Benefits and Limitations	177
10.7.4	Further reading	178
10.7.5	External links	178
10.7.6	See also	178
10.8	Comparison of usability evaluation methods	178
10.8.1	See also	178
11	Text and image sources, contributors, and licenses	179
11.1	Text	179
11.2	Images	190
11.3	Content license	192

Chapter 1

Introduction

1.1 Software testing

Software testing is an investigation conducted to provide stakeholders with information about the quality of the product or service under test.^[1] Software testing can also provide an objective, independent view of the **software** to allow the business to appreciate and understand the risks of software implementation. Test techniques include the process of executing a program or application with the intent of finding **software bugs** (errors or other defects).

It involves the execution of a software component or system component to evaluate one or more properties of interest. In general, these properties indicate the extent to which the component or system under test:

- meets the requirements that guided its design and development,
- responds correctly to all kinds of inputs,
- performs its functions within an acceptable time,
- is sufficiently usable,
- can be installed and run in its intended **environments**, and
- achieves the general result its stakeholders desire.

As the number of possible tests for even simple software components is practically infinite, all software testing uses some strategy to select tests that are feasible for the available time and resources. As a result, software testing typically (but not exclusively) attempts to execute a program or application with the intent of finding **software bugs** (errors or other defects). The job of testing is an iterative process as when one bug is fixed, it can illuminate other, deeper bugs, or can even create new ones.

Software testing can provide objective, independent information about the quality of software and risk of its failure to users and/or sponsors.^[1]

Software testing can be conducted as soon as executable software (even if partially complete) exists. The **overall approach to software development** often determines when and how testing is conducted. For example,

in a phased process, most testing occurs after system requirements have been defined and then implemented in testable programs. In contrast, under an **Agile approach**, requirements, programming, and testing are often done concurrently.

1.1.1 Overview

Although testing can determine the correctness of software under the assumption of some specific hypotheses (see **hierarchy of testing difficulty** below), testing cannot identify all the defects within software.^[2] Instead, it furnishes a *criticism* or *comparison* that compares the state and behavior of the product against **oracles**—principles or mechanisms by which someone might recognize a problem. These oracles may include (but are not limited to) specifications, **contracts**,^[3] comparable products, past versions of the same product, inferences about intended or expected purpose, user or customer expectations, relevant standards, applicable laws, or other criteria.

A primary purpose of testing is to detect software failures so that defects may be discovered and corrected. Testing cannot establish that a product functions properly under all conditions but can only establish that it does not function properly under specific conditions.^[4] The scope of software testing often includes examination of code as well as execution of that code in various environments and conditions as well as examining the aspects of code: does it do what it is supposed to do and do what it needs to do. In the current culture of software development, a testing organization may be separate from the development team. There are various roles for testing team members. Information derived from software testing may be used to correct the process by which software is developed.^[5]

Every software product has a target audience. For example, the audience for video game software is completely different from banking software. Therefore, when an organization develops or otherwise invests in a software product, it can assess whether the software product will be acceptable to its end users, its target audience, its purchasers and other stakeholders. Software testing is the process of attempting to make this assessment.

Defects and failures

Not all software defects are caused by coding errors. One common source of expensive defects is requirement gaps, e.g., unrecognized requirements which result in errors of omission by the program designer.^[6] Requirement gaps can often be **non-functional requirements** such as **testability**, **scalability**, **maintainability**, **usability**, **performance**, and **security**.

Software faults occur through the following processes. A programmer makes an **error** (mistake), which results in a **defect** (fault, bug) in the software **source code**. If this defect is executed, in certain situations the system will produce wrong results, causing a **failure**.^[7] Not all defects will necessarily result in failures. For example, defects in **dead code** will never result in failures. A defect can turn into a failure when the environment is changed. Examples of these changes in environment include the software being run on a new **computer hardware** platform, alterations in **source data**, or interacting with different software.^[7] A single defect may result in a wide range of failure symptoms.

Input combinations and preconditions

A fundamental problem with software testing is that testing under *all* combinations of inputs and preconditions (initial state) is not feasible, even with a simple product.^{[4][8]} This means that the number of **defects** in a software product can be very large and defects that occur infrequently are difficult to find in testing. More significantly, **non-functional** dimensions of quality (how it is supposed to *be* versus what it is supposed to *do*)—**usability**, **scalability**, **performance**, **compatibility**, **reliability**—can be highly subjective; something that constitutes sufficient value to one person may be intolerable to another.

Software developers can't test everything, but they can use combinatorial test design to identify the minimum number of tests needed to get the coverage they want. Combinatorial test design enables users to get greater test coverage with fewer tests. Whether they are looking for speed or test depth, they can use combinatorial test design methods to build structured variation into their test cases.^[9] Note that “coverage”, as used here, is referring to combinatorial coverage, not requirements coverage.

Economics

A study conducted by **NIST** in 2002 reports that software bugs cost the U.S. economy \$59.5 billion annually. More than a third of this cost could be avoided if better software testing was performed.^[10]

It is commonly believed that the earlier a defect is found, the cheaper it is to fix it. The following table shows the cost of fixing the defect depending on the stage it

was found.^[11] For example, if a problem in the requirements is found only post-release, then it would cost 10–100 times more to fix than if it had already been found by the requirements review. With the advent of modern **continuous deployment** practices and cloud-based services, the cost of re-deployment and maintenance may lessen over time.

The data from which this table is extrapolated is scant. Laurent Bossavit says in his analysis:

The “smaller projects” curve turns out to be from only two teams of first-year students, a sample size so small that extrapolating to “smaller projects in general” is totally indefensible. The GTE study does not explain its data, other than to say it came from two projects, one large and one small. The paper cited for the Bell Labs “Safeguard” project specifically disclaims having collected the fine-grained data that Boehm’s data points suggest. The IBM study (Fagan’s paper) contains claims which seem to contradict Boehm’s graph, and no numerical results which clearly correspond to his data points.

Boehm doesn’t even cite a paper for the TRW data, except when writing for “Making Software” in 2010, and there he cited the original 1976 article. There exists a large study conducted at TRW at the right time for Boehm to cite it, but that paper doesn’t contain the sort of data that would support Boehm’s claims.^[12]

Roles

Software testing can be done by software testers. Until the 1980s, the term “software tester” was used generally, but later it was also seen as a separate profession. Regarding the periods and the different goals in software testing,^[13] different roles have been established: *manager*, *test lead*, *test analyst*, *test designer*, *tester*, *automation developer*, and *test administrator*.

1.1.2 History

The separation of **debugging** from testing was initially introduced by Glenford J. Myers in 1979.^[14] Although his attention was on breakage testing (“a successful test is one that finds a bug”^{[14][15]}) it illustrated the desire of the software engineering community to separate fundamental development activities, such as debugging, from that of verification. **Dave Gelperin** and **William C. Hetzel** classified in 1988 the phases and goals in software testing in the following stages:^[16]

- Until 1956 – Debugging oriented^[17]
- 1957–1978 – Demonstration oriented^[18]

- 1979–1982 – Destruction oriented^[19]
- 1983–1987 – Evaluation oriented^[20]
- 1988–2000 – Prevention oriented^[21]

1.1.3 Testing methods

Static vs. dynamic testing

There are many approaches available in software testing. **Reviews**, **walkthroughs**, or **inspections** are referred to as **static testing**, whereas actually executing programmed code with a given set of **test cases** is referred to as **dynamic testing**. Static testing is often implicit, as proofreading, plus when programming tools/text editors check source code structure or compilers (pre-compilers) check syntax and data flow as **static program analysis**. Dynamic testing takes place when the program itself is run. Dynamic testing may begin before the program is 100% complete in order to test particular sections of code and are applied to discrete **functions** or modules. Typical techniques for this are either using **stubs/drivers** or execution from a **debugger** environment.

Static testing involves **verification**, whereas dynamic testing involves **validation**. Together they help improve **software quality**. Among the techniques for static analysis, **mutation testing** can be used to ensure the test-cases will detect errors which are introduced by mutating the source code.

The box approach

Software testing methods are traditionally divided into white- and black-box testing. These two approaches are used to describe the point of view that a test engineer takes when designing test cases.

White-box testing Main article: **White-box testing**

White-box testing (also known as **clear box testing**, **glass box testing**, **transparent box testing** and **structural testing**) tests internal structures or workings of a program, as opposed to the functionality exposed to the end-user. In white-box testing an internal perspective of the system, as well as programming skills, are used to design test cases. The tester chooses inputs to exercise paths through the code and determine the appropriate outputs. This is analogous to testing nodes in a circuit, e.g. **in-circuit testing (ICT)**.

While white-box testing can be applied at the **unit**, **integration** and **system** levels of the software testing process, it is usually done at the unit level. It can test paths within a unit, paths between units during integration, and between subsystems during a system-level test. Though this method of test design can uncover many errors or

problems, it might not detect unimplemented parts of the specification or missing requirements.

Techniques used in white-box testing include:

- **API testing** – testing of the application using public and private **APIs** (application programming interfaces)
- **Code coverage** – creating tests to satisfy some criteria of code coverage (e.g., the test designer can create tests to cause all statements in the program to be executed at least once)
- **Fault injection** methods – intentionally introducing faults to gauge the efficacy of testing strategies
- **Mutation testing** methods
- **Static testing** methods

Code coverage tools can evaluate the completeness of a test suite that was created with any method, including black-box testing. This allows the software team to examine parts of a system that are rarely tested and ensures that the most important **function points** have been tested.^[22] Code coverage as a **software metric** can be reported as a percentage for:

- **Function coverage**, which reports on functions executed
- **Statement coverage**, which reports on the number of lines executed to complete the test
- **Decision coverage**, which reports on whether both the True and the False branch of a given test has been executed

100% statement coverage ensures that all code paths or branches (in terms of **control flow**) are executed at least once. This is helpful in ensuring correct functionality, but not sufficient since the same code may process different inputs correctly or incorrectly.

Black-box testing Main article: **Black-box testing**
Black-box testing treats the software as a “black box”,



Black box diagram

examining functionality without any knowledge of internal implementation. The testers are only aware of what the software is supposed to do, not how it does it.^[23] Black-box testing methods include: **equivalence partitioning**, **boundary value analysis**, **all-pairs testing**,

state transition tables, decision table testing, fuzz testing, model-based testing, use case testing, exploratory testing and specification-based testing.

Specification-based testing aims to test the functionality of software according to the applicable requirements.^[24] This level of testing usually requires thorough test cases to be provided to the tester, who then can simply verify that for a given input, the output value (or behavior), either “is” or “is not” the same as the expected value specified in the test case. Test cases are built around specifications and requirements, i.e., what the application is supposed to do. It uses external descriptions of the software, including specifications, requirements, and designs to derive test cases. These tests can be functional or non-functional, though usually functional.

Specification-based testing may be necessary to assure correct functionality, but it is insufficient to guard against complex or high-risk situations.^[25]

One advantage of the black box technique is that no programming knowledge is required. Whatever biases the programmers may have had, the tester likely has a different set and may emphasize different areas of functionality. On the other hand, black-box testing has been said to be “like a walk in a dark labyrinth without a flashlight.”^[26] Because they do not examine the source code, there are situations when a tester writes many test cases to check something that could have been tested by only one test case, or leaves some parts of the program untested.

This method of test can be applied to all levels of software testing: unit, integration, system and acceptance. It typically comprises most if not all testing at higher levels, but can also dominate unit testing as well.

Visual testing The aim of visual testing is to provide developers with the ability to examine what was happening at the point of software failure by presenting the data in such a way that the developer can easily find the information she or he requires, and the information is expressed clearly.^{[27][28]}

At the core of visual testing is the idea that showing someone a problem (or a test failure), rather than just describing it, greatly increases clarity and understanding. Visual testing therefore requires the recording of the entire test process – capturing everything that occurs on the test system in video format. Output videos are supplemented by real-time tester input via picture-in-a-picture webcam and audio commentary from microphones.

Visual testing provides a number of advantages. The quality of communication is increased drastically because testers can show the problem (and the events leading up to it) to the developer as opposed to just describing it and the need to replicate test failures will cease to exist in many cases. The developer will have all the evidence he or she requires of a test failure and can instead focus on

the cause of the fault and how it should be fixed.

Visual testing is particularly well-suited for environments that deploy agile methods in their development of software, since agile methods require greater communication between testers and developers and collaboration within small teams.

Ad hoc testing and exploratory testing are important methodologies for checking software integrity, because they require less preparation time to implement, while the important bugs can be found quickly. In ad hoc testing, where testing takes place in an improvised, impromptu way, the ability of a test tool to visually record everything that occurs on a system becomes very important in order to document the steps taken to uncover the bug.

Visual testing is gathering recognition in customer acceptance and usability testing, because the test can be used by many individuals involved in the development process. For the customer, it becomes easy to provide detailed bug reports and feedback, and for program users, visual testing can record user actions on screen, as well as their voice and image, to provide a complete picture at the time of software failure for the developer.

Further information: [Graphical user interface testing](#)

Grey-box testing Main article: [Gray box testing](#)

Grey-box testing (American spelling: **gray-box testing**) involves having knowledge of internal data structures and algorithms for purposes of designing tests, while executing those tests at the user, or black-box level. The tester is not required to have full access to the software’s source code.^[29] Manipulating input data and formatting output do not qualify as grey-box, because the input and output are clearly outside of the “black box” that we are calling the system under test. This distinction is particularly important when conducting integration testing between two modules of code written by two different developers, where only the interfaces are exposed for test.

However, tests that require modifying a back-end data repository such as a database or a log file does qualify as grey-box, as the user would not normally be able to change the data repository in normal production operations. Grey-box testing may also include reverse engineering to determine, for instance, boundary values or error messages.

By knowing the underlying concepts of how the software works, the tester makes better-informed testing choices while testing the software from outside. Typically, a grey-box tester will be permitted to set up an isolated testing environment with activities such as seeding a database. The tester can observe the state of the product being tested after performing certain actions such as executing SQL statements against the database and then executing queries to ensure that the expected changes have been re-

flected. Grey-box testing implements intelligent test scenarios, based on limited information. This will particularly apply to data type handling, **exception handling**, and so on.^[30]

1.1.4 Testing levels

There are generally four recognized levels of tests: unit testing, integration testing, component interface testing, and system testing. Tests are frequently grouped by where they are added in the software development process, or by the level of specificity of the test. The main levels during the development process as defined by the **SWEBOK** guide are unit-, integration-, and system testing that are distinguished by the test target without implying a specific process model.^[31] Other test levels are classified by the testing objective.^[31]

Unit testing

Main article: **Unit testing**

Unit testing, also known as component testing, refers to tests that verify the functionality of a specific section of code, usually at the function level. In an object-oriented environment, this is usually at the class level, and the minimal unit tests include the constructors and destructors.^[32]

These types of tests are usually written by developers as they work on code (white-box style), to ensure that the specific function is working as expected. One function might have multiple tests, to catch **corner cases** or other branches in the code. Unit testing alone cannot verify the functionality of a piece of software, but rather is used to ensure that the building blocks of the software work independently from each other.

Unit testing is a software development process that involves synchronized application of a broad spectrum of defect prevention and detection strategies in order to reduce software development risks, time, and costs. It is performed by the software developer or engineer during the construction phase of the software development life-cycle. Rather than replace traditional QA focuses, it augments it. Unit testing aims to eliminate construction errors before code is promoted to QA; this strategy is intended to increase the quality of the resulting software as well as the efficiency of the overall development and QA process.

Depending on the organization's expectations for software development, unit testing might include **static code analysis**, data flow analysis, metrics analysis, peer code reviews, code coverage analysis and other software verification practices.

Integration testing

Main article: **Integration testing**

Integration testing is any type of software testing that seeks to verify the interfaces between components against a software design. Software components may be integrated in an iterative way or all together ("big bang"). Normally the former is considered a better practice since it allows interface issues to be located more quickly and fixed.

Integration testing works to expose defects in the interfaces and interaction between integrated components (modules). Progressively larger groups of tested software components corresponding to elements of the architectural design are integrated and tested until the software works as a system.^[33]

Component interface testing

The practice of component interface testing can be used to check the handling of data passed between various units, or subsystem components, beyond full integration testing between those units.^{[34][35]} The data being passed can be considered as "message packets" and the range or data types can be checked, for data generated from one unit, and tested for validity before being passed into another unit. One option for interface testing is to keep a separate log file of data items being passed, often with a timestamp logged to allow analysis of thousands of cases of data passed between units for days or weeks. Tests can include checking the handling of some extreme data values while other interface variables are passed as normal values.^[34] Unusual data values in an interface can help explain unexpected performance in the next unit. Component interface testing is a variation of **black-box testing**,^[35] with the focus on the data values beyond just the related actions of a subsystem component.

System testing

Main article: **System testing**

System testing, or end-to-end testing, tests a completely integrated system to verify that it meets its requirements.^[36] For example, a system test might involve testing a logon interface, then creating and editing an entry, plus sending or printing results, followed by summary processing or deletion (or archiving) of entries, then logoff.

Operational Acceptance testing

Main article: **Operational acceptance testing**

Operational Acceptance is used to conduct operational readiness (pre-release) of a product, service or system as part of a **quality management system**. OAT is a common type of non-functional software testing, used mainly in **software development** and **software maintenance** projects. This type of testing focuses on the **operational readiness** of the system to be supported, and/or to become part of the production environment. Hence, it is also known as operational readiness testing (ORT) or **Operations Readiness and Assurance (OR&A)** testing. **Functional testing** within OAT is limited to those tests which are required to verify the *non-functional* aspects of the system.

In addition, the software testing should ensure that the portability of the system, as well as working as expected, does not also damage or partially corrupt its operating environment or cause other processes within that environment to become inoperative.^[37]

1.1.5 Testing Types

Installation testing

Main article: [Installation testing](#)

An installation test assures that the system is installed correctly and working at actual customer's hardware.

Compatibility testing

Main article: [Compatibility testing](#)

A common cause of software failure (real or perceived) is a lack of its **compatibility** with other **application software**, **operating systems** (or operating system versions, old or new), or target environments that differ greatly from the original (such as a **terminal** or **GUI** application intended to be run on the **desktop** now being required to become a **web application**, which must render in a **web browser**). For example, in the case of a lack of **backward compatibility**, this can occur because the programmers develop and test software only on the latest version of the target environment, which not all users may be running. This results in the unintended consequence that the latest work may not function on earlier versions of the target environment, or on older hardware that earlier versions of the target environment was capable of using. Sometimes such issues can be fixed by proactively **abstracting** operating system functionality into a separate program **module** or **library**.

Smoke and sanity testing

Sanity testing determines whether it is reasonable to proceed with further testing.

Smoke testing consists of minimal attempts to operate the software, designed to determine whether there are any basic problems that will prevent it from working at all. Such tests can be used as **build verification test**.

Regression testing

Main article: [Regression testing](#)

Regression testing focuses on finding defects after a major code change has occurred. Specifically, it seeks to uncover **software regressions**, as degraded or lost features, including old bugs that have come back. Such regressions occur whenever software functionality that was previously working, correctly, stops working as intended. Typically, regressions occur as an **unintended consequence** of program changes, when the newly developed part of the software collides with the previously existing code. Common methods of regression testing include re-running previous sets of test-cases and checking whether previously fixed faults have re-emerged. The depth of testing depends on the phase in the release process and the **risk** of the added features. They can either be complete, for changes added late in the release or deemed to be risky, or be very shallow, consisting of positive tests on each feature, if the changes are early in the release or deemed to be of low risk. Regression testing is typically the largest test effort in commercial software development,^[38] due to checking numerous details in prior software features, and even new software can be developed while using some old test-cases to test parts of the new design to ensure prior functionality is still supported.

Acceptance testing

Main article: [Acceptance testing](#)

Acceptance testing can mean one of two things:

1. A **smoke test** is used as an acceptance test prior to introducing a new build to the main testing process, i.e. before **integration** or **regression**.
2. Acceptance testing performed by the customer, often in their lab environment on their own hardware, is known as **user acceptance testing (UAT)**. Acceptance testing may be performed as part of the hand-off process between any two phases of development.

Alpha testing

Alpha testing is simulated or actual operational testing by potential users/customers or an independent test team at the developers' site. Alpha testing is often employed for off-the-shelf software as a form of internal acceptance testing, before the software goes to beta testing.^[39]

Beta testing

Beta testing comes after alpha testing and can be considered a form of external **user acceptance testing**. Versions of the software, known as **beta versions**, are released to a limited audience outside of the programming team known as beta testers. The software is released to groups of people so that further testing can ensure the product has few faults or **bugs**. Beta versions can be made available to the open public to increase the **feedback** field to a maximal number of future users and to deliver value earlier, for an extended or even infinite period of time (**perpetual beta**).

Functional vs non-functional testing

Functional testing refers to activities that verify a specific action or function of the code. These are usually found in the code requirements documentation, although some development methodologies work from use cases or user stories. Functional tests tend to answer the question of “can the user do this” or “does this particular feature work.”

Non-functional testing refers to aspects of the software that may not be related to a specific function or user action, such as **scalability** or other **performance**, behavior under certain **constraints**, or **security**. Testing will determine the breaking point, the point at which extremes of scalability or performance leads to unstable execution. Non-functional requirements tend to be those that reflect the quality of the product, particularly in the context of the suitability perspective of its users.

Destructive testing

Main article: **Destructive testing**

Destructive testing attempts to cause the software or a sub-system to fail. It verifies that the software functions properly even when it receives invalid or unexpected inputs, thereby establishing the **robustness** of input validation and error-management routines. **Software fault injection**, in the form of **fuzzing**, is an example of failure testing. Various commercial non-functional testing tools are linked from the **software fault injection** page; there are also numerous open-source and free software tools available that perform destructive testing.

Further information: **Exception handling** and **Recovery testing**

Software performance testing

Performance testing is generally executed to determine how a system or sub-system performs in terms of respon-

siveness and stability under a particular workload. It can also serve to investigate, measure, validate or verify other quality attributes of the system, such as scalability, reliability and resource usage.

Load testing is primarily concerned with testing that the system can continue to operate under a specific load, whether that be large quantities of data or a large number of **users**. This is generally referred to as software **scalability**. The related load testing activity of when performed as a non-functional activity is often referred to as **endurance testing**. **Volume testing** is a way to test software functions even when certain components (for example a file or database) increase radically in size. **Stress testing** is a way to test reliability under unexpected or rare workloads. **Stability testing** (often referred to as load or endurance testing) checks to see if the software can continuously function well in or above an acceptable period.

There is little agreement on what the specific goals of performance testing are. The terms load testing, performance testing, **scalability testing**, and volume testing, are often used interchangeably.

Real-time software systems have strict timing constraints. To test if timing constraints are met, **real-time testing** is used.

Usability testing

Usability testing is to check if the user interface is easy to use and understand. It is concerned mainly with the use of the application.

Accessibility testing

Accessibility testing may include compliance with standards such as:

- **Americans with Disabilities Act of 1990**
- **Section 508 Amendment to the Rehabilitation Act of 1973**
- **Web Accessibility Initiative (WAI) of the World Wide Web Consortium (W3C)**

Security testing

Security testing is essential for software that processes confidential data to prevent **system intrusion** by **hackers**.

The International Organization for Standardization (ISO) defines this as a “type of testing conducted to evaluate the degree to which a test item, and associated data and information, are protected to that unauthorised persons or systems cannot use, read or modify them, and authorized persons or systems are not denied access to them.”^[40]

Internationalization and localization

The general ability of software to be **internationalized and localized** can be automatically tested without actual translation, by using **pseudolocalization**. It will verify that the application still works, even after it has been translated into a new language or adapted for a new culture (such as different currencies or time zones).^[41]

Actual translation to human languages must be tested, too. Possible localization failures include:

- Software is often localized by translating a list of **strings** out of context, and the translator may choose the wrong translation for an ambiguous source string.
- Technical terminology may become inconsistent if the project is translated by several people without proper coordination or if the translator is imprudent.
- Literal word-for-word translations may sound inappropriate, artificial or too technical in the target language.
- Untranslated messages in the original language may be left **hard coded** in the source code.
- Some messages may be created automatically at **run time** and the resulting string may be ungrammatical, functionally incorrect, misleading or confusing.
- Software may use a **keyboard shortcut** which has no function on the source language's **keyboard layout**, but is used for typing characters in the layout of the target language.
- Software may lack support for the **character encoding** of the target language.
- Fonts and font sizes which are appropriate in the source language may be inappropriate in the target language; for example, **CJK characters** may become unreadable if the font is too small.
- A string in the target language may be longer than the software can handle. This may make the string partly invisible to the user or cause the software to crash or malfunction.
- Software may lack proper support for reading or writing **bi-directional text**.
- Software may display images with text that was not localized.
- Localized operating systems may have differently named system **configuration files** and **environment variables** and different **formats for date and currency**.

Development testing

Main article: **Development testing**

Development Testing is a software development process that involves synchronized application of a broad spectrum of defect prevention and detection strategies in order to reduce software development risks, time, and costs. It is performed by the software developer or engineer during the construction phase of the software development lifecycle. Rather than replace traditional QA focuses, it augments it. Development Testing aims to eliminate construction errors before code is promoted to QA; this strategy is intended to increase the quality of the resulting software as well as the efficiency of the overall development and QA process.

Depending on the organization's expectations for software development, Development Testing might include **static code analysis**, data flow analysis, metrics analysis, peer code reviews, unit testing, code coverage analysis, traceability, and other software verification practices.

A/B testing

Main article: **A/B testing**

A/B testing is basically a comparison of two outputs, generally when only one variable has changed: run a test, change one thing, run the test again, compare the results. This is more useful with more small-scale situations, but very useful in fine-tuning any program. With more complex projects, multivariant testing can be done.

Concurrent testing

Main article: **Concurrent testing**

In concurrent testing, the focus is more on what the performance is like when continuously running with normal input and under normal operation as opposed to stress testing, or fuzz testing. Memory leak is more easily found and resolved using this method, as well as more basic faults.

Conformance testing or type testing

Main article: **Conformance testing**

In software testing, conformance testing verifies that a product performs according to its specified standards. Compilers, for instance, are extensively tested to determine whether they meet the recognized standard for that language.

1.1.6 Testing process

Traditional waterfall development model

A common practice of software testing is that testing is performed by an independent group of testers after the functionality is developed, before it is shipped to the customer.^[42] This practice often results in the testing phase being used as a **project** buffer to compensate for project delays, thereby compromising the time devoted to testing.^[43]

Another practice is to start software testing at the same moment the project starts and it is a continuous process until the project finishes.^[44]

Further information: **Capability Maturity Model Integration** and **Waterfall model**

Agile or Extreme development model

In contrast, some emerging software disciplines such as **extreme programming** and the **agile software development** movement, adhere to a "test-driven software development" model. In this process, **unit tests** are written first, by the **software engineers** (often with **pair programming** in the extreme programming methodology). Of course these tests fail initially; as they are expected to. Then as code is written it passes incrementally larger portions of the test suites. The test suites are continuously updated as new failure conditions and corner cases are discovered, and they are integrated with any regression tests that are developed. Unit tests are maintained along with the rest of the software source code and generally integrated into the build process (with inherently interactive tests being relegated to a partially manual build acceptance process). The ultimate goal of this test process is to achieve **continuous integration** where software updates can be published to the public frequently.^{[45] [46]}

This methodology increases the testing effort done by development, before reaching any formal testing team. In some other development models, most of the test execution occurs after the requirements have been defined and the coding process has been completed.

Top-down and bottom-up

Bottom Up Testing is an approach to integrated testing where the lowest level components (modules, procedures, and functions) are tested first, then integrated and used to facilitate the testing of higher level components. After the integration testing of lower level integrated modules, the next level of modules will be formed and can be used for integration testing. The process is repeated until the components at the top of the hierarchy are tested. This approach is helpful only when all or most of the modules of the same development level are ready. This method

also helps to determine the levels of software developed and makes it easier to report testing progress in the form of a percentage.

Top Down Testing is an approach to integrated testing where the top integrated modules are tested and the branch of the module is tested step by step until the end of the related module.

In both, **method stubs** and **drivers** are used to stand-in for missing components and are replaced as the levels are completed.

A sample testing cycle

Although variations exist between organizations, there is a typical cycle for testing.^[47] The sample below is common among organizations employing the **Waterfall development** model. The same practices are commonly found in other development models, but might not be as clear or explicit.

- **Requirements analysis:** Testing should begin in the requirements phase of the **software development life cycle**. During the design phase, testers work to determine what aspects of a design are testable and with what parameters those tests work.
- **Test planning:** **Test strategy**, **test plan**, **testbed** creation. Since many activities will be carried out during testing, a plan is needed.
- **Test development:** Test procedures, **test scenarios**, **test cases**, test datasets, test scripts to use in testing software.
- **Test execution:** Testers execute the software based on the plans and test documents then report any errors found to the development team.
- **Test reporting:** Once testing is completed, testers generate metrics and make final reports on their **test effort** and whether or not the software tested is ready for release.
- **Test result analysis:** Or Defect Analysis, is done by the development team usually along with the client, in order to decide what defects should be assigned, fixed, rejected (i.e. found software working properly) or deferred to be dealt with later.
- **Defect Retesting:** Once a defect has been dealt with by the development team, it is retested by the testing team. AKA **Resolution testing**.
- **Regression testing:** It is common to have a small test program built of a subset of tests, for each integration of new, modified, or fixed software, in order to ensure that the latest delivery has not ruined anything, and that the software product as a whole is still working correctly.

- **Test Closure:** Once the test meets the exit criteria, the activities such as capturing the key outputs, lessons learned, results, logs, documents related to the project are archived and used as a reference for future projects.

1.1.7 Automated testing

Main article: [Test automation](#)

Many programming groups are relying more and more on [automated testing](#), especially groups that use [test-driven development](#). There are many frameworks to write tests in, and [continuous integration](#) software will run tests automatically every time code is checked into a [version control](#) system.

While automation cannot reproduce everything that a human can do (and all the ways they think of doing it), it can be very useful for regression testing. However, it does require a well-developed [test suite](#) of testing scripts in order to be truly useful.

Testing tools

Program testing and fault detection can be aided significantly by testing tools and [debuggers](#). Testing/debug tools include features such as:

- Program monitors, permitting full or partial monitoring of program code including:
 - [Instruction set simulator](#), permitting complete instruction level monitoring and trace facilities
 - [Hypervisor](#), permitting complete control of the execution of program code including:-
 - [Program animation](#), permitting step-by-step execution and conditional [breakpoint](#) at source level or in [machine code](#)
 - [Code coverage](#) reports
- Formatted dump or [symbolic debugging](#), tools allowing inspection of program variables on error or at chosen points
- Automated functional GUI testing tools are used to repeat system-level tests through the GUI
- [Benchmarks](#), allowing run-time performance comparisons to be made
- [Performance analysis](#) (or profiling tools) that can help to highlight [hot spots](#) and resource usage

Some of these features may be incorporated into a single composite tool or an [Integrated Development Environment](#) (IDE).

Measurement in software testing

Main article: [Software quality](#)

Usually, quality is constrained to such topics as [correctness](#), completeness, [security](#), but can also include more technical requirements as described under the [ISO standard ISO/IEC 9126](#), such as capability, [reliability](#), [efficiency](#), [portability](#), [maintainability](#), compatibility, and [usability](#).

There are a number of frequently used [software metrics](#), or measures, which are used to assist in determining the state of the software or the adequacy of the testing.

Hierarchy of testing difficulty Based on the amount of test cases required to construct a complete test suite in each context (i.e. a test suite such that, if it is applied to the implementation under test, then we collect enough information to precisely determine whether the system is correct or incorrect according to some specification), a hierarchy of testing difficulty has been proposed.^{[48] [49]} It includes the following [testability](#) classes:

- Class I: there exists a finite complete test suite.
- Class II: any partial distinguishing rate (i.e. any incomplete capability to distinguish correct systems from incorrect systems) can be reached with a finite test suite.
- Class III: there exists a countable complete test suite.
- Class IV: there exists a complete test suite.
- Class V: all cases.

It has been proved that each class is strictly included into the next. For instance, testing when we assume that the behavior of the implementation under test can be denoted by a deterministic [finite-state machine](#) for some known finite sets of inputs and outputs and with some known number of states belongs to Class I (and all subsequent classes). However, if the number of states is not known, then it only belongs to all classes from Class II on. If the implementation under test must be a deterministic finite-state machine failing the specification for a single trace (and its continuations), and its number of states is unknown, then it only belongs to classes from Class III on. Testing temporal machines where transitions are triggered if inputs are produced within some real-bounded interval only belongs to classes from Class IV on, whereas testing many non-deterministic systems only belongs to Class V (but not all, and some even belong to Class I). The inclusion into Class I does not require the simplicity of the assumed computation model, as some testing cases involving implementations written in any programming language, and testing implementations defined as machines

depending on continuous magnitudes, have been proved to be in Class I. Other elaborated cases, such as the testing framework by **Matthew Hennessy** under must semantics, and temporal machines with rational timeouts, belong to Class II.

1.1.8 Testing artifacts

The software testing process can produce several artifacts.

Test plan A **test plan** is a document detailing the objectives, target market, internal beta team, and processes for a specific beta test. The developers are well aware what test plans will be executed and this information is made available to management and the developers. The idea is to make them more cautious when developing their code or making additional changes. Some companies have a higher-level document called a **test strategy**.

Traceability matrix A **traceability matrix** is a table that correlates requirements or design documents to test documents. It is used to change tests when related source documents are changed, to select test cases for execution when planning for regression tests by considering requirement coverage.

Test case A **test case** normally consists of a unique identifier, requirement references from a design specification, preconditions, events, a series of steps (also known as actions) to follow, input, output, expected result, and actual result. Clinically defined a test case is an input and an expected result.^[50] This can be as pragmatic as 'for condition x your derived result is y', whereas other test cases described in more detail the input scenario and what results might be expected. It can occasionally be a series of steps (but often steps are contained in a separate test procedure that can be exercised against multiple test cases, as a matter of economy) but with one expected result or expected outcome. The optional fields are a test case ID, test step, or order of execution number, related requirement(s), depth, test category, author, and check boxes for whether the test is automatable and has been automated. Larger test cases may also contain prerequisite states or steps, and descriptions. A test case should also contain a place for the actual result. These steps can be stored in a word processor document, spreadsheet, database, or other common repository. In a database system, you may also be able to see past test results, who generated the results, and what system configuration was used to generate those results. These past results would usually be stored in a separate table.

Test script A **test script** is a procedure, or programming code that replicates user actions. Initially the term

was derived from the product of work created by automated regression test tools. Test Case will be a baseline to create test scripts using a tool or a program.

Test suite The most common term for a collection of test cases is a **test suite**. The test suite often also contains more detailed instructions or goals for each collection of test cases. It definitely contains a section where the tester identifies the system configuration used during testing. A group of test cases may also contain prerequisite states or steps, and descriptions of the following tests.

Test fixture or test data In most cases, multiple sets of values or data are used to test the same functionality of a particular feature. All the test values and changeable environmental components are collected in separate files and stored as test data. It is also useful to provide this data to the client and with the product or a project.

Test harness The software, tools, samples of data input and output, and configurations are all referred to collectively as a **test harness**.

1.1.9 Certifications

Several certification programs exist to support the professional aspirations of software testers and quality assurance specialists. No certification now offered actually requires the applicant to show their ability to test software. No certification is based on a widely accepted body of knowledge. This has led some to declare that the testing field is not ready for certification.^[51] Certification itself cannot measure an individual's productivity, their skill, or practical knowledge, and cannot guarantee their competence, or professionalism as a tester.^[52]

Software testing certification types *Exam-based:*

Formalized exams, which need to be passed; can also be learned by self-study [e.g., for **ISTQB** or **QAI**]^[53]

Education-based: Instructor-led sessions, where each course has to be passed [e.g., International Institute for Software Testing (IIST)]

Testing certifications

ISEB offered by the Information Systems Examinations Board

ISTQB Certified Tester, Foundation Level (CTFL) offered by the International Software Testing Qualification Board^{[54][55]}

ISTQB Certified Tester, Advanced Level (CTAL) offered by the International Software Testing Qualification Board^{[54][55]}

Quality assurance certifications CSQE offered by the American Society for Quality (ASQ)^[56]

CQIA offered by the American Society for Quality (ASQ)^[56]

of the context-driven school of software testing about the ISO 29119 standard. Professional testing associations, such as The International Society for Software Testing, are driving the efforts to have the standard withdrawn.^{[65][66]}

1.1.10 Controversy

Some of the major software testing controversies include:

What constitutes responsible software testing?

Members of the “context-driven” school of testing^[57] believe that there are no “best practices” of testing, but rather that testing is a set of skills that allow the tester to select or invent testing practices to suit each unique situation.^[58]

Agile vs. traditional Should testers learn to work under conditions of uncertainty and constant change or should they aim at process “maturity”? The agile testing movement has received growing popularity since 2006 mainly in commercial circles,^{[59][60]} whereas government and military^[61] software providers use this methodology but also the traditional test-last models (e.g. in the Waterfall model).

Exploratory test vs. scripted ^[62] Should tests be designed at the same time as they are executed or should they be designed beforehand?

Manual testing vs. automated Some writers believe that test automation is so expensive relative to its value that it should be used sparingly.^[63] More in particular, test-driven development states that developers should write unit-tests, as those of XUnit, before coding the functionality. The tests then can be considered as a way to capture and implement the requirements. As a general rule, the larger the system and the greater the complexity, the greater the ROI in test automation. Also, the investment in tools and expertise can be amortized over multiple projects with the right level of knowledge sharing within an organization.

Software design vs. software implementation

Should testing be carried out only at the end or throughout the whole process?

Who watches the watchmen? The idea is that any form of observation is also an interaction — the act of testing can also affect that which is being tested.^[64]

Is the existence of the ISO 29119 software testing standard justified?

Significant opposition has formed out of the ranks

1.1.11 Related processes

Software verification and validation

Main articles: Verification and validation (software) and Software quality control

Software testing is used in association with verification and validation:^[67]

- Verification: Have we built the software right? (i.e., does it implement the requirements).
- Validation: Have we built the right software? (i.e., do the deliverables satisfy the customer).

The terms verification and validation are commonly used interchangeably in the industry; it is also common to see these two terms incorrectly defined. According to the IEEE Standard Glossary of Software Engineering Terminology:

Verification is the process of evaluating a system or component to determine whether the products of a given development phase satisfy the conditions imposed at the start of that phase.

Validation is the process of evaluating a system or component during or at the end of the development process to determine whether it satisfies specified requirements.

According to the ISO 9000 standard:

Verification is confirmation by examination and through provision of objective evidence that specified requirements have been fulfilled.

Validation is confirmation by examination and through provision of objective evidence that the requirements for a specific intended use or application have been fulfilled.

Software quality assurance (SQA)

Software testing is a part of the software quality assurance (SQA) process.^[4] In SQA, software process specialists and auditors are concerned for the software development process rather than just the artifacts such as documentation, code and systems. They examine and change the

software engineering process itself to reduce the number of faults that end up in the delivered software: the so-called “defect rate”. What constitutes an “acceptable defect rate” depends on the nature of the software; A flight simulator video game would have much higher defect tolerance than software for an actual airplane. Although there are close links with SQA, testing departments often exist independently, and there may be no SQA function in some companies.

Software testing is a task intended to detect defects in software by contrasting a computer program’s expected results with its actual results for a given set of inputs. By contrast, QA (quality assurance) is the implementation of policies and procedures intended to prevent defects from occurring in the first place.

1.1.12 See also

- Category:Software testing
- Dynamic program analysis
- Formal verification
- Independent test organization
- Manual testing
- Orthogonal array testing
- Pair testing
- Reverse semantic traceability
- Software testability
- Orthogonal Defect Classification
- Test Environment Management
- Test management tools
- Web testing

1.1.13 References

- [1] Kaner, Cem (November 17, 2006). “Exploratory Testing” (PDF). Florida Institute of Technology, *Quality Assurance Institute Worldwide Annual Software Testing Conference*, Orlando, FL. Retrieved November 22, 2014.
- [2] Software Testing by Jiantao Pan, Carnegie Mellon University
- [3] Leitner, A., Ciupa, I., Oriol, M., Meyer, B., Fiva, A., “Contract Driven Development = Test Driven Development – Writing Test Cases”, Proceedings of ESEC/FSE’07: European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering 2007, (Dubrovnik, Croatia), September 2007
- [4] Kaner, Cem; Falk, Jack; Nguyen, Hung Quoc (1999). *Testing Computer Software, 2nd Ed.* New York, et al: John Wiley and Sons, Inc. p. 480. ISBN 0-471-35846-0.
- [5] Kolawa, Adam; Huizinga, Dorota (2007). *Automated Defect Prevention: Best Practices in Software Management*. Wiley-IEEE Computer Society Press. pp. 41–43. ISBN 0-470-04212-5.
- [6] Kolawa, Adam; Huizinga, Dorota (2007). *Automated Defect Prevention: Best Practices in Software Management*. Wiley-IEEE Computer Society Press. p. 426. ISBN 0-470-04212-5.
- [7] Section 1.1.2, Certified Tester Foundation Level Syllabus, International Software Testing Qualifications Board
- [8] Principle 2, Section 1.3, Certified Tester Foundation Level Syllabus, International Software Testing Qualifications Board
- [9] “Proceedings from the 5th International Conference on Software Testing and Validation (ICST). Software Competence Center Hagenberg. “Test Design: Lessons Learned and Practical Implications.”.
- [10] Software errors cost U.S. economy \$59.5 billion annually, NIST report
- [11] McConnell, Steve (2004). *Code Complete* (2nd ed.). Microsoft Press. p. 29. ISBN 0735619670.
- [12] Bossavit, Laurent (2013-11-20). *The Leprechauns of Software Engineering--How folklore turns into fact and what to do about it*. Chapter 10: leanpub.
- [13] see D. Gelperin and W.C. Hetzel
- [14] Myers, Glenford J. (1979). *The Art of Software Testing*. John Wiley and Sons. ISBN 0-471-04328-1.
- [15] Company, People’s Computer (1987). “Dr. Dobb’s journal of software tools for the professional programmer”. *Dr. Dobb’s journal of software tools for the professional programmer* (M&T Pub) **12** (1–6): 116.
- [16] Gelperin, D.; B. Hetzel (1988). “The Growth of Software Testing”. *CACM* **31** (6): 687–695. doi:10.1145/62959.62965. ISSN 0001-0782.
- [17] *until 1956 it was the debugging oriented period, when testing was often associated to debugging: there was no clear difference between testing and debugging*. Gelperin, D.; B. Hetzel (1988). “The Growth of Software Testing”. *CACM* **31** (6). ISSN 0001-0782.
- [18] *From 1957–1978 there was the demonstration oriented period where debugging and testing was distinguished now – in this period it was shown, that software satisfies the requirements*. Gelperin, D.; B. Hetzel (1988). “The Growth of Software Testing”. *CACM* **31** (6). ISSN 0001-0782.
- [19] *The time between 1979–1982 is announced as the destruction oriented period, where the goal was to find errors*. Gelperin, D.; B. Hetzel (1988). “The Growth of Software Testing”. *CACM* **31** (6). ISSN 0001-0782.

- [20] 1983–1987 is classified as the evaluation oriented period: intention here is that during the software lifecycle a product evaluation is provided and measuring quality. Gelperin, D.; B. Hetzel (1988). “The Growth of Software Testing”. *CACM* **31** (6). ISSN 0001-0782.
- [21] From 1988 on it was seen as prevention oriented period where tests were to demonstrate that software satisfies its specification, to detect faults and to prevent faults. Gelperin, D.; B. Hetzel (1988). “The Growth of Software Testing”. *CACM* **31** (6). ISSN 0001-0782.
- [22] Introduction, Code Coverage Analysis, Steve Cornett
- [23] Ron, Patton. *Software Testing*.
- [24] Laycock, G. T. (1993). “The Theory and Practice of Specification Based Software Testing” (PostScript). Dept of Computer Science, Sheffield University, UK. Retrieved 2008-02-13.
- [25] Bach, James (June 1999). “Risk and Requirements-Based Testing” (PDF). *Computer* **32** (6): 113–114. Retrieved 2008-08-19.
- [26] Savenkov, Roman (2008). *How to Become a Software Tester*. Roman Savenkov Consulting. p. 159. ISBN 978-0-615-23372-7.
- [27] “Visual testing of software – Helsinki University of Technology” (PDF). Retrieved 2012-01-13.
- [28] “Article on visual testing in Test Magazine”. Test-magazine.co.uk. Retrieved 2012-01-13.
- [29] Patton, Ron. *Software Testing*.
- [30] “SOA Testing Tools for Black, White and Gray Box SOA Testing Techniques”. Crosschecknet.com. Retrieved 2012-12-10.
- [31] “SWEBOK Guide – Chapter 5”. Computer.org. Retrieved 2012-01-13.
- [32] Binder, Robert V. (1999). *Testing Object-Oriented Systems: Objects, Patterns, and Tools*. Addison-Wesley Professional. p. 45. ISBN 0-201-80938-9.
- [33] Beizer, Boris (1990). *Software Testing Techniques* (Second ed.). New York: Van Nostrand Reinhold. pp. 21,430. ISBN 0-442-20672-0.
- [34] Clapp, Judith A. (1995). *Software Quality Control, Error Analysis, and Testing*. p. 313. ISBN 0815513631.
- [35] Mathur, Aditya P. (2008). *Foundations of Software Testing*. Purdue University. p. 18. ISBN 978-8131716601.
- [36] IEEE (1990). *IEEE Standard Computer Dictionary: A Compilation of IEEE Standard Computer Glossaries*. New York: IEEE. ISBN 1-55937-079-3.
- [37] Whitepaper: Operational Acceptance – an application of the ISO 29119 Software Testing standard. May 2015 Anthony Woods, Capgemini
- [38] Paul Ammann; Jeff Offutt (2008). *Introduction to Software Testing*. p. 215 of 322 pages.
- [39] van Veenendaal, Erik. “Standard glossary of terms used in Software Testing”. Retrieved 4 January 2013.
- [40] ISO/IEC/IEEE 29119-1:2013 – Software and Systems Engineering – Software Testing – Part 1 – Concepts and Definitions; Section 4.38
- [41] “Globalization Step-by-Step: The World-Ready Approach to Testing. Microsoft Developer Network”. Msdn.microsoft.com. Retrieved 2012-01-13.
- [42] EtestingHub-Online Free Software Testing Tutorial. “e)Testing Phase in Software Testing”. Etestinghub.com. Retrieved 2012-01-13.
- [43] Myers, Glenford J. (1979). *The Art of Software Testing*. John Wiley and Sons. pp. 145–146. ISBN 0-471-04328-1.
- [44] Dustin, Elfriede (2002). *Effective Software Testing*. Addison Wesley. p. 3. ISBN 0-201-79429-2.
- [45] Marchenko, Artem (November 16, 2007). “XP Practice: Continuous Integration”. Retrieved 2009-11-16.
- [46] Gurses, Levent (February 19, 2007). “Agile 101: What is Continuous Integration?”. Retrieved 2009-11-16.
- [47] Pan, Jiantao (Spring 1999). “Software Testing (18-849b Dependable Embedded Systems)”. *Topics in Dependable Embedded Systems*. Electrical and Computer Engineering Department, Carnegie Mellon University.
- [48] Rodríguez, Ismael; Llana, Luis; Rabanal, Pablo (2014). “A General Testability Theory: Classes, properties, complexity, and testing reductions”. *IEEE Transactions on Software Engineering* **40** (9): 862–894. doi:10.1109/TSE.2014.2331690. ISSN 0098-5589.
- [49] Rodríguez, Ismael (2009). “A General Testability Theory”. *CONCUR 2009 - Concurrency Theory, 20th International Conference, CONCUR 2009, Bologna, Italy, September 1–4, 2009. Proceedings*. pp. 572–586. doi:10.1007/978-3-642-04081-8_38. ISBN 978-3-642-04080-1.
- [50] IEEE (1998). *IEEE standard for software test documentation*. New York: IEEE. ISBN 0-7381-1443-X.
- [51] Kaner, Cem (2001). “NSF grant proposal to “lay a foundation for significant improvements in the quality of academic and commercial courses in software testing”” (PDF).
- [52] Kaner, Cem (2003). “Measuring the Effectiveness of Software Testers” (PDF).
- [53] Black, Rex (December 2008). *Advanced Software Testing- Vol. 2: Guide to the ISTQB Advanced Certification as an Advanced Test Manager*. Santa Barbara: Rocky Nook Publisher. ISBN 1-933952-36-9.
- [54] “ISTQB”.
- [55] “ISTQB in the U.S.”.
- [56] “American Society for Quality”. Asq.org. Retrieved 2012-01-13.

- [57] “context-driven-testing.com”. context-driven-testing.com. Retrieved 2012-01-13.
- [58] “Article on taking agile traits without the agile method”. Technicat.com. Retrieved 2012-01-13.
- [59] “We're all part of the story” by David Strom, July 1, 2009
- [60] IEEE article about differences in adoption of agile trends between experienced managers vs. young students of the Project Management Institute. See also Agile adoption study from 2007
- [61] Willison, John S. (April 2004). “Agile Software Development for an Agile Force”. *CrossTalk* (STSC) (April 2004). Archived from the original on October 29, 2005.
- [62] “IEEE article on Exploratory vs. Non Exploratory testing” (PDF). Ieeexplore.ieee.org. Retrieved 2012-01-13.
- [63] An example is Mark Fewster, Dorothy Graham: *Software Test Automation*. Addison Wesley, 1999, ISBN 0-201-33140-3.
- [64] Microsoft Development Network Discussion on exactly this topic
- [65] Stop 29119
- [66] Infoworld.com
- [67] Tran, Eushuan (1999). “Verification/Validation/Certification”. In Koopman, P. *Topics in Dependable Embedded Systems*. USA: Carnegie Mellon University. Retrieved 2008-01-13.

1.1.14 Further reading

- Bertrand Meyer, “Seven Principles of Software Testing,” *Computer*, vol. 41, no. 8, pp. 99–101, Aug. 2008, doi:10.1109/MC.2008.306; available online.

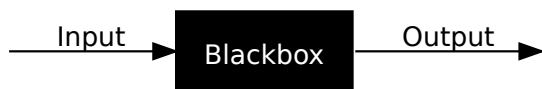
1.1.15 External links

- Software testing tools and products at DMOZ
- “Software that makes Software better” Economist.com
- “What You Need to Know About Software Beta Tests” Centercode.com

Chapter 2

Black-box testing

2.1 Black-box testing



Black-box diagram

Black-box testing is a method of software testing that examines the functionality of an application without peering into its internal structures or workings. This method of test can be applied to virtually every level of software testing: unit, integration, system and acceptance. It typically comprises most if not all higher level testing, but can also dominate unit testing as well.

2.1.1 Test procedures

Specific knowledge of the application's code/internal structure and programming knowledge in general is not required. The tester is aware of *what* the software is supposed to do but is not aware of *how* it does it. For instance, the tester is aware that a particular input returns a certain, invariable output but is not aware of *how* the software produces the output in the first place.^[1]

Test cases

Test cases are built around specifications and requirements, i.e., what the application is supposed to do. Test cases are generally derived from external descriptions of the software, including specifications, requirements and design parameters. Although the tests used are primarily *functional* in nature, *non-functional* tests may also be used. The test designer selects both valid and invalid inputs and determines the correct output, often with the help of an oracle or a previous result that is known to be good, without any knowledge of the test object's internal structure.

Test design techniques

Typical black-box test design techniques include:

- Decision table testing
- All-pairs testing
- Equivalence partitioning
- Boundary value analysis
- Cause–effect graph
- Error guessing

2.1.2 Hacking

In penetration testing, black-box testing refers to a methodology where an ethical hacker has no knowledge of the system being attacked. The goal of a black-box penetration test is to simulate an external hacking or cyber warfare attack.

2.1.3 See also

- Acceptance testing
- Boundary testing
- Fuzz testing
- Metasploit Project
- Sanity testing
- Smoke testing
- Software testing
- Stress testing
- Test automation
- Web Application Security Scanner
- White hat hacker
- White-box testing

- Grey box testing
- Blind experiment
- ABX test
- Performance Testing

2.1.4 References

[1] Ron, Patton. *Software Testing*.

2.1.5 External links

- BCS SIGIST (British Computer Society Specialist Interest Group in Software Testing): *Standard for Software Component Testing*, Working Draft 3.4, 27. April 2001.

2.2 Exploratory testing

Exploratory testing is an approach to **software testing** that is concisely described as simultaneous learning, **test design** and test execution. **Cem Kaner**, who coined the term in 1993,^[1] now defines exploratory testing as “a style of software testing that emphasizes the personal freedom and responsibility of the individual tester to continually optimize the quality of his/her work by treating test-related learning, test design, test execution, and test result interpretation as mutually supportive activities that run in parallel throughout the project.”^[2]

While the software is being tested, the tester learns things that together with experience and **creativity** generates new good tests to run. Exploratory testing is often thought of as a **black box testing** technique. Instead, those who have studied it consider it a test *approach* that can be applied to any test technique, at any stage in the development process. The key is not the test technique nor the item being tested or reviewed; the key is the cognitive engagement of the tester, and the tester’s responsibility for managing his or her time.^[3]

2.2.1 History

Exploratory testing has always been performed by skilled testers. In the early 1990s, **ad hoc** was too often synonymous with sloppy and careless work. As a result, a group of test methodologists (now calling themselves the **Context-Driven School**) began using the term “exploratory” seeking to emphasize the dominant thought process involved in unscripted testing, and to begin to develop the practice into a teachable discipline. This new terminology was first published by Cem Kaner in his book *Testing Computer Software*^[1] and expanded upon in *Lessons Learned in Software Testing*.^[4] Exploratory testing can be as disciplined as any other intellectual activity.

2.2.2 Description

Exploratory testing seeks to find out how the software actually works, and to ask questions about how it will handle difficult and easy cases. The quality of the testing is dependent on the tester’s skill of inventing **test cases** and finding **defects**. The more the tester knows about the product and different **test methods**, the better the testing will be.

To further explain, comparison can be made of freestyle exploratory testing to its antithesis **scripted testing**. In this activity test cases are designed in advance. This includes both the individual steps and the expected results. These tests are later performed by a tester who compares the actual result with the expected. When performing exploratory testing, expectations are open. Some results may be predicted and expected; others may not. The tester configures, operates, observes, and evaluates the product and its behaviour, critically investigating the result, and reporting information that seems likely to be a bug (which threatens the value of the product to some person) or an issue (which threatens the quality of the testing effort).

In reality, testing almost always is a combination of exploratory and scripted testing, but with a tendency towards either one, depending on context.

According to **Cem Kaner & James Marcus Bach**, exploratory testing is more a **mindset** or “...a way of thinking about testing” than a methodology.^[5] They also say that it crosses a continuum from slightly exploratory (slightly ambiguous or vaguely scripted testing) to highly exploratory (freestyle exploratory testing).^[6]

The documentation of exploratory testing ranges from documenting all tests performed to just documenting the **bugs**. During **pair testing**, two persons create test cases together; one performs them, and the other documents. **Session-based testing** is a method specifically designed to make exploratory testing auditable and measurable on a wider scale.

Exploratory testers often use tools, including screen capture or video tools as a record of the exploratory session, or tools to quickly help generate situations of interest, e.g. James Bach’s Perlclip.

2.2.3 Benefits and drawbacks

The main advantage of exploratory testing is that less preparation is needed, important bugs are found quickly, and at execution time, the approach tends to be more intellectually stimulating than execution of scripted tests.

Another major benefit is that testers can use **deductive reasoning** based on the results of previous results to guide their future testing on the fly. They do not have to complete a current series of scripted tests before focusing in on or moving on to exploring a more target rich environ-

ment. This also accelerates bug detection when used intelligently.

Another benefit is that, after initial testing, most bugs are discovered by some sort of exploratory testing. This can be demonstrated logically by stating, “Programs that pass certain tests tend to continue to pass the same tests and are more likely to fail other tests or scenarios that are yet to be explored.”

Disadvantages are that tests invented and performed on the fly can’t be reviewed in advance (and by that prevent errors in code and test cases), and that it can be difficult to show exactly which tests have been run.

Freestyle exploratory test ideas, when revisited, are unlikely to be performed in exactly the same manner, which can be an advantage if it is important to find new errors; or a disadvantage if it is more important to repeat specific details of the earlier tests. This can be controlled with specific instruction to the tester, or by preparing automated tests where feasible, appropriate, and necessary, and ideally as close to the unit level as possible.

2.2.4 Usage

Exploratory testing is particularly suitable if **requirements** and **specifications** are incomplete, or if there is lack of time.^{[7][8]} The approach can also be used to verify that previous testing has found the most important defects.^[7]

2.2.5 See also

- **Ad hoc testing**

2.2.6 References

- [1] Kaner, Falk, and Nguyen, *Testing Computer Software (Second Edition)*, Van Nostrand Reinhold, New York, 1993. p. 6, 7-11.
- [2] Cem Kaner, *A Tutorial in Exploratory Testing*, p. 36.
- [3] Cem Kaner, *A Tutorial in Exploratory Testing*, p. 37-39, 40- .
- [4] Kaner, Cem; Bach, James; Pettichord, Bret (2001). *Lessons Learned in Software Testing*. John Wiley & Sons. ISBN 0-471-08112-4.
- [5] Cem Kaner, James Bach, *Exploratory & Risk Based Testing*, www.testingeducation.org, 2004, p. 10
- [6] Cem Kaner, James Bach, *Exploratory & Risk Based Testing*, www.testingeducation.org, 2004, p. 14
- [7] Bach, James (2003). “Exploratory Testing Explained” (PDF). satisfice.com. p. 7. Retrieved October 23, 2010.
- [8] Kaner, Cem (2008). “A Tutorial in Exploratory Testing” (PDF). kaner.com. p. 37, 118. Retrieved October 23, 2010.

2.2.7 External links

- James Bach, *Exploratory Testing Explained*
- Cem Kaner, James Bach, *The Nature of Exploratory Testing*, 2004
- Cem Kaner, James Bach, *The Seven Basic Principles of the Context-Driven School*
- Jonathan Kohl, *Exploratory Testing: Finding the Music of Software Investigation*, Kohl Concepts Inc., 2007
- Chris Agruss, Bob Johnson, *Ad Hoc Software Testing*

2.3 Session-based testing

Session-based testing is a **software test** method that aims to combine **accountability** and **exploratory testing** to provide rapid defect discovery, creative on-the-fly **test design**, management control and metrics reporting. The method can also be used in conjunction with **scenario testing**. Session-based testing was developed in 2000 by Jonathan and **James Bach**.

Session-based testing can be used to introduce measurement and control to an immature test process and can form a foundation for significant improvements in productivity and error detection. Session-based testing can offer benefits when formal **requirements** are not present, incomplete, or changing rapidly.

2.3.1 Elements of session-based testing

Mission

The mission in Session Based Test Management identifies the purpose of the session, helping to focus the session while still allowing for exploration of the system under test. According to Jon Bach, one of the co-founders of the methodology, the mission tells us “what we are testing or what problems we are looking for.”^[1]

Charter

A charter is a goal or agenda for a test session. Charters are created by the test team prior to the start of testing, but they may be added or changed at any time. Often charters are created from a **specification**, **test plan**, or by examining results from previous sessions.

Session

An uninterrupted period of time spent testing, ideally lasting one to two hours. Each session is focused on a

charter, but testers can also explore new opportunities or issues during this time. The tester creates and executes tests based on ideas, heuristics or whatever frameworks to guide them and records their progress. This might be through the use of written notes, video capture tools or by whatever method as deemed appropriate by the tester.

Session report

The session report records the test session. Usually this includes:

- Charter.
- Area tested.
- Detailed notes on how testing was conducted.
- A list of any bugs found.
- A list of issues (open questions, product or project concerns)
- Any files the tester used or created to support their testing
- Percentage of the session spent on the charter vs investigating new opportunities.
- Percentage of the session spent on:
 - Testing - creating and executing tests.
 - Bug investigation / reporting.
 - Session setup or other non-testing activities.
- Session Start time and duration.

Debrief

A debrief is a short discussion between the manager and tester (or testers) about the session report. Jonathan Bach uses the acronym PROOF to help structure his debriefing. PROOF stands for:-

- Past. What happened during the session?
- Results. What was achieved during the session?
- Obstacles. What got in the way of good testing?
- Outlook. What still needs to be done?
- Feelings. How does the tester feel about all this?^[2]

Parsing results

With a standardized Session Report, software tools can be used to parse and store the results as aggregate data for reporting and metrics. This allows reporting on the number of sessions per area or a breakdown of time spent on testing, bug investigation, and setup / other activities.

2.3.2 Planning

Testers using session-based testing can adjust their testing daily to fit the needs of the project. Charters can be added or dropped over time as tests are executed and/or requirements change.

2.3.3 See also

- Software testing
- Test case
- Test script
- Exploratory testing
- Scenario testing

2.3.4 References

[1] First published 11/2000 in STQE magazine, today known as Better Software <http://www.stickyminds.com/BetterSoftware/magazine.asp>

[2] <http://www.satisfice.com/articles/sbtm.pdf>

2.3.5 External links

- Session-Based Test Management Site
- How to Manage and Measure ET
- Session-Based Test Lite
- Adventures in Session-Based Testing
- Session-Based Test Management
- Better Software Magazine

2.4 Scenario testing

Scenario testing is a software testing activity that uses scenarios: hypothetical stories to help the tester work through a complex problem or test system. The ideal scenario test is a credible, complex, compelling or motivating story the outcome of which is easy to evaluate.^[1] These tests are usually different from test cases in that test cases are single steps whereas scenarios cover a number of steps.^{[2][3]}

2.4.1 History

Kaner coined the phrase scenario test by October 2003.^[1] He commented that one of the most difficult aspects of testing was maintaining step-by-step test cases along with their expected results. His paper attempted to find a way to reduce the re-work of complicated written tests and incorporate the ease of use cases.^[1]

A few months later, Buwalda wrote about a similar approach he had been using that he called “soap opera testing”. Like television soap operas these tests were both exaggerated in activity and condensed in time.^[2] The key to both approaches was to avoid step-by-step testing instructions with expected results and instead replaced them with a narrative that gave freedom to the tester while confining the scope of the test.^[3]

2.4.2 Methods

System scenarios

In this method only those sets of realistic, user activities that cover several components in the system are used as scenario tests. Development of system scenario can be done using:

1. Story lines
2. State transitions
3. Business verticals
4. Implementation story from customers

Use-case and role-based scenarios

In this method the focus is on how a user uses the system with different roles and environment.^[4]

2.4.3 See also

- Test script
- Test suite
- Session-based testing

2.4.4 References

- [1] “An Introduction to Scenario Testing” (PDF). Cem Kaner. Retrieved 2009-05-07.
- [2] Buwalda, Hans (2004). “Soap Opera Testing” (PDF). *Better Software* (Software Quality Engineering) (February 2004): 30–7. Retrieved 2011-11-16.
- [3] Crispin, Lisa; Gregory, Janet (2009). *Agile Testing: A Practical Guide for Testers and Agile Teams*. Addison-Wesley. pp. 192–5. ISBN 81-317-3068-9.

- [4] Gopalaswamy, Srinivasan Desikan. *Software Testing: Principles and Practice*.

2.5 Equivalence partitioning

Equivalence partitioning (also called **Equivalence Class Partitioning** or **ECP**^[1]) is a **software testing** technique that divides the input data of a software unit into partitions of equivalent data from which test cases can be derived. In principle, test cases are designed to cover each partition at least once. This technique tries to define test cases that uncover classes of errors, thereby reducing the total number of test cases that must be developed. An advantage of this approach is reduction in the time required for testing a software due to lesser number of test cases.

Equivalence partitioning is typically applied to the inputs of a tested component, but may be applied to the outputs in rare cases. The equivalence partitions are usually derived from the requirements specification for input attributes that influence the processing of the test object.

The fundamental concept of ECP comes from **equivalence class** which in turn comes from **equivalence relation**. A software system is in effect a **computable function** implemented as an **algorithm** in some implementation **programming language**. Given an input test vector some instructions of that algorithm get covered, (see **code coverage** for details) others do not. This gives the interesting relationship between input test vectors:- aC_b is an **equivalence relation** between test vectors a, b if and only if the coverage foot print of the vectors a, b are exactly the same, that is, they cover the same instructions, at same step. This would evidently mean that the relation cover C would partition the input **vector space** of the **test vector** into multiple **equivalence class**. This partitioning is called **equivalence class partitioning** of test input. If there are N equivalent classes, only N vectors are sufficient to fully cover the system.

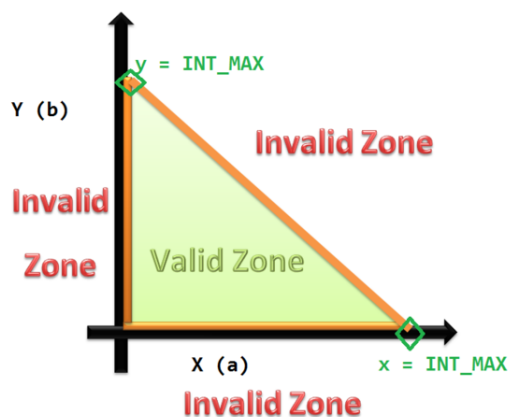
The demonstration can be done using a function written in C:

```
int safe_add( int a, int b ) { int c = a + b; if ( a >= 0
&& b >= 0 && c < 0 ) { fprintf ( stderr, "Overflow!\n"
); } if ( a < 0 && b < 0 && c >= 0 ) { fprintf ( stderr,
"Underflow!\n" ); } return c; }
```

On the basis of the code, the input vectors of $[a, b]$ are partitioned. The blocks we need to cover are the overflow statement and the underflow statement and neither of these 2. That gives rise to 3 equivalent classes, from the code review itself.

To solve the input problem, we take refuge in the **inequation** $z_{min} \leq x + y \leq z_{max}$

we note that there is a fixed size of **Integer** (computer science) hence, the z can be replaced with:- $INT_MIN \leq x + y \leq INT_MAX$



Demonstrating Equivalence Class Partitioning

and

with $x \in \{INT_MIN, \dots, INT_MAX\}$ and $y \in \{INT_MIN, \dots, INT_MAX\}$

The values of the **test vector** at the strict condition of the equality that is $INT_MIN = x + y$ and $INT_MAX = x + y$ are called the boundary values, **Boundary-value analysis** has detailed information about it. Note that the graph only covers the overflow case, first quadrant for X and Y positive values.

In general an input has certain ranges which are valid and other ranges which are invalid. Invalid data here does not mean that the data is incorrect, it means that this data lies outside of specific partition. This may be best explained by the example of a function which takes a parameter “month”. The valid range for the month is 1 to 12, representing January to December. This valid range is called a partition. In this example there are two further partitions of invalid ranges. The first invalid partition would be ≤ 0 and the second invalid partition would be ≥ 13 .

... -2 -1 0 1 12 13 14 15 -----|-----
 -----|----- invalid partition 1 valid
 partition invalid partition 2

The testing theory related to equivalence partitioning says that only one test case of each partition is needed to evaluate the behaviour of the program for the related partition. In other words it is sufficient to select one test case out of each partition to check the behaviour of the program. To use more or even all test cases of a partition will not find new faults in the program. The values within one partition are considered to be “equivalent”. Thus the number of test cases can be reduced considerably.

An additional effect of applying this technique is that you also find the so-called “dirty” test cases. An inexperienced tester may be tempted to use as test cases the input data 1 to 12 for the month and forget to select some out of the invalid partitions. This would lead to a huge number of unnecessary test cases on the one hand, and a lack of test cases for the dirty ranges on the other hand.

The tendency is to relate equivalence partitioning to so called **black box testing** which is strictly checking a software component at its interface, without consideration of internal structures of the software. But having a closer look at the subject there are cases where it applies to **grey box testing** as well. Imagine an interface to a component which has a valid range between 1 and 12 like the example above. However internally the function may have a differentiation of values between 1 and 6 and the values between 7 and 12. Depending upon the input value the software internally will run through different paths to perform slightly different actions. Regarding the input and output interfaces to the component this difference will not be noticed, however in your grey-box testing you would like to make sure that both paths are examined. To achieve this it is necessary to introduce additional equivalence partitions which would not be needed for black-box testing. For this example this would be:

... -2 -1 0 1 6 7 12 13 14 15 -----|-----
 -----|----- invalid partition 1 P1
 P2 invalid partition 2 valid partitions

To check for the expected results you would need to evaluate some internal intermediate values rather than the output interface. It is not necessary that we should use multiple values from each partition. In the above scenario we can take -2 from invalid partition 1, 6 from valid partition P1, 7 from valid partition P2 and 15 from invalid partition 2.

Equivalence partitioning is not a stand alone method to determine test cases. It has to be supplemented by **boundary value analysis**. Having determined the partitions of possible inputs the method of boundary value analysis has to be applied to select the most effective test cases out of these partitions.

2.5.1 Further reading

- The Testing Standards Working Party website
- Parteg, a free test generation tool that is combining test path generation from UML state machines with equivalence class generation of input values.
-

2.5.2 References

- [1] Burnstein, Ilene (2003), *Practical Software Testing*, Springer-Verlag, p. 623, ISBN 0-387-95131-8

2.6 Boundary-value analysis

Boundary value analysis is a software testing technique in which tests are designed to include representatives of boundary values in a range. The idea comes from the

boundary. Given that we have a set of **test vectors** to test the system, a topology can be defined on that set. Those inputs which belong to the same **equivalence class** as defined by the **equivalence partitioning** theory would constitute the **basis**. Given that the basis sets are **neighbors**, there would exist a boundary between them. The test vectors on either side of the boundary are called boundary values. In practice this would require that the test vectors can be ordered, and that the individual parameters follows some kind of order (either **partial order** or **total order**).

2.6.1 Formal Definition

Formally the boundary values can be defined as below:- Let the set of the **test vectors** be $X_1, \dots, X_n$. Let's assume that there is an ordering relation defined over them, as $\leq$. Let C_1, C_2 be two equivalent classes. Assume that test vector $X_1 \in C_1$ and $X_2 \in C_2$. If $X_1 \leq X_2$ or $X_2 \leq X_1$ then the classes C_1, C_2 are in the same **neighborhood** and the values X_1, X_2 are boundary values.

In plainer English, values on the minimum and maximum edges of an **equivalence partition** are tested. The values could be input or output ranges of a software component, can also be the internal implementation. Since these boundaries are common locations for errors that result in software **faults** they are frequently exercised in **test cases**.

2.6.2 Application

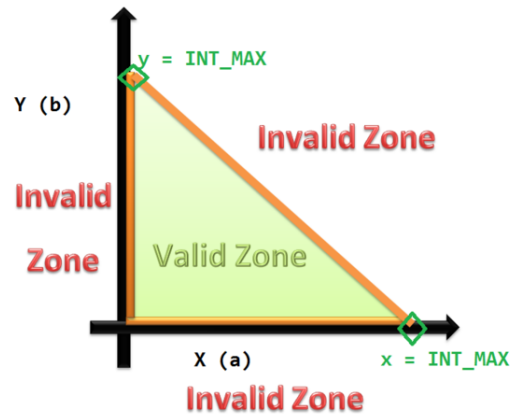
The expected input and output values to the software component should be extracted from the component specification. The values are then grouped into sets with identifiable boundaries. Each set, or partition, contains values that are expected to be processed by the component in the same way. Partitioning of test data ranges is explained in the **equivalence partitioning** test case design technique. It is important to consider both valid and invalid partitions when designing test cases.

The demonstration can be done using a function written in C

```
int safe_add( int a, int b ) { int c = a + b ; if ( a >= 0
&& b >= 0 && c < 0 ) { fprintf ( stderr, "Overflow!\n");
} if ( a < 0 && b < 0 && c >= 0 ) { fprintf ( stderr,
"Underflow!\n"); } return c; }
```

On the basis of the code, the input vectors of $[a, b]$ are partitioned. The blocks we need to cover are the overflow statement and the underflow statement and neither of these 2. That gives rise to 3 equivalent classes, from the code review itself.

we note that there is a fixed size of **integer** hence:-
 $\text{INT_MIN} \leq x + y \leq \text{INT_MAX}$



Demonstrating Boundary Values (Orange)

We note that the input parameter a and b both are integers, hence **total order** exists on them. When we compute the equalities:-

$$x + y = \text{INT_MAX}$$

$$\text{INT_MIN} = x + y$$

we get back the values which are on the boundary, inclusive, that is these pairs of (a, b) are valid combinations, and no underflow or overflow would happen for them.

On the other hand:-

$x + y = \text{INT_MAX} + 1$ gives pairs of (a, b) which are invalid combinations, Overflow would occur for them. In the same way:-

$x + y = \text{INT_MIN} - 1$ gives pairs of (a, b) which are invalid combinations, Underflow would occur for them.

Boundary values (drawn only for the overflow case) are being shown as the orange line in the right hand side figure.

For another example, if the input values were months of the year, expressed as integers, the input parameter 'month' might have the following partitions:

```
... -2 -1 0 1 ..... 12 13 14 15 ..... -----|-----|-----
invalid partition 1 valid partition invalid partition 2
```

The boundary between two partitions is the place where the behavior of the application changes and is not a real number itself. The boundary value is the minimum (or maximum) value that is at the boundary. The number 0 is the maximum number in the first partition, the number 1 is the minimum value in the second partition, both are boundary values. Test cases should be created to generate inputs or outputs that will fall on and to either side of each boundary, which results in two cases per boundary. The test cases on each side of a boundary should be in the smallest increment possible for the component under test, for an integer this is 1, but if the input was a decimal with 2 places then it would be .01. In the example above there

are boundary values at 0,1 and 12,13 and each should be tested.

Boundary value analysis does not require invalid partitions. Take an example where a heater is turned on if the temperature is 10 degrees or colder. There are two partitions (temperature $\leq$ 10, temperature $>$ 10) and two boundary values to be tested (temperature=10, temperature=11).

Where a boundary value falls within the invalid partition the test case is designed to ensure the software component handles the value in a controlled manner. Boundary value analysis can be used throughout the testing cycle and is equally applicable at all testing phases.

2.6.3 References

- The Testing Standards Working Party website.

2.7 All-pairs testing

In computer science, **all-pairs testing** or **pairwise testing** is a combinatorial method of software testing that, for each pair of input parameters to a system (typically, a software algorithm), tests all possible discrete combinations of those parameters. Using carefully chosen test vectors, this can be done much faster than an exhaustive search of all combinations of all parameters, by “parallelizing” the tests of parameter pairs.

2.7.1 Rationale

The most common bugs in a program are generally triggered by either a single input parameter or an interactions between pairs of parameters.^[1] Bugs involving interactions between three or more parameters are both progressively less common^[2] and also progressively more expensive to find---such testing has as its limit the testing of all possible inputs.^[3] Thus, a combinatorial technique for picking test cases like all-pairs testing is a useful cost-benefit compromise that enables a significant reduction in the number of test cases without drastically compromising functional coverage.^[4]

More rigorously, assume that the test function has N parameters given in a set $\{P_i\} = \{P_1, P_2, \dots, P_N\}$. The range of the parameters are given by $R(P_i) = R_i$. Let's assume that $|R_i| = n_i$. We note that the all possible conditions that can be used is an exponentiation, while imagining that the code deals with the conditions taking only two pair at a time, might reduce the number of conditionals.

To demonstrate, suppose there are X,Y,Z parameters. We can use a predicate of the form $P(X, Y, Z)$ of order 3, which takes all 3 as input, or rather three different order 2 predicates of the form $p(u, v)$

. $P(X, Y, Z)$ can be written in an equivalent form of $p_{xy}(X, Y), p_{yz}(Y, Z), p_{zx}(Z, X)$ where comma denotes any combination. If the code is written as conditions taking “pairs” of parameters: then, the set of choices of ranges $X = \{n_i\}$ can be a multiset, because there can be multiple parameters having same number of choices.

$\max(S)$ is one of the maximum of the multiset S . The number of pair-wise test cases on this test function would be:- $T = \max(X) \times \max(X \setminus \max(X))$

Plainly that would mean, if the $n = \max(X)$ and $m = \max(X \setminus \max(X))$ then the number of tests is typically $O(nm)$, where n and m are the number of possibilities for each of the two parameters with the most choices, and it can be quite a lot less than the exhaustive $\prod n_i$

2.7.2 N-wise testing

N-wise testing can be considered the generalized form of pair-wise testing.

The idea is to apply sorting to the set $X = \{n_i\}$ so that $P = \{P_i\}$ gets ordered too. Let the sorted set be a N tuple :-

$$P_s = \langle P_i \rangle ; i < j \implies |R(P_i)| < |R(P_j)|$$

Now we can take the set $X(2) = \{P_{N-1}, P_{N-2}\}$ and call it the pairwise testing. Generalizing further we can take the set $X(3) = \{P_{N-1}, P_{N-2}, P_{N-3}\}$ and call it the 3-wise testing. Eventually, we can say $X(T) = \{P_{N-1}, P_{N-2}, \dots, P_{N-T}\}$ T-wise testing.

The N-wise testing then would just be, all possible combinations from the above formula.

2.7.3 Example

Consider the parameters shown in the table below.

'Enabled', 'Choice Type' and 'Category' have a choice range of 2, 3 and 4, respectively. An exhaustive test would involve 24 tests (2 x 3 x 4). Multiplying the two largest values (3 and 4) indicates that a pair-wise tests would involve 12 tests. The pict tool generated pairwise test cases is shown below.

2.7.4 Notes

- [1] Black, Rex (2007). *Pragmatic Software Testing: Becoming an Effective and Efficient Test Professional*. New York: Wiley. p. 240. ISBN 978-0-470-12790-2.
- [2] D.R. Kuhn, D.R. Wallace, A.J. Gallo, Jr. (June 2004). “Software Fault Interactions and Implications for Software Testing” (PDF). *IEEE Trans. on Software Engineering* **30** (6).
- [3] Practical Combinatorial Testing. SP 800-142. (PDF) (Report). Natl. Inst. of Standards and Technology. 2010.

- [4] “IEEE 12. Proceedings from the 5th International Conference on Software Testing and Validation (ICST). Software Competence Center Hagenberg. “Test Design: Lessons Learned and Practical Implications.”.

2.7.5 See also

- Software testing
- Orthogonal array testing

2.7.6 External links

- Pairwise testing
- All-pairs testing
- Pairwise and generalized t-way combinatorial testing
- Pairwise Testing in the Real World: Practical Extensions to Test-Case Scenarios

2.8 Fuzz testing

“Fuzzing” redirects here. For other uses, see [Fuzz \(disambiguation\)](#).

Fuzz testing or **fuzzing** is a software testing technique, often automated or semi-automated, that involves providing invalid, unexpected, or random data to the inputs of a computer program. The program is then monitored for exceptions such as crashes, or failing built-in code assertions or for finding potential memory leaks. Fuzzing is commonly used to test for security problems in software or computer systems. It is a form of random testing which has been used for testing hardware or software.

The field of fuzzing originated with Barton Miller at the University of Wisconsin in 1988. This early work includes not only the use of random unstructured testing, but also a systematic set of tools to evaluate a wide variety of software utilities on a variety of platforms, along with a systematic analysis of the kinds of errors that were exposed by this kind of testing. In addition, they provided public access to their tool source code, test procedures and raw result data.

There are two forms of fuzzing program, *mutation-based* and *generation-based*, which can be employed as white-, grey-, or black-box testing.^[1] File formats and network protocols are the most common targets of testing, but any type of program input can be fuzzed. Interesting inputs include environment variables, keyboard and mouse events, and sequences of API calls. Even items not normally considered “input” can be fuzzed, such as the contents of databases, shared memory, or the precise interleaving of threads.

For the purpose of security, input that crosses a trust boundary is often the most interesting.^[2] For example, it is more important to fuzz code that handles the upload of a file by any user than it is to fuzz the code that parses a configuration file that is accessible only to a privileged user.

2.8.1 History

The term “fuzz” or “fuzzing” originates from a 1988 class project, taught by Barton Miller at the University of Wisconsin.^{[3][4]} The project developed a basic command-line fuzzer to test the reliability of Unix programs by bombarding them with random data until they crashed. The test was repeated in 1995, expanded to include testing of GUI-based tools (such as the X Window System), network protocols, and system library APIs.^[1] Follow-on work included testing command- and GUI-based applications on both Windows and Mac OS X.

One of the earliest examples of fuzzing dates from before 1983. “The Monkey” was a Macintosh application developed by Steve Capps prior to 1983. It used journaling hooks to feed random events into Mac programs, and was used to test for bugs in MacPaint.^[5]

Another early fuzz testing tool was *crashme*, first released in 1991, which was intended to test the robustness of Unix and Unix-like operating systems by executing random machine instructions.^[6]

2.8.2 Uses

Fuzz testing is often employed as a black-box testing methodology in large software projects where a budget exists to develop test tools. Fuzz testing offers a cost benefit for many programs.^[7]

The technique can only provide a random sample of the system’s behavior, and in many cases passing a fuzz test may only demonstrate that a piece of software can handle exceptions without crashing, rather than behaving correctly. This means fuzz testing is an assurance of overall quality, rather than a bug-finding tool, and not a substitute for exhaustive testing or formal methods.

As a gross measurement of reliability, fuzzing can suggest which parts of a program should get special attention, in the form of a code audit, application of static code analysis, or partial rewrites.

Types of bugs

As well as testing for outright crashes, fuzz testing is used to find bugs such as assertion failures and memory leaks (when coupled with a memory debugger). The methodology is useful against large applications, where any bug affecting memory safety is likely to be a severe

vulnerability.

Since fuzzing often generates invalid input it is used for testing error-handling routines, which are important for software that does not control its input. Simple fuzzing can be thought of as a way to automate negative testing.

Fuzzing can also find some types of “correctness” bugs. For example, it can be used to find incorrect-serialization bugs by complaining whenever a program’s serializer emits something that the same program’s parser rejects.^[8] It can also find unintentional differences between two versions of a program^[9] or between two implementations of the same specification.^[10]

2.8.3 Techniques

Fuzzing programs fall into two different categories. Mutation-based fuzzers mutate existing data samples to create test data while generation-based fuzzers define new test data based on models of the input.^[1]

The simplest form of fuzzing technique is sending a stream of random bits to software, either as command line options, randomly mutated protocol packets, or as events. This technique of random inputs continues to be a powerful tool to find bugs in command-line applications, network protocols, and GUI-based applications and services. Another common technique that is easy to implement is mutating existing input (e.g. files from a *test suite*) by flipping bits at random or moving blocks of the file around. However, the most successful fuzzers have detailed understanding of the format or protocol being tested.

The understanding can be based on a *specification*. A specification-based fuzzer involves writing the entire array of specifications into the tool, and then using model-based test generation techniques in walking through the specifications and adding anomalies in the data contents, structures, messages, and sequences. This “smart fuzzing” technique is also known as robustness testing, syntax testing, grammar testing, and (input) fault injection.^{[11][12][13][14]} The protocol awareness can also be created *heuristically* from examples using a tool such as *Sequitur*.^[15] These fuzzers can *generate test cases* from scratch, or they can *mutate* examples from *test suites* or real life. They can concentrate on *valid* or *invalid* input, with *mostly-valid* input tending to trigger the “deepest” error cases.

There are two limitations of protocol-based fuzzing based on protocol implementations of published specifications: 1) Testing cannot proceed until the specification is relatively mature, since a specification is a prerequisite for writing such a fuzzer; and 2) Many useful protocols are proprietary, or involve proprietary extensions to published protocols. If fuzzing is based only on published specifications, test coverage for new or proprietary protocols will be limited or nonexistent.

Fuzz testing can be combined with other testing techniques. White-box fuzzing uses *symbolic execution* and *constraint solving*.^[16] Evolutionary fuzzing leverages feedback from an heuristic (E.g., *code coverage* in grey-box harnessing,^[17] or a modeled attacker behavior in black-box harnessing^[18]) effectively automating the approach of *exploratory testing*.

2.8.4 Reproduction and isolation

Test case reduction is the process of extracting minimal *test cases* from an initial test case.^{[19][20]} Test case reduction may be done manually, or using software tools, and usually involves a *divide-and-conquer algorithm*, wherein parts of the test are removed one by one until only the essential core of the test case remains.

So as to be able to reproduce errors, fuzzing software will often record the input data it produces, usually before applying it to the software. If the computer crashes outright, the test data is preserved. If the fuzz stream is *pseudo-random number*-generated, the seed value can be stored to reproduce the fuzz attempt. Once a bug is found, some fuzzing software will help to build a *test case*, which is used for *debugging*, using test case reduction tools such as Delta or Lithium.

2.8.5 Advantages and disadvantages

The main problem with fuzzing to find program faults is that it generally only finds very simple faults. The computational complexity of the software testing problem is of *exponential order* ($O(c^n)$, $c > 1$) and every fuzzer takes shortcuts to find something interesting in a time-frame that a human cares about. A primitive fuzzer may have poor *code coverage*; for example, if the input includes a *checksum* which is not properly updated to match other random changes, only the checksum validation code will be verified. Code coverage tools are often used to estimate how “well” a fuzzer works, but these are only guidelines to fuzzer quality. Every fuzzer can be expected to find a different set of bugs.

On the other hand, bugs found using fuzz testing are sometimes severe, exploitable bugs that could be used by a real attacker. Discoveries have become more common as fuzz testing has become more widely known, as the same techniques and tools are now used by attackers to exploit deployed software. This is a major advantage over binary or source auditing, or even fuzzing’s close cousin, *fault injection*, which often relies on artificial fault conditions that are difficult or impossible to exploit.

The randomness of inputs used in fuzzing is often seen as a disadvantage, as catching a *boundary value* condition with random inputs is highly unlikely but today most of the fuzzers solve this problem by using *deterministic algorithms* based on user inputs.

Fuzz testing enhances software security and software safety because it often finds odd oversights and defects which human testers would fail to find, and even careful human test designers would fail to create tests for.

2.8.6 See also

- Boundary value analysis

2.8.7 References

- [1] Michael Sutton, Adam Greene, Pedram Amini (2007). *Fuzzing: Brute Force Vulnerability Discovery*. Addison-Wesley. ISBN 0-321-44611-9.
- [2] John Neystadt (February 2008). "Automated Penetration Testing with White-Box Fuzzing". Microsoft. Retrieved 2009-05-14.
- [3] Barton Miller (2008). "Preface". In Ari Takanen, Jared DeMott and Charlie Miller, *Fuzzing for Software Security Testing and Quality Assurance*, ISBN 978-1-59693-214-2
- [4] "Fuzz Testing of Application Reliability". University of Wisconsin-Madison. Retrieved 2009-05-14.
- [5] "Macintosh Stories: Monkey Lives". Folklore.org. 1999-02-22. Retrieved 2010-05-28.
- [6] "crashme". *CodePlex*. Retrieved 2012-06-26.
- [7] Justin E. Forrester and Barton P. Miller. "An Empirical Study of the Robustness of Windows NT Applications Using Random Testing".
- [8] Jesse Ruderman. "Fuzzing for correctness".
- [9] Jesse Ruderman. "Fuzzing TraceMonkey".
- [10] Jesse Ruderman. "Some differences between JavaScript engines".
- [11] "Robustness Testing Of Industrial Control Systems With Achilles" (PDF). Retrieved 2010-05-28.
- [12] "Software Testing Techniques by Boris Beizer. International Thomson Computer Press; 2 Sub edition (June 1990)". Amazon.com. Retrieved 2010-05-28.
- [13] "Kaksonen, Rauli. (2001) A Functional Method for Assessing Protocol Implementation Security (Licentiate thesis). Espoo. Technical Research Centre of Finland, VTT Publications 447. 128 p. + app. 15 p. ISBN 951-38-5873-1 (soft back ed.) ISBN 951-38-5874-X (on-line ed.)." (PDF). Retrieved 2010-05-28.
- [14] "Software Fault Injection: Inoculating Programs Against Errors by Jeffrey M. Voas and Gary McGraw". John Wiley & Sons. January 28, 1998.
- [15] Dan Kaminski (2006). "Black Ops 2006" (PDF).
- [16] Patrice Godefroid, Adam Kiezun, Michael Y. Levin. "Grammar-based Whitebox Fuzzing" (PDF). Microsoft Research.
- [17] "VDA Labs".
- [18] "XSS Vulnerability Detection Using Model Inference Assisted Evolutionary Fuzzing".
- [19] "Test Case Reduction". 2011-07-18.
- [20] "IBM Test Case Reduction Techniques". 2011-07-18.

2.8.8 Further reading

- Ari Takanen, Jared D. DeMott, Charles Miller, *Fuzzing for Software Security Testing and Quality Assurance*, 2008, ISBN 978-1-59693-214-2
- Michael Sutton, Adam Greene, and Pedram Amini. *Fuzzing: Brute Force Vulnerability Discovery*, 2007, ISBN 0-32-144611-9.
- H. Pohl, *Cost-Effective Identification of Zero-Day Vulnerabilities with the Aid of Threat Modeling and Fuzzing*, 2011
- Bratus, S., Darley, T., Locasto, M., Patterson, M.L., Shapiro, R.B., Shubina, A., *Beyond Planted Bugs in "Trusting Trust": The Input-Processing Frontier*, IEEE Security & Privacy Vol 12, Issue 1, (Jan-Feb 2014), pp. 83-87 -- Basically highlights why fuzzing works so well: because the input is the controlling program of the interpreter.

2.8.9 External links

- University of Wisconsin Fuzz Testing (the original fuzz project) Source of papers and fuzz software.
- Look out! It's the Fuzz! (IATAC IAnewsletter 10-1)
- Designing Inputs That Make Software Fail, conference video including fuzzy testing
- Link to the Oulu (Finland) University Secure Programming Group
- Building 'Protocol Aware' Fuzzing Frameworks
- Video training series about Fuzzing, Fuzz testing, and unknown vulnerability management

2.9 Cause-effect graph

In software testing, a **cause-effect graph** is a directed graph that maps a set of causes to a set of effects. The causes may be thought of as the input to the program, and the effects may be thought of as the output. Usually the graph shows the nodes representing the causes on the left side and the nodes representing the effects on the right side. There may be intermediate nodes in between that

combine inputs using logical operators such as AND and OR.

Constraints may be added to the causes and effects. These are represented as edges labeled with the constraint symbol using a dashed line. For causes, valid constraint symbols are E (exclusive), O (one and only one), I (at least one), and R (Requires). The exclusive constraint states that at most one of the causes 1 and 2 can be true, i.e. both cannot be true simultaneously. The Inclusive (at least one) constraint states that at least one of the causes 1, 2 or 3 must be true, i.e. all cannot be false simultaneously. The one and only one (OaOO or simply O) constraint states that only one of the causes 1, 2 or 3 can be true. The Requires constraint states that if cause 1 is true, then cause 2 must be true, and it is impossible for 1 to be true and 2 to be false.

For effects, valid constraint symbol is M (Mask). The mask constraint states that if effect 1 is true then effect 2 is false. Note that the mask constraint relates to the effects and not the causes like the other constraints.

The graph's direction is as follows:

Causes --> intermediate nodes --> Effects

The graph can always be rearranged so there is only one node between any input and any output. See **conjunctive normal form** and **disjunctive normal form**.

A cause–effect graph is useful for generating a reduced decision table.

2.9.1 See also

- Causal diagram
- Decision table
- Why–because graph

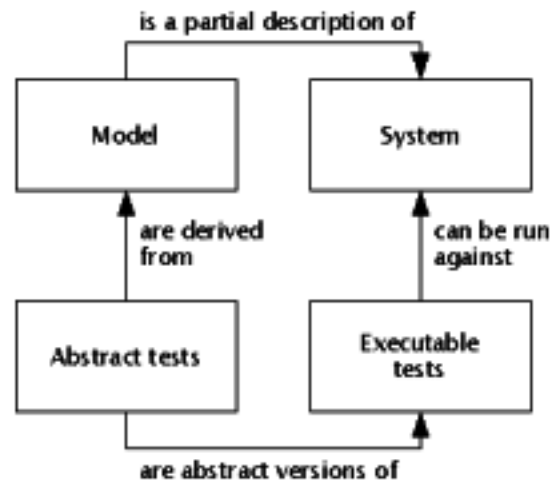
2.9.2 Further reading

- Myers, Glenford J. (1979). *The Art of Software Testing*. John Wiley & Sons. ISBN 0-471-04328-1.

2.10 Model-based testing

Model-based testing is an application of **model-based design** for designing and optionally also executing artifacts to perform **software testing** or **system testing**. Models can be used to represent the desired behavior of a System Under Test (SUT), or to represent testing strategies and a test environment. The picture on the right depicts the former approach.

A model describing a SUT is usually an abstract, partial presentation of the SUT's desired behavior. Test cases derived from such a model are functional tests on the



General model-based testing setting

same level of abstraction as the model. These test cases are collectively known as an **abstract test suite**. An abstract test suite cannot be directly executed against an SUT because the suite is on the wrong level of abstraction. An **executable test suite** needs to be derived from a corresponding abstract test suite. The executable test suite can communicate directly with the system under test. This is achieved by mapping the abstract test cases to concrete test cases suitable for execution. In some model-based testing environments, models contain enough information to generate executable test suites directly. In others, elements in the **abstract test suite** must be mapped to specific statements or method calls in the software to create a **concrete test suite**. This is called solving the “mapping problem”.^[1] In the case of online testing (see below), abstract test suites exist only conceptually but not as explicit artifacts.

Tests can be derived from models in different ways. Because testing is usually experimental and based on heuristics, there is no known single best approach for test derivation. It is common to consolidate all test derivation related parameters into a package that is often known as “test requirements”, “test purpose” or even “use case(s)”. This package can contain information about those parts of a model that should be focused on, or the conditions for finishing testing (test stopping criteria).

Because test suites are derived from models and not from source code, model-based testing is usually seen as one form of **black-box testing**.

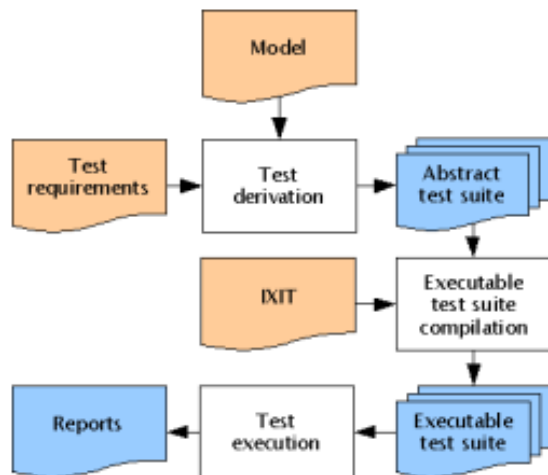
Model-based testing for complex software systems is still an evolving field.

2.10.1 Models

Especially in **Model Driven Engineering** or in Object Management Group's (OMG's) **model-driven architecture**, models are built before or parallel with the corre-

sponding systems. Models can also be constructed from completed systems. Typical modeling languages for test generation include UML, SysML, mainstream programming languages, finite machine notations, and mathematical formalisms such as Z, B, Event-B, Alloy or coq.

2.10.2 Deploying model-based testing



An example of a model-based testing workflow (offline test case generation). *IXIT* refers to implementation extra information and refers to information needed to convert an abstract test suite into an executable one. Typically, *IXIT* contains information on the test harness, data mappings and SUT configuration.

There are various known ways to deploy model-based testing, which include **online testing**, **offline generation of executable tests**, and **offline generation of manually deployable tests**.^[2]

Online testing means that a model-based testing tool connects directly to an SUT and tests it dynamically.

Offline generation of executable tests means that a model-based testing tool generates test cases as computer-readable assets that can be later run automatically; for example, a collection of **Python** classes that embodies the generated testing logic.

Offline generation of manually deployable tests means that a model-based testing tool generates test cases as human-readable assets that can later assist in manual testing; for instance, a PDF document describing the generated test steps in a human language.

2.10.3 Deriving tests algorithmically

The effectiveness of model-based testing is primarily due to the potential for automation it offers. If a model is machine-readable and formal to the extent that it has a well-defined behavioral interpretation, test cases can in principle be derived mechanically.

From finite state machines

Often the model is translated to or interpreted as a **finite state automaton** or a **state transition system**. This automaton represents the possible configurations of the system under test. To find test cases, the automaton is searched for executable paths. A possible execution path can serve as a test case. This method works if the model is **deterministic** or can be transformed into a deterministic one. Valuable off-nominal test cases may be obtained by leveraging unspecified transitions in these models.

Depending on the complexity of the system under test and the corresponding model the number of paths can be very large, because of the huge amount of possible configurations of the system. To find test cases that can cover an appropriate, but finite, number of paths, test criteria are needed to guide the selection. This technique was first proposed by Offutt and Abdurazik in the paper that started model-based testing.^[3] Multiple techniques for test case generation have been developed and are surveyed by Rushby.^[4] Test criteria are described in terms of general graphs in the testing textbook.^[1]

Theorem proving

Theorem proving has been originally used for automated proving of logical formulas. For model-based testing approaches the system is modeled by a set of logical expressions (**predicates**) specifying the system's behavior.^[5] For selecting test cases the model is partitioned into **equivalence classes** over the valid interpretation of the set of the logical expressions describing the system under development. Each class is representing a certain system behavior and can therefore serve as a test case. The simplest partitioning is done by the disjunctive normal form approach. The logical expressions describing the system's behavior are transformed into the **disjunctive normal form**.

Constraint logic programming and symbolic execution

Constraint programming can be used to select test cases satisfying specific constraints by solving a set of constraints over a set of variables. The system is described by the means of constraints.^[6] Solving the set of constraints can be done by Boolean solvers (e.g. SAT-solvers based on the **Boolean satisfiability problem**) or by **numerical analysis**, like the **Gaussian elimination**. A solution found by solving the set of constraints formulas can serve as a test cases for the corresponding system.

Constraint programming can be combined with symbolic execution. In this approach a system model is executed symbolically, i.e. collecting data constraints over different control paths, and then using the constraint programming method for solving the constraints and producing

test cases.^[7]

Model checking

Model checkers can also be used for test case generation.^[8] Originally model checking was developed as a technique to check if a property of a specification is valid in a model. When used for testing, a model of the system under test, and a property to test is provided to the model checker. Within the procedure of proofing, if this property is valid in the model, the model checker detects witnesses and counterexamples. A witness is a path, where the property is satisfied, whereas a counterexample is a path in the execution of the model, where the property is violated. These paths can again be used as test cases.

Test case generation by using a Markov chain test model

Markov chains are an efficient way to handle Model-based Testing. Test models realized with Markov chains can be understood as a usage model: it is referred to as Usage/Statistical Model Based Testing. Usage models, so Markov chains, are mainly constructed of 2 artifacts: the **Finite State Machine (FSM)** which represents all possible usage scenario of the tested system and the **Operational Profiles (OP)** which qualify the FSM to represent how the system is or will be used statistically. The first (FSM) helps to know what can be or has been tested and the second (OP) helps to derive operational test cases. Usage/Statistical Model-based Testing starts from the facts that is not possible to exhaustively test a system and that failure can appear with a very low rate.^[9] This approach offers a pragmatic way to statically derive test cases which are focused on improving the reliability of the system under test. Usage/Statistical Model Based Testing was recently extended to be applicable to embedded software systems.^{[10][11]}

Input space modeling

Abstract test cases can be generated automatically from a model of the “input space” of the SUT. The input space is defined by all of the input variables that affect SUT behavior, including not only explicit input parameters but also relevant internal state variables and even the internal state of external systems used by the SUT. For example, SUT behavior may depend on state of a file system or a database. From a model that defines each input variable and its value domain, it is possible to generate abstract test cases that describe various input combinations. Input space modeling is a common element in combinatorial testing techniques.^[12] Combinatorial testing provides a useful quantification of test adequacy known as “N-tuple coverage”. For example, 2-tuple coverage (all-pairs test-

ing) means that for each pair of input variables, every 2-tuple of value combinations is used in the test suite. Tools that generate test cases from input space models^[13] often use a “coverage model” that allows for selective tuning of the desired level of N-tuple coverage.

2.10.4 Solutions

- Conformiq Tool Suite
- MaTeLo (Markov Test Logic) - All4tec
- Smartesting CertifyIt

2.10.5 See also

- Domain Specific Language (DSL)
- Domain Specific Modeling (DSM)
- Model Driven Architecture (MDA)
- Model Driven Engineering (MDE)
- Object Oriented Analysis and Design (OOAD)
- Time Partition Testing (TPT)

2.10.6 References

- [1] Paul Ammann and Jeff Offutt. Introduction to Software Testing. Cambridge University Press, 2008.
- [2] *Practical Model-Based Testing: A Tools Approach*, Mark Utting and Bruno Legeard, ISBN 978-0-12-372501-1, Morgan-Kaufmann 2007
- [3] Jeff Offutt and Aynur Abdurazik. Generating Tests from UML Specifications. Second International Conference on the Unified Modeling Language (UML '99), pages 416-429, Fort Collins, CO, October 1999.
- [4] John Rushby. Automated Test Generation and Verified Software. Verified Software: Theories, Tools, Experiments: First IFIP TC 2/WG 2.3 Conference, VSTTE 2005, Zurich, Switzerland, October 10–13. pp. 161-172, Springer-Verlag
- [5] Brucker, Achim D.; Wolff, Burkhart (2012). “On Theorem Prover-based Testing”. *Formal Aspects of Computing*. doi:10.1007/s00165-012-0222-y.
- [6] Jefferson Offutt. Constraint-Based Automatic Test Data Generation. IEEE Transactions on Software Engineering, 17:900-910, 1991
- [7] Antti Huima. Implementing Conformiq Qtronic. Testing of Software and Communicating Systems, Lecture Notes in Computer Science, 2007, Volume 4581/2007, 1-12, DOI: 10.1007/978-3-540-73066-8_1

- [8] Gordon Fraser, Franz Wotawa, and Paul E. Ammann. Testing with model checkers: a survey. *Software Testing, Verification and Reliability*, 19(3):215–261, 2009. URL: <http://www3.interscience.wiley.com/journal/121560421/abstract>
- [9] Helene Le Guen. Validation d'un logiciel par le test statistique d'usage : de la modelisation de la decision à la livraison, 2005. URL: <ftp://ftp.irisa.fr/techreports/theses/2005/leguen.pdf>
- [10] http://ieeexplore.ieee.org/xpls/abs_all.jsp?arnumber=5954385&tag=1
- [11] http://www.amazon.de/Model-Based-Statistical-Continuous-Concurrent-Environment/dp/3843903484/ref=sr_1_1?ie=UTF8&qid=1334231267&sr=8-1
- [12] “Combinatorial Methods In Testing”, National Institute of Standards and Technology
- [13] “Tcases: A Model-Driven Test Case Generator”, The Cornutum Project
- Roodenrijs, E. (Spring 2010). “Model-Based Testing Adds Value”. *Methods & Tools* **18** (1): 33–39. ISSN 1661-402X.
- *A Systematic Review of Model Based Testing Tool Support*, Muhammad Shafique, Yvan Labiche, Carleton University, Technical Report, May 2010.
- Zander, Justyna; Schieferdecker, Ina; Mosterman, Pieter J., eds. (2011). *Model-Based Testing for Embedded Systems*. Computational Analysis, Synthesis, and Design of Dynamic Systems **13**. Boca Raton: CRC Press. ISBN 978-1-4398-1845-9.
- Online Community for Model-based Testing
- *2011 Model-based Testing User Survey: Results and Analysis*. Robert V. Binder. System Verification Associates, February 2012
- *2014 Model-based Testing User Survey: Results*. Robert V. Binder, Anne Kramer, Bruno Legeard, 2014

2.10.7 Further reading

- OMG UML 2 Testing Profile;
- Bringmann, E.; Krämer, A. (2008). “Model-Based Testing of Automotive Systems” (PDF). *2008 International Conference on Software Testing, Verification, and Validation*. International Conference on Software Testing, Verification, and Validation (ICST). pp. 485–493. doi:10.1109/ICST.2008.45. ISBN 978-0-7695-3127-4.
- *Practical Model-Based Testing: A Tools Approach*, Mark Utting and Bruno Legeard, ISBN 978-0-12-372501-1, Morgan-Kaufmann 2007.
- *Model-Based Software Testing and Analysis with C#*, Jonathan Jacky, Margus Veanes, Colin Campbell, and Wolfram Schulte, ISBN 978-0-521-68761-4, Cambridge University Press 2008.
- *Model-Based Testing of Reactive Systems* Advanced Lecture Series, LNCS 3472, Springer-Verlag, 2005. ISBN 978-3-540-26278-7.
- Hong Zhu et al. (2008). *AST '08: Proceedings of the 3rd International Workshop on Automation of Software Test*. ACM Press. ISBN 978-1-60558-030-2.
- Santos-Neto, P.; Resende, R.; Pádua, C. (2007). “Requirements for information systems model-based testing”. *Proceedings of the 2007 ACM symposium on Applied computing - SAC '07*. Symposium on Applied Computing. pp. 1409–1415. doi:10.1145/1244002.1244306. ISBN 1-59593-480-4.

2.11 Web testing

Web testing is the name given to software testing that focuses on web applications. Complete testing of a web-based system before going live can help address issues before the system is revealed to the public. Issues such as the security of the web application, the basic functionality of the site, its accessibility to handicapped users and fully able users, as well as readiness for expected traffic and number of users and the ability to survive a massive spike in user traffic, both of which are related to load testing.

2.11.1 Web application performance tool

A web application performance tool (WAPT) is used to test web applications and web related interfaces. These tools are used for performance, load and stress testing of web applications, web sites, web servers and other web interfaces. WAPT tends to simulate virtual users which will repeat either recorded URLs or specified URL and allows the users to specify number of times or iterations that the virtual users will have to repeat the recorded URLs. By doing so, the tool is useful to check for bottleneck and performance leakage in the website or web application being tested.

A WAPT faces various challenges during testing and should be able to conduct tests for:

- Browser compatibility
- Operating System compatibility
- Windows application compatibility where required

WAPT allows a user to specify how virtual users are involved in the testing environment, i.e. either increasing users or constant users or periodic users load. Increasing user load, step by step is called RAMP where virtual users are increased from 0 to hundreds. Constant user load maintains specified user load at all time. Periodic user load tends to increase and decrease the user load from time to time.

2.11.2 Web security testing

Web security testing tells us whether Web based applications requirements are met when they are subjected to malicious input data.^[1]

- Web Application Security Testing Plug-in Collection for FireFox: <https://addons.mozilla.org/en-US/firefox/collection/webappsec>

2.11.3 Testing the user interface of web applications

See also: [List of web testing tools](#)

Some frameworks give a toolbox for testing Web applications.

2.11.4 Open Source web testing tools

- **Apache JMeter**: Java program for load testing and performance measurement.
- **Curl-loader**: C-written powerful tool for load testing in different scenarios.
- **Selenium**: Suite of tools for automating web browsers. Available in many languages.
- **Watir**: Web Automation Testing In Ruby for automating web browsers.

2.11.5 Windows-based web testing tools

Main article: [List of web testing tools](#)

- **CSE HTML Validator** - Test HTML (including HTML5), XHTML, CSS (including CSS3), accessibility; software from AI Internet Solutions LLC.
- **HP LoadRunner** - Automated performance and load testing software from HP.
- **HP QuickTest Professional** - Automated functional and regression testing software from HP.

- **IBM Rational Functional Tester**
- **NeoLoad** - Load and performance testing tool from Neotys.
- **Soatest** - API testing tool from Parasoft
- **Ranorex** - Automated cross-browser functional testing software from Ranorex.
- **Silk Performer** - Performance testing tool from Borland.
- **SilkTest** - Automation tool for testing the functionality of enterprise applications.
- **TestComplete** - Automated testing tool, developed by SmartBear Software.
- **Testing Anywhere** - Automation testing tool for all types of testing from Automation Anywhere.
- **Test Studio** - Software testing tool for functional web testing from Telerik.
- **WebLOAD** - Load testing tool for web and mobile applications, from RadView Software.

2.11.6 Cloud-based testing tools

- **BlazeMeter**: a commercial, self-service load testing platform-as-a-service (PaaS), which is fully compatible with open-source **Apache JMeter** the performance testing framework by the Apache Software Foundation.
- **Blitz**: Load and performance testing of websites, mobile, web apps and REST APIs.
- **SOASTA**: a provider of cloud-based testing solutions, and created the industry's first browser-based website testing product. Website tests include load testing, software performance testing, functional testing and user interface testing.
- **Testdroid**: Smoke, compatibility and functional testing of websites, mobile, and web apps on real Android and iOS devices.

2.11.7 See also

- [Software performance testing](#)
- [Software testing](#)
- [Web server benchmarking](#)

2.11.8 References

- [1] Hope, Paco; Walther, Ben (2008), *Web Security Testing Cookbook*, Sebastopol, CA: O'Reilly Media, Inc., ISBN 978-0-596-51483-9

2.11.9 Further reading

- Hung Nguyen, Robert Johnson, Michael Hackett: *Testing Applications on the Web (2nd Edition): Test Planning for Mobile and Internet-Based Systems* ISBN 0-471-20100-6
- James A. Whittaker: *How to Break Web Software: Functional and Security Testing of Web Applications and Web Services*, Addison-Wesley Professional, February 2, 2006. ISBN 0-321-36944-0
- Lydia Ash: *The Web Testing Companion: The Insider's Guide to Efficient and Effective Tests*, Wiley, May 2, 2003. ISBN 0-471-43021-8
- S. Sampath, R. Bryce, Gokulanand Viswanath, Vani Kandimalla, A. Gunes Koru. Prioritizing User-Session-Based Test Cases for Web Applications Testing. Proceedings of the International Conference on Software Testing, Verification, and Validation (ICST), Lillehammer, Norway, April 2008.
- “An Empirical Approach to Testing Web Applications Across Diverse Client Platform Configurations” by Cyntrica Eaton and Atif M. Memon. International Journal on Web Engineering and Technology (IJWET), Special Issue on Empirical Studies in Web Engineering, vol. 3, no. 3, 2007, pp. 227–253, Inderscience Publishers.

In distributed systems, particularly where software is to be released into an already live target environment (such as an operational website) installation (or **software deployment** as it is sometimes called) can involve database schema changes as well as the installation of new software. Deployment plans in such circumstances may include back-out procedures whose use is intended to roll the target environment back if the deployment is unsuccessful. Ideally, the deployment plan itself should be tested in an environment that is a replica of the live environment. A factor that can increase the organizational requirements of such an exercise is the need to synchronize the data in the test deployment environment with that in the live environment with minimum disruption to live operation. This type of implementation may include testing of the processes which take place during the installation or upgrade of a multi-tier application. This type of testing is commonly compared to a dress rehearsal or may even be called a “**dry run**”.

2.12 Installation testing

Installation testing is a kind of **quality assurance** work in the **software** industry that focuses on what customers will need to do to install and set up the new software successfully. The testing process may involve full, partial or upgrades install/uninstall processes.

This testing is typically performed in **Operational Acceptance testing**, by a **software testing engineer** in conjunction with the **configuration manager**. Implementation testing is usually defined as testing which places a compiled version of code into the testing or pre-production environment, from which it may or may not progress into production. This generally takes place outside of the software development environment to limit code corruption from other future or past releases (or from the use of the wrong version of dependencies such as shared libraries) which may reside on the development environment.

The simplest installation approach is to run an install program, sometimes called **package software**. This package software typically uses a setup program which acts as a multi-configuration wrapper and which may allow the software to be installed on a variety of machine and/or operating environments. Every possible configuration should receive an appropriate level of testing so that it can be released to customers with confidence.

Chapter 3

White-box testing

3.1 White-box testing

White-box testing (also known as **clear box testing**, **glass box testing**, **transparent box testing**, and **structural testing**) is a method of testing **software** that tests internal structures or workings of an application, as opposed to its functionality (i.e. **black-box testing**). In white-box testing an internal perspective of the system, as well as programming skills, are used to design test cases. The tester chooses inputs to exercise paths through the code and determine the appropriate outputs. This is analogous to testing nodes in a circuit, e.g. **in-circuit testing** (ICT).

White-box testing can be applied at the **unit**, **integration** and **system** levels of the **software testing** process. Although traditional testers tended to think of white-box testing as being done at the unit level, it is used for integration and system testing more frequently today. It can test paths within a unit, paths between units during integration, and between subsystems during a system-level test. Though this method of test design can uncover many errors or problems, it has the potential to miss unimplemented parts of the specification or missing requirements.

White-box test design techniques include the following **code coverage** criteria:

- **Control flow testing**
- **Data flow testing**
- **Branch testing**
- **Statement coverage**
- **Decision coverage**
- **Modified condition/decision coverage**
- **Prime path testing**
- **Path testing**

3.1.1 Overview

White-box testing is a method of testing the application at the level of the source code. These test cases are derived through the use of the design techniques mentioned above: **control flow** testing, data flow testing, branch testing, path testing, statement coverage and decision coverage as well as modified condition/decision coverage. White-box testing is the use of these techniques as guidelines to create an error free environment by examining any fragile code. These White-box testing techniques are the building blocks of white-box testing, whose essence is the careful testing of the application at the source code level to prevent any hidden errors later on.^[1] These different techniques exercise every visible path of the source code to minimize errors and create an error-free environment. The whole point of white-box testing is the ability to know which line of the code is being executed and being able to identify what the correct output should be.^[1]

3.1.2 Levels

1. **Unit testing.** White-box testing is done during unit testing to ensure that the code is working as intended, before any integration happens with previously tested code. White-box testing during unit testing catches any defects early on and aids in any defects that happen later on after the code is integrated with the rest of the application and therefore prevents any type of errors later on.^[1]
2. **Integration testing.** White-box testing at this level are written to test the interactions of each interface with each other. The Unit level testing made sure that each code was tested and working accordingly in an isolated environment and integration examines the correctness of the behaviour in an open environment through the use of white-box testing for any interactions of interfaces that are known to the programmer.^[1]
3. **Regression testing.** White-box testing during regression testing is the use of recycled white-box test cases at the unit and integration testing levels.^[1]

3.1.3 Basic procedure

White-box testing's basic procedures involves the tester having a deep level of understanding of the source code being tested. The programmer must have a deep understanding of the application to know what kinds of test cases to create so that every visible path is exercised for testing. Once the source code is understood then the source code can be analyzed for test cases to be created. These are the three basic steps that white-box testing takes in order to create test cases:

1. Input involves different types of requirements, functional specifications, detailed designing of documents, proper source code, security specifications.^[2] This is the preparation stage of white-box testing to layout all of the basic information.
2. Processing involves performing risk analysis to guide whole testing process, proper test plan, execute test cases and communicate results.^[2] This is the phase of building test cases to make sure they thoroughly test the application the given results are recorded accordingly.
3. Output involves preparing final report that encompasses all of the above preparations and results.^[2]

3.1.4 Advantages

White-box testing is one of the two biggest testing methodologies used today. It has several major advantages:

1. Side effects of having the knowledge of the source code is beneficial to thorough testing.^[3]
2. Optimization of code by revealing hidden errors and being able to remove these possible defects.^[3]
3. Gives the programmer introspection because developers carefully describe any new implementation.^[3]
4. Provides traceability of tests from the source, allowing future changes to the software to be easily captured in changes to the tests.^[4]
5. White box tests are easy to automate.^[5]
6. White box testing give clear, engineering-based, rules for when to stop testing.^{[6][5]}

3.1.5 Disadvantages

Although white-box testing has great advantages, it is not perfect and contains some disadvantages:

1. White-box testing brings complexity to testing because the tester must have knowledge of the program, including being a programmer. White-box testing requires a programmer with a high level of knowledge due to the complexity of the level of testing that needs to be done.^[3]
2. On some occasions, it is not realistic to be able to test every single existing condition of the application and some conditions will be untested.^[3]
3. The tests focus on the software as it exists, and missing functionality may not be discovered.

3.1.6 Modern view

A more modern view is that the dichotomy between white-box testing and black-box testing has blurred and is becoming less relevant. Whereas “white-box” originally meant using the source code, and black-box meant using requirements, tests are now derived from many documents at various levels of abstraction. The real point is that tests are usually designed from an abstract structure such as the input space, a graph, or logical predicates, and the question is what level of abstraction we derive that abstract structure from.^[5] That can be the source code, requirements, input space descriptions, or one of dozens of types of design models. Therefore, the “white-box / black-box” distinction is less important and the terms are less relevant.

3.1.7 Hacking

In **penetration testing**, white-box testing refers to a methodology where a **white hat hacker** has full knowledge of the system being attacked. The goal of a white-box penetration test is to simulate a malicious insider who has knowledge of and possibly basic credentials for the target system.

3.1.8 See also

- **Black-box testing**
- **Grey-box testing**
- **White-box cryptography**

3.1.9 References

- [1] Williams, Laurie. “White-Box Testing” (PDF). pp. 60–61, 69. Retrieved 13 February 2013.
- [2] Ehmer Khan, Mohd (July 2011). “Different Approaches to White Box Testing Technique for Finding Errors” (PDF). *International Journal of Software Engineering and Its Applications* **5**: 1–6. Retrieved 12 February 2013.

- [3] Ehmer Khan, Mohd (May 2010). "Different Forms of Software Testing Techniques for Finding Errors" (PDF). *IJCSI International Journal of Computer Science Issues* 7 (3): 12. Retrieved 12 February 2013.
- [4] Binder, Bob (2000). *Testing Object-oriented Systems*. Addison-Wesley Publishing Company Inc.
- [5] Ammann, Paul; Offutt, Jeff (2008). *Introduction to Software Testing*. Cambridge University Press. ISBN 9780521880381.
- [6] Myers, Glenford (1979). *The Art of Software Testing*. John Wiley and Sons.

3.1.10 External links

- BCS SIGIST (British Computer Society Specialist Interest Group in Software Testing): <http://www.testingstandards.co.uk/Component%20Testing.pdf> [Standard for Software Component Testing], Working Draft 3.4, 27. April 2001.
- <http://agile.csc.ncsu.edu/SEMaterials/WhiteBox.pdf> has more information on control flow testing and data flow testing.
- <http://research.microsoft.com/en-us/projects/pex/> Pex - Automated white-box testing for .NET

3.2 Code coverage

In computer science, **code coverage** is a measure used to describe the degree to which the **source code** of a program is tested by a particular **test suite**. A program with high code coverage has been more thoroughly tested and has a lower chance of containing **software bugs** than a program with low code coverage. Many different metrics can be used to calculate code coverage; some of the most basic are the percent of program **subroutines** and the percent of program **statements** called during execution of the test suite.

Code coverage was among the first methods invented for systematic **software testing**. The first published reference was by Miller and Maloney in *Communications of the ACM* in 1963.^[1]

3.2.1 Coverage criteria

To measure what percentage of code has been exercised by a test suite, one or more *coverage criteria* are used. Coverage criteria is usually defined as a rule or requirement, which test suite needs to satisfy.^[2]

Basic coverage criteria

There are a number of coverage criteria, the main ones being:^[3]

- **Function coverage** - Has each function (or subroutine) in the program been called?
- **Statement coverage** - Has each **statement** in the program been executed?
- **Branch coverage** - Has each branch (also called **DD-path**) of each control structure (such as in *if* and *case* statements) been executed? For example, given an *if* statement, have both the true and false branches been executed? Another way of saying this is, has every **edge** in the program been executed?
- **Condition coverage** (or predicate coverage) - Has each Boolean sub-expression evaluated both to true and false?

For example, consider the following C function:

```
int foo (int x, int y) { int z = 0; if ((x>0) && (y>0)) { z = x; } return z; }
```

Assume this function is a part of some bigger program and this program was run with some test suite.

- If during this execution function 'foo' was called at least once, then *function coverage* for this function is satisfied.
- *Statement coverage* for this function will be satisfied if it was called e.g. as foo(1,1), as in this case, every line in the function is executed including z = x;.
- Tests calling foo(1,1) and foo(0,1) will satisfy *branch coverage* because, in the first case, the 2 if conditions are met and z = x; is executed, while in the second case, the first condition (x>0) is not satisfied, which prevents executing z = x;.
- *Condition coverage* can be satisfied with tests that call foo(1,1), foo(1,0) and foo(0,0). These are necessary because in the first two cases, (x>0) evaluates to true, while in the third, it evaluates false. At the same time, the first case makes (y>0) true, while the second and third make it false.

Condition coverage does not necessarily imply branch coverage. For example, consider the following fragment of code:

```
if a and b then
```

Condition coverage can be satisfied by two tests:

- a=true, b=false
- a=false, b=true

However, this set of tests does not satisfy branch coverage since neither case will meet the if condition.

Fault injection may be necessary to ensure that all conditions and branches of **exception handling** code have adequate coverage during testing.

Modified condition/decision coverage

Main article: [Modified Condition/Decision Coverage](#)

A combination of function coverage and branch coverage is sometimes also called **decision coverage**. This criterion requires that every point of entry and exit in the program have been invoked at least once, and every decision in the program have taken on all possible outcomes at least once. In this context the decision is a boolean expression composed of conditions and zero or more boolean operators. This definition is not the same as branch coverage,^[4] however, some do use the term *decision coverage* as a synonym for *branch coverage*.^[5]

Condition/decision coverage requires that both decision and condition coverage been satisfied. However, for **safety-critical** applications (e.g., for avionics software) it is often required that **modified condition/decision coverage (MC/DC)** be satisfied. This criterion extends condition/decision criteria with requirements that each condition should affect the decision outcome independently. For example, consider the following code:

```
if (a or b) and c then
```

The condition/decision criteria will be satisfied by the following set of tests:

- a=true, b=true, c=true
- a=false, b=false, c=false

However, the above tests set will not satisfy modified condition/decision coverage, since in the first test, the value of 'b' and in the second test the value of 'c' would not influence the output. So, the following test set is needed to satisfy MC/DC:

- a=false, b=false, c=true
- a=true, b=false, c=true
- a=false, b=true, c=true
- a=false, b=true, c=false

Multiple condition coverage

This criterion requires that all combinations of conditions inside each decision are tested. For example, the code fragment from the previous section will require eight tests:

- a=false, b=false, c=false

- a=false, b=false, c=true
- a=false, b=true, c=false
- a=false, b=true, c=true
- a=true, b=false, c=false
- a=true, b=false, c=true
- a=true, b=true, c=false
- a=true, b=true, c=true

Parameter value coverage

Parameter value coverage (PVC) requires that in a method taking parameters, all the common values for such parameters been considered. The idea is that all common possible values for a parameter are tested.^[6] For example, common values for a string are: 1) null, 2) empty, 3) whitespace (space, tabs, newline), 4) valid string, 5) invalid string, 6) single-byte string, 7) double-byte string. It may also be appropriate to use very long strings. Failure to test each possible parameter value may leave a bug. Testing only one of these could result in 100% code coverage as each line is covered, but as only one of seven options are tested, there is only 14.2% PVC.

Other coverage criteria

There are further coverage criteria, which are used less often:

- **Linear Code Sequence and Jump (LCSAJ) coverage** a.k.a. **JJ-Path coverage** - has every LCSAJ/JJ-path been executed?^[7]
- **Path coverage** - Has every possible route through a given part of the code been executed?
- **Entry/exit coverage** - Has every possible call and return of the function been executed?
- **Loop coverage** - Has every possible loop been executed zero times, once, and more than once?
- **State coverage** - Has each state in a **finite-state machine** been reached and explored?

Safety-critical applications are often required to demonstrate that testing achieves 100% of some form of code coverage.

Some of the coverage criteria above are connected. For instance, path coverage implies decision, statement and entry/exit coverage. Decision coverage implies statement coverage, because every statement is part of a branch.

Full path coverage, of the type described above, is usually impractical or impossible. Any module with a succession

of n decisions in it can have up to 2^n paths within it; loop constructs can result in an infinite number of paths. Many paths may also be infeasible, in that there is no input to the program under test that can cause that particular path to be executed. However, a general-purpose algorithm for identifying infeasible paths has been proven to be impossible (such an algorithm could be used to solve the **halting problem**).^[8] **Basis path testing** is for instance a method of achieving complete branch coverage without achieving complete path coverage.^[9]

Methods for practical path coverage testing instead attempt to identify classes of code paths that differ only in the number of loop executions, and to achieve “basis path” coverage the tester must cover all the path classes.

3.2.2 In practice

The target software is built with special options or libraries and/or run under a special environment such that every function that is exercised (executed) in the program(s) is mapped back to the function points in the source code. This process allows developers and quality assurance personnel to look for parts of a system that are rarely or never accessed under normal conditions (error handling and the like) and helps reassure test engineers that the most important conditions (function points) have been tested. The resulting output is then analyzed to see what areas of code have not been exercised and the tests are updated to include these areas as necessary. Combined with other code coverage methods, the aim is to develop a rigorous, yet manageable, set of regression tests.

In implementing code coverage policies within a software development environment, one must consider the following:

- What are coverage requirements for the end product certification and if so what level of code coverage is required? The typical level of rigor progression is as follows: Statement, Branch/Decision, Modified Condition/Decision Coverage(MC/DC), LCSAJ (Linear Code Sequence and Jump)
- Will code coverage be measured against tests that verify requirements levied on the system under test (DO-178B)?
- Is the object code generated directly traceable to source code statements? Certain certifications, (i.e. DO-178B Level A) require coverage at the assembly level if this is not the case: “Then, additional verification should be performed on the object code to establish the correctness of such generated code sequences” (DO-178B) para-6.4.4.2.^[10]

Test engineers can look at code coverage test results to help them devise test cases and input or configuration sets that will increase the code coverage over vital functions.

Two common forms of code coverage used by testers are statement (or line) coverage and branch (or edge) coverage. Line coverage reports on the execution footprint of testing in terms of which lines of code were executed to complete the test. Edge coverage reports which branches or code decision points were executed to complete the test. They both report a coverage metric, measured as a percentage. The meaning of this depends on what form(s) of code coverage have been used, as 67% branch coverage is more comprehensive than 67% statement coverage.

Generally, code coverage tools incur computation and logging in addition to the actual program thereby slowing down the application, so typically this analysis is not done in production. As one might expect, there are classes of software that cannot be feasibly subjected to these coverage tests, though a degree of coverage mapping can be approximated through analysis rather than direct testing.

There are also some sorts of defects which are affected by such tools. In particular, some **race conditions** or similar **real time** sensitive operations can be masked when run under code coverage environments; and conversely, and reliably, some of these defects may become easier to find as a result of the additional overhead of the testing code.

3.2.3 Usage in industry

Code coverage is one consideration in the safety certification of avionics equipment. The guidelines by which avionics gear is certified by the **Federal Aviation Administration** (FAA) is documented in DO-178B^[10] and the recently released DO-178C.^[11]

Code coverage is also a requirement in part 6 of the automotive safety standard **ISO 26262 Road Vehicles - Functional Safety**.^[12]

3.2.4 See also

- Cyclomatic complexity
- Intelligent verification
- Linear Code Sequence and Jump
- Modified Condition/Decision Coverage
- Mutation testing
- Regression testing
- Software metric
- Static code analysis
- White box testing

3.2.5 References

- [1] Joan C. Miller, Clifford J. Maloney (February 1963). “Systematic mistake analysis of digital computer programs”. *Communications of the ACM* (New York, NY, USA: ACM) **6** (2): 58–63. doi:10.1145/366246.366248. ISSN 0001-0782.
- [2] Paul Ammann, Jeff Offutt (2013). *Introduction to Software Testing*. Cambridge University Press.
- [3] Glenford J. Myers (2004). *The Art of Software Testing, 2nd edition*. Wiley. ISBN 0-471-46912-2.
- [4] Position Paper CAST-10 (June 2002). *What is a “Decision” in Application of Modified Condition/Decision Coverage (MC/DC) and Decision Coverage (DC)?*
- [5] MathWorks. *Types of Model Coverage*.
- [6] Unit Testing with Parameter Value Coverage (PVC)
- [7] M. R. Woodward, M. A. Hennell, “On the relationship between two control-flow coverage criteria: all JJ-paths and MCDC”, *Information and Software Technology* 48 (2006) pp. 433-440
- [8] Dorf, Richard C.: *Computers, Software Engineering, and Digital Devices*, Chapter 12, pg. 15. CRC Press, 2006. ISBN 0-8493-7340-9, ISBN 978-0-8493-7340-4; via [Google Book Search](#)
- [9] Y.N. Srikant; Priti Shankar (2002). *The Compiler Design Handbook: Optimizations and Machine Code Generation*. CRC Press. p. 249. ISBN 978-1-4200-4057-9.
- [10] RTCA/DO-178B, *Software Considerations in Airborne Systems and Equipment Certification*, Radio Technical Commission for Aeronautics, December 1, 1992
- [11] RTCA/DO-178C, *Software Considerations in Airborne Systems and Equipment Certification*, Radio Technical Commission for Aeronautics, January, 2012.
- [12] *ISO 26262-6:2011(en) Road vehicles -- Functional safety -- Part 6: Product development at the software level*. International Standardization Organization.

3.3 Modified Condition/Decision Coverage

The **modified condition/decision coverage (MC/DC)** is a **code coverage** criterion that requires all of the below during testing:^[1]

1. Each entry and exit point is invoked
2. Each decision tries every possible outcome
3. Each condition in a decision takes on every possible outcome
4. Each condition in a decision is shown to independently affect the outcome of the decision.

Independence of a condition is shown by proving that only one condition changes at a time.

MC/DC is used in avionics software development guidance **DO-178B** and **DO-178C** to ensure adequate testing of the most critical (Level A) software, which is defined as that software which could *provide (or prevent failure of)* continued safe flight and landing of an aircraft. It's also highly recommended for ASIL D in part 6 of automotive standard **ISO 26262**.

3.3.1 Definitions

Condition A condition is a leaf-level Boolean expression (it cannot be broken down into a simpler Boolean expression).

Decision A Boolean expression composed of conditions and zero or more Boolean operators. A decision without a Boolean operator is a condition.

Condition coverage Every condition in a decision in the program has taken all possible outcomes at least once.

Decision coverage Every point of entry and exit in the program has been invoked at least once, and every decision in the program has taken all possible outcomes at least once.

Condition/decision coverage Every point of entry and exit in the program has been invoked at least once, every condition in a decision in the program has taken all possible outcomes at least once, and every decision in the program has taken all possible outcomes at least once.

Modified condition/decision coverage Every point of entry and exit in the program has been invoked at least once, every condition in a decision in the program has taken on all possible outcomes at least once, and each condition has been shown to affect that decision outcome independently. A condition is shown to affect a decision's outcome independently by varying just that condition while holding fixed all other possible conditions. The condition/decision criterion does not guarantee the coverage of all conditions in the module because in many test cases, some conditions of a decision are masked by the other conditions. Using the modified condition/decision criterion, each condition must be shown to be able to act on the decision outcome by itself, everything else being held fixed. The MC/DC criterion is thus much stronger than the condition/decision coverage.

3.3.2 Criticism

The MC/DC coverage criterion is controversial. Purely syntactic rearrangements of decisions (breaking them into several independently evaluated conditions using temporary variables, the values of which are then used in the decision) which do not change the semantics of a program will dramatically lower the difficulty of obtaining complete MC/DC coverage.^[2] This is because MC/DC does not consider the dataflow coming together in a decision, but it is driven by the program syntax. It is thus easy to “cheat” either deliberately or involuntarily.

3.3.3 References

- [1] Hayhurst, Kelly; Veerhusen, Dan; Chilenski, John; Rier-son, Leanna (May 2001). “A Practical Tutorial on Modified Condition/ Decision Coverage” (PDF). NASA.
- [2] Rajan, Ajitha; Heimdahl, Mats; Whalen, Michael (March 2003). “The Effect of Program and Model Structure on MCDC Test Adequacy Coverage” (PDF).

3.3.4 External links

- What is a “Decision” in Application of Modified Condition/Decision Coverage (MC/DC) and Decision Coverage (DC)?
- An Investigation of Three Forms of the Modified Condition Decision Coverage (MCDC) Criterion

3.4 Fault injection

In software testing, **fault injection** is a technique for improving the coverage of a test by introducing faults to test code paths, in particular error handling code paths, that might otherwise rarely be followed. It is often used with stress testing and is widely considered to be an important part of developing robust software.^[1] Robustness testing^[2] (also known as Syntax Testing, Fuzzing or Fuzz testing) is a type of fault injection commonly used to test for vulnerabilities in communication interfaces such as protocols, command line parameters, or APIs.

The propagation of a fault through to an observable failure follows a well defined cycle. When executed, a fault may cause an error, which is an invalid state within a system boundary. An error may cause further errors within the system boundary, therefore each new error acts as a fault, or it may propagate to the system boundary and be observable. When error states are observed at the system boundary they are termed failures. This mechanism is termed the **fault-error-failure cycle**^[3] and is a key mechanism in dependability.

3.4.1 History

The technique of fault injection dates back to the 1970s^[4] when it was first used to induce faults at a hardware level. This type of fault injection is called Hardware Implemented Fault Injection (HWIFI) and attempts to simulate hardware failures within a system. The first experiments in hardware fault injection involved nothing more than shorting connections on circuit boards and observing the effect on the system (bridging faults). It was used primarily as a test of the dependability of the hardware system. Later specialised hardware was developed to extend this technique, such as devices to bombard specific areas of a circuit board with heavy radiation. It was soon found that faults could be induced by software techniques and that aspects of this technique could be useful for assessing software systems. Collectively these techniques are known as Software Implemented Fault Injection (SWIFI).

3.4.2 Software Implemented fault injection

SWIFI techniques for software fault injection can be categorized into two types: compile-time injection and run-time injection.

Compile-time injection is an injection technique where source code is modified to inject simulated faults into a system. One method is called **mutation testing** which changes existing lines of code so that they contain faults. A simple example of this technique could be changing

$a = a + 1$ to $a = a - 1$

Code mutation produces faults which are very similar to those unintentionally added by programmers.

A refinement of code mutation is *Code Insertion Fault Injection* which adds code, rather than modifying existing code. This is usually done through the use of perturbation functions which are simple functions which take an existing value and perturb it via some logic into another value, for example

```
int pFunc(int value) { return value + 20; }
int main(int argc, char * argv[]) { int a =
pFunc(aFunction(atoi(argv[1]))); if (a > 20) { /*
do something */ } else { /* do something else */ } }
```

In this case pFunc is the perturbation function and it is applied to the return value of the function that has been called introducing a fault into the system.

Runtime Injection techniques use a software trigger to inject a fault into a running software system. Faults can be injected via a number of physical methods and triggers can be implemented in a number of ways, such as: Time Based triggers (When the timer reaches a specified time an interrupt is generated and the interrupt handler

associated with the timer can inject the fault.); Interrupt Based Triggers (Hardware exceptions and software trap mechanisms are used to generate an interrupt at a specific place in the system code or on a particular event within the system, for instance access to a specific memory location).

Runtime injection techniques can use a number of different techniques to insert faults into a system via a trigger.

- **Corruption of memory space:** This technique consists of corrupting RAM, processor registers, and I/O map.
- **Syscall interposition techniques:** This is concerned with the fault propagation from operating system kernel interfaces to executing systems software. This is done by intercepting operating system calls made by user-level software and injecting faults into them.
- **Network Level fault injection:** This technique is concerned with the corruption, loss or reordering of network packets at the network interface.

These techniques are often based around the debugging facilities provided by computer processor architectures.

Protocol software fault injection

Complex software systems, especially multi-vendor distributed systems based on open standards, perform input/output operations to exchange data via stateful, structured exchanges known as "**protocols**." One kind of fault injection that is particularly useful to test protocol implementations (a type of software code that has the unusual characteristic in that it cannot predict or control its input) is **fuzzing**. Fuzzing is an especially useful form of **Black-box testing** since the various invalid inputs that are submitted to the software system do not depend on, and are not created based on knowledge of, the details of the code running inside the system.

3.4.3 Fault injection tools

Although these types of faults can be injected by hand the possibility of introducing an unintended fault is high, so tools exist to parse a program automatically and insert faults.

Research tools

A number of SWIFI Tools have been developed and a selection of these tools is given here. Six commonly used fault injection tools are Ferrari, FTAPE, Doctor, Orchestra, Xception and Grid-FIT.

- **MODIFI (MODEL-Implemented Fault Injection)** is a fault injection tool for robustness evaluation of Simulink behavior models. It supports fault modelling in XML for implementation of domain-specific fault models.^[5]
- **Ferrari (Fault and ERROR Automatic Real-time Injection)** is based around software traps that inject errors into a system. The traps are activated by either a call to a specific memory location or a time-out. When a trap is called the handler injects a fault into the system. The faults can either be transient or permanent. Research conducted with Ferrari shows that error detection is dependent on the fault type and where the fault is inserted.^[6]
- **FTAPE (Fault Tolerance and Performance Evaluator)** can inject faults, not only into memory and registers, but into disk accesses as well. This is achieved by inserting a special disk driver into the system that can inject faults into data sent and received from the disk unit. FTAPE also has a synthetic load unit that can simulate specific amounts of load for robustness testing purposes.^[7]
- **DOCTOR (Integrated Software Fault InjeCTiOn EnviRonment)** allows injection of memory and register faults, as well as network communication faults. It uses a combination of time-out, trap and code modification. Time-out triggers inject transient memory faults and traps inject transient emulated hardware failures, such as register corruption. Code modification is used to inject permanent faults.^[8]
- **Orchestra** is a script driven fault injector which is based around Network Level Fault Injection. Its primary use is the evaluation and validation of the fault-tolerance and timing characteristics of distributed protocols. Orchestra was initially developed for the Mach Operating System and uses certain features of this platform to compensate for latencies introduced by the fault injector. It has also been successfully ported to other operating systems.^[9]
- **Xception** is designed to take advantage of the advanced debugging features available on many modern processors. It is written to require no modification of system source and no insertion of software traps, since the processor's exception handling capabilities trigger fault injection. These triggers are based around accesses to specific memory locations. Such accesses could be either for data or fetching instructions. It is therefore possible to accurately reproduce test runs because triggers can be tied to specific events, instead of timeouts.^[10]
- **Grid-FIT (Grid – Fault Injection Technology)**^[11] is a dependability assessment method and tool for assessing Grid services by fault injection. Grid-FIT is derived from an earlier fault injector WS-FIT^[12]

which was targeted towards Java Web Services implemented using Apache Axis transport. Grid-FIT utilises a novel fault injection mechanism that allows network level fault injection to be used to give a level of control similar to Code Insertion fault injection whilst being less invasive.^[13]

- LFI (Library-level Fault Injector) ^[14] is an automatic testing tool suite, used to simulate in a controlled testing environment, exceptional situations that programs need to handle at runtime but that are not easy to check via input testing alone. LFI automatically identifies the errors exposed by shared libraries, finds potentially buggy error recovery code in program binaries and injects the desired faults at the boundary between shared libraries and applications.

Commercial tools

- **Beyond Security beSTORM** ^[15] is a commercial **black box** software security analysis tool. It is often used during development by original equipment manufacturers but is also used for testing products prior to implementation, notably in aerospace, banking and defense. beSTORM's test process starts with the most likely attack scenarios, then resorts to exhaustive generation based **fuzzing**. beSTORM provides modules for common protocols and 'auto learns' new or proprietary protocols, including mutation-based attacks. Highlights: binary and textual analysis, custom protocol testing, debugging and stack tracing, development language independent, CVE compliant.
- ExhaustiF is a commercial software tool used for **grey box testing** based on software fault injection (SWIFI) to improve reliability of software intensive systems. The tool can be used during system integration and system testing phases of any software development lifecycle, complementing other testing tools as well. ExhaustiF is able to inject faults into both software and hardware. When injecting simulated faults in software, ExhaustiF offers the following fault types: Variable Corruption and Procedure Corruption. The catalogue for hardware fault injections includes faults in Memory (I/O, RAM) and CPU (Integer Unit, Floating Unit). There are different versions available for RTEMS/ERC32, RTEMS/Pentium, Linux/Pentium and MS-Windows/Pentium.^[16]
- Holodeck^[17] is a test tool developed by Security Innovation that uses fault injection to simulate real-world application and system errors for Windows applications and services. Holodeck customers include many major commercial software development companies, including Microsoft, Symantec, EMC and Adobe. It provides a controlled, repeatable environment in which to analyze and debug error-handling code and application attack surfaces for fragility and security testing. It simulates file and network fuzzing faults as well as a wide range of other resource, system and custom-defined faults. It analyzes code and recommends test plans and also performs function call logging, API interception, stress testing, code coverage analysis and many other application security assurance functions.
- **Codenomicon Defensics** ^[18] is a blackbox test automation framework that does fault injection to more than 150 different interfaces including network protocols, API interfaces, files, and XML structures. The commercial product was launched in 2001, after five years of research at University of Oulu in the area of software fault injection. A thesis work explaining the used fuzzing principles was published by VTT, one of the PROTOS consortium members.^[19]
- The Mu Service Analyzer^[20] is a commercial service testing tool developed by **Mu Dynamics**.^[21] The Mu Service Analyzer performs **black box** and **white box** testing of services based on their exposed software interfaces, using denial-of-service simulations, service-level traffic variations (to generate invalid inputs) and the replay of known vulnerability triggers. All these techniques exercise input validation and error handling and are used in conjunction with valid protocol monitors and SNMP to characterize the effects of the test traffic on the software system. The Mu Service Analyzer allows users to establish and track system-level reliability, availability and security metrics for any exposed protocol implementation. The tool has been available in the market since 2005 by customers in the North America, Asia and Europe, especially in the critical markets of network operators (and their vendors) and **Industrial control systems** (including **Critical infrastructure**).
- Xception^[22] is a commercial software tool developed by Critical Software SA^[23] used for **black box** and **white box** testing based on software fault injection (SWIFI) and Scan Chain fault injection (SCIFI). Xception allows users to test the robustness of their systems or just part of them, allowing both Software fault injection and Hardware fault injection for a specific set of architectures. The tool has been used in the market since 1999 and has customers in the American, Asian and European markets, especially in the critical market of aerospace and the telecom market. The full Xception product family includes: a) The main Xception tool, a state-of-the-art leader in Software Implemented Fault Injection (SWIFI) technology; b) The Easy Fault Definition (EFD) and Xtract (Xception Analysis Tool)

add-on tools; c) The extended Xception tool (eXception), with the fault injection extensions for Scan Chain and pin-level forcing.

Libraries

- **libfiu** (Fault injection in userspace), C library to simulate faults in POSIX routines without modifying the source code. An API is included to simulate arbitrary faults at run-time at any point of the program.
- **TestApi** is a shared-source API library, which provides facilities for fault injection testing as well as other testing types, data-structures and algorithms for .NET applications.
- **Fuzzino** is an open source library, which provides a rich set of fuzzing heuristics that are generated from a type specification and/or valid values.

3.4.4 Fault Injection in Functional Properties or Test Cases

In contrast to traditional mutation testing where mutant faults are generated and injected into the code description of the model, application of a series of newly defined mutation operators directly to the model properties rather than to the model code has also been investigated.^[24] Mutant properties that are generated from the initial properties (or test cases) and validated by the model checker should be considered as new properties that have been missed during the initial verification procedure. Therefore, adding these newly identified properties to the existing list of properties improves the coverage metric of the formal verification and consequently lead to a more reliable design.

3.4.5 Application of fault injection

Fault injection can take many forms. In the testing of **operating systems** for example, fault injection is often performed by a *driver* (kernel-mode software) that intercepts *system calls* (calls into the kernel) and randomly returning a failure for some of the calls. This type of fault injection is useful for testing low level user mode software. For higher level software, various methods inject faults. In **managed code**, it is common to use **instrumentation**. Although fault injection can be undertaken by hand a number of fault injection tools exist to automate the process of fault injection.^[25]

Depending on the complexity of the **API** for the level where faults are injected, fault injection tests often must be carefully designed to minimise the number of false positives. Even a well designed fault injection test can sometimes produce situations that are impossible in the

normal operation of the software. For example, imagine there are two **API functions**, **Commit** and **PrepareForCommit**, such that alone, each of these functions can possibly fail, but if **PrepareForCommit** is called and succeeds, a subsequent call to **Commit** is guaranteed to succeed. Now consider the following code:

```
error = PrepareForCommit(); if (error == SUCCESS) {
error = Commit(); assert(error == SUCCESS); }
```

Often, it will be infeasible for the fault injection implementation to keep track of enough state to make the guarantee that the **API functions** make. In this example, a fault injection test of the above code might hit the **assert**, whereas this would never happen in normal operation.

3.4.6 See also

- **Debugging**
- **Mutation testing**

3.4.7 References

- [1] J. Voas, "Fault Injection for the Masses," *Computer*, vol. 30, pp. 129–130, 1997.
- [2] Kaksonen, Rauli. *A Functional Method for Assessing Protocol Implementation Security*. 2001.
- [3] A. Avizienis, J.-C. Laprie, B. Randell, and C. Landwehr, "Basic Concepts and Taxonomy of Dependable and Secure Computing," *Dependable and Secure Computing*, vol. 1, pp. 11–33, 2004.
- [4] J. V. Carreira, D. Costa, and S. J. G., "Fault Injection Spot-Checks Computer System Dependability," *IEEE Spectrum*, pp. 50–55, 1999.
- [5] Rickard Svenningsson, Jonny Vinter, Henrik Eriksson and Martin Torngren, "MODIFI: A MODEL-Implemented Fault Injection Tool," *Lecture Notes in Computer Science*, 2010, Volume 6351/2010, 210-222.
- [6] G. A. Kanawati, N. A. Kanawati, and J. A. Abraham, "FERRARI: A Flexible Software-Based Fault and Error Injection System," *IEEE Transactions on Computers*, vol. 44, pp. 248, 1995.
- [7] T. Tsai and R. Iyer, "FTAPE: A Fault Injection Tool to Measure Fault Tolerance," presented at *Computing in aerospace*, San Antonio; TX, 1995.
- [8] S. Han, K. G. Shin, and H. A. Rosenberg, "DOCTOR: An Integrated Software Fault Injection Environment for Distributed Real-time Systems," presented at *International Computer Performance and Dependability Symposium*, Erlangen; Germany, 1995.
- [9] S. Dawson, F. Jahanian, and T. Mitton, "ORCHESTRA: A Probing and Fault Injection Environment for Testing Protocol Implementations," presented at *International Computer Performance and Dependability Symposium*, Urbana-Champaign, USA, 1996.

- [10] J. V. Carreira, D. Costa, and S. J. G., “Fault Injection Spot-Checks Computer System Dependability,” *IEEE Spectrum*, pp. 50–55, 1999.
- [11] [Grid-FIT Web-site Archived 28 September 2007 at the Wayback Machine](#)
- [12] N. Looker, B. Gwynne, J. Xu, and M. Munro, “An Ontology-Based Approach for Determining the Dependability of Service-Oriented Architectures,” in the proceedings of the 10th IEEE International Workshop on Object-oriented Real-time Dependable Systems, USA, 2005.
- [13] N. Looker, M. Munro, and J. Xu, “A Comparison of Network Level Fault Injection with Code Insertion,” in the proceedings of the 29th IEEE International Computer Software and Applications Conference, Scotland, 2005.
- [14] [LFI Website](#)
- [15] [beSTORM product information](#)
- [16] [ExhaustiF SWIFI Tool Site](#)
- [17] [Holodeck product overview Archived 13 October 2008 at the Wayback Machine](#)
- [18] [Codonomicon Defensics product overview](#)
- [19] Kaksonen, Rauli. A Functional Method for Assessing Protocol Implementation Security. 2001.
- [20] [Mu Service Analyzer](#)
- [21] [Mu Dynamics, Inc.](#)
- [22] [Xception Web Site](#)
- [23] [Critical Software SA](#)
- [24] [Mutant Fault Injection in Functional Properties of a Model to Improve Coverage Metrics](#), A. Abbasinasab, M. Mohammadi, S. Mohammadi, S. Yanushkevich, M. Smith, 14th IEEE Conference Digital System Design (DSD), pp. 422-425, 2011
- [25] N. Looker, M. Munro, and J. Xu, “Simulating Errors in Web Services,” *International Journal of Simulation Systems, Science & Technology*, vol. 5, 2004.

3.4.8 External links

- [Certitude Software from Certess Inc.](#)

3.5 Bebugging

Bebugging (or **fault seeding**) is a popular software engineering technique used in the 1970s to measure test coverage. Known bugs are randomly added to a program source code and the programmer is tasked to find them. The percentage of the known bugs not found gives an indication of the real bugs that remain.

The earliest application of bebugging was [Harlan Mills's](#) fault seeding approach^[1] which was later refined by stratified fault-seeding.^[2] These techniques worked by adding a number of known faults to a software system for the purpose of monitoring the rate of detection and removal. This assumed that it is possible to estimate the number of remaining faults in a software system still to be detected by a particular test methodology.

Bebugging is a type of [fault injection](#).

3.5.1 See also

- [Fault injection](#)
- [Mutation testing](#)

3.5.2 References

- [1] H. D. Mills, “On the Statistical Validation of Computer Programs,” IBM Federal Systems Division 1972.
- [2] L. J. Morell and J. M. Voas, “Infection and Propagation Analysis: A Fault-Based Approach to Estimating Software Reliability,” College of William and Mary in Virginia, Department of Computer Science September, 1988.

3.6 Mutation testing

For the biological term, see [Gene mutation analysis](#).

Mutation testing (or *Mutation analysis* or *Program mutation*) is used to design new software tests and evaluate the quality of existing software tests. Mutation testing involves modifying a program in small ways.^[1] Each mutated version is called a *mutant* and tests detect and reject mutants by causing the behavior of the original version to differ from the mutant. This is called *killing* the mutant. Test suites are measured by the percentage of mutants that they kill. New tests can be designed to kill additional mutants. Mutants are based on well-defined *mutation operators* that either mimic typical programming errors (such as using the wrong operator or variable name) or force the creation of valuable tests (such as dividing each expression by zero). The purpose is to help the tester develop effective tests or locate weaknesses in the test data used for the program or in sections of the code that are seldom or never accessed during *execution*.

Most of this article is about “program mutation”, in which the program is modified. A more general definition of *mutation analysis* is using well-defined rules defined on syntactic structures to make systematic changes to software artifacts.^[2] Mutation analysis has been applied to other problems, but is usually applied to testing. So *mutation testing* is defined as using mutation analysis to design

new software tests or to evaluate existing software tests.^[2] Thus, mutation analysis and testing can be applied to design models, specifications, databases, tests, XML, and other types of software artifacts, although program mutation is the most common.

3.6.1 Goal

Tests can be created to verify the correctness of the implementation of a given software system, but the creation of tests still poses the question whether the tests are correct and sufficiently cover the requirements that have originated the implementation. (This technological problem is itself an instance of a deeper philosophical problem named "*Quis custodiet ipsos custodes?*" ["Who will guard the guards?"].) In this context, mutation testing was pioneered in the 1970s to locate and expose weaknesses in *test suites*. The theory was that if a mutant was introduced without the behavior (generally *output*) of the program being affected, this indicated either that the code that had been mutated was never executed (dead code) or that the test suite was unable to locate the faults represented by the mutant. For this to function at any scale, a large number of mutants usually are introduced into a large program, leading to the compilation and execution of an extremely large number of copies of the program. This problem of the expense of mutation testing had reduced its practical use as a method of software testing, but the increased use of object oriented programming languages and unit testing frameworks has led to the creation of mutation testing tools for many programming languages as a way to test individual portions of an application.

3.6.2 Historical overview

Mutation testing was originally proposed by Richard Lipton as a student in 1971,^[3] and first developed and published by DeMillo, Lipton and Sayward.^[1] The first implementation of a mutation testing tool was by Timothy Budd as part of his PhD work (titled *Mutation Analysis*) in 1980 from Yale University.^[4]

Recently, with the availability of massive computing power, there has been a resurgence of mutation analysis within the computer science community, and work has been done to define methods of applying mutation testing to object oriented programming languages and non-procedural languages such as XML, SMV, and finite state machines.

In 2004 a company called Certess Inc. (now part of Synopsys) extended many of the principles into the hardware verification domain. Whereas mutation analysis only expects to detect a difference in the output produced, Certess extends this by verifying that a checker in the test-bench will actually detect the difference. This extension means that all three stages of verification, namely: activation, propagation and detection are evaluated. They

called this functional qualification.

Fuzzing can be considered to be a special case of mutation testing. In fuzzing, the messages or data exchanged inside communication interfaces (both inside and between software instances) are mutated to catch failures or differences in processing the data. *Codenomicon*^[5] (2001) and *Mu Dynamics* (2005) evolved fuzzing concepts to a fully stateful mutation testing platform, complete with monitors for thoroughly exercising protocol implementations.

3.6.3 Mutation testing overview

Mutation testing is based on two hypotheses. The first is the *competent programmer* hypothesis. This hypothesis states that most software faults introduced by experienced programmers are due to small syntactic errors.^[1] The second hypothesis is called the *coupling effect*. The coupling effect asserts that simple faults can cascade or *couple* to form other emergent faults.^{[6][7]}

Subtle and important faults are also revealed by higher-order mutants, which further support the coupling effect.^{[8][9][10][11][12]} Higher-order mutants are enabled by creating mutants with more than one mutation.

Mutation testing is done by selecting a set of mutation operators and then applying them to the source program one at a time for each applicable piece of the source code. The result of applying one mutation operator to the program is called a *mutant*. If the test suite is able to detect the change (i.e. one of the tests fails), then the mutant is said to be *killed*.

For example, consider the following C++ code fragment:

```
if (a && b) { c = 1; } else { c = 0; }
```

The condition mutation operator would replace && with || and produce the following mutant:

```
if (a || b) { c = 1; } else { c = 0; }
```

Now, for the test to kill this mutant, the following three conditions should be met:

1. A test must *reach* the mutated statement.
2. Test input data should *infect* the program state by causing different program states for the mutant and the original program. For example, a test with $a = 1$ and $b = 0$ would do this.
3. The incorrect program state (the value of 'c') must *propagate* to the program's output and be checked by the test.

These conditions are collectively called the *RIP model*.^[3]

Weak mutation testing (or *weak mutation coverage*) requires that only the first and second conditions are satisfied. *Strong mutation testing* requires that all three conditions are satisfied. Strong mutation is more powerful, since it ensures that the test suite can really catch the problems. Weak mutation is closely related to **code coverage** methods. It requires much less computing power to ensure that the test suite satisfies weak mutation testing than strong mutation testing.

However, there are cases where it is not possible to find a test case that could kill this mutant. The resulting program is behaviorally equivalent to the original one. Such mutants are called *equivalent mutants*.

Equivalent mutants detection is one of biggest obstacles for practical usage of mutation testing. The effort needed to check if mutants are equivalent or not can be very high even for small programs.^[13] A systematic literature review of a wide range of approaches to overcome the Equivalent Mutant Problem (presented by ^[14]) identified 17 relevant techniques (in 22 articles) and three categories of techniques: detecting (DEM); suggesting (SEM); and avoiding equivalent mutant generation (AEMG). The experiment indicated that Higher Order Mutation in general and JudyDiffOp strategy in particular provide a promising approach to the Equivalent Mutant Problem.

3.6.4 Mutation operators

Many mutation operators have been explored by researchers. Here are some examples of mutation operators for imperative languages:

- Statement deletion
- Statement duplication or insertion, e.g. goto fail;^[15]
- Replacement of boolean subexpressions with *true* and *false*
- Replacement of some arithmetic operations with others, e.g. + with *, - with /
- Replacement of some boolean relations with others, e.g. > with >=, == and <=
- Replacement of variables with others from the same scope (variable types must be compatible)

mutation score = number of mutants killed / total number of mutants

These mutation operators are also called traditional mutation operators. There are also mutation operators for object-oriented languages,^[16] for concurrent constructions,^[17] complex objects like containers,^[18] etc. Operators for containers are called *class-level* mutation operators. For example the muJava tool offers various class-level mutation operators such as Access Modifier

Change, Type Cast Operator Insertion, and Type Cast Operator Deletion. Mutation operators have also been developed to perform security vulnerability testing of programs ^[19]

3.6.5 See also

- **Bebugging** (or fault seeding)
- **Sanity testing**
- **Fault injection**

3.6.6 References

- [1] Richard A. DeMillo, Richard J. Lipton, and Fred G. Sayward. Hints on test data selection: Help for the practicing programmer. *IEEE Computer*, 11(4):34-41. April 1978.
- [2] Paul Ammann and Jeff Offutt. *Introduction to Software Testing*. Cambridge University Press, 2008.
- [3] **Mutation 2000: Uniting the Orthogonal** by A. Jefferson Offutt and Roland H. Untch.
- [4] Tim A. Budd, *Mutation Analysis of Program Test Data*. PhD thesis, Yale University New Haven CT, 1980.
- [5] Kaksonen, Rauli. *A Functional Method for Assessing Protocol Implementation Security* (Licentiate thesis). Espoo. 2001.
- [6] A. Jefferson Offutt. 1992. Investigations of the software testing coupling effect. *ACM Trans. Softw. Eng. Methodol.* 1, 1 (January 1992), 5-20.
- [7] A. T. Acree, T. A. Budd, R. A. DeMillo, R. J. Lipton, and F. G. Sayward, "Mutation Analysis," Georgia Institute of Technology, Atlanta, Georgia, Technique Report GIT-ICS-79/08, 1979.
- [8] Yue Jia; Harman, M., "Constructing Subtle Faults Using Higher Order Mutation Testing," *Source Code Analysis and Manipulation*, 2008 Eighth IEEE International Working Conference on , vol., no., pp.249,258, 28-29 Sept. 2008
- [9] Maryam Umar, "An Evaluation of Mutation Operators For Equivalent Mutants," MS Thesis, 2006
- [10] Smith B., "On Guiding Augmentation of an Automated Test Suite via Mutation Analysis," 2008
- [11] Polo M. and Piattini M., "Mutation Testing: practical aspects and cost analysis," University of Castilla-La Mancha (Spain), Presentation, 2009
- [12] Anderson S., "Mutation Testing", the University of Edinburgh, School of Informatics, Presentation, 2011
- [13] P. G. Frankl, S. N. Weiss, and C. Hu. All-uses versus mutation testing: An experimental comparison of effectiveness. *Journal of Systems and Software*, 38:235-253, 1997.

- [14] Overcoming the Equivalent Mutant Problem: A Systematic Literature Review and a Comparative Experiment of Second Order Mutation by L. Madeyski, W. Orzeszyna, R. Torkar, M. Józala. *IEEE Transactions on Software Engineering*
 - [15] Apple's SSL/TLS bug by Adam Langley.
 - [16] MuJava: An Automated Class Mutation System by Yu-Seung Ma, Jeff Offutt and Yong Rae Kwo.
 - [17] Mutation Operators for Concurrent Java (J2SE 5.0) by Jeremy S. Bradbury, James R. Cordy, Juergen Dingel.
 - [18] Mutation of Java Objects by Roger T. Alexander, James M. Bieman, Sudipto Ghosh, Bixia Ji.
 - [19] Mutation-based Testing of Buffer Overflows, SQL Injections, and Format String Bugs by H. Shahriar and M. Zulkernine.
- Major Compiler-integrated mutation testing framework for Java
 - Jumble Bytecode-based mutation testing tool for Java
 - PIT Bytecode-based mutation testing tool for Java
 - Mutant AST based mutation testing tool for Ruby
 - Jester Source-based mutation testing tool for Java
 - Judy Mutation testing tool for Java
 - Heckle Mutation testing tool for Ruby
 - NinjaTurtles IL-based mutation testing tool for .NET and Mono
 - Nester Mutation testing tool for C#
 - Humbug Mutation testing tool for PHP
 - MuCheck Mutation analysis library for Haskell

3.6.7 Further reading

- Aristides Dasso, Ana Funes (2007). *Verification, Validation and Testing in Software Engineering*. Idea Group Inc. ISBN 1591408512. See Ch. VII *Test-Case Mutation* for overview on mutation testing.
- Paul Ammann, Jeff Offutt (2008). *Introduction to Software Testing*. Cambridge University Press. ISBN 978-0-521-88038-1. See Ch. V *Syntax Testing* for an overview of mutation testing.
- Yue Jia, Mark Harman (September 2009). "An Analysis and Survey of the Development of Mutation Testing" (PDF). *CREST Centre, King's College London, Technical Report TR-09-06*.
- Lech Madeyski, Norbert Radyk (2010). "Judy – A Mutation Testing Tool for Java" (PDF). *IET Software, Volume 4, Issue 1, Pages 32 – 42*.

3.6.8 External links

- Mutation testing list of tools and publications by Jeff Offutt
- muJava A mutation tool for Java that includes class-level operators
- mutate.py A Python script to mutate C-programs
- Mutator A source-based multi-language commercial mutation analyzer for concurrent Java, Ruby, JavaScript and PHP
- Bacterio Mutation testing tool for multi-class Java systems
- Javalanche Bytecode-based mutation testing tool for Java

Chapter 4

Testing of non functional software aspects

4.1 Non-functional testing

Non-functional testing is the testing of a software application or system for its **non-functional requirements**: the way a system operates, rather than specific behaviours of that system. This is contrast to **functional testing**, which tests against **functional requirements** that describe the functions of a system and its components. The names of many non-functional tests are often used interchangeably because of the overlap in scope between various non-functional requirements. For example, **software performance** is a broad term that includes many specific requirements like **reliability** and **scalability**.

Non-functional testing includes:

- Baseline testing
- Compliance testing
- Documentation testing
- Endurance testing
- Load testing
- Localization testing and Internationalization testing
- Performance testing
- Recovery testing
- Resilience testing
- Security testing
- Scalability testing
- Stress testing
- Usability testing
- Volume testing

4.2 Software performance testing

In **software engineering**, **performance testing** is in general testing performed to determine how a **system** performs in terms of responsiveness and stability under a particular workload. It can also serve to investigate, measure, validate or verify other **quality attributes** of the system, such as **scalability**, **reliability** and resource usage.

Performance testing, a subset of **performance engineering**, is a **computer science** practice which strives to build performance into the implementation, design and architecture of a system.

4.2.1 Testing types

Load testing

Load testing is the simplest form of performance testing. A load test is usually conducted to understand the behaviour of the system under a specific expected load. This load can be the expected concurrent number of users on the **application** performing a specific number of **transactions** within the set duration. This test will give out the response times of all the important business critical transactions. If the **database**, **application server**, etc. are also monitored, then this simple test can itself point towards **bottlenecks** in the application software.

Stress testing

Stress testing is normally used to understand the upper limits of capacity within the system. This kind of test is done to determine the system's robustness in terms of extreme load and helps application administrators to determine if the system will perform sufficiently if the current load goes well above the expected maximum.

Soak testing

Soak testing, also known as endurance testing, is usually done to determine if the system can sustain the continuous expected load. During soak tests, memory utiliza-

tion is monitored to detect potential leaks. Also important, but often overlooked is performance degradation, i.e. to ensure that the throughput and/or response times after some long period of sustained activity are as good as or better than at the beginning of the test. It essentially involves applying a significant load to a system for an extended, significant period of time. The goal is to discover how the system behaves under sustained use.

Spike testing

Spike testing is done by suddenly increasing the load generated by a very large number of users, and observing the behaviour of the system. The goal is to determine whether performance will suffer, the system will fail, or it will be able to handle dramatic changes in load.

Configuration testing

Rather than testing for performance from a load perspective, tests are created to determine the effects of configuration changes to the system's components on the system's performance and behaviour. A common example would be experimenting with different methods of load-balancing.

Isolation testing

Isolation testing is not unique to performance testing but involves repeating a test execution that resulted in a system problem. Such testing can often isolate and confirm the fault domain.

4.2.2 Setting performance goals

Performance testing can serve different purposes:

- It can demonstrate that the system meets performance criteria.
- It can compare two systems to find which performs better.
- It can measure which parts of the system or workload cause the system to perform badly.

Many performance tests are undertaken without setting sufficiently realistic, goal-oriented performance goals. The first question from a business perspective should always be, "why are we performance-testing?". These considerations are part of the **business** case of the testing. Performance goals will differ depending on the system's technology and purpose, but should always include some of the following:

Concurrency/throughput

If a system identifies end-users by some form of log-in procedure then a concurrency goal is highly desirable. By definition this is the largest number of concurrent system users that the system is expected to support at any given moment. The work-flow of a scripted transaction may impact true **concurrency** especially if the iterative part contains the log-in and log-out activity.

If the system has no concept of end-users, then performance goal is likely to be based on a maximum throughput or transaction rate. A common example would be casual browsing of a web site such as Wikipedia.

Server response time

This refers to the time taken for one system node to respond to the request of another. A simple example would be a HTTP 'GET' request from browser client to web server. In terms of response time this is what all **load testing** tools actually measure. It may be relevant to set server response time goals between all nodes of the system.

Render response time

Load-testing tools have difficulty measuring render-response time, since they generally have no concept of what happens within a **node** apart from recognizing a period of time where there is no activity 'on the wire'. To measure render response time, it is generally necessary to include functional **test scripts** as part of the performance test scenario. Many load testing tools do not offer this feature.

Performance specifications

It is critical to detail performance specifications (requirements) and document them in any performance test plan. Ideally, this is done during the requirements development phase of any system development project, prior to any design effort. See **Performance Engineering** for more details.

However, performance testing is frequently not performed against a specification; e.g., no one will have expressed what the maximum acceptable response time for a given population of users should be. Performance testing is frequently used as part of the process of performance profile tuning. The idea is to identify the "weakest link" – there is inevitably a part of the system which, if it is made to respond faster, will result in the overall system running faster. It is sometimes a difficult task to identify which part of the system represents this critical path, and some test tools include (or can have add-ons that provide) instrumentation that runs on the server (agents) and reports transaction times, database access times, network

overhead, and other server monitors, which can be analyzed together with the raw performance statistics. Without such instrumentation one might have to have someone crouched over **Windows Task Manager** at the server to see how much CPU load the performance tests are generating (assuming a Windows system is under test).

Performance testing can be performed across the web, and even done in different parts of the country, since it is known that the response times of the internet itself vary regionally. It can also be done in-house, although **routers** would then need to be configured to introduce the lag that would typically occur on public networks. Loads should be introduced to the system from realistic points. For example, if 50% of a system's user base will be accessing the system via a 56K modem connection and the other half over a T1, then the load injectors (computers that simulate real users) should either inject load over the same mix of connections (ideal) or simulate the network latency of such connections, following the same user profile.

It is always helpful to have a statement of the likely peak number of users that might be expected to use the system at peak times. If there can also be a statement of what constitutes the maximum allowable 95 percentile response time, then an injector configuration could be used to test whether the proposed system met that specification.

Questions to ask

Performance specifications should ask the following questions, at a minimum:

- In detail, what is the performance test scope? What subsystems, interfaces, components, etc. are in and out of scope for this test?
- For the user interfaces (UIs) involved, how many concurrent users are expected for each (specify peak vs. nominal)?
- What does the target system (hardware) look like (specify all server and network appliance configurations)?
- What is the Application Workload Mix of each system component? (for example: 20% log-in, 40% search, 30% item select, 10% checkout).
- What is the System Workload Mix? [Multiple workloads may be simulated in a single performance test] (for example: 30% Workload A, 20% Workload B, 50% Workload C).
- What are the time requirements for any/all back-end batch processes (specify peak vs. nominal)?

4.2.3 Prerequisites for Performance Testing

A stable build of the system which must resemble the production environment as closely as is possible.

To ensure consistent results, the performance testing environment should be isolated from other environments, such as **user acceptance testing** (UAT) or development. As a best practice it is always advisable to have a separate performance testing environment resembling the production environment as much as possible.

Test conditions

In performance testing, it is often crucial for the test conditions to be similar to the expected actual use. However, in practice this is hard to arrange and not wholly possible, since production systems are subjected to unpredictable workloads. Test workloads may mimic occurrences in the production environment as far as possible, but only in the simplest systems can one exactly replicate this workload variability.

Loosely-coupled architectural implementations (e.g.: **SOA**) have created additional complexities with performance testing. To truly replicate production-like states, enterprise services or assets that share a common **infrastructure** or platform require coordinated performance testing, with all consumers creating production-like transaction volumes and load on shared infrastructures or platforms. Because this activity is so complex and costly in money and time, some organizations now use tools to monitor and simulate production-like conditions (also referred as “noise”) in their performance testing environments (**PTE**) to understand capacity and resource requirements and verify / validate quality attributes.

Timing

It is critical to the cost performance of a new system, that performance test efforts begin at the inception of the development project and extend through to deployment. The later a performance defect is detected, the higher the cost of remediation. This is true in the case of functional testing, but even more so with performance testing, due to the end-to-end nature of its scope. It is crucial for a performance test team to be involved as early as possible, because it is time-consuming to acquire and prepare the testing environment and other key performance requisites.

4.2.4 Tools

In the diagnostic case, **software engineers** use tools such as **profilers** to measure what parts of a device or software contribute most to the poor performance, or to establish

throughput levels (and thresholds) for maintained acceptable response time.

4.2.5 Technology

Performance testing technology employs one or more PCs or Unix servers to act as injectors, each emulating the presence of numbers of users and each running an automated sequence of interactions (recorded as a script, or as a series of scripts to emulate different types of user interaction) with the host whose performance is being tested. Usually, a separate PC acts as a test conductor, coordinating and gathering metrics from each of the injectors and collating performance data for reporting purposes. The usual sequence is to ramp up the load: to start with a few virtual users and increase the number over time to a predetermined maximum. The test result shows how the performance varies with the load, given as number of users vs. response time. Various tools are available to perform such tests. Tools in this category usually execute a suite of tests which emulate real users against the system. Sometimes the results can reveal oddities, e.g., that while the average response time might be acceptable, there are outliers of a few key transactions that take considerably longer to complete – something that might be caused by inefficient database queries, pictures, etc.

Performance testing can be combined with **stress testing**, in order to see what happens when an acceptable load is exceeded. Does the system crash? How long does it take to recover if a large load is reduced? Does its failure cause collateral damage?

Analytical Performance Modeling is a method to model the behaviour of a system in a spreadsheet. The model is fed with measurements of transaction resource demands (CPU, disk I/O, LAN, WAN), weighted by the transaction-mix (business transactions per hour). The weighted transaction resource demands are added up to obtain the hourly resource demands and divided by the hourly resource capacity to obtain the resource loads. Using the responsetime formula ($R=S/(1-U)$, R =responsetime, S =servicetime, U =load), responsetimes can be calculated and calibrated with the results of the performance tests. Analytical performance modeling allows evaluation of design options and system sizing based on actual or anticipated business use. It is therefore much faster and cheaper than performance testing, though it requires thorough understanding of the hardware platforms.

4.2.6 Tasks to undertake

Tasks to perform such a test would include:

- Decide whether to use internal or external resources to perform the tests, depending on inhouse expertise (or lack of it)

- Gather or elicit performance requirements (specifications) from users and/or business analysts
- Develop a high-level **plan** (or project charter), including requirements, resources, timelines and milestones
- Develop a detailed performance **test plan** (including detailed scenarios and **test cases**, workloads, environment info, etc.)
- Choose **test tool(s)**
- Specify test data needed and charter effort (often overlooked, but vital to carrying out a valid performance test)
- Develop **proof-of-concept** scripts for each application/component under test, using chosen test tools and strategies
- Develop detailed performance test project plan, including all dependencies and associated timelines
- Install and configure injectors/controller
- Configure the test environment (ideally identical hardware to the production platform), router configuration, quiet network (we don't want results upset by other users), deployment of server instrumentation, database test sets developed, etc.
- Dry run the tests - before actually executing the load test with predefined users, a dry run is carried out in order to check the correctness of the script
- Execute tests – probably repeatedly (iteratively) in order to see whether any unaccounted-for factor might affect the results
- Analyze the results - either pass/fail, or investigation of critical path and recommendation of corrective action

4.2.7 Methodology

Performance testing web applications

According to the Microsoft Developer Network the **Performance Testing Methodology** consists of the following activities:

1. **Identify the Test Environment.** Identify the physical **test environment** and the production environment as well as the tools and resources available to the test team. The physical environment includes hardware, software, and network configurations. Having a thorough understanding of the entire test environment at the outset enables more efficient **test design** and planning and helps you identify testing challenges early in the project. In some situations, this process must be revisited periodically throughout the project's **life cycle**.

2. **Identify Performance Acceptance Criteria.** Identify the response time, throughput, and resource-use goals and constraints. In general, response time is a user concern, throughput is a business concern, and resource use is a system concern. Additionally, identify project success criteria that may not be captured by those goals and constraints; for example, using performance tests to evaluate which combination of configuration settings will result in the most desirable performance characteristics.
3. **Plan and Design Tests.** Identify key scenarios, determine variability among representative users and how to simulate that variability, define test data, and establish metrics to be collected. Consolidate this information into one or more models of system usage to be implemented, executed, and analyzed.
4. **Configure the Test Environment.** Prepare the test environment, tools, and resources necessary to execute each strategy, as features and components become available for test. Ensure that the test environment is instrumented for resource monitoring as necessary.
5. **Implement the Test Design.** Develop the performance tests in accordance with the test design.
6. **Execute the Test.** Run and monitor your tests. Validate the tests, test data, and results collection. Execute validated tests for analysis while monitoring the test and the test environment.
7. **Analyze Results, Tune, and Retest.** Analyse, Consolidate and share results data. Make a tuning change and retest. Compare the results of both tests. Each improvement made will return smaller improvement than the previous improvement. When do you stop? When you reach a CPU bottleneck, the choices then are either improve the code or add more CPU.

4.2.8 See also

- [Stress testing \(software\)](#)
- [Benchmark \(computing\)](#)
- [Web server benchmarking](#)
- [Application Response Measurement](#)

4.2.9 External links

- [The Art of Application Performance Testing - O'Reilly ISBN 978-0-596-52066-3 \(Book\)](#)
- [Performance Testing Guidance for Web Applications \(MSDN\)](#)

- [Performance Testing Guidance for Web Applications \(Book\)](#)
- [Performance Testing Guidance for Web Applications \(PDF\)](#)
- [Performance Testing Guidance \(Online KB\)](#)
- [Enterprise IT Performance Testing \(Online KB\)](#)
- [Performance Testing Videos \(MSDN\)](#)
- [Open Source Performance Testing tools](#)
- [“User Experience, not Metrics” and “Beyond Performance Testing”](#)
- [“Performance Testing Traps / Pitfalls”](#)

4.3 Stress testing

Stress testing is a software testing activity that determines the robustness of software by testing beyond the limits of normal operation. Stress testing is particularly important for "mission critical" software, but is used for all types of software. Stress tests commonly put a greater emphasis on robustness, availability, and error handling under a heavy load, than on what would be considered correct behavior under normal circumstances.

4.3.1 Field experience

Failures may be related to:

- characteristics of non-production like environments, e.g. small test databases
- complete lack of load or stress testing

4.3.2 Rationale

Reasons for stress testing include:

- The software being tested is “mission critical”, that is, failure of the software (such as a crash) would have disastrous consequences.
- The amount of time and resources dedicated to testing is usually not sufficient, with traditional testing methods, to test all of the situations in which the software will be used when it is released.
- Even with sufficient time and resources for writing tests, it may not be possible to determine before hand all of the different ways in which the software will be used. This is particularly true for operating systems and middleware, which will eventually be used by software that doesn't even exist at the time of the testing.

- Customers may use the software on computers that have significantly fewer computational resources (such as **memory** or **disk space**) than the computers used for testing.
- Input data integrity cannot be guaranteed. Input data are software wide: it can be data files, streams and memory buffers, as well as arguments and options given to a command line executable or user inputs triggering actions in a GUI application. **Fuzzing** and **monkey test** methods can be used to find problems due to data corruption or incoherence.
- **Concurrency** is particularly difficult to test with traditional testing methods. Stress testing may be necessary to find **race conditions** and **deadlocks**.
- Software such as **web servers** that will be accessible over the **Internet** may be subject to **denial of service** attacks.
- Under normal conditions, certain types of **bugs**, such as **memory leaks**, can be fairly benign and difficult to detect over the short periods of time in which testing is performed. However, these bugs can still be potentially serious. In a sense, stress testing for a relatively short period of time can be seen as simulating normal operation for a longer period of time.

4.3.3 Relationship to branch coverage

Branch coverage (a specific type of **code coverage**) is a metric of the number of branches executed under test, where “100% branch coverage” means that every branch in a program has been executed at least once under some test. Branch coverage is one of the most important metrics for software testing; software for which the branch coverage is low is not generally considered to be thoroughly tested. Note that code coverage metrics are a property of the tests for a piece of software, not of the software being tested.

Achieving high branch coverage often involves writing *negative* test variations, that is, variations where the software is supposed to fail in some way, in addition to the usual *positive* test variations, which test intended usage. An example of a negative variation would be calling a **function** with illegal parameters. There is a limit to the branch coverage that can be achieved even with negative variations, however, as some branches may only be used for handling of errors that are beyond the control of the test. For example, a test would normally have no control over memory allocation, so branches that handle an “out of memory” error are difficult to test.

Stress testing can achieve higher branch coverage by producing the conditions under which certain error handling branches are followed. The coverage can be further improved by using **fault injection**.

4.3.4 Examples

- A **web server** may be stress tested using **scripts**, **bots**, and various **denial of service** tools to observe the performance of a web site during peak loads.

4.3.5 Load test vs. stress test

Stress testing tries to break the system under test by overwhelming its resources or by taking resources away from it (in which case it is sometimes called negative testing). The main purpose of this process is to make sure that the system fails and recovers gracefully—a quality known as recoverability.^[1]

Load testing implies a controlled environment moving from low loads to high. Stress testing focuses on more random events, chaos and unpredictability. Using a web application as an example here are ways stress might be introduced:^[1]

- double the baseline number for concurrent users/HTTP connections
- randomly shut down and restart ports on the network switches/routers that connect the servers (via SNMP commands for example)
- take the database offline, then restart it
- rebuild a RAID array while the system is running
- run processes that consume resources (CPU, memory, disk, network) on the Web and database servers
- observe how the system reacts to failure and recovers
 - Does it save its state?
 - Does the application hang and freeze or does it fail gracefully?
 - On restart, is it able to recover from the last good state?
 - Does the system output meaningful error messages to the user and to the logs?
 - Is the security of the system compromised because of unexpected failures?

4.3.6 See also

- **Software testing**
- This article covers testing software reliability *under unexpected or rare* (stressed) workloads. See also the closely related:
 - **Scalability testing**
 - **Load testing**
 - List of software tools for load testing at **Load testing#Load testing tools**

- Stress test for a general discussion
- Black box testing
- Software performance testing
- Scenario analysis
- Simulation
- White box testing
- Technischer Überwachungsverein (TÜV) - product testing and certification
- Concurrency testing using the **CHESS model checker**
- Jinx automates stress testing by automatically exploring unlikely execution scenarios.
- Stress test (hardware)

4.3.7 References

- [1] Gheorghiu, Grig. “Performance vs. load vs. stress testing”. Agile Testing. Retrieved 25 February 2013.

4.4 Load testing

Load testing is the process of putting demand on a **software system** or **computing device** and measuring its response. Load testing is performed to determine a system’s behavior under both normal and anticipated peak load conditions. It helps to identify the maximum operating capacity of an application as well as any bottlenecks and determine which element is causing degradation. When the load placed on the system is raised beyond normal usage patterns, in order to test the system’s response at unusually high or peak loads, it is known as **stress testing**. The load is usually so great that error conditions are the expected result, although no clear boundary exists when an activity ceases to be a load test and becomes a stress test.

There is little agreement on what the specific goals of load testing are. The term is often used synonymously with concurrency testing, **software performance testing**, **reliability testing**, and **volume testing**. *Load testing* is usually a type of **non-functional testing** although it can be used as a functional test to validate suitability for use.

4.4.1 Software load testing

Main article: **Software performance testing**

The term *load testing* is used in different ways in the professional **software testing** community. *Load testing* generally refers to the practice of modeling the expected usage

of a software program by simulating multiple users accessing the program concurrently.^[1] As such, this testing is most relevant for multi-user systems; often one built using a client/server model, such as web servers. However, other types of software systems can also be load tested. For example, a word processor or graphics editor can be forced to read an extremely large document; or a financial package can be forced to generate a report based on several years’ worth of data. The most accurate load testing simulates actual use, as opposed to testing using theoretical or analytical modeling.

Load testing lets you measure your website’s **QOS** performance based on actual customer behavior. Nearly all the load testing tools and frame-works follow the classical load testing paradigm: when customers visit your web site, a script recorder records the communication and then creates related interaction scripts. A load generator tries to replay the recorded scripts, which could possibly be modified with different test parameters before replay. In the replay procedure, both the hardware and software statistics will be monitored and collected by the conductor, these statistics include the CPU, memory, disk IO of the physical servers and the response time, throughput of the System Under Test (short as SUT), etc. And at last, all these statistics will be analyzed and a load testing report will be generated.

Load and performance testing analyzes **software** intended for a multi-user audience by subjecting the software to different numbers of virtual and live users while monitoring performance measurements under these different loads. Load and performance testing is usually conducted in a test environment identical to the production environment before the software system is permitted to go live.

As an example, a web site with **shopping cart** capability is required to support 100 concurrent users broken out into following activities:

- 25 Virtual Users (VUsers) log in, browse through items and then log off
- 25 VUsers log in, add items to their shopping cart, check out and then log off
- 25 VUsers log in, return items previously purchased and then log off
- 25 VUsers just log in without any subsequent activity

A test analyst can use various load testing tools to create these VUsers and their activities. Once the test has started and reached a steady state, the application is being tested at the 100 VUser load as described above. The application’s performance can then be monitored and captured.

The specifics of a **load test plan** or **script** will generally vary across organizations. For example, in the bulleted list above, the first item could represent 25 VUsers browsing unique items, random items, or a selected set of items

depending upon the test plan or script developed. However, all load test plans attempt to simulate system performance across a range of anticipated peak workflows and volumes. The criteria for passing or failing a load test (pass/fail criteria) are generally different across organizations as well. There are no standards specifying acceptable load testing performance metrics.

A common misconception is that load testing software provides record and playback capabilities like **regression testing** tools. Load testing tools analyze the entire **OSI protocol stack** whereas most regression testing tools focus on **GUI** performance. For example, a regression testing tool will record and playback a mouse click on a button on a web browser, but a load testing tool will send out hypertext the web browser sends after the user clicks the button. In a multiple-user environment, load testing tools can send out hypertext for multiple users with each user having a unique login ID, password, etc.

The popular load testing tools available also provide insight into the causes for slow performance. There are numerous possible causes for slow system performance, including, but not limited to, the following:

- Application server(s) or software
- Database server(s)
- Network – latency, congestion, etc.
- Client-side processing
- Load balancing between multiple servers

Load testing is especially important if the application, system or service will be subject to a **service level agreement** or SLA.

User Experience Under Load test

In the example above, while the device under test (DUT) is under production load - 100 VUsers, run the target application. The performance of the target application here would be the User Experience Under Load. It describes how fast or slow the DUT responds, and how satisfied or how the user actually perceives performance.

Load testing tools

4.4.2 Physical load testing

Many types of machinery, engines^[4], structures^[5], and motors^[6] are load tested. The load may be at a designated safe working load (SWL), full load, or at an aggravated level of load. The governing contract, **technical specification** or **test method** contains the details of conducting the test. The purpose of a mechanical load test is to verify that all the component parts of a structure including

materials, base-fixings are fit for task and loading it is designed for.

Several types of load testing are employed

- Static testing is when a designated constant load is applied for a specified time.
- Dynamic testing is when a variable or moving load is applied.
- Cyclical testing consists of repeated loading and unloading for specified cycles, durations and conditions.

The *Supply of Machinery (Safety) Regulation 1992 UK* state that load testing is undertaken before the equipment is put into service for the first time. **Performance testing** applies a safe working load (SWL), or other specified load, for a designated time in a governing test method, specification, or contract. Under the *Lifting Operations and Lifting Equipment Regulations 1998 UK* load testing after the initial test is required if a major component is replaced, if the item is moved from one location to another or as dictated by the *Competent Person*

4.4.3 Car charging system

A load test can be used to evaluate the health of a car's **battery**. The tester consists of a large resistor that has a resistance similar to a car's **starter motor** and a meter to read the battery's output voltage both in the unloaded and loaded state. When the tester is used, the battery's open circuit voltage is checked first. If the open circuit voltage is below spec (12.6 volts for a fully charged battery), the battery is charged first. After reading the battery's open circuit voltage, the load is applied. When applied, it draws approximately the same current the car's starter motor would draw during cranking. Based on the specified **cold cranking amperes** of the battery, if the voltage under load falls below a certain point, the battery is bad. Load tests are also used on running cars to check the output of the car's **alternator**.

4.4.4 See also

- **Web testing**
- **Web server benchmarking**

4.4.5 References

- [1] Wescott, Bob (2013). *The Every Computer Performance Book, Chapter 6: Load Testing*. CreateSpace. ISBN 1482657759.
- [2] "Load Testing ASP.NET Applications with Visual Studio 2010". Eggheadcafe.com. Retrieved 2013-01-13.

- [3] “WebLOAD Review-Getting Started with WebLOAD Load Testing Tool”
- [4] Harper, David; Devin Martin, Harold Miller, Robert Grimley and Frédéric Greiner (2003), *Design of the 6C Heavy-Duty Gas Turbine*, ASME Turbo Expo 2003, collocated with the 2003 International Joint Power Generation Conference, Volume 2: Turbo Expo 2003, Atlanta GA: ASME 1., pp. 833–841, ISBN 0-7918-3685-1, retrieved 14 July 2013
- [5] Raines, Richard; Garnier, Jacques (2004), *Physical Modeling of Suction Piles in Clay*, 23rd International Conference on Offshore Mechanics and Arctic Engineering 1, Vancouver BC: ASME, pp. 621–631, doi:10.1115/OMAE2004-51343, retrieved 14 July 2013
- [6] *DETERMINING ELECTRIC MOTOR LOAD AND EFFICIENCY* (PDF), DOE/GO-10097-517, US Department of Energy, 2010, ISBN 978-0-9709500-6-2, retrieved 14 July 2013

4.4.6 External links

- Modeling the Real World for Load Testing Web Sites by Steven Splaine
- What is Load Testing? by Tom Huston
- 4 Types of Load Testing and when each should be used by David Buch

4.5 Volume testing

Volume Testing belongs to the group of **non-functional tests**, which are often misunderstood and/or used interchangeably. Volume testing refers to testing a software application with a certain amount of data. This amount can, in generic terms, be the database size or it could also be the size of an interface file that is the subject of volume testing. For example, if you want to volume test your application with a specific database size, you will expand your database to that size and then test the application’s **performance** on it. Another example could be when there is a requirement for your application to interact with an interface file (could be any file such as .dat, .xml); this interaction could be reading and/or writing on to/from the file. You will create a sample file of the size you want and then test the application’s functionality with that file in order to test the performance.

4.6 Scalability testing

Scalability Testing, part of the battery of **non-functional tests**, is the testing of a software application for measuring its capability to **scale up or scale out** in terms of any of its non-functional capability.

Performance, scalability and reliability are usually considered together by **software quality analysts**.

Scalability testing tools exist (often leveraging scalable resources themselves) in order to test user load, concurrent connections, transactions, and throughput of many internet services. Of the available testing services, those offering API support suggest that environment of continuous deployment also continuously test how recent changes may impact scalability.

4.6.1 External links

- Designing Distributed Applications with Visual Studio .NET: Scalability

4.7 Compatibility testing

Compatibility testing, part of **software non-functional tests**, is testing conducted on the application to evaluate the application’s **compatibility** with the computing environment. Computing environment may contain some or all of the below mentioned elements:

- Computing capacity of Hardware Platform (IBM 360, HP 9000, etc.)..
- Bandwidth handling capacity of networking hardware
- Compatibility of **peripherals** (Printer, DVD drive, etc.)
- Operating systems (**Linux**, **Windows**, **Mac** etc.)
- **Database** (Oracle, SQL Server, MySQL, etc.)
- Other System Software (Web server, networking/messaging tool, etc.)
- **Browser** compatibility (Chrome, Firefox, Netscape, Internet Explorer, Safari, etc.)

Browser compatibility testing, can be more appropriately referred to as **user experience testing**. This requires that the web applications are tested on different web browsers, to ensure the following:

- Users have the same visual experience irrespective of the browsers through which they view the web application.
- In terms of functionality, the application must behave and respond the same way across different browsers.
- **Carrier** compatibility (Verizon, Sprint, Orange, O2, AirTel, etc.)

- **Backwards compatibility.**
- Hardware (different phones)
- Different **Compilers** (compile the code correctly)
- Runs on multiple host/guest **Emulators**
-

Certification testing falls within the scope of compatibility testing. Product Vendors run the complete suite of testing on the newer computing environment to get their application certified for a specific Operating Systems or Databases.

4.8 Portability testing

Portability testing is the process of determining the degree of ease or difficulty to which a **software component** or application can be effectively and efficiently transferred from one hardware, software or other operational or usage environment to another.^[1] The test results, defined by the individual needs of the system, are some measurement of how easily the component or application will be to integrate into the environment and these results will then be compared to the **software system's non-functional requirement** of **portability**^[2] for correctness. The levels of correctness are usually measured by the **cost** to adapt the software to the new environment^[3] compared to the cost of redevelopment.^[4]

4.8.1 Use cases

When multiple subsystems share components of a larger system, portability testing can be used to help prevent propagation of errors throughout the system.^[5] Changing or upgrading to a newer system, adapting to a new interface or interfacing a new system in an existing environment are all problems that software systems with longevity will face sooner or later and properly testing the environment for portability can save on overall cost throughout the life of the system.^[5] A general guideline for portability testing is that it should be done if the software system is designed to move from one hardware platform, operating system, or web browser to another.^[6]

Examples

- Software designed to run on Macintosh OS X and Microsoft Windows operating systems.^[7]
- Applications developed to be compatible with Google Android and Apple iOS phones.^[7]
- Video Games or other graphic intensive software intended to work with OpenGL and DirectX **API's**.^[7]

- Software that should be compatible with Google Chrome and Mozilla Firefox browsers.^[7]

4.8.2 Attributes

There are four testing attributes included in portability testing. The **ISO 9126 (1991)** standard breaks down portability testing attributes^[5] as Installability, Compatibility, Adaptability and Replaceability. The **ISO 29119 (2013)** standard describes Portability with the attributes of Compatibility, Installability, Interoperability and Localization testing.^[8]

- **Adaptability testing-** Functional test to verify that the software can perform all of its intended behaviors in each of the target environments.^{[9][10]} Using communication standards, such as HTML can help with adaptability. Adaptability may include testing in the following areas: **hardware dependency**, **software dependency**, representation dependency, standard language conformance, **dependency encapsulation** and/or text convertibility.^[5]
- **Compatibility/ Co-existence-** Testing the compatibility of multiple, unrelated software systems to co-exist in the same environment, without effecting each other's behavior.^{[9][11][12]} This is a growing issue with advanced systems, increased functionality and interconnections between systems and subsystems who share components. Components that fail this requirement could have profound effects on a system. For example, if 2 sub-systems share memory or a **stack**, an error in one could propagate to the other and in some cases cause complete failure of the entire system.^[5]
- **Installability testing-** Installation software is tested on its ability to effectively install the target software in the intended environment.^{[5][9][13][14]} Installability may include tests for: space demand, checking prerequisites, installation procedures, completeness, installation interruption, customization, initialization, and/or deinstallation.^[5]
- **Interoperability testing-** Testing the capability to communicate, execute programs, or transfer data among various functional units in a manner that requires the user to have little or no knowledge of the unique characteristics of those units.^[1]
- **Localization testing-** Localization is also known as internationalization. Its purpose is to test if the software can be understood in using the local language where the software is being used.^[8]
- **Replaceability testing-** Testing the capability of one software component to be replaced by another soft-

ware component within a single system. The system, in regards to the replaced component, should produce the same results that it produced before the replacement.^{[9][15][16]} The issues for adaptability also apply for replaceability, but for replaceability you may also need to test for data load-ability and/or convertibility.^[5]

4.8.3 See also

- Porting
- Software portability
- Software system
- Software testing
- Software testability
- Application portability
- Operational Acceptance

4.8.4 References

- [1] "ISO/IEC/IEEE 29119-4 Software and Systems Engineering - Software Testing -Part 4- Test Techniques url=http://www.iso.org/iso/home/store/catalogue_tc/catalogue_detail.htm?csnumber=60245".
- [2] "Portability Testing". OPEN Process Framework Repository Organization. Retrieved 29 April 2014.
- [3] Rouse, Margaret. "DEFINITION environment". Retrieved 29 April 2014.
- [4] Mooney, James. "Bringing Portability to the Software Process" (PDF). Retrieved 29 April 2014.
- [5] Hass, Anne Mette Jonassen (2008). *Guide to advanced software testing* ([Online-Ausg.] ed.). Boston: Artech House. pp. 271–272. ISBN 978-1596932852.
- [6] Salonen, Ville. "Automatic Portability Testing" (PDF). Retrieved 29 April 2014.
- [7] Salonen, Ville (October 17, 2012). "Automatic Portability Testing" (PDF). Ville Salonen. pp. 11–18. Retrieved 15 May 2014.
- [8] Woods, Anthony (2015). "Operational Acceptance - an application of the ISO 29119 Software Testing standard".
- [9] "ISTQB Advanced Level Syllabi". ASTQB. Retrieved 29 April 2014.
- [10] Hass, Anne Mette Jonassen (2008). *Guide to advanced software testing* ([Online-Ausg.] ed.). Boston: Artech House. pp. 272–273. ISBN 978-1596932852.
- [11] "What is Compatibility testing in Software testing?". Mindstream Theme on Genesis Framework. Retrieved 29 April 2014.
- [12] Hass, Anne Mette Jonassen (2008). *Guide to advanced software testing* ([Online-Ausg.] ed.). Boston: Artech House. p. 272. ISBN 978-1596932852.
- [13] "Installability Guidelines". Retrieved 29 April 2014.
- [14] "What is Portability testing in software?". Mindstream Theme. Retrieved 29 April 2014.
- [15] "Replaceability". Retrieved 29 April 2014.
- [16] Hass, Anne Mette Jonassen (2008). *Guide to advanced software testing* ([Online-Ausg.] ed.). Boston: Artech House. p. 273. ISBN 978-1596932852.

4.9 Security testing

Security testing is a process intended to reveal flaws in the security mechanisms of an information system that protect data and maintain functionality as intended. Due to the logical limitations of security testing, passing security testing is not an indication that no flaws exist or that the system adequately satisfies the security requirements.

Typical security requirements may include specific elements of confidentiality, integrity, authentication, availability, authorization and non-repudiation. Actual security requirements tested depend on the security requirements implemented by the system. Security testing as a term has a number of different meanings and can be completed in a number of different ways. As such a Security Taxonomy helps us to understand these different approaches and meanings by providing a base level to work from.

4.9.1 Confidentiality

- A security measure which protects against the disclosure of information to parties other than the intended recipient is by no means the only way of ensuring the security.

4.9.2 Integrity

Integrity of information refers to protecting information from being modified by unauthorized parties

- A measure intended to allow the receiver to determine that the information provided by a system is correct.
- Integrity schemes often use some of the same underlying technologies as confidentiality schemes, but they usually involve adding information to a communication, to form the basis of an algorithmic check, rather than the encoding all of the communication.
- To check if the correct information is transferred from one application to other

4.9.3 Authentication

This might involve confirming the identity of a person, tracing the origins of an artifact, ensuring that a product is what its packaging and labeling claims to be, or assuring that a computer program is a trusted one.

4.9.4 Authorization

- The process of determining that a requester is allowed to receive a service or perform an operation.
- **Access control** is an example of authorization.

4.9.5 Availability

- Assuring information and communications services will be ready for use when expected.
- Information must be kept available to authorized persons when they need it.

4.9.6 Non-repudiation

- In reference to digital security, nonrepudiation means to ensure that a transferred message has been sent and received by the parties claiming to have sent and received the message. Nonrepudiation is a way to guarantee that the sender of a message cannot later deny having sent the message and that the recipient cannot deny having received the message.

4.9.7 Security Testing Taxonomy

Common terms used for the delivery of security testing:

- **Discovery** - The purpose of this stage is to identify systems within scope and the services in use. It is not intended to discover vulnerabilities, but version detection may highlight deprecated versions of software / firmware and thus indicate potential vulnerabilities.
- **Vulnerability Scan** - Following the discovery stage this looks for known security issues by using automated tools to match conditions with known vulnerabilities. The reported risk level is set automatically by the tool with no manual verification or interpretation by the test vendor. This can be supplemented with credential based scanning that looks to remove some common false positives by using supplied credentials to authenticate with a service (such as local windows accounts).

- **Vulnerability Assessment** - This uses discovery and vulnerability scanning to identify security vulnerabilities and places the findings into the context of the environment under test. An example would be removing common false positives from the report and deciding risk levels that should be applied to each report finding to improve business understanding and context.

- **Security Assessment** - Builds upon Vulnerability Assessment by adding manual verification to confirm exposure, but does not include the exploitation of vulnerabilities to gain further access. Verification could be in the form of authorised access to a system to confirm system settings and involve examining logs, system responses, error messages, codes, etc. A Security Assessment is looking to gain a broad coverage of the systems under test but not the depth of exposure that a specific vulnerability could lead to.

- **Penetration Test** - **Penetration test** simulates an attack by a malicious party. Building on the previous stages and involves exploitation of found vulnerabilities to gain further access. Using this approach will result in an understanding of the ability of an attacker to gain access to confidential information, affect data integrity or availability of a service and the respective impact. Each test is approached using a consistent and complete methodology in a way that allows the tester to use their problem solving abilities, the output from a range of tools and their own knowledge of networking and systems to find vulnerabilities that would/ could not be identified by automated tools. This approach looks at the depth of attack as compared to the Security Assessment approach that looks at the broader coverage.

- **Security Audit** - Driven by an Audit / Risk function to look at a specific control or compliance issue. Characterised by a narrow scope, this type of engagement could make use of any of the earlier approaches discussed (vulnerability assessment, security assessment, penetration test).

- **Security Review** - Verification that industry or internal security standards have been applied to system components or product. This is typically completed through gap analysis and utilises build / code reviews or by reviewing design documents and architecture diagrams. This activity does not utilise any of the earlier approaches (Vulnerability Assessment, Security Assessment, Penetration Test, Security Audit)

4.9.8 See also

- **National Information Assurance Glossary**

- White Paper: Security Testing for Financial Institutions

4.10 Attack patterns

In computer science, **attack patterns** are a group of rigorous methods for finding **bugs** or errors in code related to computer security.

Attack patterns are often used for testing purposes and are very important for ensuring that potential **vulnerabilities** are prevented. The attack patterns themselves can be used to highlight areas which need to be considered for security **hardening** in a **software application**. They also provide, either physically or in reference, the common solution pattern for preventing the attack. Such a practice can be termed *defensive coding patterns*.

Attack patterns define a series of repeatable steps that can be applied to simulate an attack against the security of a **system**.

4.10.1 Categories

There are several different ways to categorize attack patterns. One way is to group them into general categories, such as: Architectural, Physical, and External (see details below). Another way of categorizing attack patterns is to group them by a specific technology or type of technology (e.g. database attack patterns, web application attack patterns, network attack patterns, etc. or SQL Server attack patterns, Oracle Attack Patterns, .Net attack patterns, Java attack patterns, etc.)

Using General Categories

Architectural attack patterns are used to attack flaws in the architectural design of the system. These are things like weaknesses in **protocols**, **authentication** strategies, and system modularization. These are more logic-based attacks than actual bit-manipulation attacks.

Physical attack patterns are targeted at the code itself. These are things such as **SQL injection attacks**, **buffer overflows**, **race conditions**, and some of the more common forms of attacks that have become popular in the news.

External attack patterns include attacks such as **trojan horse** attacks, **viruses**, and **worms**. These are not generally solvable by software-design approaches, because they operate relatively independently from the attacked program. However, vulnerabilities in a piece of software can lead to these attacks being successful on a system running the vulnerable code. An example of this is the vulnerable edition of **Microsoft SQL Server**, which allowed the **Slammer worm** to propagate itself.^[1] The approach taken to these attacks is generally to revise the vulnerable code.

4.10.2 Structure

Attack Patterns are structured very much like **structure of Design patterns**. Using this format is helpful for standardizing the development of attack patterns and ensures that certain information about each pattern is always documented the same way.

A recommended structure for recording Attack Patterns is as follows:

- **Pattern Name**

The label given to the pattern which is commonly used to refer to the pattern in question.

- **Type & Subtypes**

The pattern type and its associated subtypes aid in classification of the pattern. This allows users to rapidly locate and identify pattern groups that they will have to deal with in their security efforts.

Each pattern will have a type, and zero or more subtypes that identify the category of the attack pattern. Typical types include **Injection Attack**, **Denial of Service Attack**, **Cryptanalysis Attack**, etc. Examples of typical subtypes for Denial Of Service for example would be: **DOS – Resource Starvation**, **DOS-System Crash**, **DOS-Policy Abuse**.

Another important use of this field is to ensure that true patterns are not repeated unnecessarily. Often it is easy to confuse a new exploit with a new attack. New exploits are created all the time for the same attack patterns. The Buffer Overflow Attack Pattern is a good example. There are many known exploits, and viruses that take advantage of a Buffer Overflow vulnerability. But they all follow the same pattern. Therefore the Type and Subtype classification mechanism provides a way to classify a pattern. If the pattern you are creating doesn't have a unique Type and Subtype, chances are it's a new exploit for an existing pattern.

This section is also used to indicate if it is possible to automate the attack. If it is possible to automate the attack, it is recommended to provide a sample in the Sample Attack Code section which is described below.

- **Also Known As**

Certain attacks may be known by several different names. This field is used to list those other names.

- **Description**

This is a description of the attack itself, and where it may have originated from. It is essentially a free-form field that can be used to record information that doesn't easily fit into the other fields.

- **Attacker Intent**

This field identifies the intended result of the attacker. This indicates the attacker's main target and goal for the attack itself. For example, The Attacker Intent of a DOS – Bandwidth Starvation attack is to make the target web site unreachable to legitimate traffic.

- **Motivation**

This field records the attacker's reason for attempting this attack. It may be to crash a system in order to cause financial harm to the organization, or it may be to execute the theft of critical data in order to create financial gain for the attacker.

This field is slightly different from the Attacker Intent field in that it describes why the attacker may want to achieve the Intent listed in the Attacker Intent field, rather than the physical result of the attack.

- **Exploitable Vulnerability**

This field indicates the specific or type of vulnerability that creates the attack opportunity in the first place. An example of this in an **Integer Overflow** attack would be that the integer based input field is not checking size of the value of the incoming data to ensure that the target variable is capable of managing the incoming value. This is the vulnerability that the associated exploit will take advantage of in order to carry out the attack.

- **Participants**

The Participants are one or more entities that are required for this attack to succeed. This includes the victim systems as well as the attacker and the attacker's tools or system components. The name of the entity should be accompanied by a brief description of their role in the attack and how they interact with each other.

- **Process Diagram**

These are one or more diagrams of the attack to visually explain how the attack is executed. This diagram can take whatever form is appropriate but it is recommended that the diagram be similar to a system or **class diagram** showing data flows and the components involved.

- **Dependencies and Conditions**

Every attack must have some context to operate in and the conditions that make the attack possible. This section describes what conditions are required and what other systems or situations need to be in place in order for the attack to succeed. For example, for the attacker to be able

to execute an Integer Overflow attack, they must have access to the vulnerable application. That will be common amongst most of the attacks. However if the vulnerability only exposes itself when the target is running on a remote RPC server, that would also be a condition that would be noted here.

- **Sample Attack Code**

If it is possible to demonstrate the exploit code, this section provides a location to store the demonstration code. In some cases, such as a Denial of Service attack, specific code may not be possible. However in Overflow, and **Cross Site Scripting** type attacks, sample code would be very useful.

- **Existing Exploits**

Exploits can be automated or manual. Automated exploits are often found as viruses, worms and hacking tools. If there are any existing exploits known for the attack this section should be used to list a reference to those exploits. These references can be internal such as corporate knowledge bases, or external such as the various CERT, and Virus databases.

Exploits are not to be confused with vulnerabilities. An Exploit is an automated or manual attack that utilises the vulnerability. It is not a listing of a vulnerability found in a particular product for example.

- **Follow-On Attacks**

Follow-on attacks are any other attacks that may be enabled by this particular attack pattern. For example, a Buffer Overflow attack pattern, is usually followed by Escalation of Privilege attacks, Subversion attacks or setting up for Trojan Horse / Backdoor attacks. This field can be particularly useful when researching an attack and identifying what other potential attacks may have been carried out or set up.

- **Mitigation Types**

The mitigation types are the basic types of mitigation strategies that would be used to prevent the attack pattern. This would commonly refer to Security Patterns and Defensive Coding Patterns. Mitigation Types can also be used as a means of classifying various attack patterns. By classifying Attack Patterns in this manner, libraries can be developed to implement particular mitigation types which can then be used to mitigate entire classes of Attack Patterns. This libraries can then be used and reused throughout various applications to ensure consistent and reliable coverage against particular types of attacks.

- **Recommended Mitigation**

Since this is an attack pattern, the recommended mitigation for the attack can be listed here in brief. Ideally this will point the user to a more thorough mitigation pattern for this class of attack.

• Related Patterns

This section will have a few subsections such as Related Patterns, Mitigation Patterns, Security Patterns, and Architectural Patterns. These are references to patterns that can support, relate to or mitigate the attack and the listing for the related pattern should note that.

An example of related patterns for an Integer Overflow Attack Pattern is:

Mitigation Patterns – Filtered Input Pattern, Self Defending Properties pattern

Related Patterns – Buffer Overflow Pattern

• Related Alerts, Listings and Publications

This section lists all the references to related alerts listings and publications such as listings in the **Common Vulnerabilities and Exposures** list, CERT, SANS, and any related vendor alerts. These listings should be hyperlinked to the online alerts and listings in order to ensure it references the most up to date information possible.

- CVE:
- CWE:
- CERT:

Various Vendor Notification Sites.

4.10.3 Further reading

- Alexander, Christopher; Ishikawa, Sara; & Silverstein, Murray. *A Pattern Language*. New York, NY: Oxford University Press, 1977
- Gamma, E.; Helm, R.; Johnson, R.; & Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software* ISBN 0-201-63361-2, Addison-Wesley, 1995
- Thompson, Herbert; Chase, Scott, *The Software Vulnerability Guide* ISBN 1-58450-358-0, Charles River Media, 2005
- Gegick, Michael & Williams, Laurie. “Matching Attack Patterns to Security Vulnerabilities in Software-Intensive System Designs.” ACM SIGSOFT Software Engineering Notes, Proceedings of the 2005 workshop on Software engineering for secure systems—building trustworthy applications SESS '05, Volume 30, Issue 4, ACM Press, 2005

- Howard, M.; & LeBlanc, D. *Writing Secure Code* ISBN 0-7356-1722-8, Microsoft Press, 2002.
- Moore, A. P.; Ellison, R. J.; & Linger, R. C. *Attack Modeling for Information Security and Survivability*, Software Engineering Institute, Carnegie Mellon University, 2001
- Hoglund, Greg & McGraw, Gary. *Exploiting Software: How to Break Code* ISBN 0-201-78695-8, Addison-Wesley, 2004
- McGraw, Gary. *Software Security: Building Security In* ISBN 0-321-35670-5, Addison-Wesley, 2006
- Viega, John & McGraw, Gary. *Building Secure Software: How to Avoid Security Problems the Right Way* ISBN 0-201-72152-X, Addison-Wesley, 2001
- Schumacher, Markus; Fernandez-Buglioni, Eduardo; Hybertson, Duane; Buschmann, Frank; Sommerlad, Peter *Security Patterns* ISBN 0-470-85884-2, John Wiley & Sons, 2006
- Koizol, Jack; Litchfield, D.; Aitel, D.; Anley, C.; Eren, S.; Mehta, N.; & Riley, H. *The Shell-coder's Handbook: Discovering and Exploiting Security Holes* ISBN 0-7645-4468-3, Wiley, 2004
- Schneier, Bruce. *Attack Trees: Modeling Security Threats* Dr. Dobb's Journal, December, 1999

4.10.4 References

- [1] PSS Security Response Team Alert - New Worm: W32.Slammer

- fuzzdb:

4.11 Pseudolocalization

Pseudolocalization (or **pseudo-localization**) is a software testing method used for testing internationalization aspects of software. Instead of translating the text of the software into a foreign language, as in the process of **localization**, the textual elements of an application are replaced with an altered version of the original language.

Example:

These specific alterations make the original words appear readable, but include the most problematic characteristics of the world's languages: varying length of text or characters, language direction, and so on.

4.11.1 Localization process

Traditionally, localization of software is independent of the software development process. In a typical scenario, software would be built and tested in one base language (such as English), with any *localizable* elements being extracted into external resources. Those resources are handed off to a localization team for translation into different target languages.^[2] The problem with this approach is that many subtle software bugs may be found during the process of localization, when it is too late (or more likely, too expensive) to fix them.^[2]

The types of problems that can arise during localization involve differences in how written text appears in different languages. These problems include:

- Translated text that is significantly longer than the source language, and does not fit within the UI constraints, or which causes text breaks at awkward positions.
- Font glyphs that are significantly larger than, or possess *diacritic* marks not found in, the source language, and which may be cut off vertically.
- Languages for which the reading order is not *left-to-right*, which is especially problematic for user input.
- Application code that assumes all characters fit into a limited character set, such as ASCII or ANSI, which can produce actual logic bugs if left uncaught.

In addition, the localization process may uncover places where an element should be localizable, but is hard coded in a source language. Similarly, there may be elements that were designed to be localized, but should not be (e.g. the element names in an **XML** or **HTML** document.)^[3]

Pseudolocalization is designed to catch these types of bugs during the development cycle, by mechanically replacing *all* localizable elements with a pseudo-language that is readable by native speakers of the source language, but which contains most of the troublesome elements of other languages and scripts. This is why pseudolocalization is to be considered an engineering or internationalization tool more than a localization one.

4.11.2 Pseudolocalization in Microsoft Windows

Pseudolocalization was introduced at Microsoft during the Windows Vista development cycle.^[4] The type of pseudo-language invented for this purpose is called a **pseudo locale** in Windows parlance. These locales were designed to use character sets and scripts characteristics from one of the three broad classes of foreign languages used by Windows at the time—basic (“Western”), mirrored (“Near-Eastern”), and CJK (“Far-Eastern”).^[2]

Prior to Vista, each of these three languages had their own separate builds of Windows, with potentially different code bases (and thus, different behaviors and bugs.) The pseudo locales created for each of these language families would produce text that still “reads” as English, but is made up of script from another languages. For example, the text string

Edit program settings

would be rendered in the “basic” pseudo-locale as

[!!! εΔιτ ΠrOḡRām sαTτuIḡ§ !!!]

This process produces translated strings that are longer, include non-ASCII characters, and (in the case of the “mirrored” pseudo-locale), are written right-to-left.^[4]

Note that the brackets on either side of the text in this example help to spot the following issues:

- text that is cut off (truncation)
- strings that are formed by combining (concatenation)
- strings that are not made localizable (hard-coding)

4.11.3 Pseudolocalization process at Microsoft

Michael Kaplan (a Microsoft program manager) explains the process of pseudo-localization similar to:

an eager and hardworking yet naive intern localizer, who is eager to prove himself [or herself] and who is going to translate every single string that you don't say shouldn't get translated.^[3]

One of the key features of the pseudolocalization process is that it happens automatically, during the development cycle, as part of a routine build. The process is almost identical to the process used to produce true localized builds, but is done *before* a build is tested, much earlier in the development cycle. This leaves time for any bugs that are found to be fixed in the base code, which is much easier than bugs not found until a release date is near.^[2]

The builds that are produced by the pseudolocalization process are tested using the same QA cycle as a non-localized build. Since the pseudo-locales are mimicking English text, they can be tested by an English speaker. Recently, beta version of Windows (7 and 8) have been released with some pseudo-localized strings intact.^{[5][6]} For these recent version of Windows, the pseudo-localized build is the primary staging build (the one created routinely for testing), and the final English language build is a “localized” version of that.^[3]

4.11.4 Pseudolocalization tools for other platforms

Besides the tools used internally by Microsoft, other internationalization tools now include pseudolocalization options. These tools include **Alchemy Catalyst** from Alchemy Software Development, and **SDL Passolo** from SDL. Such tools include pseudo-localization capability, including ability to view rendered Pseudo-localized dialog's and forms in the tools themselves. The process of creating a pseudolocalised build is fairly easy and can be done by running a custom made pseudolocalisation script on the extracted text resources.

There are a variety of free pseudolocalization resources on the Internet that will create pseudolocalized versions of common localization formats like iOS strings, Android xml, Gettext po, and others. These sites, like **Pseudolocalize.com** and **Babble-on**, allow developers to upload strings file to a Web site and download the resulting pseudolocalized file.

4.11.5 See also

- **Fuzz testing**

4.11.6 External links

- **Pseudolocalize.com** - Free online pseudolocalization tool

4.11.7 References

- [1] Benjamin Zadik (12 April 2013). "Pseudolocalization: Prepare your app for localization". Retrieved 13 April 2013.
- [2] Raymond Chen (26 July 2012). "A brief and also incomplete history of Windows localization". Retrieved 26 July 2012.
- [3] Michael Kaplan (11 April 2011). "One of my colleagues is the "Pseudo Man"". Retrieved 26 July 2012.
- [4] Shawn Steele (27 June 2006). "Pseudo Locales in Windows Vista Beta 2". Retrieved 26 July 2012.
- [5] Steven Sinofsky (7 July 2009). "Engineering Windows 7 for a Global Market". Retrieved 26 July 2012.
- [6] Kriti Jindal (16 March 2012). "Install PowerShell Web Access on non-English machines". Retrieved 26 July 2012.

4.12 Recovery testing

In **software testing**, **recovery testing** is the activity of testing how well an application is able to recover from crashes, hardware failures and other similar problems.

Recovery testing is the forced failure of the software in a variety of ways to verify that recovery is properly performed. Recovery testing should not be confused with **reliability testing**, which tries to discover the specific point at which failure occurs. Recovery testing is basically done in order to check how fast and better the application can recover against any type of crash or hardware failure etc. Type or extent of recovery is specified in the requirement specifications. It is basically testing how well a system recovers from crashes, hardware failures, or other catastrophic problems

Examples of recovery testing:

1. While an application is running, suddenly restart the computer, and afterwards check the validness of the application's data integrity.
2. While an application is receiving data from a **network**, unplug the connecting cable. After some time, plug the cable back in and analyze the application's ability to continue receiving data from the point at which the network connection disappeared.
3. Restart the system while a **browser** has a definite number of sessions. Afterwards, check that the **browser** is able to recover all of them.

4.12.1 See also

- **Fault injection**
- **Failsafe**

4.13 Soak testing

Soak testing involves testing a system with a typical production load, over a continuous availability period, to validate system behavior under production use.

It may be required to extrapolate the results, if not possible to conduct such as extended test. For example if the system is required to process 10,000 transactions over 100 hours, it may be possible to complete processing the same 10,000 transactions in a shorter duration (say 50 hours) as representative (and conservative estimate) of the actual production use. A good soak test would also include the ability to simulate peak loads as opposed to just average loads. If manipulating the load over specific periods of time is not possible, alternatively (and conservatively) allow the system to run at peak production loads for the duration of the test.

For example, in **software testing**, a system may behave exactly as expected when tested for one hour. However, when it is tested for three hours, problems such as **memory leaks** cause the system to fail or behave unexpectedly.

Soak tests are used primarily to check the reaction of a subject under test under a possible simulated environment for a given duration and for a given threshold. Observations made during the soak test are used to improve the characteristics of the subject under further tests.

In electronics, soak testing may involve testing a system up to or above its maximum ratings for a long period of time. Some companies may soak test a product for a period of many months, while also applying external stresses such as elevated temperatures.

This falls under [load testing](#).

4.13.1 See also

4.14 Characterization test

In [computer programming](#), a **characterization test** is a means to describe (characterize) the **actual** behavior of an existing piece of software, and therefore protect existing behavior of [legacy code](#) against unintended changes via [automated testing](#). This term was coined by Michael Feathers. ^[1]

The goal of characterization tests is to help developers verify that the modifications made to a reference version of a software system did not modify its behavior in unwanted or undesirable ways. They enable, and provide a safety net for, extending and [refactoring](#) code that does not have adequate [unit tests](#).

When creating a characterization test, one must observe what outputs occur for a given set of inputs. Given an observation that the legacy code gives a certain output based on given inputs, then a test can be written that asserts that the output of the legacy code matches the observed result for the given inputs. For example, if one observes that $f(3.14) == 42$, then this could be created as a characterization test. Then, after modifications to the system, the test can determine if the modifications caused changes in the results when given the same inputs.

Unfortunately, as with any testing, it is generally not possible to create a characterization test for every possible input and output. As such, many people opt for either statement or branch coverage. However, even this can be difficult. Test writers must use their judgment to decide how much testing is appropriate. It is often sufficient to write characterization tests that only cover the specific inputs and outputs that are known to occur, paying special attention to edge cases.

Unlike [regression tests](#), to which they are very similar, characterization tests do not verify the *correct* behavior of the code, which can be impossible to determine. Instead they verify the behavior that was observed when they were written. Often no specification or test suite is available, leaving only characterization tests as an option, since the conservative path is to assume that the old behavior is the

required behavior. Characterization tests are, essentially, change detectors. It is up to the person analyzing the results to determine if the detected change was expected and/or desirable, or unexpected and/or undesirable.

One of the interesting aspects of characterization tests is that, since they are based on existing code, it's possible to generate some characterization tests automatically. An automated characterization test tool will exercise existing code with a wide range of relevant and/or random input values, record the output values (or state changes) and generate a set of characterization tests. When the generated tests are executed against a new version of the code, they will produce one or more failures/warnings if that version of the code has been modified in a way that changes a previously established behavior.

4.14.1 References

- [1] Feathers, Michael C. *Working Effectively with Legacy Code* (ISBN 0-13-117705-2).

4.14.2 External links

- [Characterization Tests](#)
- [Working Effectively With Characterization Tests](#) first in a blog-based series of tutorials on characterization tests.
- [Change Code Without Fear](#) DDJ article on characterization tests.

Chapter 5

Unit testing

5.1 Unit testing

In computer programming, **unit testing** is a software testing method by which individual units of source code, sets of one or more computer program modules together with associated control data, usage procedures, and operating procedures, are tested to determine whether they are fit for use.^[1] Intuitively, one can view a unit as the smallest testable part of an application. In **procedural programming**, a unit could be an entire module, but it is more commonly an individual function or procedure. In **object-oriented programming**, a unit is often an entire interface, such as a class, but could be an individual method.^[2] Unit tests are short code fragments^[3] created by programmers or occasionally by **white box testers** during the development process. It forms the basis for **component testing**.^[4]

Ideally, each **test case** is independent from the others. Substitutes such as **method stubs**, **mock objects**,^[5] **fakes**, and **test harnesses** can be used to assist testing a module in isolation. Unit tests are typically written and run by **software developers** to ensure that code meets its design and behaves as intended.

5.1.1 Benefits

The goal of unit testing is to isolate each part of the program and show that the individual parts are correct.^[1] A unit test provides a strict, written **contract** that the piece of code must satisfy. As a result, it affords several benefits.

Finds problems early

Unit testing finds problems early in the **development cycle**. This includes both bugs in the programmer's implementation and flaws or missing parts of the specification for the unit. The process of writing a thorough set of tests forces the author to think through inputs, outputs, and error conditions, and thus more crisply define the unit's desired behavior. The cost of finding a bug before coding begins or when the code is first written is considerably lower than the cost of detecting, identifying, and correct-

ing the bug later; bugs may also cause problems for the end-users of the software. Some argue that code that is impossible or difficult to test is poorly written, thus unit testing can force developers to structure functions and objects in better ways.

In **test-driven development** (TDD), which is frequently used in both **extreme programming** and **scrum**, unit tests are created before the code itself is written. When the tests pass, that code is considered complete. The same unit tests are run against that function frequently as the larger code base is developed either as the code is changed or via an automated process with the build. If the unit tests fail, it is considered to be a bug either in the changed code or the tests themselves. The unit tests then allow the location of the fault or failure to be easily traced. Since the unit tests alert the development team of the problem before handing the code off to testers or clients, it is still early in the development process.

Facilitates change

Unit testing allows the programmer to **refactor** code or upgrade system libraries at a later date, and make sure the module still works correctly (e.g., in **regression testing**). The procedure is to write test cases for all **functions** and **methods** so that whenever a change causes a fault, it can be quickly identified. Unit tests detect changes which may break a **design contract**.

Simplifies integration

Unit testing may reduce uncertainty in the units themselves and can be used in a **bottom-up** testing style approach. By testing the parts of a program first and then testing the sum of its parts, **integration testing** becomes much easier.

Documentation

Unit testing provides a sort of living documentation of the system. Developers looking to learn what functionality is provided by a unit, and how to use it, can look at the unit

tests to gain a basic understanding of the unit's interface (API).

Unit **test cases** embody characteristics that are critical to the success of the unit. These characteristics can indicate appropriate/inappropriate use of a unit as well as negative behaviors that are to be trapped by the unit. A unit test case, in and of itself, documents these critical characteristics, although many software development environments do not rely solely upon code to document the product in development.

Design

When software is developed using a test-driven approach, the combination of writing the unit test to specify the interface plus the refactoring activities performed after the test is passing, may take the place of formal design. Each unit test can be seen as a design element specifying classes, methods, and observable behaviour. The following **Java** example will help illustrate this point.

Here is a set of test cases that specify a number of elements of the implementation. First, that there must be an interface called **Adder**, and an implementing class with a zero-argument constructor called **AdderImpl**. It goes on to **assert** that the **Adder** interface should have a method called **add**, with two integer parameters, which returns another integer. It also specifies the behaviour of this method for a small range of values over a number of test methods.

```
public class TestAdder { // can it add the positive
    numbers 1 and 1? public void testSumPositiveNum-
    bersOneAndOne() { Adder adder = new AdderImpl();
    assert(adder.add(1, 1) == 2); } // can it add the positive
    numbers 1 and 2? public void testSumPositiveNum-
    bersOneAndTwo() { Adder adder = new AdderImpl();
    assert(adder.add(1, 2) == 3); } // can it add the positive
    numbers 2 and 2? public void testSumPositiveNum-
    bersTwoAndTwo() { Adder adder = new AdderImpl();
    assert(adder.add(2, 2) == 4); } // is zero neutral?
    public void testSumZeroNeutral() { Adder adder =
    new AdderImpl(); assert(adder.add(0, 0) == 0); } //
    can it add the negative numbers -1 and -2? public
    void testSumNegativeNumbers() { Adder adder = new
    AdderImpl(); assert(adder.add(-1, -2) == -3); } // can
    it add a positive and a negative? public void testSumPos-
    itiveAndNegative() { Adder adder = new AdderImpl();
    assert(adder.add(-1, 1) == 0); } // how about larger
    numbers? public void testSumLargeNumbers() { Adder
    adder = new AdderImpl(); assert(adder.add(1234, 988)
    == 2222); } }
```

In this case the unit tests, having been written first, act as a design document specifying the form and behaviour of a desired solution, but not the implementation details, which are left for the programmer. Following the “do the simplest thing that could possibly work” practice, the eas-

iest solution that will make the test pass is shown below.

```
interface Adder { int add(int a, int b); } class AdderImpl
implements Adder { int add(int a, int b) { return a + b; } }
```

Unlike other diagram-based design methods, using unit-tests as a design specification has one significant advantage. The design document (the unit-tests themselves) can be used to verify that the implementation adheres to the design. With the unit-test design method, the tests will never pass if the developer does not implement the solution according to the design.

It is true that unit testing lacks some of the accessibility of a diagram, but **UML** diagrams are now easily generated for most modern languages by free tools (usually available as extensions to **IDEs**). Free tools, like those based on the **xUnit** framework, outsource to another system the graphical rendering of a view for human consumption.

5.1.2 Separation of interface from implementation

Because some classes may have **references** to other classes, testing a class can frequently spill over into testing another class. A common example of this is classes that depend on a **database**: in order to test the class, the tester often writes code that interacts with the database. This is a mistake, because a unit test should usually not go outside of its own class boundary, and especially should not cross such process/network boundaries because this can introduce unacceptable performance problems to the unit test-suite. Crossing such unit boundaries turns unit tests into integration tests, and when test cases fail, makes it less clear which component is causing the failure. See also **Fakes, mocks and integration tests**.

Instead, the software developer should create an abstract interface around the database queries, and then implement that interface with their own **mock object**. By abstracting this necessary attachment from the code (temporarily reducing the net effective coupling), the independent unit can be more thoroughly tested than may have been previously achieved. This results in a higher quality unit that is also more maintainable.

5.1.3 Parameterized unit testing

Parameterized unit tests (PUTs) are tests that take parameters. Unlike traditional unit tests, which are usually closed methods, PUTs take any set of parameters. PUTs have been supported by **TestNG**, **JUnit** and various .NET test frameworks. Suitable parameters for the unit tests may be supplied manually or in some cases are automatically generated by the test framework. Testing tools like **QuickCheck** exist to generate test inputs for PUTs.

5.1.4 Unit testing limitations

Testing will not catch every error in the program, since it cannot evaluate every execution path in any but the most trivial programs. The same is true for unit testing. Additionally, unit testing by definition only tests the functionality of the units themselves. Therefore, it will not catch integration errors or broader system-level errors (such as functions performed across multiple units, or non-functional test areas such as **performance**). Unit testing should be done in conjunction with other **software testing** activities, as they can only show the presence or absence of particular errors; they cannot prove a complete absence of errors. In order to guarantee correct behavior for every execution path and every possible input, and ensure the absence of errors, other techniques are required, namely the application of **formal methods** to proving that a software component has no unexpected behavior.

An elaborate hierarchy of unit tests does not equal integration testing. Integration with peripheral units should be included in integration tests, but not in unit tests. Integration testing typically still relies heavily on humans **testing manually**; high-level or global-scope testing can be difficult to automate, such that manual testing often appears faster and cheaper.

Software testing is a combinatorial problem. For example, every boolean decision statement requires at least two tests: one with an outcome of “true” and one with an outcome of “false”. As a result, for every line of code written, programmers often need 3 to 5 lines of test code.^[6] This obviously takes time and its investment may not be worth the effort. There are also many problems that cannot easily be tested at all – for example those that are **nondeterministic** or involve multiple **threads**. In addition, code for a unit test is likely to be at least as buggy as the code it is testing. Fred Brooks in *The Mythical Man-Month* quotes: “Never go to sea with two chronometers; take one or three.”^[7] Meaning, if two **chronometers** contradict, how do you know which one is correct?

Another challenge related to writing the unit tests is the difficulty of setting up realistic and useful tests. It is necessary to create relevant initial conditions so the part of the application being tested behaves like part of the complete system. If these initial conditions are not set correctly, the test will not be exercising the code in a realistic context, which diminishes the value and accuracy of unit test results.^[8]

To obtain the intended benefits from unit testing, rigorous discipline is needed throughout the software development process. It is essential to keep careful records not only of the tests that have been performed, but also of all changes that have been made to the source code of this or any other unit in the software. Use of a **version control** system is essential. If a later version of the unit fails a particular test that it had previously passed, the version-control software can provide a list of the source

code changes (if any) that have been applied to the unit since that time.

It is also essential to implement a sustainable process for ensuring that test case failures are reviewed daily and addressed immediately.^[9] If such a process is not implemented and ingrained into the team’s workflow, the application will evolve out of sync with the unit test suite, increasing false positives and reducing the effectiveness of the test suite.

Unit testing embedded system software presents a unique challenge: Since the software is being developed on a different platform than the one it will eventually run on, you cannot readily run a test program in the actual deployment environment, as is possible with desktop programs.^[10]

5.1.5 Applications

Extreme programming

Unit testing is the cornerstone of **extreme programming**, which relies on an automated **unit testing framework**. This automated unit testing framework can be either third party, e.g., **xUnit**, or created within the development group.

Extreme programming uses the creation of unit tests for **test-driven development**. The developer writes a unit test that exposes either a software requirement or a defect. This test will fail because either the requirement isn’t implemented yet, or because it intentionally exposes a defect in the existing code. Then, the developer writes the simplest code to make the test, along with other tests, pass.

Most code in a system is unit tested, but not necessarily all paths through the code. Extreme programming mandates a “test everything that can possibly break” strategy, over the traditional “test every execution path” method. This leads developers to develop fewer tests than classical methods, but this isn’t really a problem, more a restatement of fact, as classical methods have rarely ever been followed methodically enough for all execution paths to have been thoroughly tested. Extreme programming simply recognizes that testing is rarely exhaustive (because it is often too expensive and time-consuming to be economically viable) and provides guidance on how to effectively focus limited resources.

Crucially, the test code is considered a first class project artifact in that it is maintained at the same quality as the implementation code, with all duplication removed. Developers release unit testing code to the code repository in conjunction with the code it tests. Extreme programming’s thorough unit testing allows the benefits mentioned above, such as simpler and more confident code development and **refactoring**, simplified code integration, accurate documentation, and more modular designs. These unit tests are also constantly run as a form of **regression test**.

Unit testing is also critical to the concept of **emergent design**. As emergent design is heavily dependent upon refactoring, unit tests are an integral component.^[11]

Techniques

Unit testing is commonly **automated**, but may still be performed manually. The **IEEE** does not favor one over the other.^[12] The objective in unit testing is to isolate a unit and validate its correctness. A manual approach to unit testing may employ a step-by-step instructional document. However, automation is efficient for achieving this, and enables the many benefits listed in this article. Conversely, if not planned carefully, a careless manual unit test case may execute as an integration test case that involves many software components, and thus preclude the achievement of most if not all of the goals established for unit testing.

To fully realize the effect of isolation while using an automated approach, the unit or code body under test is executed within a **framework** outside of its natural environment. In other words, it is executed outside of the product or calling context for which it was originally created. Testing in such an isolated manner reveals unnecessary dependencies between the code being tested and other units or data spaces in the product. These dependencies can then be eliminated.

Using an automation framework, the developer codes criteria, or an **oracle** or result that is known to be good, into the test to verify the unit's correctness. During test case execution, the framework **logs** tests that fail any criterion. Many frameworks will also automatically flag these failed test cases and report them in a summary. Depending upon the severity of a failure, the framework may halt subsequent testing.

As a consequence, unit testing is traditionally a motivator for programmers to create **decoupled** and **cohesive** code bodies. This practice promotes healthy habits in software development. **Design patterns**, unit testing, and refactoring often work together so that the best solution may emerge.

Unit testing frameworks

See also: [List of unit testing frameworks](#)

Unit testing frameworks are most often third-party products that are not distributed as part of the compiler suite. They help simplify the process of unit testing, having been developed for a **wide variety of languages**. Examples of testing frameworks include open source solutions such as the various code-driven testing frameworks known collectively as **xUnit**, and proprietary/commercial solutions such as **Typemock Isolator.NET/Isolator++**, **TBrun**, **JustMock**, **Parasoft Development Testing (Jtest**,

Parasoft C/C++test, **dotTEST)**, **Testwell CTA++** and **VectorCAST/C++**.

It is generally possible to perform unit testing without the support of a specific framework by writing client code that exercises the units under test and uses **assertions**, **exception handling**, or other **control flow** mechanisms to signal failure. Unit testing without a framework is valuable in that there is a **barrier to entry** for the adoption of unit testing; having scant unit tests is hardly better than having none at all, whereas once a framework is in place, adding unit tests becomes relatively easy.^[13] In some frameworks many advanced unit test features are missing or must be hand-coded.

Language-level unit testing support

Some programming languages directly support unit testing. Their grammar allows the direct declaration of unit tests without importing a library (whether third party or standard). Additionally, the boolean conditions of the unit tests can be expressed in the same syntax as boolean expressions used in non-unit test code, such as what is used for if and while statements.

Languages that support unit testing include:

- **ABAP**
- **C#**
- **Clojure**^[14]
- **D**
- **Go**^[15]
- **Java**
- **Obix**
- **Python**^[16]
- **Racket**^[17]
- **Ruby**^[18]
- **Rust**^[19]
- **Scala**
- **Objective-C**
- **Visual Basic .NET**
- **PHP**
- **tcl**

5.1.6 See also

- Acceptance testing
- Characterization test
- Component-based usability testing
- Design predicates
- Design by contract
- Extreme programming
- Integration testing
- List of unit testing frameworks
- Regression testing
- Software archaeology
- Software testing
- Test case
- Test-driven development
- xUnit – a family of unit testing frameworks.

5.1.7 Notes

- [1] Kolawa, Adam; Huizinga, Dorota (2007). *Automated Defect Prevention: Best Practices in Software Management*. Wiley-IEEE Computer Society Press. p. 426. ISBN 0-470-04212-5.
- [2] Xie, Tao. "Towards a Framework for Differential Unit Testing of Object-Oriented Programs" (PDF). Retrieved 2012-07-23.
- [3] "Unit Testing". Retrieved 2014-01-06.
- [4] "ISTQN Exam Certification". *ISTQB Exam Certification*. Retrieved 12 March 2015.
- [5] Fowler, Martin (2007-01-02). "Mocks aren't Stubs". Retrieved 2008-04-01.
- [6] Cramblitt, Bob (2007-09-20). "Alberto Savoia sings the praises of software testing". Retrieved 2007-11-29.
- [7] Brooks, Frederick J. (1995) [1975]. *The Mythical Man-Month*. Addison-Wesley. p. 64. ISBN 0-201-83595-9.
- [8] Kolawa, Adam (2009-07-01). "Unit Testing Best Practices". Retrieved 2012-07-23.
- [9] daVeiga, Nada (2008-02-06). "Change Code Without Fear: Utilize a regression safety net". Retrieved 2008-02-08.
- [10] Kucharski, Marek (2011-11-23). "Making Unit Testing Practical for Embedded Development". Retrieved 2012-05-08.
- [11] "Agile Emergent Design". Agile Sherpa. 2010-08-03. Retrieved 2012-05-08.

- [12] IEEE Standards Board, "IEEE Standard for Software Unit Testing: An American National Standard, ANSI/IEEE Std 1008-1987" in *IEEE Standards: Software Engineering, Volume Two: Process Standards; 1999 Edition; published by The Institute of Electrical and Electronics Engineers, Inc.* Software Engineering Technical Committee of the IEEE Computer Society.
- [13] Bullseye Testing Technology (2006–2008). "Intermediate Coverage Goals". Retrieved 24 March 2009.
- [14] Sierra, Stuart. "API for clojure.test - Clojure v1.6 (stable)". Retrieved 11 February 2015.
- [15] golang.org. "testing - The Go Programming Language". Retrieved 3 December 2013.
- [16] Python Documentation (1999–2012). "unittest -- Unit testing framework". Retrieved 15 November 2012.
- [17] Welsh, Noel; Culpepper, Ryan. "RackUnit: Unit Testing". PLT Design Inc. Retrieved 11 February 2015.
- [18] Ruby-Doc.org. "Module: Test::Unit::Assertions (Ruby 2.0)". Retrieved 19 August 2013.
- [19] The Rust Project Developers (2011–2014). "The Rust Testing Guide (Rust 0.12.0-pre-nightly)". Retrieved 12 August 2014.

5.1.8 External links

- Test Driven Development (Ward Cunningham's Wiki)

5.2 Self-testing code

Self-testing code is software which incorporates built-in tests (see **test-first development**).

In Java, to execute a unit test from the command line, a class can have methods like the following.

```
// Executing <code>main</code> runs the unit test.
public static void main(String[] args) { test(); }
static void test() { assert foo == bar; }
```

To invoke a full **system test**, a class can incorporate a method call.

```
public static void main(String[] args) { test();
TestSuite.test(); // invokes full system test }
```

5.2.1 See also

- Software development
- Extreme programming

5.2.2 Further reading

Self-testing code explained by Martin Fowler

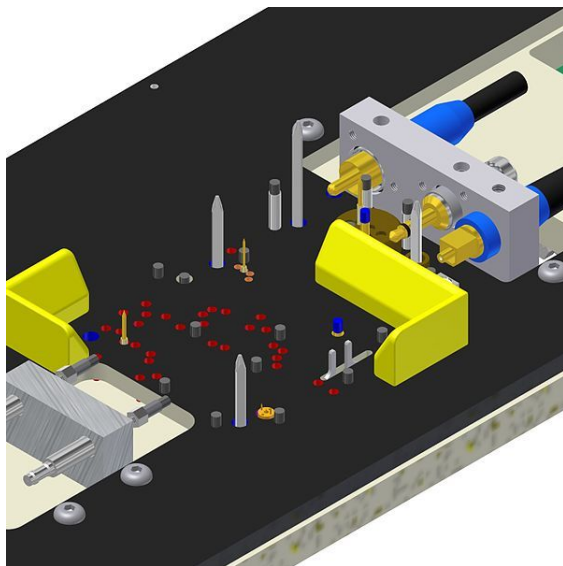
5.3 Test fixture

This article is about testing fixtures. For other uses, see [Fixture \(disambiguation\)](#).

A **test fixture** is something used to consistently test some item, device, or piece of software.

5.3.1 Electronics

In testing electronic equipment such as circuit boards, electronic components, and chips, a **test fixture** is a device or setup designed to hold the **device under test** in place and allow it to be tested by being subjected to controlled electronic test signals.



Side connectors, centering pins, test needles, pre-centering parts.

Examples are a **bed of nails tester** or **SmartFixture**.

5.3.2 Software

In **software testing**, a **test fixture** is a fixed state of the software under test used as a baseline for running tests; also known as the **test context**. It may also refer to the actions performed in order to bring the system into such a state.

Examples of fixtures:

- Loading a database with a specific, known set of data
- Erasing a hard disk and installing a known clean operating system installation



A Functional Test Fixture is a complex device to interface the DUT to the ATE

- Copying a specific known set of files
- Preparation of input data and set-up/creation of **fake** or **mock** objects

Software used to systematically run reproducible tests on a piece of software under test is known as a **test harness**; part of its job is to set up suitable test fixtures.

Test fixture in xUnit

In generic **xUnit**, a **test fixture** is all the things that must be in place in order to run a test and expect a particular outcome.

Frequently fixtures are created by handling *setUp()* and *tearDown()* events of the **unit testing framework**. In *setUp()* one would create the expected state for the test, and in *tearDown()* it would clean up what had been set up.

Four phases of a test:

1. **Set up** -- Setting up the *test fixture*.
2. **Exercise** -- Interact with the *system under test*.
3. **Verify** -- Determine whether the expected outcome has been obtained.
4. **Tear down** -- Tear down the *test fixture* to return to the original state.

Use of fixtures

Some advantages of *fixtures* include separation of the test initialization (and destruction) from the testing, reusing a known state for more than one test, and special assumption by the testing framework that the fixture set up works.

5.3.3 Physical testing

In **physical testing**, a fixture is a device or apparatus to hold or support the test **specimen** during the test. The influence of test fixtures on test results is important and is an ongoing subject of research.^[1]

Many **test methods** detail the requirements of test fixtures in the text of the document.^{[2][3]}

- Test fixture on **universal testing machine** for three point flex test
- Hydraulic system testing on fixture
- jet engine fixtures for operational testing

Some fixtures employ clamps, wedge grips and pincer grips.

- pincer clamps max. 50 kN spring-biased
- offset compensated wedge grip max.50 kN
- different vice and screw grips of a German manufacturer

Further types of construction are eccentric roller fixtures, thread grips and button head grips as well as rope grips.

- symmetric roller grip, self-closing and self-adjusting
- multiple button head grip for speedy tests on series
- small rope grip 200N to test fine wires
- very compact wedge grip for temperature chambers providing extreme temperatures

Mechanical holding apparatus provide the clamping force via arms, wedges or eccentric wheel to the jaws. Additionally there are pneumatic and hydraulic fixtures for tensile testing that do allow very fast clamping procedures and very high clamping forces

- pneumatic grip, symmetrical, clamping force 2.4 kN
- heavy duty hydraulic clamps, clamping force 700 kN
- Bending device for tensile testing machines
- Equipment to test peeling forces up to 10 kN

5.3.4 See also

- **Unit testing**

5.3.5 References

- [1] Abadallah, MG; Gascoigne, HE (1989). *The Influence of Test Fixture Design on the Shear Test for Fiber Composite Materials*. ASTM STP.
 - [2] **ASTM B829** Test for Determining the Formability of copper Strip
 - [3] **ASTM D6641** Compressive Properties of Polymer Matrix Using a Combined Loading Compression Test Fixture
- **Unit Testing with JUnit**, by Yoonsik Cheon.
 - **The Low-Down on fixtures**, from A Guide to Testing Rails Applications.

5.4 Method stub

A **method stub** or simply **stub** in **software development** is a piece of code used to stand in for some other programming functionality. A stub may **simulate** the behavior of existing code (such as a **procedure** on a remote machine) or be a temporary substitute for yet-to-be-developed code. Stubs are therefore most useful in **porting**, **distributed computing** as well as general software development and **testing**.

An example of a stub in **pseudocode** might be as follows:

```
BEGIN Temperature = ThermometerRead(Outside)
IF Temperature > 40 THEN PRINT "It's HOT!"
END IF
BEGIN ThermometerRead(Source insideOrOutside)
RETURN 28
END ThermometerRead
```

The above pseudocode utilises the function **ThermometerRead**, which returns a temperature. While **ThermometerRead** would be intended to read some hardware device, this function currently does not contain the necessary code. So **ThermometerRead** does not, in essence, **simulate** any process, yet it *does* return a legal value, allowing the main program to be at least partially tested. Also note that although it accepts the **parameter** of type **Source**, which determines whether inside or outside temperature is needed, it does not use the actual value passed (**argument** **insideOrOutside**) by the caller in its logic.

A stub ^[1] is a routine that doesn't actually do anything other than declaring itself and the parameters it accepts and returning something that is usually the values expected in one of the "happy scenarios" for the caller. Stubs are used commonly as placeholders for implementation of a known interface, where the **interface** is finalized/known but the implementation is not yet known/finalized. The stub contains just enough code to allow it to be compiled and linked with the rest of the program. In **RMI** nomenclature, a stub communicates on the server-side with a **skeleton**.^[2]

5.4.1 See also

- Abstract method
- Mock object
- Dummy code
- Test stub

5.4.2 References

- [1] “stub”. WEBOPEDIA. Retrieved 2012-08-28.
- [2] Freeman, Eric; Freeman, Elisabeth; Kathy, Sierra; Bert, Bates (2004). Hendrickson, Mike; Loukides, Mike, eds. “Head First Design Patterns” (paperback) 1. O'REILLY. p. 440. ISBN 978-0-596-00712-6. Retrieved 2012-08-28.

5.4.3 External links

- A Stub Generation System For C++ (PDF)
- Stub/mock frameworks for Java Review and comparison of stub & mock frameworks for Java

5.5 Mock object

In object-oriented programming, **mock objects** are simulated objects that mimic the behavior of real objects in controlled ways. A programmer typically creates a mock object to test the behavior of some other object, in much the same way that a car designer uses a **crash test dummy** to simulate the dynamic behavior of a human in vehicle impacts.

5.5.1 Reasons for use

In a **unit test**, mock objects can simulate the behavior of complex, real objects and are therefore useful when a real object is impractical or impossible to incorporate into a unit test. If an actual object has any of the following characteristics, it may be useful to use a mock object in its place:

- the object supplies **non-deterministic** results (e.g., the current time or the current temperature);
- it has states that are difficult to create or reproduce (e.g., a network error);
- it is slow (e.g., a complete **database**, which would have to be initialized before the test);
- it does not yet exist or may change behavior;

- it would have to include information and methods exclusively for testing purposes (and not for its actual task).

For example, an alarm clock program which causes a bell to ring at a certain time might get the current time from the outside world. To test this, the test must wait until the alarm time to know whether it has rung the bell correctly. If a mock object is used in place of the real object, it can be programmed to provide the bell-ringing time (whether it is actually that time or not) so that the alarm clock program can be tested in isolation.

5.5.2 Technical details

Mock objects have the same **interface** as the real objects they mimic, allowing a client object to remain unaware of whether it is using a real object or a mock object. Many available mock object frameworks allow the programmer to specify which, and in what order, **methods** will be invoked on a mock object and what **parameters** will be passed to them, as well as what values will be returned. Thus, the behavior of a complex object such as a network socket can be mimicked by a mock object, allowing the programmer to discover whether the object being tested responds appropriately to the wide variety of states such mock objects may be in.

Mocks, Fakes and Stubs

Classification between mocks, fakes, and stubs is highly inconsistent across literature.^{[1][2][3][4][5][6]} Consistent among the literature, though, is that they all represent a production object in a testing environment by exposing the same interface.

Which of the mock, fake, or stub is the simplest is inconsistent, but the simplest always returns pre-arranged responses (as in a **method stub**). On the other side of the spectrum, the most complex object will fully simulate a production object with complete logic, exceptions, etc. Whether or not any of the mock, fake, or stub trio fits such a definition is, again, inconsistent across the literature.

For example, a mock, fake, or stub method implementation between the two ends of the complexity spectrum might contain **assertions** to examine the context of each call. For example, a mock object might assert the order in which its methods are called, or assert consistency of data across method calls.

In the book “The Art of Unit Testing”^[7] mocks are described as a fake object that helps decide whether a test failed or passed by verifying whether an interaction with an object occurred. Everything else is defined as a stub. In that book, “Fakes” are anything that is not real. Based on their usage, they are either stubs or mocks.

Setting expectations

Consider an example where an authorization sub-system has been mocked. The mock object implements an `isUserAllowed(task : Task) : boolean`^[8] method to match that in the real authorization class. Many advantages follow if it also exposes an `isAllowed : boolean` property, which is not present in the real class. This allows test code easily to set the expectation that a user will, or will not, be granted permission in the next call and therefore readily to test the behavior of the rest of the system in either case.

Similarly, a mock-only setting could ensure that subsequent calls to the sub-system will cause it to **throw an exception**, or **hang** without responding, or return **null** etc. Thus it is possible to develop and test **client** behaviors for all realistic fault conditions in **back-end** sub-systems as well as for their expected responses. Without such a simple and flexible mock system, testing each of these situations may be too laborious for them to be given proper consideration.

Writing log strings

A mock database object's `save(person : Person)` method may not contain much (if any) implementation code. It might or might not check the existence and perhaps the **validity** of the `Person` object passed in for saving (see fake vs. mock discussion above), but beyond that there might be no other implementation.

This is a missed opportunity. The mock method could add an entry to a public log string. The entry need be no more than "Person saved",^{[9]:146-7} or it may include some details from the person object instance, such as a name or ID. If the test code also checks the final contents of the log string after various series of operations involving the mock database then it is possible to verify that in each case exactly the expected number of database saves have been performed. This can find otherwise invisible performance-sapping bugs, for example, where a developer, nervous of losing data, has coded repeated calls to `save()` where just one would have sufficed.

5.5.3 Use in test-driven development

Programmers working with the **test-driven development** (TDD) method make use of mock objects when writing software. Mock objects meet the **interface** requirements of, and stand in for, more complex real ones; thus they allow programmers to write and **unit-test** functionality in one area without actually calling complex underlying or collaborating **classes**.^{[9]:144-5} Using mock objects allows developers to focus their tests on the behavior of the system under test (SUT) without worrying about its dependencies. For example, testing a complex algorithm based on multiple objects being in particular states can

be clearly expressed using mock objects in place of real objects.

Apart from complexity issues and the benefits gained from this **separation of concerns**, there are practical speed issues involved. Developing a realistic piece of software using TDD may easily involve several hundred unit tests. If many of these induce communication with databases, web services and other **out-of-process** or **networked** systems, then the suite of unit tests will quickly become too slow to be run regularly. This in turn leads to bad habits and a reluctance by the developer to maintain the basic tenets of TDD.

When mock objects are replaced by real ones then the end-to-end functionality will need further testing. These will be **integration tests** rather than unit tests.

5.5.4 Limitations

The use of mock objects can closely couple the **unit tests** to the actual implementation of the code that is being tested. For example, many mock object frameworks allow the developer to check the order of and number of times that mock object methods were invoked by the real object being tested; subsequent **refactoring** of the code that is being tested could therefore cause the test to fail even though all mocked object methods still obey the contract of the previous implementation. This illustrates that unit tests should test a method's external behavior rather than its internal implementation. Over-use of mock objects as part of a suite of unit tests can result in a dramatic increase in the amount of maintenance that needs to be performed on the tests themselves during system evolution as refactoring takes place. The improper maintenance of such tests during evolution could allow bugs to be missed that would otherwise be caught by unit tests that use instances of real classes. Conversely, simply mocking one method might require far less configuration than setting up an entire real class and therefore reduce maintenance needs.

Mock objects have to accurately model the behavior of the object they are mocking, which can be difficult to achieve if the object being mocked comes from another developer or project or if it has not even been written yet. If the behavior is not modeled correctly then the unit tests may register a pass even though a failure would occur at run time under the same conditions that the unit test is exercising, thus rendering the unit test inaccurate.^[10]

5.5.5 See also

- **Abstract method**
- **Dummy code**
- **Hamcrest**
- **Method stub**

- Test double

5.5.6 References

- [1] <https://msdn.microsoft.com/en-us/library/ff798400.aspx>
- [2] <http://hamletdarcy.blogspot.ca/2007/10/mocks-and-stubs-arent-spies.html>
- [3] <http://xunitpatterns.com/Mocks,%20Fakes,%20Stubs%20and%20Dummies.html>
- [4] <http://stackoverflow.com/questions/3459287/whats-the-difference-between-a-mock-stub?q=1>
- [5] <http://stackoverflow.com/questions/346372/whats-the-difference-between-faking-mocking-and-stubbing>
- [6] Feathers, Michael (2005). “Sensing and separation”. *Working effectively with legacy code*. NJ: Prentice Hall. p. 23 et seq. ISBN 0-13-117705-2.
- [7] Osheroove, Roy (2009). “Interaction testing with mock objects et seq”. *The art of unit testing*. Manning. ISBN 978-1-933988-27-6.
- [8] These examples use a nomenclature that is similar to that used in Unified Modeling Language
- [9] Beck, Kent (2003). *Test-Driven Development By Example*. Boston: Addison Wesley. ISBN 0-321-14653-0.
- [10] InJava.com to Mocking ! O'Reilly Media

5.5.7 External links

- Tim Mackinnon (8 September 2009). “A Brief History of Mock Objects”. Mockobjects.com/.
- Test Doubles: a section of a book on unit testing patterns.
- All about mock objects! Portal concerning mock objects
- “Using mock objects for complex unit tests”. IBM developerWorks. 16 October 2006. Archived from the original on 4 May 2007.
- Unit testing with mock objects IBM developerWorks
- Mocks Aren't Stubs (Martin Fowler) Article about developing tests with Mock objects. Identifies and compares the “classical” and “mockist” schools of testing. Touches on points about the impact on design and maintenance.

5.6 Lazy systematic unit testing

Lazy Systematic Unit Testing^[1] is a software **unit testing** method based on the two notions of *lazy specification*, the ability to infer the evolving specification of a unit on-the-fly by dynamic analysis, and *systematic testing*, the ability to explore and test the unit's state space exhaustively to bounded depths. A testing toolkit JWalk exists to support lazy systematic unit testing in the Java programming language.^[2]

5.6.1 Lazy Specification

Lazy specification refers to a flexible approach to software **specification**, in which a specification evolves rapidly in parallel with frequently modified code.^[1] The specification is inferred by a semi-automatic analysis of a prototype software unit. This can include **static analysis** (of the unit's interface) and **dynamic analysis** (of the unit's behaviour). The dynamic analysis is usually supplemented by limited interaction with the programmer.

The term **Lazy specification** is coined by analogy with *lazy evaluation* in functional programming. The latter describes the delayed evaluation of sub-expressions, which are only evaluated on demand. The analogy is with the late stabilization of the specification, which evolves in parallel with the changing code, until this is deemed stable.

5.6.2 Systematic Testing

Systematic testing refers to a complete, **conformance testing** approach to **software testing**, in which the tested unit is shown to conform exhaustively to a specification, up to the testing assumptions.^[3] This contrasts with exploratory, incomplete or random forms of testing. The aim is to provide repeatable guarantees of correctness after testing is finished.

Examples of systematic testing methods include the **Stream X-Machine** testing method^[4] and **equivalence partition testing** with full **boundary value analysis**.

5.6.3 References

- [1] A J H Simons, JWalk: Lazy systematic unit testing of Java classes by design introspection and user interaction, *Automated Software Engineering*, 14 (4), December, ed. B. Nuseibeh, (Boston: Springer, 2007), 369-418.
- [2] *The JWalk Home Page*, <http://www.dcs.shef.ac.uk/~{ajhs/jwalk/>
- [3] A J H Simons, A theory of regression testing for behaviourally compatible object types, *Software Testing, Verification and Reliability*, 16 (3), UKTest 2005 Special

Issue, September, eds. M Woodward, P McMinn, M Holcombe and R Hierons (Chichester: John Wiley, 2006), 133-156.

- [4] F Ipate and W M L Holcombe, Specification and testing using generalised machines: a presentation and a case study, *Software Testing, Verification and Reliability*, 8 (2), (Chichester: John Wiley, 1998), 61-81.

5.7 Test Anything Protocol

The **Test Anything Protocol** (TAP) is a protocol to allow communication between **unit tests** and a **test harness**. It allows individual tests (TAP producers) to communicate test results to the testing harness in a **language-agnostic** way. Originally developed for unit testing of the **Perl** interpreter in 1987, producers and parsers are now available for many development platforms.

5.7.1 History

TAP was created for the first version of the **Perl programming language** (released in 1987), as part of the Perl's core test harness (**t/TEST**). The **Test::Harness** module was written by **Tim Bunce** and **Andreas König** to allow Perl module authors to take advantage of TAP.

Development of TAP, including standardization of the protocol, writing of test producers and consumers, and evangelizing the language is coordinated at the **TestAnything** website.^[1]

5.7.2 Specification

A formal specification for this protocol exists in the **TAP::Spec::Parser** and **TAP::Parser::Grammar** modules. The behavior of the **Test::Harness** module is the de facto TAP standard implementation, along with a writeup of the specification on <http://testanything.org>.

A project to produce an **IETF** standard for TAP was initiated in August 2008, at **YAPC::Europe 2008**.^[1]

5.7.3 Usage examples

Here's an example of TAP's general format:

```
1..48 ok 1 Description # Directive # Diagnostic .... ok 47
Description ok 48 Description
```

For example, a test file's output might look like:

```
1..4 ok 1 - Input file opened not ok 2 - First line of the
input valid. More output from test 2. There can be arbitrary
number of lines for any output so long as there is at least
some kind of whitespace at beginning of line. ok 3
- Read the rest of the file #TAP meta information not ok 4
- Summarized correctly # TODO Not written yet
```

5.7.4 References

- [1] "The Test Anything Protocol website". Retrieved September 4, 2008.

5.7.5 External links

- <http://testanything.org/> is a site dedicated to the discussion, development and promotion of TAP.

5.8 xUnit

*For the particular .NET testing framework, see **xUnit.net**.*

*For the unit of measurement, see **x unit**.*

xUnit is the collective name for several **unit testing frameworks** that derive their structure and functionality from **Smalltalk's SUnit**. **SUnit**, designed by **Kent Beck** in 1998, was written in a highly structured **object-oriented** style, which lent easily to contemporary languages such as **Java** and **C#**. Following its introduction in **Smalltalk** the framework was **ported** to **Java** by Beck and **Erich Gamma** and gained wide popularity, eventually gaining ground in the majority of programming languages in current use. The names of many of these frameworks are a variation on "SUnit", usually substituting the "S" for the first letter (or letters) in the name of their intended language ("**JUnit**" for **Java**, "**RUnit**" for **R** etc.). These frameworks and their common architecture are collectively known as "xUnit".

5.8.1 xUnit architecture

All xUnit frameworks share the following basic component architecture, with some varied implementation details.^[1]

Test runner

A **test runner** is an executable program that runs tests implemented using an **xUnit** framework and reports the test results.^[2]

Test case

A **test case** is the most elemental class. All unit tests are inherited from here.

Test fixtures

A **test fixture** (also known as a test context) is the set of **preconditions** or state needed to run a test. The developer

should set up a known good state before the tests, and return to the original state after the tests.

Test suites

A **test suite** is a set of tests that all share the same fixture. The order of the tests shouldn't matter.

Test execution

The execution of an individual unit test proceeds as follows:

```
setup(); /* First, we should prepare our 'world' to make
an isolated environment for testing */ ... /* Body of test
- Here we make all the tests */ ... teardown(); /* At the
end, whether we succeed or fail, we should clean up our
'world' to not disturb other tests or code */
```

The `setup()` and `teardown()` methods serve to initialize and clean up test fixtures.

Test result formatter

A **test runner** produces results in one or more output formats. In addition to a plain, human-readable format, there is often a test result formatter that produces **XML** output. The **XML** test result format originated with **JUnit** but is also used by some other **xUnit** testing frameworks, for instance build tools such as **Jenkins** and **Atlassian Bamboo**.

Assertions

An **assertion** is a function or macro that verifies the behavior (or the state) of the unit under test. Usually an assertion expresses a **logical condition** that is true for results expected in a correctly running **system under test** (SUT). Failure of an assertion typically throws an **exception**, aborting the execution of the current test.

5.8.2 xUnit frameworks

Many xUnit frameworks exist for various programming languages and development platforms.

- [List of unit testing frameworks](#)

5.8.3 See also

Unit testing in general:

- [Unit testing](#)
- [Software testing](#)

Programming approach to unit testing:

- [Test-driven development](#)
- [Extreme programming](#)

5.8.4 References

- [1] Beck, Kent. "Simple Smalltalk Testing: With Patterns". Archived from the original on 15 March 2015. Retrieved 25 June 2015.
- [2] Meszaros, Gerard (2007) *xUnit Test Patterns*, Pearson Education, Inc./Addison Wesley

5.8.5 External links

- [Other list of various unit testing frameworks](#)
- [OpenSourceTesting.org](#) lists many unit testing frameworks, performance testing tools and other tools programmers/developers may find useful
- [Test automation patterns for writing tests/specs in xUnit.](#)
- [Martin Fowler on the background of xUnit.](#)

5.9 List of unit testing frameworks

This page is a list of tables of code-driven unit testing frameworks for various programming languages. Some but not all of these are based on xUnit.

5.9.1 Columns (Classification)

- **Name:** This column contains the name of the framework and will usually link to it.
- **xUnit:** This column indicates whether a framework should be considered of **xUnit** type.
- **TAP:** This column indicates whether a framework can emit **TAP** output for TAP-compliant testing harnesses.
- **Generators:** Indicates whether a framework supports **data generators**. Data generators generate input data for a test and the test is run for each input data that the generator produces.
- **Fixtures:** Indicates whether a framework supports test-local fixtures. Test-local fixtures ensure a specified environment for a single test.
- **Group fixtures:** Indicates whether a framework supports group fixtures. Group fixtures ensure a specified environment for a whole group of Tests

- **MPI:** Indicates whether a framework supports message passing via MPI - commonly used for high-performance scientific computing.
- **Other columns:** These columns indicate whether a specific language / tool feature is available / used by a framework.
- **Remarks:** Any remarks.

C++**Cg****CFML (ColdFusion)****Clojure****Cobol****Common Lisp****5.9.2 Languages****Curl****ABAP****Delphi****ActionScript / Adobe Flex****Ada****Emacs Lisp****AppleScript****Erlang****ASCET****Fortran****ASP****BPEL****F#****C****Groovy****C#**

See .NET programming languages below.

All entries under Java may also be used in Groovy.

Genexus	SQL
Haskell	MySQL
Haxe	PL/SQL
HLSL	IBM DB2 SQL-PL
ITT IDL	PostgreSQL
Internet	Transact-SQL
Java	Swift
JavaScript	SystemVerilog
Lasso	TargetLink
LaTeX	Tcl
LabVIEW	TinyOS/necC
LISP	TypeScript
Logtalk	Visual FoxPro
Lua	Visual Basic (VB6.0)
MATLAB	For unit testing frameworks for VB.NET, see the .NET programming languages section.
.NET programming languages	Visual Lisp
Objective-C	XML
OCaml	XSLT
Object Pascal (Free Pascal)	Other
PegaRULES Process Commander	5.9.3 See also
Perl	Unit testing in general:
PHP	• Unit testing • Software testing
PowerBuilder	Extreme programming approach to unit testing:
Progress 4GL	• xUnit • Test-driven development (TDD) • Behavior-driven development (BDD) • Extreme programming
Prolog	
Python	
R programming language	
Racket	
REALbasic	
Rebol	
RPG	
Ruby	
SAS	

5.9.4 References

- [1] "SAP Library - ABAP - Analysis Tools". [Help.sap.com](http://help.sap.com). Retrieved 2012-11-12.
- [2] "as3flexunitlib - ActionScript 3.0 framework for unit testing. - Google Project Hosting". [Code.google.com](http://code.google.com). 2008-08-20. Retrieved 2012-11-12.
- [3] <http://www.flexunit.org/>
- [4] "reflex-unit - Unit testing framework for Flex 2/3 - Google Project Hosting". [Code.google.com](http://code.google.com). Retrieved 2012-11-12.
- [5] "astuce - a JUnit clone for ECMAScript - Google Project Hosting". [Code.google.com](http://code.google.com). Retrieved 2012-11-12.
- [6] "AsUnit". asunit.org. Retrieved 23 June 2015.
- [7] "dpuint - Unit and Integration Testing Framework for Flex 2 and 3 - Google Project Hosting". [Code.google.com](http://code.google.com). Retrieved 2012-11-12.
- [8] "fluint - Flex Unit and Integration Testing Framework - Google Project Hosting". fluint.googlecode.com. Retrieved 23 June 2015.
- [9] "loomis / morefluent / wiki / Home". [Bitbucket.org](http://bitbucket.org). 2011-02-25. Retrieved 2012-11-12.
- [10] "mojotest - A very simple and easy to use ActionScript 3 Unit Test framework - Google Project Hosting". [Code.google.com](http://code.google.com). Retrieved 2012-11-12.
- [11] "Aunit". [Libre.adacore.com](http://libre.adacore.com). Retrieved 2012-11-12.
- [12] "AdaTEST95 – efficient implementation of unit and integration testing". [Qa-systems.com](http://qa-systems.com). 2012-03-16. Retrieved 2012-11-12.
- [13] "Ahven - Unit Testing Library for Ada Programming Language". stronglytyped.org. Retrieved 23 June 2015.
- [14] "LDRA - LDRA Tool Suite". ldra.com. Retrieved 23 June 2015.
- [15] "Embedded Software Testing - Vector Software". vector-cast.com. Retrieved 23 June 2015.
- [16] "ASUnit". freeshell.org. Retrieved 23 June 2015.
- [17]
- [18] "TPT - real time testing embedded control software". Piketec.com. Retrieved 2012-11-12.
- [19] "ASPUint: an ASP Unit Testing Framework". sourceforge.net. Retrieved 23 June 2015.
- [20] Mayer, Philip; Daniel Lübke (2006). "Towards a BPEL unit testing framework". *TAV-WEB '06 Proceedings of the 2006 workshop on Testing, analysis, and verification of web services and applications* (New York, NY, USA: ACM): 33–42. doi:10.1145/1145718.1145723. ISBN 1595934588.
- [21] "nassersala/cbdd". [GitHub](https://github.com). Retrieved 23 June 2015.
- [22] "AceUnit". sourceforge.net. Retrieved 23 June 2015.
- [23] "Overview - API Sanity Checker - Open-Source Projects". github.com. Retrieved 2015-06-25.
- [24] "Automated Testing Framework (ATF)". [Netbsd.org](http://netbsd.org). Retrieved 2012-11-12.
- [25] "autounit.tigris.org". tigris.org. Retrieved 23 June 2015.
- [26] "C and C++ testing tools: Static code analysis, code review, unit testing". [Parasoft](http://parasoft.com). 2012-09-24. Retrieved 2012-11-12.
- [27] "Dynamic testing with Cantata: automated and easy". [Qa-systems.com](http://qa-systems.com). 2012-03-16. Retrieved 2012-11-12.
- [28]
- [29] "cfix ¶ C and C++ Unit Testing Framework for Windows". cfix-testing.org. Retrieved 23 June 2015.
- [30] Marcus Baker et al. "Cgreen is a unit testing framework for the C programming language". Retrieved 2013-05-15.
- [31] "Check". sourceforge.net. Retrieved 23 June 2015.
- [32] "cmockery - A lightweight library to simplify and generalize the process of writing unit tests for C applications. - Google Project Hosting". [Code.google.com](http://code.google.com). Retrieved 2012-11-12.
- [33] "CppUTest (Moved!) | Free Development software downloads at". [Sourceforge.net](http://sourceforge.net). Retrieved 2012-11-12.
- [34] "DanFis - CU - C Unit Testing Framework". danfis.cz. Retrieved 23 June 2015.
- [35] "bvdberg/ctest · GitHub". [Github.com](https://github.com). Retrieved 2012-11-12.
- [36] "CUnit". sourceforge.net. Retrieved 23 June 2015.
- [37] "cunitwin32 - CUnitWin32 is a unit testing framework for C/C++ for Microsoft Windows - Google Project Hosting". [Code.google.com](http://code.google.com). Retrieved 2012-11-12.
- [38] "CUT 2.6 - 10th Anniversary Version!". [Falvotech.com](http://falvotech.com). Retrieved 2012-11-12.
- [39] "CuTest: The Cutest C Unit Testing Framework". sourceforge.net. Retrieved 23 June 2015.
- [40] "a Unit Testing Framework for C and C++ - Cutter". sourceforge.net. Retrieved 23 June 2015.
- [41] "Embedded Unit". sourceforge.net. Retrieved 23 June 2015.
- [42] "Unit Testing Tool - Embunit". embunit.com. Retrieved 23 June 2015.
- [43] "imb/fctx". [GitHub](https://github.com). Retrieved 23 June 2015.
- [44]
- [45] "garage: GUnit: Project Info". [Garage.maemo.org](http://garage.maemo.org). Retrieved 2012-11-12.
- [46] "lcut - a Lightweight C Unit Testing framework - Google Project Hosting". google.com. Retrieved 23 June 2015.
- [47] "LibU". koanlogic.com. Retrieved 23 June 2015.

- [48] “JTN002 - MinUnit - a minimal unit testing framework for C”. Jera.com. Retrieved 2012-11-12.
- [49] “galvedro/mut”. *GitHub*. Retrieved 23 June 2015.
- [50] “opmock | Free software downloads at”. Sourceforge.net. Retrieved 2012-11-12.
- [51] “jecklgamis/rcunit”. *GitHub*. Retrieved 23 June 2015.
- [52] “IBM Rational software”. *rational.com*. Retrieved 23 June 2015.
- [53] “keithn/seatest”. *GitHub*. Retrieved 23 June 2015.
- [54] “Accord - Dynamic Analyzer - C Unit Test Tool”. Accord-soft.com. Retrieved 2012-11-12.
- [55] Lingua-Systems Software GmbH (27 March 2015). “Sput Unit Testing Framework for C/C++”. *lingua-systems.com*. Retrieved 23 June 2015.
- [56] “STRIDE Wiki”. *stridewiki.com*. Retrieved 23 June 2015.
- [57] “Redir”. HiteX.de. Retrieved 2012-11-12.
- [58] “TestApe - Unit testing for embedded software”. *testape.com*. Retrieved 23 June 2015.
- [59] “test-dept - Unit Test Framework for C with Stubbing - Google Project Hosting”. *test-dept.googlecode.com*. Retrieved 23 June 2015.
- [60] “tf-unit-test - unit testing framework for ansi c - Google Project Hosting”. *google.com*. Retrieved 23 June 2015.
- [61] <http://unity.sourceforge.net>
- [62] “Embedded Software Testing - Vector Software”. *vector-cast.com*. Retrieved 23 June 2015.
- [63] <http://www.visualassert.com/>
- [64] “ccosmin/tinytest”. *GitHub*. Retrieved 23 June 2015.
- [65] “xTests - Multi-language, Lightweight Test-suites”. *sourceforge.net*. Retrieved 23 June 2015.
- [66] “Login”. *tigris.org*. Retrieved 23 June 2015.
- [67] “bandit”. *banditcpp.org*. Retrieved 23 June 2015.
- [68] Llopis, Noel. “Exploring the C++ Unit Testing Framework Jungle”, 2004-12-28. Retrieved on 2010-2-13.
- [69] Rozental, Gennadiy “Boost Test Fixture Documentation”. Retrieved on 2010-2-13.
- [70] Rozental, Gennadiy “Boost Test Suite Level Fixture Documentation”. Retrieved on 2010-2-13.
- [71] “Turtle”. *sourceforge.net*.
- [72] “Boost Test Library”. Boost.org. Retrieved 2012-11-12.
- [73] “philsquared/Catch · GitHub”. Github.com. Retrieved 2012-11-12.
- [74] “martinmoene/Catch · GitHub”. Github.com. Retrieved 2013-09-03.
- [75] “cput”. *CodePlex*. Retrieved 23 June 2015.
- [76] “CppTest - A C++ Unit Testing Framework”. *sourceforge.net*. Retrieved 23 June 2015.
- [77] “cppunit”. SourceForge.net. 2009-11-23. Retrieved 2012-11-12.
- [78] “cppunit”. *Freedesktop.org*. 18 May 2013. Retrieved 6 November 2013.
- [79] “Cpp Unit Lite”. C2.com. 2011-04-21. Retrieved 2012-11-12.
- [80] “CUnit project page”. *sourceforge.net*. Retrieved 23 June 2015.
- [81] “crpcut - the Compartmented Robust Posix C++ Unit Test system”. *sourceforge.net*. Retrieved 23 June 2015.
- [82] “Wiki - CUTE - C++ Unit Testing Easier”. *cute-test.com*. Retrieved 23 June 2015.
- [83] “cutee, C++ Unit Testing Easy Environment”. Codesink.org. Retrieved 2012-11-12.
- [84] “CxxTest”. *cxctest.com*.
- [85] “Exercisix | Alexander Churanov | Personal WebSite”. Alexander Churanov. 2011-07-14. Retrieved 2012-11-12.
- [86] “eranpeer/FakeIt”. *GitHub*. Retrieved 23 June 2015.
- [87] <http://fctx.wildbearsoftware.com>
- [88] “FRUCTOSE | Free Development software downloads at”. Sourceforge.net. Retrieved 2012-11-12.
- [89] “googlemock - Google C++ Mocking Framework - Google Project Hosting”. Code.google.com. Retrieved 2012-11-12.
- [90] “googletest - Google C++ Testing Framework - Google Project Hosting”. Code.google.com. Retrieved 2012-11-12.
- [91] “Hestia | Free Development software downloads at”. Sourceforge.net. Retrieved 2012-11-12.
- [92] “Hestia | Free Development software downloads at”. Sourceforge.net. Retrieved 2012-11-12.
- [93] “Igloo - BDD Style Unit Testing for C++”. *igloo-testing.org*. Retrieved 23 June 2015.
- [94] “martinmoene/lest · GitHub”. Github.com. Retrieved 2013-09-03.
- [95] “etr/liblittletest”. *GitHub*. Retrieved 23 June 2015.
- [96] “libunittest C++ library”. *sourceforge.net*. Retrieved 23 June 2015.
- [97] “Smart Unit Testing for C/C++”. *typemock.org*.
- [98] “An Eclipse CDT plug-in for C++ Seams and Mock Objects”. IFS. Retrieved 2012-11-18.
- [99] “mockcpp - A C++ Mock Framework - Google Project Hosting”. Code.google.com. Retrieved 2012-11-12.

- [100] "mockitopp - Simple mocking for C++". *github.com*. Retrieved 2015-03-19.
- [101] "Software Patent Mine Field: Danger! Using this website is risky!". *sourceforge.net*. Retrieved 23 June 2015.
- [102]
- [103] "jdmclark/nullunit". *GitHub*. Retrieved 23 June 2015.
- [104] "Service temporarily unavailable". *oaklib.org*. Retrieved 23 June 2015.
- [105] "since Qt5".
- [106] "Qt 4.7: QTestLib Tutorial". *Doc.qt.nokia.com*. Retrieved 2012-11-12.
- [107] "QuickTest". *sf.net*. Retrieved 23 June 2015.
- [108] "SafetyNet, C++ Unit Testing Framework". *devmentor.org*. Retrieved 23 June 2015.
- [109] "ShortCUT - A Short C++ Unit Testing Framework". *CodeProject*. 2007-02-15. Retrieved 2012-11-12.
- [110] "STRIDE Wiki". *stridewiki.com*. Retrieved 23 June 2015.
- [111] charlesweir. "Symbian OS C++ Unit Testing Framework". *symbianosunit.co.uk*. Retrieved 23 June 2015.
- [112] <http://www.ldra.co.uk/tbrun.asp>
- [113]
- [114] "Test soon: Test soon documentation". *sourceforge.net*. Retrieved 23 June 2015.
- [115] "Testwell CTA++ Description". *Testwell.fi*. Retrieved 2012-11-12.
- [116] "tpounds/tpunitpp · GitHub". *Github.com*. 2012-05-20. Retrieved 2012-11-12.
- [117] "rollbear/Trompeloeil". *GitHub*. Retrieved 23 July 2015.
- [118] <http://tut-framework.sourceforge.net/>
- [119] "The unit++ Testing Framework". *sourceforge.net*. Retrieved 23 June 2015.
- [120] "burner/sweet.hpp". *GitHub*. Retrieved 23 June 2015.
- [121] "unittest-cpp/unittest-cpp". *GitHub*. Retrieved 23 June 2015.
- [122] "DronMDF/uppl1". *GitHub*.
- [123] "UquoniTest: a unit testing library for C". *Q-mentum.com*. Retrieved 2012-11-12.
- [124] "WinUnit". *CodePlex*. Retrieved 23 June 2015.
- [125] "moswald / xUnit++ / wiki / Home — Bitbucket". *Bitbucket.org*. 2012-11-06. Retrieved 2012-11-12.
- [126] "SourceForge: Welcome". *sourceforge.net*. Retrieved 23 June 2015.
- [127] "Source Checkout - unittestcg - UnitTestCg is a unittest framework for Cg and HLSL programs. - Google Project Hosting". *google.com*. Retrieved 23 June 2015.
- [128] "MXUnit - Unit Test Framework and Eclipse Plugin for Adobe ColdFusion". *mxunit.org*.
- [129] "clojure.test - Clojure v1.4 API documentation". *Clojure.github.com*. Retrieved 2012-11-12.
- [130] weavejester. "weavejester/fact · GitHub". *Github.com*. Retrieved 2012-11-12.
- [131] "marick/Midje · GitHub". *Github.com*. Retrieved 2012-11-12.
- [132] "Clojure Testing Framework - Speclj". *speclj.com*.
- [133] "COBOLUnit". *Sites.google.com*. Retrieved 2012-11-12.
- [134] "cobol-unit-test". *Github.com*. Retrieved 2015-08-20.
- [135] savignano software solutions. "Better Software In Less Time: - savignano software solutions". *Savignano.net*. Retrieved 2012-11-12.
- [136] "z/OS Automated Unit Testing Framework (zUnit)". *ibm.com*.
- [137] "CLiki: CLUnit". *cliki.net*.
- [138] <http://cybertiggyr.com/gene/lut/>
- [139] "FiveAM". *Common-lisp.net*. 2004-11-16. Retrieved 2012-11-12.
- [140] "FRet". *Common-lisp.net*. Retrieved 2012-11-12.
- [141] "Grand-prix". *Common-lisp.net*. Retrieved 2012-11-12.
- [142] "HEUTE - Common LISP Unit Test Package". *Rdrop.com*. Retrieved 2012-11-12.
- [143] "LIFT - the LIsp Framework for Testing". *Common-lisp.net*. Retrieved 2012-11-12.
- [144] "lisp-unit". *Cs.northwestern.edu*. Retrieved 2012-11-12.
- [145] "Package: lang/lisp/code/testing/rt/". *Cs.cmu.edu*. Retrieved 2012-11-12.
- [146] "stefil". *Common-lisp.net*. Retrieved 2012-11-12.
- [147] "CLiki: xlunit". *cliki.net*.
- [148] "CurlUnit 1.0". *sourceforge.net*.
- [149] "DUNIT: An Xtreme testing framework for Delphi programs". *sourceforge.net*.
- [150] "DUnit2 | Free software downloads at". *Sourceforge.net*. Retrieved 2012-11-12.
- [151] "DUnitX". Retrieved 2014-07-09.
- [152] Last edited 2010-12-11 11:44 UTC by JariAalto (diff) (2010-12-11). "El Unit". *EmacsWiki*. Retrieved 2012-11-12.

- [153] Last edited 2010-03-18 14:38 UTC by LennartBorgman (diff) (2010-03-18). “Elk Test”. EmacsWiki. Retrieved 2012-11-12.
- [154] Last edited 2009-05-13 06:57 UTC by Free Ekanayaka (diff) (2009-05-13). “unit-test.el”. EmacsWiki. Retrieved 2012-11-12.
- [155]
- [156] “nasarb’s funit-0.11.1 Documentation”. *rubyforge.org*.
- [157] “FORTRAN Unit Test Framework (FRUIT) | Free Development software downloads at”. Sourceforge.net. Retrieved 2012-11-12.
- [158] “flibs/ftnunit - flibs”. Flibs.sf.net. Retrieved 2012-11-12.
- [159] “pFUnit | Free Development software downloads at”. Sourceforge.net. Retrieved 2014-01-16.
- [160] “ObjexxFTK - Objexx Fortran ToolKit | Objexx Engineering”. Objexx.com. Retrieved 2012-11-12.
- [161] “Foq”. *CodePlex*.
- [162] “FsCheck: A random testing framework - Home”. Codeplex.com. Retrieved 2012-11-12.
- [163] “andriniaina/FsMocks”. *GitHub*.
- [164] “FsTest”. *CodePlex*.
- [165] “FsUnit”. *CodePlex*.
- [166]
- [167] “unquote - Write F# unit test assertions as quoted expressions, get step-by-step failure messages for free - Google Project Hosting”. Code.google.com. Retrieved 2012-11-12.
- [168] “easyb”. *easyb.org*.
- [169] “spock - the enterprise ready specification framework - Google Project Hosting”. Code.google.com. Retrieved 2012-11-12.
- [170] “gmock - A Mocking Framework for Groovy - Google Project Hosting”. Code.google.com. 2011-12-13. Retrieved 2012-11-12.
- [171] “GXUnit”. Wiki.gxtechnical.com. Retrieved 2012-11-12.
- [172] “HUnit -- Haskell Unit Testing”. *sourceforge.net*.
- [173] “HUnit-Plus: A test framework building on HUnit. - Hackage”. *haskell.org*.
- [174] “nick8325/quickcheck”. *GitHub*.
- [175] “feuerbach/smallcheck”. *GitHub*.
- [176] “hspec/hspec”. *GitHub*.
- [177] “marcotmarcot/chuchu”. *GitHub*.
- [178] “massiveinteractive/MassiveUnit · GitHub”. Github.com. Retrieved 2012-11-12.
- [179] “mgunit project site moved!”. *idlddev.com*.
- [180]
- [181] Mike Bowler. “HtmlUnit – Welcome to HtmlUnit”. *sourceforge.net*.
- [182] “ieunit - Unit test framework for web pages. - Google Project Hosting”. Code.google.com. Retrieved 2012-11-12.
- [183] “Canoo WebTest”. *canoo.com*.
- [184] “SoapUI - The Home of Functional Testing”. *soapui.org*.
- [185] “API Testing”. *Parasoft*.
- [186] “API Testing”. Parasoft.com. Retrieved 2015-04-15.
- [187] “Arquillian · Write Real Tests”. *arquillian.org*.
- [188] “beanSpec | Free Development software downloads at”. Sourceforge.net. Retrieved 2012-11-12.
- [189] “abreksa4/BeanTest”. *GitHub*.
- [190] “Specification by Example - Concordion”. *concordion.org*.
- [191] “Concutest”. *concutest.org*.
- [192] “cucumber/cucumber-jvm · GitHub”. Github.com. Retrieved 2012-11-12.
- [193] “?????????????????????????????”. *dbunit.org*.
- [194] “EasyMock”. *easymock.org*.
- [195] “10. Testing”. *springsource.org*. Retrieved 23 June 2015.
- [196] “ETLUNIT Home”. *atlassian.net*.
- [197] “Etl-unit Home Page.”.
- [198] Tim Lavers. “GrandTestAuto”. *grandtestauto.org*.
- [199] “GroboUtils - GroboUtils Home Page”. *sourceforge.net*.
- [200] “havarunner/havarunner”. *GitHub*.
- [201] “instinct - Instinct is a Behaviour Driven Development (BDD) framework for Java - Google Project Hosting”. Code.google.com. Retrieved 2012-11-12.
- [202] shyiko (2010-11-17). “Home · shyiko/jsst Wiki · GitHub”. Github.com. Retrieved 2012-11-12.
- [203] “What is JBehave?”. *jbehave.org*.
- [204] “JDave”. *jdave.org*.
- [205] “SCG: JExample”. Scg.unibe.ch. 2009-04-21. doi:10.1007/978-3-540-68255-4_8. Retrieved 2012-11-12.
- [206] “JGiven”. *jgiven.org*.
- [207] “jMock - An Expressive Mock Object Library for Java”. *jmock.org*.
- [208] “Google Project Hosting”. *google.com*.

- [209] Sebastian Benz. “Jnario”. *jnario.org*.
- [210] “Java testing tools: static code analysis, code review, unit testing”. Parasoft. 2012-10-08. Retrieved 2012-11-12.
- [211] <http://jukito.org/>
- [212] “JUnit - About”. *junit.org*.
- [213] “junitee.org”. *junitee.org*.
- [214] “JWalk software testing tool suite - Lazy systematic unit testing for agile methods”. The University of Sheffield. Retrieved 2014-09-04.
- [215] “mockito - simpler & better mocking - Google Project Hosting”. Code.google.com. 2008-01-14. Retrieved 2012-11-12.
- [216] “Mock classes for enterprise application testing”. Retrieved 2014-09-04.
- [217] “Needle - Effective Unit Testing for Java EE - Overview”. *spree.de*.
- [218] “JavaLib”. *neu.edu*.
- [219] <http://openpojo.com/>
- [220] “powermock - PowerMock is a Java framework that allows you to unit test code normally regarded as untestable. - Google Project Hosting”. *powermock.org*.
- [221] “Randoop”. *mernst.github.io*. Retrieved 23 June 2015.
- [222] “Sprystone.com”. *sprystone.com*.
- [223] “Sureassert UC”. *sureassert.com*.
- [224] “Test NG Website”. Retrieved 2014-09-04.
- [225] “TestNG makes Java unit testing a breeze”. *ibm.com*. 2005-01-06. Retrieved 2012-11-12.
- [226] “Google Testing Blog: TotT: TestNG on the Toilet”. *Googletesting.blogspot.com*. Retrieved 2012-11-12.
- [227] “Unitils – Index”. *unitils.org*.
- [228] “<XmlUnit/>”. *sourceforge.net*.
- [229] “monolithed/Suittest · GitHub”. *Github.com*. Retrieved 2012-11-12.
- [230] Authors:. “D.O.H.: Dojo Objective Harness — The Dojo Toolkit - Reference Guide”. *Dojotoolkit.org*. Retrieved 2012-11-12.
- [231] “lbrtw/ut”. *GitHub*.
- [232] “JavaScript unit test framework, part 1”. *lbrtw.com*.
- [233] “jsunit.net”. *jsunit.net*.
- [234] Steve Fenton. “JavaScript Enhance Test Framework - Steve Fenton : The Internet, Web Development, JavaScript, Photography”. Steve Fenton. Retrieved 2012-11-12.
- [235] “QUnit”. *qunitjs.com*.
- [236] “Unit testing framework for Javascript”. *unitjs.com*.
- [237] <http://www.iankent.co.uk/rhunit/>
- [238]
- [239] “J3Unit”. *sourceforge.net*.
- [240] “Mocha”. *mochajs.org*.
- [241] <https://github.com/theintern/inter>
- [242] “Specification Frameworks and Tools”. *Valleyhighlands.com*. 2010-11-26. Retrieved 2012-11-12.
- [243] “YUI 2: YUI Test”. *Developer.yahoo.com*. 2011-04-13. Retrieved 2012-11-12.
- [244] <http://jania.pe.kr/aw/moin.cgi/JSSpec>
- [245] “Home — Scriptaculous Documentation”. *Github.com*. Retrieved 2012-11-12.
- [246] <http://visionmedia.github.com/jspec>
- [247] <http://pivotal.github.com/jasmine>
- [248] “nkallen/screw-unit · GitHub”. *Github.com*. Retrieved 2012-11-12.
- [249] “substack/tape”. Retrieved 2015-01-29.
- [250] TAP output can easily be transformed into JUnit XML via the CPAN module TAP::Formatter::JUnit.
- [251] “JSAN - Test.Simple”. *Openjsan.org*. 2009-08-21. Retrieved 2012-11-12.
- [252] “JSAN - Test.More 0.21”. *Openjsan.org*. Retrieved 2012-11-12.
- [253] Bruce Williams <<http://codefluency.com>>, for Ruby Central <<http://rubycentral.org>>. “TestCase: Project Info”. *RubyForge*. Retrieved 2012-11-12.
- [254] “DouglasMeyer/test_it · GitHub”. *Github.com*. Retrieved 2012-11-12.
- [255] <https://code.google.com/p/jsunity/source/browse/trunk/jsunity/jsunity.js>
- [256] “willurd/JSTest · GitHub”. *Github.com*. Retrieved 2012-11-12.
- [257] “JSTest.NET - Browserless JavaScript Unit Test Runner”. *CodePlex*.
- [258] <http://jsunity.com/>
- [259] “rhinounit - Javascript Testing Framework using Rhino - Google Project Hosting”. *Code.google.com*. Retrieved 2012-11-12.
- [260] “jasproject - Javascript Agile Suite - Google Project Hosting”. *Code.google.com*. Retrieved 2012-11-12.
- [261] “FireUnit: Firebug Unit Testing for Firefox”. *fireunit.org*.
- [262] “js-test-driver - Remote javascript console - Google Project Hosting”. *Code.google.com*. Retrieved 2012-11-12.

- [263] <http://js-testrunner.codehaus.org/>
- [264] <http://cjohansen.no/sinon/>
- [265] “Vows”. vowsjs.org.
- [266] “caolan/nodeunit · GitHub”. [Github.com](https://github.com/caolan/nodeunit). Retrieved 2012-11-12.
- [267] “Tyrte :: Javascript Unit Testing Framework”. [github.com](https://github.com/tyrte).
- [268] “WebReflection/wru · GitHub”. [Github.com](https://github.com/WebReflection/wru). Retrieved 2012-11-12.
- [269] “Welcome! Buster.JS is... — Buster.JS 0.7 documentation”. busterjs.org.
- [270] “asvd/lighttest”. [GitHub](https://github.com/asvd/lighttest).
- [271] “Home - Chai”. chaijs.com.
- [272] “JSUS”. crisstanza.github.io.
- [273] <http://wallabyjs.com/>
- [274] “zeroloop/l-unit8”. [GitHub](https://github.com/zeroloop/l-unit8).
- [275] “Comprehensive TEX Archive Network: Package qstest”. [Ctan.org](http://ctan.org). Retrieved 2013-07-04.
- [276] JKI (2012-11-07). “VI Tester - Home Page - JKI Discussion Forums”. [Jkisoft.com](http://jkisoft.com). Retrieved 2012-11-12.
- [277] “lgtunit”. logtalk.org. Retrieved 2013-10-14.
- [278] “Luaunit”. [Phil.freehackers.org](http://phil.freehackers.org). Retrieved 2012-11-12.
- [279] “lunit - Unit Testing Framework for Lua - Homepage”. [Nessie.de](http://nessie.de). 2009-11-05. Retrieved 2012-11-12.
- [280] axelberres. “mlUnit”. [SourceForge](https://sourceforge.net).
- [281] “mlunit_2008a - File Exchange - MATLAB Central”. [Mathworks.com](https://mathworks.com). Retrieved 2012-11-12.
- [282] “MUnit: a unit testing framework in Matlab - File Exchange - MATLAB Central”. [Mathworks.com](https://mathworks.com). Retrieved 2012-11-12.
- [283] “MUnit: a unit testing framework in Matlab - File Exchange - MATLAB Central”. [Mathworks.com](https://mathworks.com). Retrieved 2012-11-12.
- [284] “MATLAB xUnit Test Framework - File Exchange - MATLAB Central”. [Mathworks.com](https://mathworks.com). Retrieved 2012-11-12.
- [285] “tgs / Doctest for Matlab — Bitbucket”. bitbucket.org.
- [286] Smith, Thomas. “Doctest - embed testable examples in your function’s help comments”. Retrieved 5 August 2011.
- [287] “Unit Testing Framework”. mathworks.com.
- [288] “DbUnit.NET”. sourceforge.net.
- [289] “fixie/fixie”. [GitHub](https://github.com).
- [290] “mb-unit - The Gallio test automation platform and MbUnit unit testing framework. - Google Project Hosting”. gallio.org.
- [291] “mb-unit - The Gallio test automation platform and MbUnit unit testing framework. - Google Project Hosting”. mbunit.com.
- [292] “moq - The simplest mocking library for .NET and Silverlight - Google Project Hosting”. google.com.
- [293] “NBi”. [CodePlex](http://codeplex.com).
- [294] “nmate - Open Source Unit-Test Code Generation and Integration Add-in for Visual Studio - Google Project Hosting”. google.com.
- [295] “Pex, Automated White box Testing for .NET - Microsoft Research”. microsoft.com. Microsoft. Retrieved 23 June 2015.
- [296] “Home”. qgonestudio.com. Retrieved 23 June 2015.
- [297] <http://www.quickunit.com/>
- [298] “abb-iss/Randoop.NET”. [GitHub](https://github.com). Retrieved 23 June 2015.
- [299] Next Page. “Ayende @ Rahien”. [Ayende.com](http://ayende.com). Retrieved 2012-11-12.
- [300] “Roaster unit test”. [CodePlex](http://codeplex.com). Retrieved 23 June 2015.
- [301] TechTalk. “SpecFlow”. [SpecFlow](http://specflow.com). Retrieved 23 June 2015.
- [302] “Specter Framework”. sf.net. Retrieved 23 June 2015.
- [303] “TestDriven.Net > Home”. testdriven.net.
- [304] “NET testing tools: Static code analysis, code review, unit testing with Parasoft dotTEST”. [Parasoft.com](http://parasoft.com). Retrieved 2012-11-12.
- [305] “TickSpec: An F# BDD Framework”. [CodePlex](http://codeplex.com).
- [306] “Smart Unit Testing - Made easy with Typemock”. typemock.org.
- [307]
- [308] “xUnit.net - Unit testing framework for C# and .NET (a successor to NUnit) - Home”. [CodePlex](http://codeplex.com).
- [309] “gabriel/gh-unit · GitHub”. [Github.com](https://github.com). Retrieved 2012-11-12.
- [310] philsquared (2012-06-02). “Home · philsquared/Catch Wiki · GitHub”. [Github.com](https://github.com). Retrieved 2012-11-12.
- [311] “pivotal/cedar · GitHub”. [Github.com](https://github.com). Retrieved 2012-11-12.
- [312] “kiwi-bdd/Kiwi”. [GitHub](https://github.com).
- [313] “specta/specta”. [GitHub](https://github.com).
- [314] “modocache/personal-fork-of-Quick”. [GitHub](https://github.com).
- [315] “ObjcUnit”. [Oops.se](http://oops.se). Retrieved 2012-11-12.

- [316] "Sen:te - OUnit". Sente.ch. Retrieved 2012-11-12.
- [317] "witebox - A more visually-oriented Unit Testing system exclusively for iPhone development! - Google Project Hosting". Code.google.com. Retrieved 2012-11-12.
- [318] "WOTest". *wincent.com*.
- [319] "Xcode - Features - Apple Developer". Apple Inc. Retrieved 2014-11-04.
- [320] "OUnit". *ocamlcore.org*.
- [321] Xavier Clerc (30 August 2012). "Kaputt - Introduction". *x9c.fr*.
- [322] <http://www.iinteractive.com/ocaml/>
- [323] "FORT | Free Development software downloads at". Sourceforge.net. Retrieved 2012-11-12.
- [324] "Index". Camelos.sourceforge.net. Retrieved 2012-11-12.
- [325] "Pascal TAP Unit Testing Suite | Free software downloads at". Sourceforge.net. Retrieved 2012-11-12.
- [326] "graemeg/fptest · GitHub". Github.com. Retrieved 2012-11-12.
- [327] "PRUnit SourceForge Project Homepage". *sourceforge.net*.
- [328]
- [329] "Test::Harness". metacpan.org. Retrieved 2012-11-12.
- [330] "Test::More". metacpan.org. Retrieved 2012-11-12.
- [331] "Test::Class". metacpan.org. Retrieved 2012-11-12.
- [332] "Test::Builder". metacpan.org. Retrieved 2012-11-12.
- [333] "Test::Unit". metacpan.org. Retrieved 2012-11-12.
- [334] "PerlUnit: unit testing framework for Perl". *sourceforge.net*.
- [335] "Re: Test::Unit, ::Class, or ::Inline?". nntp.perl.org. Retrieved 2012-11-12.
- [336] "Re: Test::Unit, ::Class, or ::Inline?". nntp.perl.org. Retrieved 2012-11-12.
- [337] "Test::DBUnit". metacpan.org. Retrieved 2012-11-12.
- [338] "Test::Unit::Lite". metacpan.org. Retrieved 2012-11-12.
- [339] "Test::Able". metacpan.org. Retrieved 2012-11-12.
- [340] "PHPUnit – The PHP Testing Framework". *phpunit.de*.
- [341] "PHP Unit Testing Framework". *sourceforge.net*.
- [342] "SimpleTest - Unit Testing for PHP". *simpletest.org*.
- [343] "/tools/lime/trunk - symfony - Trac". Trac.symfony-project.com. Retrieved 2012-11-12.
- [344] "shiflett/testmore · GitHub". Shiflett.org. Retrieved 2012-11-12.
- [345] Chris Shiflett. "Test::Simple for PHP". *shiflett.org*.
- [346] "OjesUnit". *ojesunit.blogspot.com*.
- [347] "Jakobo/snaptest". *GitHub*.
- [348] "atoum/atoum · GitHub". Github.com. Retrieved 2012-11-12.
- [349] README. "jamm/Tester · GitHub". Github.com. Retrieved 2012-11-12.
- [350] "ptrofimov/phpinlinetest · GitHub". Github.com. Retrieved 2012-11-12.
- [351] "phpspec". *phpspec.net*.
- [352] "nette/tester · GitHub". Github.com. Retrieved 2014-04-22.
- [353] "cryslead/kahlan · GitHub". Github.com. Retrieved 2015-03-19.
- [354] "Internet Archive Wayback Machine". Web.archive.org. 2009-07-28. Retrieved 2012-11-12.
- [355] "Welcome to ProUnit! -- The Progress - OpenEdge unit tests framework". *sourceforge.net*.
- [356] "CameronWills/OEUnit". *GitHub*.
- [357] "Prolog Unit Tests". Swi-prolog.org. Retrieved 2012-11-12.
- [358] <http://www.autotest.github.io/>
- [359] "25.3. unittest — Unit testing framework — Python 2.7.10 documentation". *python.org*. Retrieved 23 June 2015.
- [360] "Installation and quick start — nose 1.2.1 documentation". Somethingaboutorange.com. Retrieved 2012-11-12.
- [361] "pytest: helps you write better programs". *pytest.org*. Retrieved 23 June 2015.
- [362] "TwistedTrial – Twisted". Twistedmatrix.com. Retrieved 2012-11-12.
- [363] "Should-DSL documentation". *should-dsl.info*. Retrieved 23 June 2015.
- [364] "R Unit Test Framework | Free software downloads at". Sourceforge.net. Retrieved 2012-11-12.
- [365] "CRAN - Package testthat". Cran.r-project.org. 2012-06-27. Retrieved 2012-11-12.
- [366] "3 RackUnit API". Docs.racket-lang.org. Retrieved 2012-11-12.
- [367] Neil Van Dyke. "Overeasy: Racket Language Test Engine". Neilvandyke.org. Retrieved 2012-11-12.
- [368] "RBUnit is now Free!". LogicalVue. Retrieved 2012-11-12.
- [369] "REBOL.org". *rebol.org*.
- [370] "RPGUnit.org - Summary". *sourceforge.net*.

- [371] “Module: Test::Unit (Ruby 1.9.3)”. Ruby-doc.org. 2012-11-08. Retrieved 2012-11-12.
- [372] “Community, open source ruby on rails development”. thoughtbot. Retrieved 2012-11-12.
- [373] “Documentation for minitest (2.0.2)”. Rubydoc.info. Retrieved 2012-11-12.
- [374]
- [375] “Github page for TMF”. Github.com. Retrieved 2013-01-24.
- [376] “FUTS - Framework for Unit Testing SAS”. ThotWave. Retrieved 2012-11-12.
- [377] “ScUnit”. sasCommunity. 2008-10-26. Retrieved 2012-11-12.
- [378] “SASUnit | Free Development software downloads at”. Sourceforge.net. Retrieved 2012-11-12.
- [379] “ScalaTest”. scalatest.org.
- [380] “Rehersal - A testing framework for Scala”. sourceforge.net.
- [381] “scunit - A unit testing framework for Scala. - Google Project Hosting”. Code.google.com. Retrieved 2012-11-12.
- [382] “specs - a BDD library for Scala - Google Project Hosting”. Code.google.com. 2011-09-04. Retrieved 2012-11-12.
- [383] “scalacheck - A powerful tool for automatic unit testing - Google Project Hosting”. Code.google.com. Retrieved 2012-11-12.
- [384] “test_run - Launch tests”. Help.scilab.org. 2011-11-21. Retrieved 2012-11-12.
- [385] main.ss. “PLaneT Package Repository : PLaneT > schematics > schemeunit.plt”. Planet.plt-scheme.org. Retrieved 2012-11-12.
- [386] Neil Van Dyke. “Testeez: Lightweight Unit Test Mechanism for R5RS Scheme”. Neilvandyke.org. Retrieved 2012-11-12.
- [387] “lehmannro/assert.sh · GitHub”. Github.com. Retrieved 2012-11-12.
- [388] “sstephenson/bats · GitHub”. Github.com. Retrieved 2012-11-12.
- [389] shadowfen. “jshu”. SourceForge.
- [390] “Roundup - Prevent shell bugs. (And: Are you a model Unix citizen?) - It's Bonus”. Itsbonus.heroku.com. 2010-11-01. Retrieved 2012-11-12.
- [391] haran. “ShUnit”. sourceforge.net.
- [392] “shunit2 - shUnit2 - xUnit based unit testing for Unix shell scripts - Google Project Hosting”. Code.google.com. Retrieved 2012-11-12.
- [393] Stefan Merten. “filterunit”. Merten-home.de. Retrieved 2012-11-12.
- [394] http://mlunit.sourceforge.net/index.php/The_slUnit_Testing_Framework
- [395] “SQLUnit Project Home Page”. sourceforge.net.
- [396] “fitnesse.info”. fitnesse.info.
- [397] “STK Documentation”. wikidot.com.
- [398] “MyTAP”. github.com.
- [399] “utMySQL”. sourceforge.net.
- [400] “Welcome to the utPLSQL Project”. sourceforge.net.
- [401] “Code Tester for Oracle”. <http://software.dell.com/>. Retrieved 2014-02-13.
- [402] “Automated PL SQL Code Testing – Code Tester from Quest Software”. quest.com. Retrieved 2013-09-30.
- [403] “Unit Testing with SQL Developer”. Docs.oracle.com. Retrieved 2012-11-12.
- [404] “PL/Unit - Test Driven Development for Oracle”. plunit.com.
- [405] “pluto-test-framework - PL/SQL Unit Testing for Oracle - Google Project Hosting”. Code.google.com. Retrieved 2012-11-12.
- [406] “rsim/ruby-plsql-spec · GitHub”. Github.com. Retrieved 2012-11-12.
- [407] Jake Benilov. “DbFit”. benilovj.github.io.
- [408] “angoca/db2unit”. GitHub.
- [409] <http://www.epictest.org/>
- [410] “pgTAP”. pgtap.org.
- [411] “pgtools | Free Development software downloads at”. Sourceforge.net. Retrieved 2012-11-12.
- [412] “dkLab | Constructor | PGUnit: stored procedures unit-test framework for PostgreSQL 8.3”. En.dklab.ru. Retrieved 2012-11-12.
- [413] “tSQLt - Database Unit Testing for SQL Server”. tSQLt - Database Unit Testing for SQL Server.
- [414] Red Gate Software Ltd. “SQL Test - Unit Testing for SQL Server”. Red-gate.com. Retrieved 2012-11-12.
- [415] aevdokimenko. “TSQLUnit unit testing framework”. SourceForge.
- [416] “TSQLUnit”. Sourceforge.net. Retrieved 2012-11-12.
- [417] “utTSQL”. sourceforge.net.
- [418] “Download Visual Studio 2005 Team Edition for Database Professionals Add-on from Official Microsoft Download Center”. Microsoft.com. 2007-01-08. Retrieved 2012-11-12.
- [419] “Download Alcyone SQL Unit”. Retrieved 2014-08-18.

- [420] "T.S.T. the T-SQL Test Tool". *CodePlex*.
- [421] vassilvk (2012-06-15). "Home · vassilvk/slacker Wiki · GitHub". Github.com. Retrieved 2012-11-12.
- [422] "Quick/Quick". *GitHub*.
- [423] "railsware/Sleipnir". *GitHub*.
- [424] "SVUnit Sourceforge page". Retrieved 2014-05-06.
- [425] "Tcl Bundled Packages - tcltest manual page". Tcl.tk. Retrieved 2012-11-12.
- [426] "TclUnit | Free Development software downloads at". Sourceforge.net. Retrieved 2012-11-12.
- [427] "t-unit - a unit test framework for the tcl programming language - Google Project Hosting". Code.google.com. Retrieved 2012-11-12.
- [428] <http://www.lavalampmotemasters.com/>
- [429] "tsUnit - TypeScript Unit Testing Framework". *CodePlex*.
- [430] "Oscar - Test harness for TypeScript". *adriencadet.com*.
- [431] <http://www.foxunit.org/>
- [432] Maass Computertechnik. "vbUnit 3 - Unit Test Framework for Visual Basic and COM objects". *vbunit.com*.
- [433] <http://vbunitfree.sourceforge.net/>
- [434] "Vba Unit". C2.com. 2007-05-15. Retrieved 2012-11-12.
- [435] "excelvbaunit - xUnit type test harness for Excel VBA code - Google Project Hosting". Code.google.com. Retrieved 2012-11-12.
- [436] "TinyUnit: The Simplest Unit Test Framework that Can Possibly Work". W-p.dds.nl. Retrieved 2012-11-12.
- [437] "SimplyVBUnit". *sourceforge.net*.
- [438] "VB Lite Unit Home Page". *sourceforge.net*.
- [439] "vl-unit - Visual Lisp Unit testing framework - Google Project Hosting". Code.google.com. Retrieved 2012-11-12.
- [440] "RefleX". Reflex.gforge.inria.fr. Retrieved 2012-11-12.
- [441] "RefleX". Reflex.gforge.inria.fr. Retrieved 2012-11-12.
- [442] "vauto - Extensible - Data driven - Automation framework. - Google Project Hosting". Code.google.com. Retrieved 2012-11-12.
- [443] "Apache Ant - Apache AntUnit". Ant.apache.org. 2011-08-16. Retrieved 2012-11-12.
- [444] "juxy.tigris.org". *tigris.org*.
- [445] "Tennison Tests (XSLT Unit Testing) - Build the Site". *sourceforge.net*.
- [446] "Unit Testing Framework - XSLT". *sourceforge.net*.
- [447] "XSLTunit". *xsltunit.org*.
- [448] "expath/xspec". *GitHub*.
- [449] White, L.J. (27–30 Sep 1993). "Test Manager: A regression testing tool". *Software Maintenance ,1993. CSM-93, Proceedings., Conference on:* 338. doi:10.1109/ICSM.1993.366928. Retrieved 2012-11-12.
- [450] TriVir. "IdMUnit.org". *sourceforge.net*.

5.9.5 External links

- Oracle Unit Testing - tutorial site
- Other list of various unit testing frameworks
- OpenSourceTesting.org lists many unit testing frameworks, performance testing tools and other tools programmers/developers may find useful
- Testing Framework

5.10 SUnit

SUnit is a unit testing framework for the programming language Smalltalk. It is the original source of the xUnit design, originally written by the creator of Extreme Programming, Kent Beck. SUnit allows writing tests and checking results in Smalltalk. The resulting tests are very stable, but this method has the disadvantage that testers must be able to write simple Smalltalk programs.

5.10.1 History

SUnit was originally described by Beck in "Simple Smalltalk Testing: With Patterns" (1989), then published as chapter 30 "Simple Smalltalk Testing", in the book **Kent Beck's Guide to Better Smalltalk** by Kent Beck, Donald G. Firesmith (Editor) (Publisher: Cambridge University Press, Pub. Date: December 1998, ISBN 978-0-521-64437-2, 408pp)

5.10.2 External links

- Official website @ Camp Smalltalk
- SUnit @ Ward Cunningham's Wiki

5.11 JUnit

Not to be confused with G-Unit.

"JUnit" redirects here. For the Egyptian goddess, see Junit (goddess).

JUnit is a **unit testing framework** for the **Java programming language**. JUnit has been important in the development of **test-driven development**, and is one of a family of **unit testing frameworks** which is collectively known as **xUnit** that originated with **SUnit**.

JUnit is linked as a **JAR** at compile-time; the framework resides under package `junit.framework` for JUnit 3.8 and earlier, and under package `org.junit` for JUnit 4 and later.

A research survey performed in 2013 across 10,000 Java projects hosted on GitHub found that JUnit, (in a tie with `slf4j-api`), was the most commonly included external library. Each library was used by 30.7% of projects. ^[3]

5.11.1 Example of JUnit test fixture

A JUnit **test fixture** is a Java object. With older versions of JUnit, fixtures had to inherit from `junit.framework.TestCase`, but the new tests using JUnit 4 should not do this. ^[4] Test methods must be annotated by the **@Test annotation**. If the situation requires it, ^[5] it is also possible to define a method to execute before (or after) each (or all) of the test methods with the **@Before** (or **@After**) and **@BeforeClass** (or **@AfterClass**) annotations. ^[4]

```
import org.junit.*; public class TestFoobar { @BeforeClass public static void setUpClass() throws Exception {
// Code executed before the first test method } @Before public void setUp() throws Exception { // Code executed before each test } @Test public void testOneThing() { // Code that tests one thing } @Test public void testAnotherThing() { // Code that tests another thing } @Test public void testSomethingElse() { // Code that tests something else } @After public void tearDown() throws Exception { // Code executed after each test } @AfterClass public static void tearDownClass() throws Exception { // Code executed after the last test method } }
```

5.11.2 Ports

JUnit alternatives have been written in other languages including:

- **Actionscript** (FlexUnit)
- **Ada** (AUnit)
- **C** (CUnit)
- **C#** (NUnit)
- **C++** (CPPUnit, CxxTest)
- **Coldfusion** (MXUnit)
- **Erlang** (EUnit)

- **Eiffel (Auto-Test)** - JUnit inspired getest (from GoboSoft), which led to Auto-Test in Eiffel Studio.
- **Fortran** (fUnit, pFUnit)
- **Delphi** (DUnit)
- **Free Pascal** (FPCUnit)
- **Haskell** (HUnit)
- **JavaScript** (JSUnit)
- **Microsoft .NET** (NUnit)
- **Objective-C** (OCUnit)
- **OCaml** (OUnit)
- **Perl** (Test::Class and Test::Unit)
- **PHP** (PHPUnit)
- **Python** (PyUnit)
- **Qt** (QTestLib)
- **R** (RUnit)
- **Ruby** (Test::Unit)

5.11.3 See also

- **TestNG**, another test framework for Java
- **Mock object**, a technique used during unit testing
- **Mockito** and **PowerMock**, mocking extensions to JUnit

5.11.4 References

- [1] **JUnit Releases**
- [2] “Relicense JUnit from CPL to EPL”. Philippe Marschall. 18 May 2013. Retrieved 20 September 2013.
- [3] “We Analyzed 30,000 GitHub Projects – Here Are The Top 100 Libraries in Java, JS and Ruby”.
- [4] Kent Beck, Erich Gamma. “JUnit Cookbook”. `junit.sourceforge.net`. Retrieved 2011-05-21.
- [5] Kent Beck. “Expensive Setup Smell”. C2 Wiki. Retrieved 2011-11-28.

5.11.5 External links

- **Official website**
- **JUnit antipatterns** (developerWorks) and **JUnit antipatterns** (Exubero)
- **An early look at JUnit 4**
- **JUnit Presentation**
- **JUnit different APIs with Examples**
- **JUnit Tutorials**

5.12 CppUnit

CppUnit is a unit testing framework module for the C++ programming language. It allows unit-testing of C sources as well as C++ with minimal source modification. It was started around 2000 by Michael Feathers as a C++ port of JUnit for Windows and ported to Unix by Jerome Lacoste.^[2] The library is released under the GNU Lesser General Public License.

The framework runs tests in suites. Test result output is sent to a filter, the most basic being a simple pass or fail count printed out, or more advanced filters allowing XML output compatible with continuous integration reporting systems.^[3]

The project has been forked several times.^{[4][5]} The freedesktop.org version, maintained by Markus Mohrhard of the LibreOffice project (which uses CppUnit heavily), is actively maintained, and is used in Linux distributions such as Debian, Ubuntu, Gentoo and Arch.^[6]

5.12.1 See also

- List of unit testing frameworks

5.12.2 Further reading

- Madden, Blake (6 April 2006). "1.7: Using CPUnit to implement unit testing". In Dickheiser, Mike. *Game Programming Gems 6*. Charles River Media. ISBN 1-58450-450-1.

5.12.3 References

- [1] Mohrhard, Markus (12 November 2013). "Cppunit 1.13.2 released". Retrieved 18 November 2013.
- [2] Mohrhard, Markus. "CppUnit Documentation". freedesktop.org.
- [3] Jenkins plug-in for CppUnit and other Unit Test tools
- [4] freedesktop.org fork presented as CppUnit v1.13
- [5] fork presented as CppUnit2; not modified since 2009
- [6] Mohrhard, Markus (22 October 2013). "cppunit framework". *LibreOffice mailing list*. Retrieved 20 March 2014.

5.12.4 External links

- Official website (freedesktop.org version)

5.13 Test::More

Test::More is a unit testing module for Perl. Created and maintained by Michael G Schwern with help from Barrie Slaimaker, Tony Bowden, chromatic, Fergal Daly and perl-qa. Introduced in 2001 to replace Test.pm, Test::More simplified and re-energized the culture of testing in Perl leading to an explosion of new testing modules and a strongly test driven community.

Test::More is the most popular Perl testing module, as of this writing about 80% of all CPAN distributions make use of it. Unlike other testing systems, Test::More is not a framework but can be used in concert with other testing libraries via a shared Test::Builder object. As a result, Test::More provides only the baseline testing functions leaving other libraries to implement more specific and sophisticated functionality. This removes what would otherwise be a development bottleneck and allows a rich eco-system of specialized niche testing functions.

Test::More is not a complete testing framework. Rather, test programs written with Test::More output their results as TAP which can then either be interpreted by a human, or more usually run through a TAP parser such as Test::Harness. It is this separation between test program and test result interpreter via a common protocol which allows Perl programmers to develop so many different testing modules and use them in combination. Additionally, the TAP output can be stored and reinterpreted later providing a historical record of test results.

5.13.1 External links

- Test::More documentation
- Test::More tutorial

5.14 NUnit

NUnit is an open source unit testing framework for Microsoft .NET. It serves the same purpose as JUnit does in the Java world, and is one of many programs in the xUnit family.

5.14.1 Features

Every test case can be added to one or more categories, to allow for selective running.^[1]

5.14.2 Runners

NUnit provides a console runner (nunit-console.exe), which is used for batch execution of tests. The console runner works through the NUnit Test Engine, which provides it with the ability to load, explore and execute tests.

When tests are to be run in a separate process, the engine makes use of the nunit-agent program to run them.

The NUnitLite runner may be used in situations where a simpler runner is more suitable.

5.14.3 Assertions

NUnit provides a rich set of assertions as static methods of the Assert class. If an assertion fails, the method call does not return and an error is reported. If a test contains multiple assertions, any that follow the one that failed will not be executed. For this reason, it's usually best to try for one assertion per test.

Classical

Before NUnit 2.4, a separate method of the Assert class was used for each different assertion. It continues to be supported in NUnit, since many people prefer it.

Each assert method may be called without a message, with a simple text message or with a message and arguments. In the last case the message is formatted using the provided text and arguments.

```
// Equality asserts Assert.AreEqual(object expected,
object actual); Assert.AreEqual(object expected, ob-
ject actual, string message, params object[] parms);
Assert.AreNotEqual(object expected, object actual);
Assert.AreNotEqual(object expected, object actual,
string message, params object[] parms); // Identity
asserts Assert.AreSame(object expected, object
actual); Assert.AreSame(object expected, object
actual, string message, params object[] parms); As-
sert.AreNotSame(object expected, object actual);
Assert.AreNotSame(object expected, object actual,
string message, params object[] parms); // Condi-
tion asserts // (For simplicity, methods with mes-
sage signatures are omitted.) Assert.IsTrue(bool
condition); Assert.IsFalse(bool condition); As-
sert.IsNull(object anObject); Assert.IsNotNull(object
anObject); Assert.IsNaN(double aDouble); As-
sert.IsEmpty(string aString); Assert.IsNotEmpty(string
aString); Assert.IsEmpty(ICollection collection); As-
sert.IsNotEmpty(ICollection collection);
```

Constraint based

Beginning with NUnit 2.4, a new *Constraint-based* model was introduced. This approach uses a single method of the Assert class for all assertions, passing a Constraint object that specifies the test to be performed. This constraint-based model is now used internally by NUnit for all assertions. The methods of the classic approach have been re-implemented on top of this new model.

5.14.4 Example

Example of an NUnit **test fixture**:

```
using NUnit.Framework; [TestFixture] public class
ExampleTestOfNUnit { [Test] public void TestMulti-
plication() { Assert.AreEqual(4, 2*2, "Multiplication");
// Equivalently, since version 2.4 NUnit offers a
new and // more intuitive assertion syntax based on
constraint objects // [http://www.nunit.org/index.
php?p=constraintModel&r=2.4.7]: Assert.That(2*2,
Is.EqualTo(4), "Multiplication constraint-based"); }
} // The following example shows different ways of
writing the same exception test. [TestFixture] public
class AssertThrowsTests { [Test] public void Tests() { //
.NET 1.x Assert.Throws(typeof(ArgumentException),
new TestDelegate(MethodThatThrows)); // .NET 2.0 As-
sert.Throws<ArgumentException>(MethodThatThrows);
Assert.Throws<ArgumentException>( delegate { throw
new ArgumentException(); }); // Using C# 3.0 As-
sert.Throws<ArgumentException>( () => { throw new
ArgumentException(); }); } void MethodThatThrows()
{ throw new ArgumentException(); } } // This example
shows use of the return value to perform additional
verification of the exception. [TestFixture] public class
UsingReturnValue { [Test] public void TestException()
{ MyException ex = Assert.Throws<MyException>(
delegate { throw new MyException("message", 42);
}); Assert.That(ex.Message, Is.EqualTo("message"));
Assert.That(ex.MyParam, Is.EqualTo(42)); } } //
This example does the same thing using the over-
load that includes a constraint. [TestFixture] public
class UsingConstraint { [Test] public void TestExcep-
tion() { Assert.Throws(Is.TypeOf<MyException>()
.And.Message.EqualTo("message")
.And.Property("MyParam").EqualTo(42), delegate
{ throw new MyException("message", 42); }); } }
```

The NUnit framework discovers the method ExampleTestOfNUnit.TestMultiplication() automatically by reflection.

5.14.5 Extensions

FireBenchmarks is an **addin** able to record execution time of unit tests and generate **XML**, **CSV**, **XHTML** performances reports with charts and history tracking. Its main purpose is to enable a developer or a team that work with an **agile methodology** to integrate **performance metrics** and analysis into the **unit testing** environment, to easily control and monitor the evolution of a software system in terms of **algorithmic complexity** and system resources load.

NUnit.Forms is an expansion to the core NUnit framework and is also open source. It specifically looks at expanding NUnit to be able to handle testing user interface elements in **Windows Forms**. As of January 2013,

NUnit.Forms is in Alpha release, and no versions have been released since May 2006.

NUnit.ASP is a discontinued^[2] expansion to the core NUnit framework and is also open source. It specifically looks at expanding NUnit to be able to handle testing user interface elements in ASP.Net

5.14.6 See also

- [JUnit](#)
- [Test automation](#)

5.14.7 References

- [1] “CategoryAttribute - NUnit documentation”. Retrieved 2008-04-15.
- [2] “NUnit.ASP website main page”. Sourceforge. Retrieved 2008-04-15.

5.14.8 Further reading

- Andrew Hunt, David Thomas: *Pragmatic Unit Testing in C# with NUnit, 2nd Ed.* The Pragmatic Bookshelf, Raleigh 2007, ISBN 0-9776166-7-3
- Jim Newkirk, Alexei Vorontsov: *Test-Driven Development in Microsoft .NET.* Microsoft Press, Redmond 2004, ISBN 0-7356-1948-4
- Bill Hamilton: *NUnit Pocket Reference.* O'Reilly, Cambridge 2004, ISBN 0-596-00739-6

5.14.9 External links

- [Official website](#)
- [GitHub Site](#)
- [Launchpad Site](#) (no longer maintained)
- [Test-driven Development with NUnit & Test-driven.NET video demonstration](#)
- [NUnit.Forms home page](#)
- [NUnitAsp homepage](#)
- [Article Improving Application Quality Using Test-Driven Development](#) provides an introduction to TDD with concrete examples using NUnit
- [Open source tool, which can execute nunit tests in parallel](#)

5.15 NUnitAsp

NUnitAsp is a tool for automatically testing ASP.NET web pages. It's an extension to **NUnit**, a tool for [test-driven development](#) in .NET.

5.15.1 How It Works

NUnitAsp is a class library for use within your NUnit tests. It provides NUnit with the ability to download, parse, and manipulate ASP.NET web pages.

With NUnitASP, your tests don't need to know how ASP.NET renders controls into HTML. Instead, you can rely on the NUnitASP library to do this for you, keeping your test code simple and clean. For example, your tests don't need to know that a DataGrid control renders as an HTML table. You can rely on NUnitASP to handle the details. This gives you the freedom to focus on functionality questions, like whether the DataGrid holds the expected values.

```
[Test] public void TestExample() { // First, instantiate "Tester" objects:
    LabelTester label = new LabelTester("textLabel", CurrentWebForm);
    LinkButtonTester link = new LinkButtonTester("linkButton", CurrentWebForm);
    // Second, visit the page being tested:
    Browser.GetPage("http://localhost/example/example.aspx");
    // Third, use tester objects to test the page:
    AssertEquals("Not clicked.", label.Text);
    link.Click();
    AssertEquals("Clicked once.", label.Text);
    link.Click();
    AssertEquals("Clicked twice.", label.Text);
}
```

NUnitAsp can test complex web sites involving multiple pages and nested controls.

5.15.2 Credits & History

NUnitAsp was created by Brian Knowles as a simple way to read and manipulate web documents with NUnit. Jim Shore (known at the time as “Jim Little”) took over the project shortly afterwards and refactored it to the Tester-based approach used for the first release. Since then, more than a dozen people have contributed to the product. In November 2003, Levi Khatskevitch joined the team as “patch king” and brought new energy to the project, leading to the long-anticipated release of version 1.4. On January 31, 2008, Jim Shore announced the end of its development.

5.15.3 See also

- [NUnit](#)
- [Test automation](#)

5.15.4 External links

- [NunitAsp Homepage](#)

5.16 csUnit

csUnit is a unit testing framework for the .NET Framework. It is designed to work with any .NET compliant language. It has specifically been tested with C#, Visual Basic .NET, Managed C++, and J#. csUnit is open source and comes with a flexible license that allows cost-free inclusion in commercial closed-source products as well.

csUnit follows the concepts of other unit testing frameworks in the xUnit family and has had several releases since 2002. The tool offers a native GUI application, a command line, and addins for Visual Studio 2005 and Visual Studio 2008.

Starting with version 2.4 it also supports execution of NUnit tests without recompiling. This feature works for NUnit 2.4.7 (.NET 2.0 version).

csUnit supports .NET 3.5 and earlier versions, but does not support .NET 4.

csUnit has been integrated with ReSharper.

5.16.1 Special features

Along with the standard features, csUnit offers abilities that are uncommon in other unit testing frameworks for .NET:

- Categories to group included, excluded tests
- ExpectedException working with concrete instances rather than type only
- Out of the box addins for Visual Studio 2005 and 2008
- A tab for simple performance base lining
- A very rich set of assertions, continuously expanded
- Rich set of attributes for implementing tests
- Parameterized testing, data-driven testing
- Search abilities, saving time when test suites have thousands of tests

5.16.2 See also

- Test automation
- List of unit testing frameworks

5.16.3 External links

- www.csunit.org
- [SourceForge Site](#)

5.17 HtmlUnit

HtmlUnit is a headless web browser written in Java. It allows high-level manipulation of websites from other Java code, including filling and submitting forms and clicking hyperlinks. It also provides access to the structure and the details within received web pages. HtmlUnit emulates parts of browser behaviour including the lower-level aspects of TCP/IP and HTTP. A sequence such as getPage(url), getLinkWith("Click here"), click() allows a user to navigate through hypertext and obtain web pages that include HTML, JavaScript, Ajax and cookies. This headless browser can deal with HTTPS security, basic http authentication, automatic page redirection and other HTTP headers. It allows Java test code to examine returned pages either as text, an XML DOM, or as collections of forms, tables, and links.^[1]

The most common use of HtmlUnit is test automation of web pages, but sometimes it can be used for web scraping, or downloading website content.

Version 2.0 includes many new enhancements such as a W3C DOM implementation, Java 5 features, better XPath support, and improved handling for incorrect HTML, in addition to various JavaScript enhancements, while version 2.1 mainly focuses on tuning some performance issues reported by users.

5.17.1 See also

- Headless system
- PhantomJS a headless WebKit with JavaScript API
- CasperJS is a navigation scripting & testing utility for PhantomJS, written in JavaScript
- ENVJS is a simulated browser environment written in JavaScript
- Web scraping
- Web testing
- SimpleTest
- xUnit
- River Trail
- Selenium WebDriver

5.17.2 References

[1] [“HtmlUnit Home”](#). Retrieved 23 December 2010.

5.17.3 External links

- [HtmlUnit](#)

Chapter 6

Test automation

6.1 Test automation framework

See also: [Manual testing](#)

In [software testing](#), **test automation** is the use of special [software](#) (separate from the software being tested) to control the execution of tests and the comparison of actual outcomes with predicted outcomes.^[1] Test automation can automate some repetitive but necessary tasks in a formalized testing process already in place, or add additional testing that would be difficult to perform manually.

6.1.1 Overview

Some [software testing](#) tasks, such as extensive low-level interface [regression testing](#), can be laborious and time consuming to do manually. In addition, a manual approach might not always be effective in finding certain classes of defects. Test automation offers a possibility to perform these types of testing effectively. Once automated tests have been developed, they can be run quickly and repeatedly. Many times, this can be a cost-effective method for regression testing of software products that have a long maintenance life. Even minor patches over the lifetime of the application can cause existing features to break which were working at an earlier point in time.

There are many approaches to test automation, however below are the general approaches used widely:

- **Code-driven testing.** The [public \(usually\) interfaces](#) to classes, modules or libraries are tested with a variety of input arguments to validate that the results that are returned are correct.
- **Graphical user interface testing.** A testing framework generates user interface events such as keystrokes and mouse clicks, and observes the changes that result in the user interface, to validate that the observable behavior of the program is correct.
- **API driven testing.** A testing framework that uses a programming interface to the application to validate the behaviour under test. Typically API

driven testing bypasses application user interface altogether.

Test automation tools can be expensive, and are usually employed in combination with manual testing. Test automation can be made cost-effective in the long term, especially when used repeatedly in [regression testing](#).

In automated testing the Test Engineer or [Software quality assurance](#) person must have software coding ability, since the test cases are written in the form of source code which, when run, produce output according to the [assertions](#) that are a part of it.

One way to generate test cases automatically is [model-based testing](#) through use of a model of the system for test case generation, but research continues into a variety of alternative methodologies for doing so. In some cases, the model-based approach enables non-technical users to create automated business test cases in plain English so that no programming of any kind is needed in order to configure them for multiple operating systems, browsers, and smart devices.^[2]

What to automate, when to automate, or even whether one really needs automation are crucial decisions which the testing (or development) team must make. Selecting the correct features of the product for automation largely determines the success of the automation. Automating unstable features or features that are undergoing changes should be avoided.^[3]

6.1.2 Code-driven testing

A growing trend in software development is the use of testing frameworks such as the [xUnit](#) frameworks (for example, [JUnit](#) and [NUnit](#)) that allow the execution of [unit tests](#) to determine whether various sections of the [code](#) are acting as expected under various circumstances. [Test cases](#) describe tests that need to be run on the program to verify that the program runs as expected.

Code driven test automation is a key feature of [agile software development](#), where it is known as test-driven development (TDD). Unit tests are written to define the functionality *before* the code is written. However, these unit tests evolve and are extended as coding progresses, issues

are discovered and the code is subjected to refactoring.^[4] Only when all the tests for all the demanded features pass is the code considered complete. Proponents argue that it produces software that is both more reliable and less costly than code that is tested by manual exploration. It is considered more reliable because the code coverage is better, and because it is run constantly during development rather than once at the end of a **waterfall** development cycle. The developer discovers defects immediately upon making a change, when it is least expensive to fix. Finally, **code refactoring** is safer; transforming the code into a simpler form with less code duplication, but equivalent behavior, is much less likely to introduce new defects.

6.1.3 Graphical User Interface (GUI) testing

Many test automation tools provide record and playback features that allow users to interactively record user actions and replay them back any number of times, comparing actual results to those expected. The advantage of this approach is that it requires little or no **software development**. This approach can be applied to any application that has a **graphical user interface**. However, reliance on these features poses major reliability and maintainability problems. Relabelling a button or moving it to another part of the window may require the test to be re-recorded. Record and playback also often adds irrelevant activities or incorrectly records some activities.

A variation on this type of tool is for testing of web sites. Here, the “interface” is the web page. This type of tool also requires little or no software development. However, such a framework utilizes entirely different techniques because it is reading **HTML** instead of observing window events.

Another variation is scriptless test automation that does not use record and playback, but instead builds a model of the Application Under Test (AUT) and then enables the tester to create test cases by simply editing in test parameters and conditions. This requires no scripting skills, but has all the power and flexibility of a scripted approach. Test-case maintenance seems to be easy, as there is no code to maintain and as the AUT changes the software objects can simply be re-learned or added. It can be applied to any GUI-based software application. The problem is the model of the AUT is actually implemented using **test scripts**, which have to be constantly maintained whenever there's change to the AUT.

6.1.4 API driven testing

API testing is also being widely used by software testers due to the difficulty of creating and maintaining GUI-based automation testing.

Programmers or testers write scripts using a programming or scripting language that calls interface exposed by the application under test. These interfaces are custom built or commonly available interfaces like COM, HTTP, command line interface. The test scripts created are executed using an automation framework or a programming language to compare test results with expected behaviour of the application.

6.1.5 What to test

Testing tools can help automate tasks such as product installation, test data creation, GUI interaction, problem detection (consider parsing or polling agents equipped with oracles), defect logging, etc., without necessarily automating tests in an end-to-end fashion.

One must keep satisfying popular requirements when thinking of test automation:

- **Platform** and OS independence
- Data driven capability (Input Data, Output Data, Metadata)
- Customizable Reporting (DB Data Base Access, Crystal Reports)
- Easy debugging and logging
- **Version control** friendly – minimal binary files
- Extensible & Customizable (Open APIs to be able to integrate with other tools)
- Common Driver (For example, in the Java development ecosystem, that means **Ant** or **Maven** and the popular IDEs). This enables tests to integrate with the developers' **workflows**.
- Support unattended test runs for integration with build processes and batch runs. **Continuous integration** servers require this.
- Email Notifications like **bounce messages**
- Support distributed execution environment (distributed **test bed**)
- Distributed application support (distributed **SUT**)

6.1.6 Framework approach in automation

A test automation framework is an integrated system that sets the rules of automation of a specific product. This system integrates the function libraries, test data sources, object details and various reusable modules. These components act as small building blocks which need to be assembled to represent a business process. The framework provides the basis of test automation and simplifies the automation effort.

The main advantage of a **framework** of assumptions, concepts and tools that provide support for automated software testing is the low cost for **maintenance**. If there is change to any **test case** then only the test case file needs to be updated and the **driver Script** and **startup script** will remain the same. Ideally, there is no need to update the scripts in case of changes to the application.

Choosing the right framework/scripting technique helps in maintaining lower costs. The costs associated with test scripting are due to development and maintenance efforts. The approach of scripting used during test automation has effect on costs.

Various framework/scripting techniques are generally used:

1. Linear (procedural code, possibly generated by tools like those that use record and playback)
2. Structured (uses control structures - typically 'if-else', 'switch', 'for', 'while' conditions/ statements)
3. **Data-driven** (data is persisted outside of tests in a database, spreadsheet, or other mechanism)
4. **Keyword-driven**
5. Hybrid (two or more of the patterns above are used)
6. Agile automation framework

The Testing framework is responsible for:^[5]

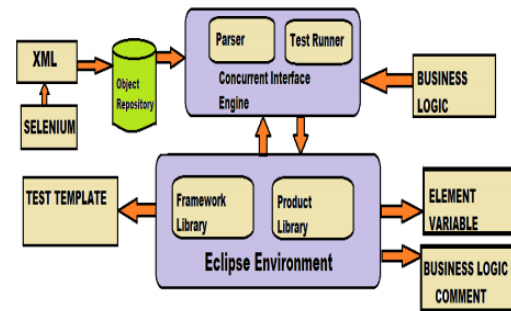
1. defining the format in which to express expectations
2. creating a mechanism to hook into or drive the application under test
3. executing the tests
4. reporting results

Test automation interface

Test automation interface are platforms that provide a single **workspace** for incorporating multiple testing tools and frameworks for **System/Integration testing** of application under test. The goal of Test Automation Interface is to simplify the process of mapping tests to business criteria without coding coming in the way of the process. Test automation interface are expected to improve the efficiency and flexibility of maintaining test scripts.^[6]

Test Automation Interface consists of the following core modules:

- Interface Engine
- Interface Environment
- Object Repository



Test Automation Interface Model

Interface engine

Interface engines are built on top of Interface Environment. Interface engine consists of a **parser** and a test runner. The parser is present to parse the object files coming from the object repository into the test specific scripting language. The test runner executes the test scripts using a **test harness**.^[6]

Object repository

Object repositories are a collection of UI/Application object data recorded by the testing tool while exploring the application under test.^[6]

6.1.7 Defining boundaries between automation framework and a testing tool

Tools are specifically designed to target some particular test environment, such as Windows and web automation tools, etc. Tools serve as a driving agent for an automation process. However, an automation framework is not a tool to perform a specific task, but rather an infrastructure that provides the solution where different tools can do their job in a unified manner. This provides a common platform for the automation engineer.

There are various types of frameworks. They are categorized on the basis of the automation component they leverage. These are:

1. **Data-driven testing**
2. **Modularity-driven testing**
3. **Keyword-driven testing**
4. **Hybrid testing**
5. **Model-based testing**
6. **Code driven testing**
7. **Behavior driven testing**

6.1.8 See also

- Software Testing portal
- List of GUI testing tools
- List of web testing tools
- Software testing
- System testing
- Unit test

6.1.9 References

- [1] Kolawa, Adam; Huizinga, Dorota (2007). *Automated Defect Prevention: Best Practices in Software Management*. Wiley-IEEE Computer Society Press. p. 74. ISBN 0-470-04212-5.
 - [2] "Proceedings from the 5th International Conference on Software Testing and Validation (ICST). Software Competence Center Hagenberg. "Test Design: Lessons Learned and Practical Implications.".
 - [3] Brian Marick. "When Should a Test Be Automated?". StickyMinds.com. Retrieved 2009-08-20.
 - [4] *Learning Test-Driven Development by Counting Lines*; Bas Vodde & Lasse Koskela; IEEE Software Vol. 24, Issue 3, 2007
 - [5] "Selenium Meet-Up 4/20/2010 Elisabeth Hendrickson on Robot Framework 1of2". Retrieved 2010-09-26.
 - [6] "Conquest: Interface for Test Automation Design" (PDF). Retrieved 2011-12-11.
- Elfriede Dustin et al. (1999). *Automated Software Testing*. Addison Wesley. ISBN 0-201-43287-0.
 - Elfriede Dustin et al. *Implementing Automated Software Testing*. Addison Wesley. ISBN 978-0-321-58051-1.
 - Mark Fewster & Dorothy Graham (1999). *Software Test Automation*. ACM Press/Addison-Wesley. ISBN 978-0-201-33140-0.
 - Roman Savenkov: *How to Become a Software Tester*. Roman Savenkov Consulting, 2008, ISBN 978-0-615-23372-7
 - Hong Zhu et al. (2008). *AST '08: Proceedings of the 3rd International Workshop on Automation of Software Test*. ACM Press. ISBN 978-1-60558-030-2.
 - Mosley, Daniel J.; Posey, Bruce. *Just Enough Software Test Automation*. ISBN 0130084689.
 - Hayes, Linda G., "Automated Testing Handbook", Software Testing Institute, 2nd Edition, March 2004
 - Kaner, Cem, "Architectures of Test Automation", August 2000

6.1.10 External links

- Practical Experience in Automated Testing
- Test Automation: Delivering Business Value
- Test Automation Snake Oil by James Bach
- When Should a Test Be Automated? by Brian Marick
- Guidelines for Test Automation framework
- Advanced Test Automation
- Success Factors for Keyword Driven Testing by Hans Buwalda

6.2 Test bench

A **test bench** or testing workbench is a virtual environment used to verify the correctness or soundness of a design or model, for example, a software product.

The term has its roots in the testing of electronic devices, where an engineer would sit at a lab bench with tools for measurement and manipulation, such as **oscilloscopes**, **multimeters**, soldering irons, wire cutters, and so on, and manually verify the correctness of the **device under test** (DUT).

In the context of software or firmware or hardware engineering, a test bench refers to an environment in which the product under development is tested with the aid of software and hardware tools. The suite of testing tools is often designed specifically for the product under test. The software may need to be modified slightly in some cases to work with the test bench but careful coding can ensure that the changes can be undone easily and without introducing bugs. ^[1]

6.2.1 Components of a test bench

A test bench has four components:

1. Input: The ece criteria or deliverables needed to perform work
2. Procedures to : The tasks or processes that will transform the input into the output
3. Procedures to check: The processes that determine that the output meets the standards
4. Output: The exit criteria or deliverables produced from the workbench

6.2.2 Kinds of test benches

The following types of test bench are the most common:

1. Stimulus only — Contains only the stimulus driver and DUT; does not contain any results verification.
2. Full test bench — Contains stimulus driver, known good results, and results comparison.
3. Simulator specific — The test bench is written in a simulator-specific format.
4. Hybrid test bench — Combines techniques from more than one test bench style.
5. Fast test bench — Test bench written to get ultimate speed from simulation.

6.2.3 An example of a software test bench

The tools used to automate the testing process in a test bench perform the following functions:

Test manager Manages the running of program tests; keeps track of test data, expected results and program facilities tested.

Test data generator Generates test data for the program to be tested.

Oracle Generates predictions of the expected test results; the oracle may be either previous program versions or prototype systems. Note that this is not Oracle Corporation, the database company.

File comparator Compares the results of the program tests with previous test results and records any differences in a document.

Report generator Provides report definition and generation facilities for the test results.

Dynamic analyzer Adds code to a program to count the number of times each statement has been executed. It generates an execution profile for the statements to show the number of times they are executed in the program run.

Simulator Simulates the testing environment where the software product is to be used.

6.2.4 References

[1] <http://www.marilynwolf.us/CaC3e/>

6.3 Test execution engine

A **test execution engine** is a type of software used to test software, hardware or complete systems.

Synonyms of test execution engine:

- Test executive
- Test manager

A test execution engine may appear in two forms:

- Module of a test software suite (test bench) or an integrated development environment
- Stand-alone application software

6.3.1 Concept

The test execution engine does not carry any information about the tested product. Only the test specification and the test data carries information about the tested product.

The test specification is software. Test specification is sometimes referred to as test sequence, which consists of test steps.

The test specification should be stored in the test repository in a text format (such as source code). Test data is sometimes generated by some test data generator tool. Test data can be stored in binary or text files. Test data should also be stored in the test repository together with the test specification.

Test specification is selected, loaded and executed by the test execution engine similarly, as application software is selected, loaded and executed by operation systems. The test execution engine should not operate on the tested object directly, but through plug-in modules similarly as an application software accesses devices through drivers which are installed on the operation system.

The difference between the concept of test execution engine and operation system is that the test execution engine monitors, presents and stores the status, results, time stamp, length and other information for every Test Step of a Test Sequence, but typically an operation system does not perform such profiling of a software execution.

Reasons for using a test execution engine:

- Test results are stored and can be viewed in a uniform way, independent of the type of the test
- Easier to keep track of the changes
- Easier to reuse components developed for testing

6.3.2 Functions

Main functions of a test execution engine:

- Select a test type to execute. Selection can be automatic or manual.
- Load the **specification** of the selected test type by opening a file from the local **file system** or downloading it from a **Server**, depending on where the **test repository** is stored.
- Execute the test through the use of testing tools (SW test) or instruments (HW test), while showing the progress and accepting control from the operator (for example to **Abort**)
- Present the outcome (such as Passed, Failed or Aborted) of test Steps and the complete Sequence to the operator
- Store the Test Results in report files

An advanced test execution engine may have additional functions, such as:

- Store the test results in a **Database**
- Load test result back from the **Database**
- Present the test results as raw data.
- Present the test results in a processed format. (**Statistics**)
- Authenticate the operators.

Advanced functions of the test execution engine maybe less important for software testing, but these advanced features could be essential when executing hardware/system tests.

6.3.3 Operations types

A test execution engine by executing a test specification, it may perform different types of operations on the product, such as:

- **Verification**
- **Calibration**
- **Programming**
 - Downloading **firmware** to the product's **nonvolatile memory** (Flash)
 - **Personalization**: programming with unique parameters, like a **serial number** or a **MAC address**

If the subject is a software, verification is the only possible operation.

6.4 Test stubs

In computer science, **test stubs** are programs that simulate the behaviors of software components (or modules) that a module undergoing tests depends on.

Test stubs are mainly used in incremental testing's **top-down approach**. Stubs are computer programs that act as temporary replacement for a called module and give the same output as the actual product or software.

6.4.1 Example

Consider a computer program that queries a database to obtain the sum price total of all products stored in the database. In this example, the query is slow and consumes a large number of system resources. This reduces the number of test runs per day. Secondly, tests may include values outside those currently in the database. The method (or call) used to perform this is *get_total()*. For testing purposes, the source code in *get_total()* can be temporarily replaced with a simple statement that returns a specific value. This would be a test stub.

Several testing frameworks are available, as is software that generates **test stubs** based on existing source code and testing requirements.

6.4.2 See also

- **Method stub**
- **Software testing**
- **Test Double**
- **Stub (distributed computing)**

6.4.3 References

- [1] Fowler, Martin (2007), *Mocks Aren't Stubs* (Online)

6.4.4 External links

- <http://xunitpatterns.com/Test%20Stub.html>

6.5 Testware

Generally speaking, **Testware** is a sub-set of **software** with a special purpose, that is, for software testing, especially for software testing automation. Automation testware for example is designed to be executed on automation frameworks. Testware is an umbrella term for all utilities and application software that serve in combination for testing a software package but not necessarily contribute to operational purposes. As such, testware is

not a standing configuration but merely a working environment for application software or subsets thereof.

It includes artifacts produced during the test process required to plan, design, and execute tests, such as documentation, scripts, inputs, expected results, set-up and clear-up procedures, files, databases, environment, and any additional software or utilities used in testing.^[1]

Testware is produced by both verification and validation testing methods. Like software, Testware includes codes and binaries as well as test cases, test plan, test report, etc. Testware should be placed under the control of a configuration management system, saved and faithfully maintained.

Compared to general software, testware is special because it has:

1. a different purpose
2. different metrics for quality and
3. different users

The different methods should be adopted when you develop testware with what you use to develop general software.

Testware is also referred as test tools in a narrow sense.^[2]

6.5.1 References

[1] Fewster, M.; Graham, D. (1999), *Software Test Automation, Effective use of test execution tools*, Addison-Wesley, ISBN 0-201-33140-3

[2] http://www.homeoftester.com/articles/what_is_testware.htm

6.5.2 See also

- Software

6.6 Test automation framework

See also: [Manual testing](#)

In **software testing**, **test automation** is the use of special **software** (separate from the software being tested) to control the execution of tests and the comparison of actual outcomes with predicted outcomes.^[1] Test automation can automate some repetitive but necessary tasks in a formalized testing process already in place, or add additional testing that would be difficult to perform manually.

6.6.1 Overview

Some **software testing** tasks, such as extensive low-level interface **regression testing**, can be laborious and time consuming to do manually. In addition, a manual approach might not always be effective in finding certain classes of defects. Test automation offers a possibility to perform these types of testing effectively. Once automated tests have been developed, they can be run quickly and repeatedly. Many times, this can be a cost-effective method for regression testing of software products that have a long maintenance life. Even minor patches over the lifetime of the application can cause existing features to break which were working at an earlier point in time.

There are many approaches to test automation, however below are the general approaches used widely:

- **Code-driven testing.** The **public** (usually) **interfaces** to classes, modules or libraries are tested with a variety of input arguments to validate that the results that are returned are correct.
- **Graphical user interface testing.** A testing framework generates user interface events such as keystrokes and mouse clicks, and observes the changes that result in the user interface, to validate that the observable behavior of the program is correct.
- **API driven testing.** A testing framework that uses a programming interface to the application to validate the behaviour under test. Typically API driven testing bypasses application user interface altogether.

Test automation tools can be expensive, and are usually employed in combination with manual testing. Test automation can be made cost-effective in the long term, especially when used repeatedly in **regression testing**.

In automated testing the Test Engineer or **Software quality assurance** person must have software coding ability, since the test cases are written in the form of source code which, when run, produce output according to the **assertions** that are a part of it.

One way to generate test cases automatically is **model-based testing** through use of a model of the system for test case generation, but research continues into a variety of alternative methodologies for doing so. In some cases, the model-based approach enables non-technical users to create automated business test cases in plain English so that no programming of any kind is needed in order to configure them for multiple operating systems, browsers, and smart devices.^[2]

What to automate, when to automate, or even whether one really needs automation are crucial decisions which the testing (or development) team must make. Selecting the correct features of the product for automation largely

determines the success of the automation. Automating unstable features or features that are undergoing changes should be avoided.^[3]

6.6.2 Code-driven testing

A growing trend in software development is the use of testing frameworks such as the **xUnit** frameworks (for example, **JUnit** and **NUnit**) that allow the execution of **unit tests** to determine whether various sections of the **code** are acting as expected under various circumstances. **Test cases** describe tests that need to be run on the program to verify that the program runs as expected.

Code driven test automation is a key feature of **agile software development**, where it is known as **test-driven development** (TDD). Unit tests are written to define the functionality *before* the code is written. However, these unit tests evolve and are extended as coding progresses, issues are discovered and the code is subjected to refactoring.^[4] Only when all the tests for all the demanded features pass is the code considered complete. Proponents argue that it produces software that is both more reliable and less costly than code that is tested by manual exploration. It is considered more reliable because the code coverage is better, and because it is run constantly during development rather than once at the end of a **waterfall** development cycle. The developer discovers defects immediately upon making a change, when it is least expensive to fix. Finally, **code refactoring** is safer; transforming the code into a simpler form with less code duplication, but equivalent behavior, is much less likely to introduce new defects.

6.6.3 Graphical User Interface (GUI) testing

Many test automation tools provide record and playback features that allow users to interactively record user actions and replay them back any number of times, comparing actual results to those expected. The advantage of this approach is that it requires little or no **software development**. This approach can be applied to any application that has a **graphical user interface**. However, reliance on these features poses major reliability and maintainability problems. Relabelling a button or moving it to another part of the window may require the test to be re-recorded. Record and playback also often adds irrelevant activities or incorrectly records some activities.

A variation on this type of tool is for testing of web sites. Here, the “interface” is the web page. This type of tool also requires little or no software development. However, such a framework utilizes entirely different techniques because it is reading **HTML** instead of observing window events.

Another variation is scriptless test automation that does

not use record and playback, but instead builds a model of the Application Under Test (AUT) and then enables the tester to create test cases by simply editing in test parameters and conditions. This requires no scripting skills, but has all the power and flexibility of a scripted approach. Test-case maintenance seems to be easy, as there is no code to maintain and as the AUT changes the software objects can simply be re-learned or added. It can be applied to any GUI-based software application. The problem is the model of the AUT is actually implemented using **test scripts**, which have to be constantly maintained whenever there’s change to the AUT.

6.6.4 API driven testing

API testing is also being widely used by software testers due to the difficulty of creating and maintaining GUI-based automation testing.

Programmers or testers write scripts using a programming or scripting language that calls interface exposed by the application under test. These interfaces are custom built or commonly available interfaces like COM, HTTP, command line interface. The test scripts created are executed using an automation framework or a programming language to compare test results with expected behaviour of the application.

6.6.5 What to test

Testing tools can help automate tasks such as product installation, test data creation, GUI interaction, problem detection (consider parsing or polling agents equipped with oracles), defect logging, etc., without necessarily automating tests in an end-to-end fashion.

One must keep satisfying popular requirements when thinking of test automation:

- **Platform** and OS independence
- Data driven capability (Input Data, Output Data, Metadata)
- Customizable Reporting (DB Data Base Access, Crystal Reports)
- Easy debugging and logging
- **Version control** friendly – minimal binary files
- Extensible & Customizable (Open APIs to be able to integrate with other tools)
- Common Driver (For example, in the Java development ecosystem, that means **Ant** or **Maven** and the popular IDEs). This enables tests to integrate with the developers’ **workflows**.

- Support unattended test runs for integration with build processes and batch runs. **Continuous integration** servers require this.
- Email Notifications like **bounce messages**
- Support distributed execution environment (distributed **test bed**)
- Distributed application support (distributed **SUT**)

6.6.6 Framework approach in automation

A test automation framework is an integrated system that sets the rules of automation of a specific product. This system integrates the function libraries, test data sources, object details and various reusable modules. These components act as small building blocks which need to be assembled to represent a business process. The framework provides the basis of test automation and simplifies the automation effort.

The main advantage of a **framework** of assumptions, concepts and tools that provide support for automated software testing is the low cost for **maintenance**. If there is change to any **test case** then only the test case file needs to be updated and the **driver Script** and **startup script** will remain the same. Ideally, there is no need to update the scripts in case of changes to the application.

Choosing the right framework/scripting technique helps in maintaining lower costs. The costs associated with test scripting are due to development and maintenance efforts. The approach of scripting used during test automation has effect on costs.

Various framework/scripting techniques are generally used:

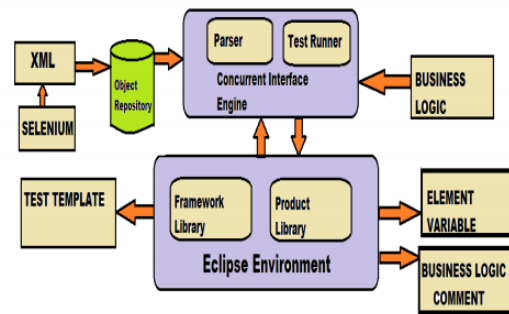
1. Linear (procedural code, possibly generated by tools like those that use record and playback)
2. Structured (uses control structures - typically 'if-else', 'switch', 'for', 'while' conditions/ statements)
3. **Data-driven** (data is persisted outside of tests in a database, spreadsheet, or other mechanism)
4. **Keyword-driven**
5. Hybrid (two or more of the patterns above are used)
6. Agile automation framework

The Testing framework is responsible for:^[5]

1. defining the format in which to express expectations
2. creating a mechanism to hook into or drive the application under test
3. executing the tests
4. reporting results

Test automation interface

Test automation interface are platforms that provide a single **workspace** for incorporating multiple testing tools and frameworks for **System/Integration testing** of application under test. The goal of Test Automation Interface is to simplify the process of mapping tests to business criteria without coding coming in the way of the process. Test automation interface are expected to improve the efficiency and flexibility of maintaining test scripts.^[6]



Test Automation Interface Model

Test Automation Interface consists of the following core modules:

- Interface Engine
- Interface Environment
- Object Repository

Interface engine

Interface engines are built on top of Interface Environment. Interface engine consists of a **parser** and a test runner. The parser is present to parse the object files coming from the object repository into the test specific scripting language. The test runner executes the test scripts using a **test harness**.^[6]

Object repository

Object repositories are a collection of UI/Application object data recorded by the testing tool while exploring the application under test.^[6]

6.6.7 Defining boundaries between automation framework and a testing tool

Tools are specifically designed to target some particular test environment, such as Windows and web automation tools, etc. Tools serve as a driving agent for an automation process. However, an automation framework is not a tool

to perform a specific task, but rather an infrastructure that provides the solution where different tools can do their job in a unified manner. This provides a common platform for the automation engineer.

There are various types of frameworks. They are categorized on the basis of the automation component they leverage. These are:

1. Data-driven testing
2. Modularity-driven testing
3. Keyword-driven testing
4. Hybrid testing
5. Model-based testing
6. Code driven testing
7. Behavior driven testing

6.6.8 See also

- Software Testing portal
- List of GUI testing tools
- List of web testing tools
- Software testing
- System testing
- Unit test

6.6.9 References

- [1] Kolawa, Adam; Huizinga, Dorota (2007). *Automated Defect Prevention: Best Practices in Software Management*. Wiley-IEEE Computer Society Press. p. 74. ISBN 0-470-04212-5.
- [2] "Proceedings from the 5th International Conference on Software Testing and Validation (ICST). Software Competence Center Hagenberg. "Test Design: Lessons Learned and Practical Implications."
- [3] Brian Marick. "When Should a Test Be Automated?". StickyMinds.com. Retrieved 2009-08-20.
- [4] *Learning Test-Driven Development by Counting Lines*; Bas Vodde & Lasse Koskela; IEEE Software Vol. 24, Issue 3, 2007
- [5] "Selenium Meet-Up 4/20/2010 Elisabeth Hendrickson on Robot Framework 1of2". Retrieved 2010-09-26.
- [6] "Conquest: Interface for Test Automation Design" (PDF). Retrieved 2011-12-11.
- Elfriede Dustin et al. (1999). *Automated Software Testing*. Addison Wesley. ISBN 0-201-43287-0.
- Elfriede Dustin et al. *Implementing Automated Software Testing*. Addison Wesley. ISBN 978-0-321-58051-1.
- Mark Fewster & Dorothy Graham (1999). *Software Test Automation*. ACM Press/Addison-Wesley. ISBN 978-0-201-33140-0.
- Roman Savenkov: *How to Become a Software Tester*. Roman Savenkov Consulting, 2008, ISBN 978-0-615-23372-7
- Hong Zhu et al. (2008). *AST '08: Proceedings of the 3rd International Workshop on Automation of Software Test*. ACM Press. ISBN 978-1-60558-030-2.
- Mosley, Daniel J.; Posey, Bruce. *Just Enough Software Test Automation*. ISBN 0130084689.
- Hayes, Linda G., "Automated Testing Handbook", Software Testing Institute, 2nd Edition, March 2004
- Kaner, Cem, "Architectures of Test Automation", August 2000

6.6.10 External links

- Practical Experience in Automated Testing
- Test Automation: Delivering Business Value
- Test Automation Snake Oil by James Bach
- When Should a Test Be Automated? by Brian Marick
- Guidelines for Test Automation framework
- Advanced Test Automation
- Success Factors for Keyword Driven Testing by Hans Buwalda

6.7 Data-driven testing

Data-driven testing (DDT) is a term used in the testing of computer software to describe testing done using a table of conditions directly as test inputs and verifiable outputs as well as the process where test environment settings and control are not hard-coded. In the simplest form the tester supplies the inputs from a row in the table and expects the outputs which occur in the same row. The table typically contains values which correspond to boundary or partition input spaces. In the control methodology, test configuration is "read" from a database.

6.7.1 Introduction

In the testing of **software** or **programs**, several methodologies are available for implementing this testing. Each of these methods co-exist because they differ in the effort required to create and subsequently maintain. The advantage of Data-driven testing is the ease to add additional inputs to the table when new partitions are discovered or added to the product or **System Under Test**. The cost aspect makes DDT cheap for automation but expensive for manual testing..

6.7.2 Methodology Overview

- **Data-driven testing** is the creation of test scripts to run together with their related data sets in a framework. The framework provides re-usable test logic to reduce maintenance and improve test coverage. Input and result (test criteria) data values can be stored in one or more central data sources or **databases**, the actual format and organisation can be implementation specific.

The data comprises variables used for both input values and output verification values. In advanced (mature) automation environments data can be harvested from a running system using a purpose-built custom tool or sniffer, the DDT framework thus performs playback of harvested data producing a powerful automated regression testing tool. Navigation through the **program**, reading of the data sources, and logging of test status and information are all coded in the test script.

6.7.3 Data Driven

Anything that has a potential to change (also called “variability,” and includes elements such as environment, end points, test data, locations, etc.) is separated out from the test logic (scripts) and moved into an 'external asset'. This can be a configuration or test dataset. The logic executed in the script is dictated by the data values. **Keyword-driven testing** is similar except that the test case is contained in the set of data values and not embedded or “hard-coded” in the test script itself. The script is simply a “driver” (or delivery mechanism) for the data that is held in the data source.

The databases used for data-driven testing can include:

- **Data pools**
- **ODBC sources**
- **CSV files**
- **Excel files**
- **DAO objects**
- **ADO objects**

6.7.4 See also

- **Control table**
- **Keyword-driven testing**
- **Test automation framework**
- **Test-driven development**
- **Metadata-driven testing**
- **Modularity-driven testing**
- **Hybrid testing**
- **Model-based testing**

6.7.5 References

- Carl Nagle: *Test Automation Frameworks*, Software Automation Framework Support on SourceForge

6.8 Modularity-driven testing

Modularity-driven testing is a term used in the testing of **software**.

6.8.1 Test Script Modularity Framework

The test script modularity framework requires the creation of small, independent scripts that represent modules, sections, and functions of the application-under-test. These small scripts are then used in a hierarchical fashion to construct larger tests, realizing a particular test case.

Of all the frameworks, this one should be the simplest to grasp and master. It is a well-known programming strategy to build an abstraction layer in front of a component to hide the component from the rest of the application. This insulates the application from modifications in the component and provides modularity in the application design. The test script modularity framework applies this principle of abstraction or encapsulation in order to improve the maintainability and scalability of automated test suites. ^[1]

6.8.2 References

- [1] Kelly, Michael. “Choosing a test automation framework”. Retrieved 2013-02-22.

6.9 Keyword-driven testing

Keyword-driven testing, also known as **table-driven testing** or **action word based testing**, is a **software testing** methodology suitable for both manual and automated

testing. This method separates the documentation of test cases -including the data to use- from the prescription of the way the test cases are executed. As a result it separates the test creation process into two distinct stages: a design and development stage, and an execution stage.

6.9.1 Overview

This methodology uses keywords (or action words) to symbolize a functionality to be tested, such as **Enter Client**. The keyword **Enter Client** is defined as the set of actions that must be executed to enter a new client in the database. Its keyword documentation would contain:

- the starting state of the system under test (SUT)
- the window or menu to start from
- the keys or mouse clicks to get to the correct data entry window
- the names of the fields to find and which arguments to enter
- the actions to perform in case additional dialogs pop up (like confirmations)
- the button to click to submit
- an assertion about what the state of the SUT should be after completion of the actions

Keyword-driven testing syntax lists test cases using a table format (see example below). The first column (column A) holds the keyword, Enter Client, which is the functionality being tested. Then the remaining columns, B-E, contain the data needed to execute the keyword: Name, Address, Postcode and City.

To enter another client, the tester would create another row in the table with Enter Client as the keyword and the new client's data in the following columns. There is no need to relist all the actions included.

6.9.2 Advantages

Keyword-driven testing reduces the sensitivity to maintenance caused by changes in the SUT. If screen layouts change or the system is migrated to another OS hardly any changes have to be made to the test cases: the changes will be made to the keyword documentation, one document for every keyword, no matter how many times the keyword is used in test cases. Also, due to the very detailed description of the way of executing the keyword (in the keyword documentation) the test can be performed by almost anyone. Thus keyword-driven testing can be used for both **manual testing** and **automated testing**.^[1]

6.9.3 Methodology

The keyword-driven testing methodology divides test process execution into several stages:

1. Test preparation: intake test basis etc.
2. Test design: analysis of test basis, test case design, test data design.
3. Manual test execution: manual execution of the test cases using keyword documentation as execution guideline.
4. Automation of test execution: creation of automated script that perform actions according to the keyword documentation.
5. Automated test execution.

6.9.4 Definition

A Keyword or Action Word is a defined combination of actions on a test object which describes how test lines must be executed. An action word contains arguments and is defined by a test analyst.

Automation of the test execution

The implementation stage differs depending on the tool or framework. Often, automation engineers implement a framework that provides keywords like “check” and “enter”.^[1] Testers or test designers (who do not need to know how to program) write **test cases** based on the keywords defined in the planning stage that have been implemented by the engineers. The test is executed using a driver that reads the keywords and executes the corresponding code.

Other methodologies use an all-in-one implementation stage. Instead of separating the tasks of test design and test engineering, the test design *is* the test automation. Keywords, such as “edit” or “check” are created using tools in which the necessary code has already been written. This removes the necessity for extra engineers in the test process, because the implementation for the keywords is already a part of the tool. Examples include GUIDancer and QTP.

Pros

- Maintenance is low in the long run:
 - Test cases are concise
 - Test cases are readable for the stake holders
 - Test cases easy to modify
 - New test cases can reuse existing keywords more easily

- Keyword re-use across multiple test cases
- Not dependent on a specific tool or programming language
- Division of Labor
 - Test case construction needs stronger domain expertise - lesser tool / programming skills
 - Keyword implementation requires stronger tool/programming skill - with relatively lower domain skill
- Abstraction of Layers

Cons

- Longer Time to Market (as compared to manual testing or record and replay technique)
- Moderately high learning curve initially

6.9.5 See also

- Data-driven testing
- Robot Framework
- Test Automation Framework
- Test-Driven Development
- TestComplete

6.9.6 References

- [1] Faight, Danny R. (November 2004). "Keyword-Driven Testing". *Sticky Minds*. Software Quality Engineering. Retrieved September 12, 2012.

6.9.7 External links

- Success Factors for Keyword Driven Testing, by Hans Buwalda
- SAFS (Software Automation Framework Support)
- Test automation frameworks
- Automation Framework - gFast: generic Framework for Automated Software Testing - QTP Framework

6.10 Hybrid testing

Hybrid testing is what most frameworks evolve into over time and multiple projects. The most successful automation frameworks generally accommodate both grammar and spelling as well as information input. This allows information given to be cross checked against existing and confirmed information. This helps to prevent false or misleading information being posted. It still however allows others to post new and relevant information to existing posts and so increases the usefulness and relevance of the site. This said, no system is perfect and it may not perform to this standard on all subjects all of the time but will improve with increasing input and increasing use.

6.10.1 Pattern

The Hybrid-Driven Testing pattern is made up of a number of reusable modules / function libraries that are developed with the following characteristics in mind:

- Maintainability – significantly reduces the test maintenance effort
- Reusability – due to modularity of test cases and library functions
- Manageability - effective test design, execution, and traceability
- Accessibility – to design, develop & modify tests whilst executing
- Availability – scheduled execution can run unattended on a 24/7 basis
- Reliability – due to advanced error handling and scenario recovery
- Flexibility – framework independent of system or environment under test
- Measurability – customisable reporting of test results ensure quality..

6.10.2 See also

- Control table
- Keyword-driven testing
- Test automation framework
- Test-driven development
- Modularity-driven testing
- Model-based testing

6.10.3 References

- Wright: *Hybrid Keyword Data Driven Frameworks*, ANZTB Conference 2010
- Dorothy Graham & Mark Fewster: *Experiences of Test Automation: Case Studies of Software Test Automation* Publication Date: January 19, 2012 ISBN 978-0321754066

6.11 Lightweight software test automation

Lightweight software test automation is the process of creating and using relatively short and simple computer programs, called lightweight test harnesses, designed to test a software system. Lightweight test automation harnesses are not tied to a particular programming language but are most often implemented with the Java, Perl, Visual Basic .NET, and C# programming languages. Lightweight test automation harnesses are generally four pages of source code or less, and are generally written in four hours or less. Lightweight test automation is often associated with Agile software development methodology.

The three major alternatives to the use of lightweight software test automation are commercial test automation frameworks, Open Source test automation frameworks, and heavyweight test automation. The primary disadvantage of lightweight test automation is manageability. Because lightweight automation is relatively quick and easy to implement, a test effort can be overwhelmed with harness programs, test case data files, test result files, and so on. However, lightweight test automation has significant advantages. Compared with commercial frameworks, lightweight automation is less expensive in initial cost and is more flexible. Compared with Open Source frameworks, lightweight automation is more stable because there are fewer updates and external dependencies. Compared with heavyweight test automation, lightweight automation is quicker to implement and modify. Lightweight test automation is generally used to complement, not replace these alternative approaches.

Lightweight test automation is most useful for regression testing, where the intention is to verify that new source code added to the system under test has not created any new software failures. Lightweight test automation may be used for other areas of software testing such as performance testing, stress testing, load testing, security testing, code coverage analysis, mutation testing, and so on. The most widely published proponent of the use of lightweight software test automation is Dr. James D. McCaffrey.

6.11.1 References

- Definition and characteristics of lightweight software test automation in: McCaffrey, James D., ".NET Test Automation Recipes", Apress Publishing, 2006. ISBN 1-59059-663-3.
- Discussion of lightweight test automation versus manual testing in: Patton, Ron, "Software Testing, 2nd ed.", Sams Publishing, 2006. ISBN 0-672-32798-8.
- An example of lightweight software test automation for .NET applications: "Lightweight UI Test Automation with .NET", MSDN Magazine, January 2005 (Vol. 20, No. 1). See <http://msdn2.microsoft.com/en-us/magazine/cc163864.aspx>.
- A demonstration of lightweight software test automation applied to stress testing: "Stress Testing", MSDN Magazine, May 2006 (Vol. 21, No. 6). See <http://msdn2.microsoft.com/en-us/magazine/cc163613.aspx>.
- A discussion of lightweight software test automation for performance testing: "Web App Diagnostics: Lightweight Automated Performance Analysis", asp.netPRO Magazine, August 2005 (Vol. 4, No. 8).
- An example of lightweight software test automation for Web applications: "Lightweight UI Test Automation for ASP.NET Web Applications", MSDN Magazine, April 2005 (Vol. 20, No. 4). See <http://msdn2.microsoft.com/en-us/magazine/cc163814.aspx>.
- A technique for mutation testing using lightweight software test automation: "Mutant Power: Create a Simple Mutation Testing System with the .NET Framework", MSDN Magazine, April 2006 (Vol. 21, No. 5). See <http://msdn2.microsoft.com/en-us/magazine/cc163619.aspx>.
- An investigation of lightweight software test automation in a scripting environment: "Lightweight Testing with Windows PowerShell", MSDN Magazine, May 2007 (Vol. 22, No. 5). See <http://msdn2.microsoft.com/en-us/magazine/cc163430.aspx>.

6.11.2 See also

- Test automation
- Microsoft Visual Test
- iMacros
- Software Testing

Chapter 7

Testing process

7.1 Software testing controversies

There is considerable **variety** among **software testing** writers and consultants about what constitutes responsible software testing. Members of the “context-driven” school of testing^[1] believe that there are no “best practices” of testing, but rather that testing is a set of skills that allow the tester to select or invent testing practices to suit each unique situation. In addition, prominent members of the community consider much of the writing about software testing to be doctrine, mythology, and folklore. Some contend that this belief directly contradicts standards such as the **IEEE 829** test documentation standard, and organizations such as the **Food and Drug Administration** who promote them. The context-driven school’s retort is that Lessons Learned in Software Testing includes one lesson supporting the use IEEE 829 and another opposing it; that not all software testing occurs in a regulated environment and that practices appropriate for such environments would be ruinously expensive, unnecessary, and inappropriate for other contexts; and that in any case the FDA generally promotes the principle of the least burdensome approach.

Some of the major controversies include:

7.1.1 Agile vs. traditional

Starting around 1990, a new style of writing about testing began to challenge what had come before. The seminal work in this regard is widely considered to be *Testing Computer Software*, by **Cem Kaner**.^[2] Instead of assuming that testers have full access to source code and complete specifications, these writers, including Kaner and **James Bach**, argued that testers must learn to work under conditions of uncertainty and constant change. Meanwhile, an opposing trend toward process “maturity” also gained ground, in the form of the **Capability Maturity Model**. The agile testing movement (which includes but is not limited to forms of testing practiced on **agile development** projects) has popularity mainly in commercial circles, whereas the CMM was embraced by government and military software providers.

However, saying that “maturity models” like CMM

gained ground against or opposing Agile testing may not be right. Agile movement is a ‘way of working’, while CMM is a process improvement idea.

But another point of view must be considered: the operational culture of an organization. While it may be true that testers must have an ability to work in a world of uncertainty, it is also true that their flexibility must have direction. In many cases test cultures are self-directed and as a result fruitless, unproductive results can ensue. Furthermore, providing positive evidence of defects may either indicate that you have found the tip of a much larger problem, or that you have exhausted all possibilities. A framework is a test of Testing. It provides a boundary that can measure (validate) the capacity of our work. Both sides have, and will continue to argue the virtues of their work. The proof however is in each and every assessment of delivery quality. It does little good to test systematically if you are too narrowly focused. On the other hand, finding a bunch of errors is not an indicator that Agile methods was the driving force; you may simply have stumbled upon an obviously poor piece of work.

7.1.2 Exploratory vs. scripted

Exploratory testing means simultaneous test design and test execution with an emphasis on learning. Scripted testing means that learning and test design happen prior to test execution, and quite often the learning has to be done again during test execution. Exploratory testing is very common, but in most writing and training about testing it is barely mentioned and generally misunderstood. Some writers consider it a primary and essential practice. Structured exploratory testing is a compromise when the testers are familiar with the software. A vague test plan, known as a test charter, is written up, describing what functionalities need to be tested but not how, allowing the individual testers to choose the method and steps of testing.

There are two main disadvantages associated with a primarily exploratory testing approach. The first is that there is no opportunity to prevent defects, which can happen when the designing of tests in advance serves as a form of structured static testing that often reveals problems

in system requirements and design. The second is that, even with test charters, demonstrating test coverage and achieving repeatability of tests using a purely exploratory testing approach is difficult. For this reason, a blended approach of scripted and exploratory testing is often used to reap the benefits while mitigating each approach's disadvantages.

7.1.3 Manual vs. automated

Some writers believe that **test automation** is so expensive relative to its value that it should be used sparingly.^[3] Others, such as advocates of **agile development**, recommend automating 100% of all tests. A challenge with automation is that automated testing requires automated test oracles (an oracle is a mechanism or principle by which a problem in the software can be recognized). Such tools have value in load testing software (by signing on to an application with hundreds or thousands of instances simultaneously), or in checking for intermittent errors in software. The success of automated software testing depends on complete and comprehensive test planning. Software development strategies such as **test-driven development** are highly compatible with the idea of devoting a large part of an organization's testing resources to automated testing. Many large software organizations perform automated testing. Some have developed their own automated testing environments specifically for internal development, and not for resale.

7.1.4 Software design vs. software implementation

Ideally, software testers should not be limited only to testing software implementation, but also to testing software design. With this assumption, the role and involvement of testers will change dramatically. In such an environment, the test cycle will change too. To test software design, testers would review requirement and design specifications together with designer and programmer, potentially helping to identify bugs earlier in software development.

7.1.5 Who watches the watchmen?

One principle in software testing is summed up by the classical Latin question posed by Juvenal: *Quis Custodiet Ipsos Custodes* (Who watches the watchmen?), or is alternatively referred informally, as the "Heisenbug" concept (a common misconception that confuses **Heisenberg's uncertainty principle** with **observer effect**). The idea is that any form of observation is also an interaction, that the act of testing can also affect that which is being tested.

In practical terms the test engineer is testing software (and sometimes hardware or firmware) with other software (and hardware and firmware). The process can fail

in ways that are not the result of defects in the target but rather result from defects in (or indeed intended features of) the testing tool.

There are metrics being developed to measure the effectiveness of testing. One method is by analyzing **code coverage** (this is highly controversial) - where everyone can agree what areas are not being covered at all and try to improve coverage in these areas.

Bugs can also be placed into code on purpose, and the number of bugs that have not been found can be predicted based on the percentage of intentionally placed bugs that were found. The problem is that it assumes that the intentional bugs are the same type of bug as the unintentional ones.

Finally, there is the analysis of historical find-rates. By measuring how many bugs are found and comparing them to predicted numbers (based on past experience with similar projects), certain assumptions regarding the effectiveness of testing can be made. While not an absolute measurement of quality, if a project is halfway complete and there have been no defects found, then changes may be needed to the procedures being employed by QA.

7.1.6 References

- [1] context-driven-testing.com
- [2] Kaner, Cem; Jack Falk; Hung Quoc Nguyen (1993). *Testing Computer Software* (Third ed.). John Wiley and Sons. ISBN 1-85032-908-7.
- [3] An example is Mark Fewster, Dorothy Graham: *Software Test Automation*. Addison Wesley, 1999, ISBN 0-201-33140-3

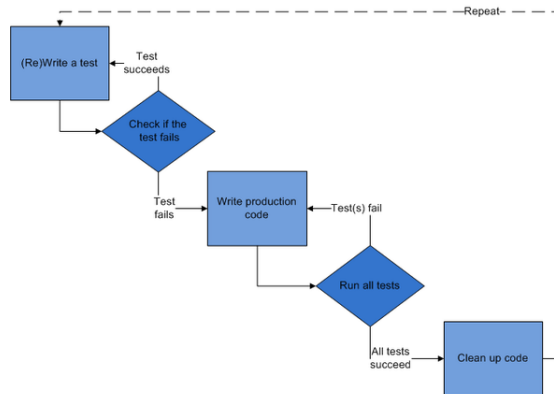
7.2 Test-driven development

Test-driven development (TDD) is a **software development process** that relies on the repetition of a very short development cycle: first the developer writes an (initially failing) automated **test case** that defines a desired improvement or new function, then produces the minimum amount of code to pass that test, and finally **refactors** the new code to acceptable standards. **Kent Beck**, who is credited with having developed or 'rediscovered'^[1] the technique, stated in 2003 that TDD encourages simple designs and inspires confidence.^[2]

Test-driven development is related to the test-first programming concepts of **extreme programming**, begun in 1999,^[3] but more recently has created more general interest in its own right.^[4]

Programmers also apply the concept to improving and **debugging legacy code** developed with older techniques.^[5]

7.2.1 Test-driven development cycle



A graphical representation of the development cycle, using a basic flowchart

The following sequence is based on the book *Test-Driven Development by Example*.^[2]

1. Add a test

In test-driven development, each new feature begins with writing a test. To write a test, the developer must clearly understand the feature's specification and requirements. The developer can accomplish this through *use cases* and *user stories* to cover the requirements and exception conditions, and can write the test in whatever testing framework is appropriate to the software environment. It could be a modified version of an existing test. This is a differentiating feature of test-driven development versus writing unit tests *after* the code is written: it makes the developer focus on the requirements *before* writing the code, a subtle but important difference.

2. Run all tests and see if the new one fails

This validates that the *test harness* is working correctly, that the new test does not mistakenly pass without requiring any new code, and that the required feature does not already exist. This step also tests the test itself, in the negative: it rules out the possibility that the new test always passes, and therefore is worthless. The new test should also fail for the expected reason. This step increases the developer's confidence that the unit test is testing the correct constraint, and passes only in intended cases.

3. Write some code

The next step is to write some code that causes the test to pass. The new code written at this stage is not perfect and may, for example, pass the test in an inelegant way. That is acceptable because it will be improved and honed in Step 5.

At this point, the only purpose of the written code is to pass the test; no further (and therefore untested) functionality should be predicted nor 'allowed for' at any stage.

4. Run tests

If all test cases now pass, the programmer can be confident that the new code meets the test requirements, and does not break or degrade any existing features. If they do not, the new code must be adjusted until they do.

5. Refactor code

The growing code base must be *cleaned up* regularly during test-driven development. New code can be moved from where it was convenient for passing a test to where it more logically belongs. *Duplication* must be removed. *Object*, *class*, *module*, *variable* and *method* names should clearly represent their current purpose and use, as extra functionality is added. As features are added, method bodies can get longer and other objects larger. They benefit from being split and their parts carefully named to improve *readability* and *maintainability*, which will be increasingly valuable later in the *software lifecycle*. *Inheritance hierarchies* may be rearranged to be more logical and helpful, and perhaps to benefit from recognised *design patterns*. There are specific and general guidelines for refactoring and for creating clean code.^{[6][7]} By continually re-running the test cases throughout each refactoring phase, the developer can be confident that process is not altering any existing functionality.

The concept of removing duplication is an important aspect of any software design. In this case, however, it also applies to the removal of any duplication between the test code and the production code—for example *magic numbers* or strings repeated in both to make the test pass in Step 3.

Repeat

Starting with another new test, the cycle is then repeated to push forward the functionality. The size of the steps should always be small, with as few as 1 to 10 edits between each test run. If new code does not rapidly satisfy a new test, or other tests fail unexpectedly, the programmer should *undo* or *revert* in preference to excessive *debugging*. *Continuous integration* helps by providing *reversible checkpoints*. When using external libraries it is important not to make increments that are so small as to be effectively merely testing the library itself,^[4] unless there is some reason to believe that the library is buggy or is not sufficiently feature-complete to serve all the needs of the software under development.

7.2.2 Development style

There are various aspects to using test-driven development, for example the principles of “keep it simple, stupid” (KISS) and “You aren't gonna need it” (YAGNI). By focusing on writing only the code necessary to pass tests, designs can often be cleaner and clearer than is achieved by other methods.^[2] In *Test-Driven Development by Example*, Kent Beck also suggests the principle “Fake it till you make it”.

To achieve some advanced design concept such as a *design pattern*, tests are written that generate that design. The code may remain simpler than the target pattern, but still pass all required tests. This can be unsettling at first but it allows the developer to focus only on what is important.

Writing the tests first: The tests should be written before the functionality that is to be tested. This has been claimed to have many benefits. It helps ensure that the application is written for testability, as the developers must consider how to test the application from the outset rather than adding it later. It also ensures that tests for every feature get written. Additionally, writing the tests first leads to a deeper and earlier understanding of the product requirements, ensures the effectiveness of the test code, and maintains a continual focus on *software quality*.^[8] When writing feature-first code, there is a tendency by developers and organisations to push the developer on to the next feature, even neglecting testing entirely. The first TDD test might not even compile at first, because the classes and methods it requires may not yet exist. Nevertheless, that first test functions as the beginning of an executable specification.^[9]

Each test case fails initially: This ensures that the test really works and can catch an error. Once this is shown, the underlying functionality can be implemented. This has led to the “test-driven development mantra”, which is “red/green/refactor,” where red means *fail* and green means *pass*. Test-driven development constantly repeats the steps of adding test cases that fail, passing them, and refactoring. Receiving the expected test results at each stage reinforces the developer’s mental model of the code, boosts confidence and increases productivity.

Keep the unit small

For TDD, a unit is most commonly defined as a class, or a group of related functions often called a module. Keeping units relatively small is claimed to provide critical benefits, including:

- Reduced debugging effort – When test failures are detected, having smaller units aids in tracking down errors.
- Self-documenting tests – Small test cases are easier to read and to understand.^[8]

Advanced practices of test-driven development can lead to *Acceptance test-driven development* (ATDD) and *Specification by example* where the criteria specified by the customer are automated into acceptance tests, which then drive the traditional unit test-driven development (UTDD) process.^[10] This process ensures the customer has an automated mechanism to decide whether the software meets their requirements. With ATDD, the development team now has a specific target to satisfy – the acceptance tests – which keeps them continuously focused on what the customer really wants from each user story.

7.2.3 Best practices

Test structure

Effective layout of a test case ensures all required actions are completed, improves the readability of the test case, and smooths the flow of execution. Consistent structure helps in building a self-documenting test case. A commonly applied structure for test cases has (1) setup, (2) execution, (3) validation, and (4) cleanup.

- Setup: Put the Unit Under Test (UUT) or the overall test system in the state needed to run the test.
- Execution: Trigger/drive the UUT to perform the target behavior and capture all output, such as return values and output parameters. This step is usually very simple.
- Validation: Ensure the results of the test are correct. These results may include explicit outputs captured during execution or state changes in the UUT & UAT.
- Cleanup: Restore the UUT or the overall test system to the pre-test state. This restoration permits another test to execute immediately after this one.^[8]

Individual best practices

Individual best practices states that one should:

- Separate common set up and teardown logic into test support services utilized by the appropriate test cases.
- Keep each *test oracle* focused on only the results necessary to validate its test.
- Design time-related tests to allow tolerance for execution in non-real time operating systems. The common practice of allowing a 5-10 percent margin for late execution reduces the potential number of false negatives in test execution.

- Treat your test code with the same respect as your production code. It also must work correctly for both positive and negative cases, last a long time, and be readable and maintainable.
- Get together with your team and review your tests and test practices to share effective techniques and catch bad habits. It may be helpful to review this section during your discussion.^[11]

Practices to avoid, or “anti-patterns”

- Having test cases depend on system state manipulated from previously executed test cases.
- Dependencies between test cases. A test suite where test cases are dependent upon each other is brittle and complex. Execution order should not be presumed. Basic refactoring of the initial test cases or structure of the UUT causes a spiral of increasingly pervasive impacts in associated tests.
- Interdependent tests. Interdependent tests can cause cascading false negatives. A failure in an early test case breaks a later test case even if no actual fault exists in the UUT, increasing defect analysis and debug efforts.
- Testing precise execution behavior timing or performance.
- Building “all-knowing oracles.” An oracle that inspects more than necessary is more expensive and brittle over time. This very common error is dangerous because it causes a subtle but pervasive time sink across the complex project.^[11]
- Testing implementation details.
- Slow running tests.

7.2.4 Benefits

A 2005 study found that using TDD meant writing more tests and, in turn, programmers who wrote more tests tended to be more productive.^[12] Hypotheses relating to code quality and a more direct correlation between TDD and productivity were inconclusive.^[13]

Programmers using pure TDD on new (“greenfield”) projects reported they only rarely felt the need to invoke a **debugger**. Used in conjunction with a **version control system**, when tests fail unexpectedly, reverting the code to the last version that passed all tests may often be more productive than debugging.^[14]

Test-driven development offers more than just simple validation of correctness, but can also drive the design of a program.^[15] By focusing on the test cases first, one must imagine how the functionality is used by clients (in the first case, the test cases). So, the programmer is

concerned with the interface before the implementation. This benefit is complementary to **Design by Contract** as it approaches code through test cases rather than through mathematical assertions or preconceptions.

Test-driven development offers the ability to take small steps when required. It allows a programmer to focus on the task at hand as the first goal is to make the test pass. Exceptional cases and error handling are not considered initially, and tests to create these extraneous circumstances are implemented separately. Test-driven development ensures in this way that all written code is covered by at least one test. This gives the programming team, and subsequent users, a greater level of confidence in the code.

While it is true that more code is required with TDD than without TDD because of the unit test code, the total code implementation time could be shorter based on a model by Müller and Padberg.^[16] Large numbers of tests help to limit the number of defects in the code. The early and frequent nature of the testing helps to catch defects early in the development cycle, preventing them from becoming endemic and expensive problems. Eliminating defects early in the process usually avoids lengthy and tedious debugging later in the project.

TDD can lead to more modularized, flexible, and extensible code. This effect often comes about because the methodology requires that the developers think of the software in terms of small units that can be written and tested independently and integrated together later. This leads to smaller, more focused classes, looser **coupling**, and cleaner interfaces. The use of the **mock object** design pattern also contributes to the overall modularization of the code because this pattern requires that the code be written so that modules can be switched easily between mock versions for unit testing and “real” versions for deployment.

Because no more code is written than necessary to pass a failing test case, automated tests tend to cover every code path. For example, for a TDD developer to add an else branch to an existing if statement, the developer would first have to write a failing test case that motivates the branch. As a result, the automated tests resulting from TDD tend to be very thorough: they detect any unexpected changes in the code’s behaviour. This detects problems that can arise where a change later in the development cycle unexpectedly alters other functionality.

Madeyski^[17] provided an empirical evidence (via a series of laboratory experiments with over 200 developers) regarding the superiority of the TDD practice over the classic Test-Last approach, with respect to the lower coupling between objects (CBO). The mean effect size represents a medium (but close to large) effect on the basis of meta-analysis of the performed experiments which is a substantial finding. It suggests a better modularization (i.e., a more modular design), easier reuse and testing of the developed software products due to the TDD pro-

gramming practice.^[17] Madeyski also measured the effect of the TDD practice on unit tests using branch coverage (BC) and mutation score indicator (MSI),^{[18][19][20]} which are indicators of the thoroughness and the fault detection effectiveness of unit tests, respectively. The effect size of TDD on branch coverage was medium in size and therefore is considered substantive effect.^[17]

7.2.5 Limitations

Test-driven development does not perform sufficient testing in situations where full functional tests are required to determine success or failure due to extensive use of unit tests.^[21] Examples of these are **user interfaces**, programs that work with **databases**, and some that depend on specific **network** configurations. TDD encourages developers to put the minimum amount of code into such modules and to maximize the logic that is in testable library code, using fakes and **mocks** to represent the outside world.^[22]

Management support is essential. Without the entire organization believing that test-driven development is going to improve the product, management may feel that time spent writing tests is wasted.^[23]

Unit tests created in a test-driven development environment are typically created by the developer who is writing the code being tested. Therefore, the tests may share blind spots with the code: if, for example, a developer does not realize that certain input parameters must be checked, most likely neither the test nor the code will verify those parameters. Another example: if the developer misinterprets the requirements for the module he is developing, the code and the unit tests he writes will both be wrong in the same way. Therefore, the tests will pass, giving a false sense of correctness.

A high number of passing unit tests may bring a false sense of security, resulting in fewer additional **software testing** activities, such as **integration testing** and **compliance testing**.

Tests become part of the maintenance overhead of a project. Badly written tests, for example ones that include hard-coded error strings or are themselves prone to failure, are expensive to maintain. This is especially the case with **fragile tests**.^[24] There is a risk that tests that regularly generate false failures will be ignored, so that when a real failure occurs, it may not be detected. It is possible to write tests for low and easy maintenance, for example by the reuse of error strings, and this should be a goal during the **code refactoring** phase described above.

Writing and maintaining an excessive number of tests costs time. Also, more-flexible modules (with limited tests) might accept new requirements without the need for changing the tests. For those reasons, testing for only extreme conditions, or a small sample of data, can be easier to adjust than a set of highly detailed tests. However, developers could be warned about overtesting to avoid the

excessive work, but it might require advanced skills in **sampling** or **factor analysis**.

The level of coverage and testing detail achieved during repeated TDD cycles cannot easily be re-created at a later date. Therefore these original, or early, tests become increasingly precious as time goes by. The tactic is to fix it early. Also, if a poor architecture, a poor design, or a poor testing strategy leads to a late change that makes dozens of existing tests fail, then it is important that they are individually fixed. Merely deleting, disabling or rashly altering them can lead to undetectable holes in the test coverage.

7.2.6 Test-driven work

Test-driven development has been adopted outside of software development, in both product and service teams, as test-driven work.^[25] Similar to TDD, non-software teams develop **quality control** checks (usually manual tests rather than automated tests) for each aspect of the work prior to commencing. These QC checks are then used to inform the design and validate the associated outcomes. The six steps of the TDD sequence are applied with minor semantic changes:

1. “Add a check” replaces “Add a test”
2. “Run all checks” replaces “Run all tests”
3. “Do the work” replaces “Write some code”
4. “Run all checks” replaces “Run tests”
5. “Clean up the work” replaces “Refactor code”
6. “Repeat”

7.2.7 TDD and ATDD

Test-Driven Development is related to, but different from Acceptance Test-Driven Development (ATDD).^[26] TDD is primarily a developer’s tool to help create well-written unit of code (function, class, or module) that correctly performs a set of operations. ATDD is a communication tool between the customer, developer, and tester to ensure that the requirements are well-defined. TDD requires test automation. ATDD does not, although automation helps with regression testing. Tests used In TDD can often be derived from ATDD tests, since the code units implement some portion of a requirement. ATDD tests should be readable by the customer. TDD tests do not need to be.

7.2.8 TDD and BDD

BDD (**Behavior-driven development**) combines practices from TDD and from ATDD.^[27] It includes the practice of writing tests first, but focuses on tests which describe

behavior, rather than tests which test a unit of implementation. Tools such as Mspec and Specflow provide a syntax which allow non-programmers to define the behaviors which developers can then translate into automated tests.

7.2.9 Code visibility

Test suite code clearly has to be able to access the code it is testing. On the other hand, normal design criteria such as **information hiding**, encapsulation and the **separation of concerns** should not be compromised. Therefore unit test code for TDD is usually written within the same project or **module** as the code being tested.

In **object oriented design** this still does not provide access to private data and methods. Therefore, extra work may be necessary for unit tests. In **Java** and other languages, a developer can use **reflection** to access private fields and methods.^[28] Alternatively, an **inner class** can be used to hold the unit tests so they have visibility of the enclosing class's members and attributes. In the **.NET Framework** and some other programming languages, **partial classes** may be used to expose private methods and data for the tests to access.

It is important that such testing hacks do not remain in the production code. In **C** and other languages, **compiler directives** such as `#if DEBUG ... #endif` can be placed around such additional classes and indeed all other test-related code to prevent them being compiled into the released code. This means the released code is not exactly the same as what was unit tested. The regular running of fewer but more comprehensive, end-to-end, integration tests on the final release build can ensure (among other things) that no production code exists that subtly relies on aspects of the test harness.

There is some debate among practitioners of TDD, documented in their blogs and other writings, as to whether it is wise to test private methods and data anyway. Some argue that private members are a mere implementation detail that may change, and should be allowed to do so without breaking numbers of tests. Thus it should be sufficient to test any class through its public interface or through its subclass interface, which some languages call the “protected” interface.^[29] Others say that crucial aspects of functionality may be implemented in private methods and testing them directly offers advantage of smaller and more direct unit tests.^{[30][31]}

7.2.10 Software for TDD

There are many testing frameworks and tools that are useful in TDD

xUnit frameworks

Developers may use computer-assisted **testing frameworks**, such as **xUnit** created in 1998, to create and automatically run the test cases. Xunit frameworks provide assertion-style test validation capabilities and result reporting. These capabilities are critical for automation as they move the burden of execution validation from an independent post-processing activity to one that is included in the test execution. The execution framework provided by these test frameworks allows for the automatic execution of all system test cases or various subsets along with other features.^[32]

TAP results

Testing frameworks may accept unit test output in the language agnostic **Test Anything Protocol** created in 1987.

7.2.11 Fakes, mocks and integration tests

Unit tests are so named because they each test *one unit* of code. A complex module may have a thousand unit tests and a simple module may have only ten. The tests used for TDD should never cross process boundaries in a program, let alone network connections. Doing so introduces delays that make tests run slowly and discourage developers from running the whole suite. Introducing dependencies on external modules or data also turns *unit tests* into *integration tests*. If one module misbehaves in a chain of interrelated modules, it is not so immediately clear where to look for the cause of the failure.

When code under development relies on a database, a web service, or any other external process or service, enforcing a unit-testable separation is also an opportunity and a driving force to design more modular, more testable and more reusable code.^[33] Two steps are necessary:

1. Whenever external access is needed in the final design, an **interface** should be defined that describes the access available. See the **dependency inversion principle** for a discussion of the benefits of doing this regardless of TDD.
2. The interface should be implemented in two ways, one of which really accesses the external process, and the other of which is a **fake or mock**. Fake objects need do little more than add a message such as “Person object saved” to a trace log, against which a test **assertion** can be run to verify correct behaviour. Mock objects differ in that they themselves contain test assertions that can make the test fail, for example, if the person's name and other data are not as expected.

Fake and mock object methods that return data, ostensibly from a data store or user, can help the test process by

always returning the same, realistic data that tests can rely upon. They can also be set into predefined fault modes so that error-handling routines can be developed and reliably tested. In a fault mode, a method may return an invalid, incomplete or **null** response, or may throw an **exception**. Fake services other than data stores may also be useful in TDD: A fake encryption service may not, in fact, encrypt the data passed; a fake random number service may always return 1. Fake or mock implementations are examples of **dependency injection**.

A Test Double is a test-specific capability that substitutes for a system capability, typically a class or function, that the UUT depends on. There are two times at which test doubles can be introduced into a system: link and execution. Link time substitution is when the test double is compiled into the load module, which is executed to validate testing. This approach is typically used when running in an environment other than the target environment that requires doubles for the hardware level code for compilation. The alternative to linker substitution is run-time substitution in which the real functionality is replaced during the execution of a test cases. This substitution is typically done through the reassignment of known function pointers or object replacement.

Test doubles are of a number of different types and varying complexities:

- **Dummy** – A dummy is the simplest form of a test double. It facilitates linker time substitution by providing a default return value where required.
- **Stub** – A stub adds simplistic logic to a dummy, providing different outputs.
- **Spy** – A spy captures and makes available parameter and state information, publishing accessors to test code for private information allowing for more advanced state validation.
- **Mock** – A mock is specified by an individual test case to validate test-specific behavior, checking parameter values and call sequencing.
- **Simulator** – A simulator is a comprehensive component providing a higher-fidelity approximation of the target capability (the thing being doubled). A simulator typically requires significant additional development effort.^[8]

A corollary of such dependency injection is that the actual database or other external-access code is never tested by the TDD process itself. To avoid errors that may arise from this, other tests are needed that instantiate the test-driven code with the “real” implementations of the interfaces discussed above. These are **integration tests** and are quite separate from the TDD unit tests. There are fewer of them, and they must be run less often than the unit tests. They can nonetheless be implemented using the same testing framework, such as **xUnit**.

Integration tests that alter any **persistent store** or database should always be designed carefully with consideration of the initial and final state of the files or database, even if any test fails. This is often achieved using some combination of the following techniques:

- The **TearDown** method, which is integral to many test frameworks.
- **try...catch...finally exception handling** structures where available.
- **Database transactions** where a transaction **atomically** includes perhaps a write, a read and a matching delete operation.
- Taking a “snapshot” of the database before running any tests and rolling back to the snapshot after each test run. This may be automated using a framework such as **Ant** or **NAnt** or a **continuous integration** system such as **CruiseControl**.
- Initialising the database to a clean state *before* tests, rather than cleaning up *after* them. This may be relevant where cleaning up may make it difficult to diagnose test failures by deleting the final state of the database before detailed diagnosis can be performed.

7.2.12 TDD for complex systems

Exercising TDD on large, challenging systems requires a modular architecture, well-defined components with published interfaces, and disciplined system layering with maximization of platform independence. These proven practices yield increased testability and facilitate the application of build and test automation.^[8]

Designing for testability

Complex systems require an architecture that meets a range of requirements. A key subset of these requirements includes support for the complete and effective testing of the system. Effective modular design yields components that share traits essential for effective TDD.

- **High Cohesion** ensures each unit provides a set of related capabilities and makes the tests of those capabilities easier to maintain.
- **Low Coupling** allows each unit to be effectively tested in isolation.
- **Published Interfaces** restrict Component access and serve as contact points for tests, facilitating test creation and ensuring the highest fidelity between test and production unit configuration.

A key technique for building effective modular architecture is Scenario Modeling where a set of sequence charts is constructed, each one focusing on a single system-level execution scenario. The Scenario Model provides an excellent vehicle for creating the strategy of interactions between components in response to a specific stimulus. Each of these Scenario Models serves as a rich set of requirements for the services or functions that a component must provide, and it also dictates the order that these components and services interact together. Scenario modeling can greatly facilitate the construction of TDD tests for a complex system.^[8]

Managing tests for large teams

In a larger system the impact of poor component quality is magnified by the complexity of interactions. This magnification makes the benefits of TDD accrue even faster in the context of larger projects. However, the complexity of the total population of tests can become a problem in itself, eroding potential gains. It sounds simple, but a key initial step is to recognize that test code is also important software and should be produced and maintained with the same rigor as the production code.

Creating and managing the architecture of test software within a complex system is just as important as the core product architecture. Test drivers interact with the UUT, test doubles and the unit test framework.^[8]

7.2.13 See also

7.2.14 References

- [1] Kent Beck (May 11, 2012). "Why does Kent Beck refer to the "rediscovery" of test-driven development?". Retrieved December 1, 2014.
- [2] Beck, K. Test-Driven Development by Example, Addison Wesley - Vaseem, 2003
- [3] Lee Copeland (December 2001). "Extreme Programming". Computerworld. Retrieved January 11, 2011.
- [4] Newkirk, JW and Vorontsov, AA. *Test-Driven Development in Microsoft .NET*, Microsoft Press, 2004.
- [5] Feathers, M. Working Effectively with Legacy Code, Prentice Hall, 2004
- [6] Beck, Kent (1999). *XP Explained, 1st Edition*. Addison-Wesley Professional. p. 57. ISBN 0201616416.
- [7] Ottinger and Langr, Tim and Jeff. "Simple Design". Retrieved 5 July 2013.
- [8] "Effective TDD for Complex Embedded Systems Whitepaper" (PDF). Pathfinder Solutions.
- [9] "Agile Test Driven Development". Agile Sherpa. 2010-08-03. Retrieved 2012-08-14.
- [10] Koskela, L. "Test Driven: TDD and Acceptance TDD for Java Developers", Manning Publications, 2007
- [11] "Test-Driven Development for Complex Systems Overview Video". Pathfinder Solutions.
- [12] Erdogmus, Hakan; Morisio, Torchiano. "On the Effectiveness of Test-first Approach to Programming". Proceedings of the IEEE Transactions on Software Engineering, 31(1). January 2005. (NRC 47445). Retrieved 2008-01-14. We found that test-first students on average wrote more tests and, in turn, students who wrote more tests tended to be more productive.
- [13] Proffitt, Jacob. "TDD Proven Effective! Or is it?". Retrieved 2008-02-21. So TDD's relationship to quality is problematic at best. Its relationship to productivity is more interesting. I hope there's a follow-up study because the productivity numbers simply don't add up very well to me. There is an undeniable correlation between productivity and the number of tests, but that correlation is actually stronger in the non-TDD group (which had a single outlier compared to roughly half of the TDD group being outside the 95% band).
- [14] Llopis, Noel (20 February 2005). "Stepping Through the Looking Glass: Test-Driven Game Development (Part 1)". Games from Within. Retrieved 2007-11-01. Comparing [TDD] to the non-test-driven development approach, you're replacing all the mental checking and debugger stepping with code that verifies that your program does exactly what you intended it to do.
- [15] Mayr, Herwig (2005). *Projekt Engineering Ingenieurmäßige Softwareentwicklung in Projektgruppen* (2., neu bearb. Aufl. ed.). München: Fachbuchverl. Leipzig im Carl-Hanser-Verl. p. 239. ISBN 978-3446400702.
- [16] Müller, Matthias M.; Padberg, Frank. "About the Return on Investment of Test-Driven Development" (PDF). Universität Karlsruhe, Germany. p. 6. Retrieved 2012-06-14.
- [17] Madeyski, L. "Test-Driven Development - An Empirical Evaluation of Agile Practice", Springer, 2010, ISBN 978-3-642-04287-4, pp. 1-245. DOI: 978-3-642-04288-1
- [18] The impact of Test-First programming on branch coverage and mutation score indicator of unit tests: An experiment. by L. Madeyski *Information & Software Technology* 52(2): 169-184 (2010)
- [19] On the Effects of Pair Programming on Thoroughness and Fault-Finding Effectiveness of Unit Tests by L. Madeyski *PROFES* 2007: 207-221
- [20] Impact of pair programming on thoroughness and fault detection effectiveness of unit test suites. by L. Madeyski *Software Process: Improvement and Practice* 13(3): 281-295 (2008)
- [21] "Problems with TDD". Dalkescientific.com. 2009-12-29. Retrieved 2014-03-25.
- [22] Hunter, Andrew (2012-10-19). "Are Unit Tests Overused?". Simple-talk.com. Retrieved 2014-03-25.

- [23] Loughran, Steve (November 6, 2006). “Testing” (PDF). HP Laboratories. Retrieved 2009-08-12.
- [24] “Fragile Tests”.
- [25] Leybourn, E. (2013) *Directing the Agile Organisation: A Lean Approach to Business Management*. London: IT Governance Publishing: 176-179.
- [26] *Lean-Agile Acceptance Test-Driven Development: Better Software Through Collaboration*. Boston: Addison Wesley Professional. 2011. ISBN 978-0321714084.
- [27] “BDD”. Retrieved 2015-04-28.
- [28] Burton, Ross (2003-11-12). “Subverting Java Access Protection for Unit Testing”. O’Reilly Media, Inc. Retrieved 2009-08-12.
- [29] van Rossum, Guido; Warsaw, Barry (5 July 2001). “PEP 8 -- Style Guide for Python Code”. Python Software Foundation. Retrieved 6 May 2012.
- [30] Newkirk, James (7 June 2004). “Testing Private Methods/Member Variables - Should you or shouldn’t you”. Microsoft Corporation. Retrieved 2009-08-12.
- [31] Stall, Tim (1 Mar 2005). “How to Test Private and Protected methods in .NET”. CodeProject. Retrieved 2009-08-12.
- [32] “Effective TDD for Complex, Embedded Systems Whitepaper” (PDF). Pathfinder Solutions.
- [33] Fowler, Martin (1999). *Refactoring - Improving the design of existing code*. Boston: Addison Wesley Longman, Inc. ISBN 0-201-48567-2.

7.2.15 External links

- TestDrivenDevelopment on WikiWikiWeb
- Bertrand Meyer (September 2004). “Test or spec? Test and spec? Test from spec!”. Archived from the original on 2005-02-09.
- Microsoft Visual Studio Team Test from a TDD approach
- Write Maintainable Unit Tests That Will Save You Time And Tears
- Improving Application Quality Using Test-Driven Development (TDD)

7.3 Agile testing

Agile testing is a software testing practice that follows the principles of agile software development. Agile testing involves all members of a cross-functional agile team, with special expertise contributed by testers, to ensure delivering the business value desired by the customer at frequent intervals, working at a sustainable pace. *Specification by example* is used to capture examples of desired and undesired behavior and guide coding.

7.3.1 Overview

Agile development recognizes that testing is not a separate phase, but an integral part of software development, along with coding. Agile teams use a “whole-team” approach to “baking quality in” to the software product. Testers on agile teams lend their expertise in eliciting examples of desired behavior from customers, collaborating with the development team to turn those into executable specifications that guide coding. Testing and coding are done incrementally and iteratively, building up each feature until it provides enough value to release to production. Agile testing covers all types of testing. The Agile Testing Quadrants provide a helpful taxonomy to help teams identify and plan the testing needed.

7.3.2 Further reading

- Lisa Crispin, Janet Gregory (2009). *Agile Testing: A Practical Guide for Testers and Agile Teams*. Addison-Wesley. ISBN 0-321-53446-8.
- Adzic, Gojko (2011). *Specification by Example: How Successful Teams Deliver the Right Software*. Manning. ISBN 978-1-61729-008-4.
- Ambler, Scott (2010). “Agile Testing and Quality Strategies: Discipline over Rhetoric”. Retrieved 2010-07-15.

7.3.3 References

- Pettichord, Bret (2002-11-11). “Agile Testing What is it? Can it work?” (PDF). Retrieved 2011-01-10.
- Hendrickson, Elisabeth (2008-08-11). “Agile Testing, Nine Principles and Six Concrete Practices for Testing on Agile Teams” (PDF). Retrieved 2011-04-26.
- Huston, Tom (2013-11-15). “What Is Agile Testing?”. Retrieved 2013-11-23.
- Crispin, Lisa (2003-03-21). “XP Testing Without XP: Taking Advantage of Agile Testing Practices”. Retrieved 2009-06-11.

7.4 Bug bash

In software development, a **bug bash** is a procedure where all the developers, testers, program managers, usability researchers, designers, documentation folks, and even sometimes marketing people, put aside their regular day-to-day duties and “pound on the product”—that is, each exercises the product in every way they can think of. Because each person will use the product in slightly

different (or very different) ways, and the product is getting a great deal of use in a short amount of time, this approach may reveal bugs relatively quickly.^[1]

The use of bug-bashing sessions is one possible tool in the testing methodology TMap (test management approach). Bug-bashing sessions are usually announced to the organization some days or weeks ahead of time. The test management team may specify that only some parts of the product need testing. It may give detailed instructions to each participant about how to test, and how to record bugs found.

In some organizations, a bug-bashing session is followed by a party and a prize to the person who finds the worst bug, and/or the person who finds the greatest total of bugs.

Bug Bash is a collaboration event, the step by step procedure has been given in the article 'Bug Bash - A Collaboration Episode',^[2] which is written by Trinadh Bonam.

7.4.1 See also

- Tiger team
- Eat one's own dog food

7.4.2 References

- [1] Ron Patton (2001). *Software Testing*. Sams. ISBN 0-672-31983-7.
- [2] {title=Testing Experience | publisher=Díaz & Hilterscheid GmbH | year=2012 | http://www.testingexperience.com/}

7.5 Pair Testing

Pair testing is a software development technique in which two team members work together at one keyboard to test the software application. One does the testing and the other analyzes or reviews the testing. This can be done between one tester and developer or business analyst or between two testers with both participants taking turns at driving the keyboard.

7.5.1 Description

This can be more related to pair programming and exploratory testing of agile software development where two team members are sitting together to test the software application. This will help both the members to learn more about the application. This will narrow down the root cause of the problem while continuous testing. Developer can find out which portion of the source code is affected by the bug. This track can help to make the solid test cases and narrowing the problem for the next time.

7.5.2 Benefits and drawbacks

The developer can learn more about the software application by exploring with the tester. The tester can learn more about the software application by exploring with the developer.

Less participation is required for testing and for important bugs root cause can be analyzed very easily. The tester can very easily test the initial bug fixing status with the developer.

This will make the developer to come up with great testing scenarios by their own

This can not be applicable to scripted testing where all the test cases are already written and one has to run the scripts. This will not help in the evolution of any issue and its impact.

7.5.3 Usage

This is more applicable where the requirements and specifications are not very clear, the team is very new, and needs to learn the application behavior quickly.

This follows the same principles of pair programming; the two team members should be in the same level.

7.5.4 See also

- Pair programming
- Exploratory testing
- Agile software development
- Software testing
- All-pairs testing
- International Software Testing Qualifications Board

7.6 Manual testing

Compare with Test automation.

Manual testing is the process of manually testing software for defects. It requires a tester to play the role of an end user and use most of all features of the application to ensure correct behavior. To ensure completeness of testing, the tester often follows a written test plan that leads them through a set of important test cases.

7.6.1 Overview

A key step in the process is, testing the software for correct behavior prior to release to end users.

For small scale engineering efforts (including prototypes), **exploratory testing** may be sufficient. With this informal approach, the tester does not follow any rigorous testing procedure, but rather explores the user interface of the application using as many of its features as possible, using information gained in prior tests to intuitively derive additional tests. The success of exploratory manual testing relies heavily on the domain expertise of the tester, because a lack of knowledge will lead to incompleteness in testing. One of the key advantages of an informal approach is to gain an intuitive insight to how it feels to use the application.

Large scale engineering projects that rely on manual software testing follow a more rigorous methodology in order to maximize the number of defects that can be found. A systematic approach focuses on predetermined test cases and generally involves the following steps.^[1]

1. Choose a high level **test plan** where a general methodology is chosen, and resources such as people, computers, and software licenses are identified and acquired.
2. Write detailed **test cases**, identifying clear and concise steps to be taken by the tester, with expected outcomes.
3. Assign the test cases to testers, who manually follow the steps and record the results.
4. Author a test report, detailing the findings of the testers. The report is used by managers to determine whether the software can be released, and if not, it is used by engineers to identify and correct the problems.

A rigorous test case based approach is often traditional for large software engineering projects that follow a **Waterfall model**.^[2] However, at least one recent study did not show a dramatic difference in defect detection efficiency between exploratory testing and test case based testing.^[3]

Testing can be through **black-**, **white-** or **grey-box testing**. In white-box testing the tester is concerned with the execution of the statements through the source code. In black-box testing the software is run to check for the defects and is less concerned with how the processing of the input is done. Black-box testers do not have access to the source code. Grey-box testing is concerned with running the software while having an understanding of the source code and algorithms.

Static and **dynamic testing** approach may also be used. Dynamic testing involves running the software. Static testing includes verifying requirements, syntax of code and any other activities that do not include actually running the code of the program.

Testing can be further divided into **functional** and **non-functional testing**. In functional testing the tester would

check the calculations, any link on the page, or any other field which on given input, output may be expected. Non-functional testing includes testing performance, compatibility and fitness of the system under test, its security and usability among other things.

7.6.2 Stages

There are several stages. They are:

Unit Testing This initial stage in testing normally carried out by the developer who wrote the code and sometimes by a peer using the white box testing technique.

Integration Testing This stage is carried out in two modes, as a complete package or as an increment to the earlier package. Most of the time black box testing technique is used. However, sometimes a combination of Black and White box testing is also used in this stage.

Software Testing After the integration have been tested, software tester who may be a manual tester or automator perform software testing on complete software build. This Software testing consists of two type of testing:

1. **Functional**(to check whether SUT(Software under testing) is working as per the Functional Software Requirement Specification[SRS=FRS+NFRS(Non-Functional Requirements Specifications)] or NOT). This is performed using White Box testing techniques like BVA, ECP, Decision Table, Orthogonal Arrays. This Testing contains four Front-End testing(GUI,Control flow,Input Domain, Output or Manipulation) and one Back-End testing i.e. Database testing.
2. **Non-Functional Testing** /System Testing/Characteristics Testing(to check whether SUT is working as per the NFRS, which contains characteristics of the Software to be developed like Usability, Compatibility, Configuration, Inter System Sharing, Performance, Security)

System Testing In this stage the software is tested from all possible dimensions for all intended purposes and platforms. In this stage Black box testing technique is normally used.

User Acceptance Testing This testing stage carried out in order to get customer sign-off of finished product. A 'pass' in this stage also ensures that the customer has accepted the software and is ready for their use.

Release or Deployment Testing Onsite team will go to customer site to install the system in customer configured environment and will check for the following points:

1. Whether SetUp.exe is running or not.
2. There are easy screens during installation
3. How much space is occupied by system on HDD
4. Is the system completely uninstalled when opted to uninstall from the system.

- Usability testing
- GUI testing
- Software testing

7.6.3 Comparison to Automated Testing

Test automation may be able to reduce or eliminate the cost of actual testing. A computer can follow a rote sequence of steps more quickly than a person, and it can run the tests overnight to present the results in the morning. However, the labor that is saved in actual testing must be spent instead authoring the test program. Depending on the type of application to be tested, and the automation tools that are chosen, this may require more labor than a manual approach. In addition, some testing tools present a very large amount of data, potentially creating a time consuming task of interpreting the results.

Things such as **device drivers** and **software libraries** must be tested using test programs. In addition, testing of large numbers of users (**performance testing** and **load testing**) is typically simulated in software rather than performed in practice.

Conversely, graphical user interfaces whose layout changes frequently are very difficult to test automatically. There are test frameworks that can be used for regression testing of user interfaces. They rely on recording of sequences of keystrokes and mouse gestures, then playing them back and observing that the user interface responds in the same way every time. Unfortunately, these recordings may not work properly when a button is moved or relabeled in a subsequent release. An automatic regression test may also be fooled if the program output varies significantly.

7.6.4 References

- [1] ANSI/IEEE 829-1983 IEEE Standard for Software Test Documentation
- [2] Craig, Rick David; Stefan P. Jaskiel (2002). *Systematic Software Testing*. Artech House. p. 7. ISBN 1-58053-508-9.
- [3] Itkonen, Juha; Mika V. Mäntylä; Casper Lassenius (2007). "Defect Detection Efficiency: Test Case Based vs. Exploratory Testing" (PDF). *First International Symposium on Empirical Software Engineering and Measurement*. Retrieved January 17, 2009.

7.6.5 See also

- Test method

7.7 Regression testing

Regression testing is a type of **software testing** that seeks to uncover new **software bugs**, or *regressions*, in existing **functional** and **non-functional** areas of a system after changes such as enhancements, **patches** or **configuration** changes, have been made to them.

The purpose of regression testing is to ensure that changes such as those mentioned above have not introduced new faults.^[1] One of the main reasons for regression testing is to determine whether a change in one part of the software affects other parts of the software.^[2]

Common methods of regression testing include rerunning previously completed tests and checking whether program behavior has changed and whether previously fixed faults have re-emerged. Regression testing can be performed to test a system efficiently by systematically selecting the appropriate minimum set of tests needed to adequately cover a particular change.

Contrast with **non-regression testing** (usually validation-test for a new issue), which aims to verify whether, after introducing or updating a given software application, the change has had the intended effect.

7.7.1 Background

Experience has shown that as software is fixed, emergence of new faults and/or re-emergence of old faults is quite common. Sometimes re-emergence occurs because a fix gets lost through poor **revision control** practices (or simple human error in revision control). Often, a fix for a problem will be "fragile" in that it fixes the problem in the narrow case where it was first observed but not in more general cases which may arise over the lifetime of the software. Frequently, a fix for a problem in one area inadvertently causes a **software bug** in another area. Finally, it may happen that, when some feature is redesigned, some of the same mistakes that were made in the original implementation of the feature are made in the redesign.

Therefore, in most software development situations, it is considered **good coding practice**, when a bug is located and fixed, to record a test that exposes the bug and re-run that test regularly after subsequent changes to the program.^[3] Although this may be done through **manual testing** procedures using programming techniques, it is often done using **automated testing tools**.^[4] Such a test suite contains software tools that allow the testing environment to execute all the regression **test cases** automatically; some projects even set up automated systems to au-

tomatically re-run all regression tests at specified intervals and report any failures (which could imply a regression or an out-of-date test).^[5] Common strategies are to run such a system after every successful compile (for small projects), every night, or once a week. Those strategies can be automated by an **external tool**.

Regression testing is an integral part of the **extreme programming** software development method. In this method, design documents are replaced by extensive, repeatable, and automated testing of the entire software package throughout each stage of the **software development process**.

In the corporate world, regression testing has traditionally been performed by a **software quality assurance** team after the development team has completed work. However, defects found at this stage are the most costly to fix. This problem is being addressed by the rise of **unit testing**. Although developers have always written test cases as part of the development cycle, these test cases have generally been either **functional tests** or **unit tests** that verify only intended outcomes. Developer testing compels a developer to focus on unit testing and to include both positive and negative test cases.^[6]

7.7.2 Uses

Regression testing can be used not only for testing the **correctness** of a program, but often also for tracking the quality of its output.^[7] For instance, in the design of a **compiler**, regression testing could track the code size, and the time it takes to compile and execute the test suite cases.

“Also as a consequence of the introduction of new bugs, program maintenance requires far more system testing per statement written than any other programming. Theoretically, after each fix one must run the entire batch of test cases previously run against the system, to ensure that it has not been damaged in an obscure way. In practice, such *regression testing* must indeed approximate this theoretical idea, and it is very costly.”

— Fred Brooks, *The Mythical Man Month*, p. 122

Regression tests can be broadly categorized as **functional tests** or **unit tests**. Functional tests exercise the complete program with various inputs. Unit tests exercise individual functions, subroutines, or object methods. Both functional testing tools and unit testing tools tend to be third-party products that are not part of the compiler suite, and both tend to be automated. A functional test may be a scripted series of program inputs, possibly even involving an automated mechanism for controlling mouse movements and clicks. A unit test may be a set of separate

functions within the code itself, or a driver layer that links to the code without altering the code being tested.

7.7.3 See also

- Characterization test
- Quality control
- Smoke testing
- Test-driven development

7.7.4 References

- [1] Myers, Glenford (2004). *The Art of Software Testing*. Wiley. ISBN 978-0-471-46912-4.
- [2] Savenkov, Roman (2008). *How to Become a Software Tester*. Roman Savenkov Consulting. p. 386. ISBN 978-0-615-23372-7.
- [3] Kolawa, Adam; Huizinga, Dorota (2007). *Automated Defect Prevention: Best Practices in Software Management*. Wiley-IEEE Computer Society Press. p. 73. ISBN 0-470-04212-5.
- [4] Automate Regression Tests When Feasible, Automated Testing: Selected Best Practices, Elfriede Dustin, Safari Books Online
- [5] daVeiga, Nada (February 2008). “Change Code Without Fear: Utilize a Regression Safety Net”. *Dr. Dobbs's Journal*.
- [6] Dudney, Bill (2004-12-08). “Developer Testing Is 'In': An interview with Alberto Savoia and Kent Beck”. Retrieved 2007-11-29.
- [7] Kolawa, Adam. “Regression Testing, Programmer to Programmer”. *Wrox*.

7.7.5 External links

- Microsoft regression testing recommendations
- Gauger performance regression visualization tool
- What is Regression Testing by Scott Barber and Tom Huston

7.8 Ad hoc testing

Ad hoc testing is a commonly used term for **software testing** performed without planning and documentation, but can be applied to early scientific experimental studies.

The tests are intended to be run only once, unless a defect is discovered. **Ad hoc** testing is the least formal test method. As such, it has been criticized because it is not structured and hence defects found using this method may

be harder to reproduce (since there are no written test cases). However, the strength of ad hoc testing is that important defects can be found quickly.

It is performed by improvisation: the tester seeks to find bugs by any means that seem appropriate. Ad hoc testing can be seen as a light version of error guessing, which itself is a light version of exploratory testing.

7.8.1 See also

7.8.2 References

- Exploratory Testing Explained
- Context-Driven School of testing

7.9 Sanity testing

A **sanity test** or **sanity check** is a basic test to quickly evaluate whether a claim or the result of a calculation can possibly be true. It is a simple check to see if the produced material is rational (that the material's creator was thinking rationally, applying **sanity**). The point of a sanity test is to rule out certain classes of obviously false results, not to catch every possible error. A **rule-of-thumb** may be checked to perform the test. The advantage of a sanity test, over performing a complete or rigorous test, is speed.

In arithmetic, for example, when multiplying by 9, using the **divisibility rule** for 9 to verify that the **sum of digits** of the result is divisible by 9 is a sanity test - it will not catch every multiplication error, however it's a quick and simple method to discover many possible errors.

In computer science, a *sanity test* is a very brief run-through of the functionality of a **computer program**, system, calculation, or other analysis, to assure that part of the system or methodology works roughly as expected. This is often prior to a more exhaustive round of testing.

7.9.1 Mathematical

A sanity test can refer to various **orders of magnitude** and other simple **rule-of-thumb** devices applied to cross-check **mathematical calculations**. For example:

- If one were to attempt to **square** 738 and calculated 53,874, a quick sanity check could show that this result cannot be true. Consider that $700 < 738$, yet $700^2 = 7^2 \times 100^2 = 490,000 > 53,874$. Since squaring positive integers preserves their **inequality**, the result cannot be true, and so the calculated result is incorrect. The correct answer, $738^2 = 544,644$, is more than 10 times higher than 53,874, and so the result had been off by an order of magnitude.

- In multiplication, 918×155 is not 142,135 since 918 is divisible by three but 142,135 is not (digits add up to 16, not a **multiple of three**). Also, the product must end in the same digit as the product of end-digits $8 \times 5 = 40$, but 142,135 does not end in "0" like "40", while the correct answer does: $918 \times 155 = 142,290$. An even quicker check is that the product of even and odd numbers is even, whereas 142,135 is odd.
- The **power** output of a car cannot be 700 kJ, since that is a measure of **energy**, not power (energy per unit time). This is a basic application of **dimensional analysis**.

7.9.2 Software development

In software development, the sanity test (a form of **software testing** which offers "quick, broad, and shallow testing"^[1]) determines whether it is possible and reasonable to proceed with further testing.

Software sanity tests are synonymous with **smoke tests**.^[2]

^[3] A sanity or smoke test determines whether it is *possible* and *reasonable* to continue testing. It exercises the smallest subset of application functions needed to determine whether the systems are accessible and the application logic is responsive. If the sanity test fails, it is not reasonable to attempt more rigorous testing. Sanity tests are ways to avoid wasting time and effort by quickly determining whether an application is too flawed to merit any rigorous testing. Many companies run sanity tests on an **automated build** as part of their software development life cycle.^[4]

Sanity testing may be a tool used while manually **debugging** software. An overall piece of software likely involves multiple subsystems between the input and the output. When the overall system is not working as expected, a sanity test can be used to make the decision on what to test next. If one subsystem is not giving the expected result, the other subsystems can be eliminated from further investigation until the problem with this one is solved.

A "Hello, World!" program is often used as a sanity test for a development environment. If the program fails to compile or execute, the supporting environment likely has a configuration problem. If it works, any problem being diagnosed likely lies in the actual application in question.

Another, possibly more common usage of 'sanity test' is to denote checks which are performed *within* program code, usually on arguments to functions or returns therefrom, to see if the answers can be assumed to be correct. The more complicated the routine, the more important that its response be checked. The trivial case is checking to see that a file opened, written to, or closed, did not fail on these activities – which is a sanity check often ignored by programmers.^[5] But more complex items can also be

sanity-checked for various reasons.

Examples of this include bank account management systems which check that withdrawals are sane in not requesting more than the account contains, and that deposits or purchases are sane in fitting in with patterns established by historical data – large deposits may be more closely scrutinized for accuracy, large purchase transactions may be double-checked with a card holder for validity against fraud, ATM withdrawals in foreign locations never before visited by the card holder might be cleared up with him, etc.; these are “runtime” sanity checks, as opposed to the “development” sanity checks mentioned above.

7.9.3 See also

- Proof of concept
- Back-of-the-envelope calculation
- Software testing
- Mental calculation
- Order of magnitude
- Fermi problem
- Checksum

7.9.4 References

- [1] M. A. Fecko and C. M. Lott, “Lessons learned from automating tests for an operations support system, Software-Practice and Experience, v. 32, October 2002.
- [2] Erik van Veenendaal (ED), *Standard glossary of terms used in Software Testing*, International Software Testing Qualification Board.
- [3] *Standard Glossary of Terms Used in Software Testing*, International Software Testing Qualification Board.
- [4] Hassan, A. E. and Zhang, K. 2006. *Using Decision Trees to Predict the Certification Result of a Build*. In *Proceedings of the 21st IEEE/ACM international Conference on Automated Software Engineering* (September 18 – 22, 2006). Automated Software Engineering. IEEE Computer Society, Washington, DC, 189–198.
- [5] Darwin, Ian F. (January 1991). *Checking C programs with lint* (1st ed., with minor revisions. ed.). Newton, Mass.: O'Reilly & Associates. p. 19. ISBN 0-937175-30-7. Retrieved 7 October 2014. A common programming habit is to ignore the return value from `fprintf(stderr, ...`

7.10 Integration testing

Integration testing (sometimes called **integration and testing**, abbreviated **I&T**) is the phase in **software testing** in which individual software modules are combined and tested as a group. It occurs after **unit testing** and before **validation testing**. Integration testing takes as its input **modules** that have been **unit tested**, groups them in larger aggregates, applies tests defined in an integration **test plan** to those aggregates, and delivers as its output the integrated system ready for **system testing**.^[1]

7.10.1 Purpose

The purpose of integration testing is to verify functional, performance, and reliability **requirements** placed on major design items. These “design items”, i.e. assemblages (or groups of units), are exercised through their interfaces using **black box testing**, success and error cases being simulated via appropriate parameter and data inputs. Simulated usage of shared data areas and **inter-process communication** is tested and individual **subsystems** are exercised through their input interface. **Test cases** are constructed to test whether all the components within assemblages interact correctly, for example across procedure calls or process activations, and this is done after testing individual modules, i.e. unit testing. The overall idea is a “building block” approach, in which verified assemblages are added to a verified base which is then used to support the integration testing of further assemblages.

Some different types of integration testing are big bang, **top-down**, and **bottom-up**. Other Integration Patterns^[2] are: Collaboration Integration, Backbone Integration, Layer Integration, Client/Server Integration, Distributed Services Integration and High-frequency Integration.

Big Bang

In this approach, most of the developed modules are coupled together to form a complete software system or major part of the system and then used for integration testing. The Big Bang method is very effective for saving time in the integration testing process. However, if the test cases and their results are not recorded properly, the entire integration process will be more complicated and may prevent the testing team from achieving the goal of integration testing.

A type of Big Bang Integration testing is called **Usage Model testing**. Usage Model Testing can be used in both software and hardware integration testing. The basis behind this type of integration testing is to run user-like workloads in integrated user-like environments. In doing the testing in this manner, the environment is proofed, while the individual components are proofed indirectly through their use. Usage Model testing takes an optimistic approach to testing, because it expects to have few

problems with the individual components. The strategy relies heavily on the component developers to do the isolated unit testing for their product. The goal of the strategy is to avoid redoing the testing done by the developers, and instead flesh-out problems caused by the interaction of the components in the environment. For integration testing, Usage Model testing can be more efficient and provides better test coverage than traditional focused functional integration testing. To be more efficient and accurate, care must be used in defining the user-like workloads for creating realistic scenarios in exercising the environment. This gives confidence that the integrated environment will work as expected for the target customers.

Top-down and Bottom-up

Bottom Up Testing is an approach to integrated testing where the lowest level components are tested first, then used to facilitate the testing of higher level components. The process is repeated until the component at the top of the hierarchy is tested.

All the bottom or low-level modules, procedures or functions are integrated and then tested. After the integration testing of lower level integrated modules, the next level of modules will be formed and can be used for integration testing. This approach is helpful only when all or most of the modules of the same development level are ready. This method also helps to determine the levels of software developed and makes it easier to report testing progress in the form of a percentage.

Top Down Testing is an approach to integrated testing where the top integrated modules are tested and the branch of the module is tested step by step until the end of the related module.

Sandwich Testing is an approach to combine top down testing with bottom up testing.

The main advantage of the Bottom-Up approach is that bugs are more easily found. With Top-Down, it is easier to find a missing branch link.

7.10.2 Limitations

Any conditions not stated in specified integration tests, outside of the confirmation of the execution of design items, will generally not be tested.

7.10.3 References

- [1] Martyn A Ould & Charles Unwin (ed), *Testing in Software Development*, BCS (1986), p71. Accessed 31 Oct 2014
- [2] Binder, Robert V.: *Testing Object-Oriented Systems: Models, Patterns, and Tools*. Addison Wesley 1999. ISBN 0-201-80938-9

7.10.4 See also

- Design predicates
- Software testing
- System testing
- Unit testing
- Continuous integration

7.11 System testing

System testing of software or hardware is testing conducted on a complete, integrated system to evaluate the system's compliance with its specified requirements. System testing falls within the scope of **black box testing**, and as such, should require no knowledge of the inner design of the code or logic. ^[1]

As a rule, system testing takes, as its input, all of the “integrated” software components that have passed **integration testing** and also the software system itself integrated with any applicable hardware system(s). The purpose of integration testing is to detect any inconsistencies between the software units that are integrated together (called *assemblages*) or between any of the *assemblages* and the hardware. System testing is a more limited type of testing; it seeks to detect defects both within the “inter-assemblages” and also within the system as a whole.

7.11.1 Testing the whole system

System testing is performed on the entire system in the context of a **Functional Requirement Specification(s)** (FRS) and/or a **System Requirement Specification (SRS)**. System testing tests not only the design, but also the behaviour and even the believed expectations of the customer. It is also intended to test up to and beyond the bounds defined in the software/hardware requirements specification(s).

7.11.2 Types of tests to include in system testing

The following examples are different types of testing that should be considered during System testing:

- Graphical user interface testing
- Usability testing
- Software performance testing
- Compatibility testing
- Exception handling

- Load testing
- Volume testing
- Stress testing
- Security testing
- Scalability testing
- Sanity testing
- Smoke testing
- Exploratory testing
- Ad hoc testing
- Regression testing
- Installation testing
- Maintenance testing
- Recovery testing and failover testing.
- Accessibility testing, including compliance with:
 - Americans with Disabilities Act of 1990
 - Section 508 Amendment to the Rehabilitation Act of 1973
 - Web Accessibility Initiative (WAI) of the World Wide Web Consortium (W3C)

Although different testing organizations may prescribe different tests as part of System testing, this list serves as a general framework or foundation to begin with.

7.11.3 See also

- Automatic test equipment
- Software testing
- Unit testing
- Integration testing
- Test case
- Test fixture
- Test plan
- Automated testing
- Quality control
- Software development process

7.11.4 References

[1] *IEEE Standard Computer Dictionary: A Compilation of IEEE Standard Computer Glossaries*; IEEE; New York, NY.; 1990.

- Black, Rex; (2002). *Managing the Testing Process* (2nd ed.). Wiley Publishing. ISBN 0-471-22398-0

7.12 System integration testing

In the context of software systems and software engineering, **system integration testing (SIT)** is a testing process that exercises a software system's coexistence with others. With multiple integrated systems, assuming that each have already passed system testing,^[1] SIT proceeds to test their required interactions. Following this, the deliverables are passed on to acceptance testing.

7.12.1 Introduction

SIT is part of the software testing life cycle for collaborative projects. Usually, a round of SIT precedes the user acceptance test (UAT) round. Software providers usually run a pre-SIT round of tests before consumers run their SIT test cases.

For example, if an integrator (company) is providing an enhancement to a customer's existing solution, then they integrate the new application layer and the new database layer with the customer's existing application and database layers. After the integration is complete, users use both the new part (extended part) and old part (pre-existing part) of the integrated application to update data. A process should exist to exchange data imports and exports between the two data layers. This data exchange process should keep both systems up-to-date. The purpose of system integration testing is to ensure all parts of these systems successfully co-exist and exchange data where necessary.

There may be more parties in the integration, for example the primary customer (consumer) can have their own customers; there may be also multiple providers.

7.12.2 Data driven method

A simple method of SIT which can be performed with minimum usage of software testing tools. Data imports and exports are exchanged before the behavior of each data field within each individual layer is investigated. After the software collaboration, there are three main states of data flow.

Data state within the integration layer

Integration layer can be a **middleware** or **web service(s)** which acts as a medium for data imports and data exports. Data imports and exports performance can be checked with the following steps.

1. Cross checking of the data properties within the Integration layer with technical/business specification documents.

- For web service involvement with the integration layer, WSDL and XSD can be used against web service request for the cross check.

- Middleware involvement with the integration layer allows for **data mappings** against middleware logs for the cross check.

2. Execute some unit tests. Cross check the data mappings (data positions, declarations) and requests (character length, data types) with technical specifications.

3. Investigate the server logs/middleware logs for troubleshooting.

(Reading knowledge of **WSDL**, **XSD**, **DTD**, **XML**, and **EDI** might be required for this)

Data state within the database layer

1. First check whether all the data have committed to the database layer from the integration layer.

2. Then check the data properties with the table and column properties with relevant to technical/business specification documents.

3. Check the data validations/constraints with business specification documents.

4. If there are any processing data within the database layer then check Stored Procedures with relevant specifications.

5. Investigate the server logs for troubleshooting.

(Knowledge in SQL and reading knowledge in [stored procedures] might be required for this)

Data state within the Application layer

There is not that much to do with the application layer when we perform a system integration testing.

1. Mark all the fields from business requirement documents which should be visible in the UI.

2. Create a data map from database fields to application fields and check whether necessary fields are visible in UI.

3. Check data properties by some positive and negative test cases.

There are many **combinations** of data imports and export which we can perform by considering the time period for

system integration testing

(We have to select best combinations to perform with the limited time). And also we have to repeat some of the above steps in order to test those combinations.

7.12.3 References

[1] What is System integration testing?

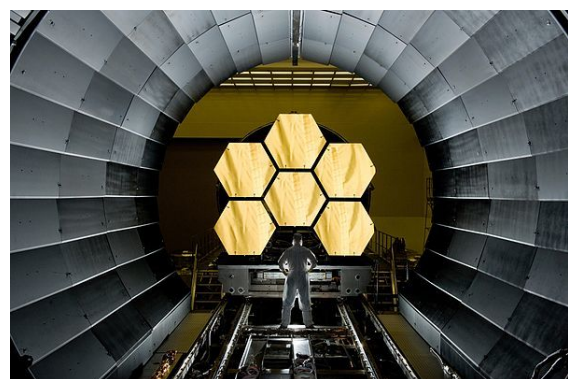
7.12.4 See also

- **Integration testing**
- **User acceptance testing (UAT)**
- **Performance acceptance testing (PAT)**

7.13 Acceptance testing



Acceptance testing of an aircraft catapult



Six of the primary mirrors of the James Webb Space Telescope being prepared for acceptance testing

In **engineering** and its various **subdisciplines**, **acceptance testing** is a test conducted to determine if the requirements of a **specification** or **contract** are met. It may involve **chemical tests**, **physical tests**, or **performance tests**.

In **systems engineering** it may involve **black-box testing** performed on a **system** (for example: a piece of software,

lots of manufactured mechanical parts, or batches of chemical products) prior to its delivery.^[1]

In **software testing** the **ISTQB** defines *acceptance* as: formal testing with respect to user needs, requirements, and business processes conducted to determine whether or not a system satisfies the acceptance criteria and to enable the user, customers or other authorized entity to determine whether or not to accept the system.^[2] Acceptance testing is also known as **user acceptance testing** (UAT), end-user testing, **operational acceptance testing** (OAT) or field (acceptance) testing.

A **smoke test** is used as an acceptance test prior to introducing a build to the main testing process.

7.13.1 Overview

Testing is a set of activities conducted to facilitate discovery and/or evaluation of properties of one or more items under test.^[3] Each individual test, known as a test case, exercises a set of predefined test activities, developed to drive the execution of the test item to meet test objectives; including correct implementation, error identification, quality verification and other valued detail.^[3] The test environment is usually designed to be identical, or as close as possible, to the anticipated production environment. It includes all facilities, hardware, software, firmware, procedures and/or documentation intended for or used to perform the testing of software.^[3]

UAT and OAT test cases are ideally derived in collaboration with business customers, business analysts, testers, and developers. It's essential that these tests include both business logic tests as well as operational environment conditions. The business customers (product owners) are the primary **stakeholders** of these tests. As the test conditions successfully achieve their acceptance criteria, the stakeholders are reassured the development is progressing in the right direction.^[4]

- User acceptance test (UAT) criteria (in **agile software development**) are usually created by business customers and expressed in a **business domain language**. These are high-level tests to verify the completeness of a **user story** or stories 'played' during any sprint/iteration.
- Operational acceptance test (OAT) criteria (regardless if using agile, iterative or sequential development) are defined in terms of functional and non-functional requirements; covering key quality attributes of **functional stability**, **portability** and **reliability**.

7.13.2 Process

The acceptance test suite may need to be performed multiple times, as all of the test cases may not be executed

within a single test iteration.^[5]

The acceptance test suite is run using predefined acceptance test procedures to direct the testers which data to use, the step-by-step processes to follow and the expected result following execution. The actual results are retained for comparison with the expected results.^[5] If the actual results match the expected results for each test case, the test case is said to pass. If the quantity of non-passing test cases does not breach the projects predetermined threshold, the test suite is said to pass. If not, the system may either be rejected or accepted on conditions previously agreed between the sponsor and the manufacturer.

The anticipated result of a successful test execution:

- test cases are executed, using predetermined data
- actual results are recorded
- actual and expected results are compared, and
- test results are determined.

The objective is to provide confidence that the developed product meets both the functional and non-functional requirements. The purpose of conducting acceptance testing is that once completed, and provided the acceptance criteria are met, it is expected the sponsors will sign-off on the product development/enhancement as satisfying the defined requirements (previously agreed between business and product provider/developer).

7.13.3 User acceptance testing

User acceptance testing (UAT) consists of a process of verifying that a solution works for the user.^[6] It is not **system testing** (ensuring software does not crash and meets documented requirements), but rather ensures that the solution will work for the user i.e. test the user accepts the solution (software vendors often refer to this as "Beta testing").

This testing should be undertaken by a **subject-matter expert** (SME), preferably the owner or client of the solution under test, and provide a summary of the findings for confirmation to proceed after trial or review. In **software development**, UAT as one of the final stages of a project often occurs before a client or customer accepts the new system. Users of the system perform tests in line with what would occur in real-life scenarios.^[7]

It is important that the materials given to the tester be similar to the materials that the end user will have. Provide testers with real-life scenarios such as the three most common tasks or the three most difficult tasks you expect an average user will undertake. Instructions on how to complete the tasks must not be provided.

The UAT acts as a final verification of the required business functionality and proper functioning of the system,

emulating real-world usage conditions on behalf of the paying client or a specific large customer. If the software works as required and without issues during normal use, one can reasonably extrapolate the same level of stability in production.^[8]

User tests, usually performed by clients or by end-users, do not normally focus on identifying simple problems such as spelling errors or cosmetic problems, nor on **showstopper** defects, such as **software crashes**; testers and developers previously identify and fix these issues during earlier **unit testing**, **integration testing**, and **system testing** phases.

UAT should be executed against test scenarios. Test scenarios usually differ from System or Functional test cases in the sense that they represent a “player” or “user” journey. The broad nature of the test scenario ensures that the focus is on the journey and not on technical or system-specific key presses, staying away from “click-by-click” test steps to allow for a variance in users’ steps through systems. Test scenarios can be broken down into logical “days”, which are usually where the actor (player/customer/operator) system (backoffice, front end) changes.

In the industrial sector, a common UAT is a factory acceptance test (FAT). This test takes place before installation of the concerned equipment. Most of the time testers not only check if the equipment meets the pre-set specification, but also if the equipment is fully functional. A FAT usually includes a check of completeness, a verification against contractual requirements, a proof of functionality (either by simulation or a conventional function test) and a final inspection.^{[9][10]}

The results of these tests give confidence to the client(s) as to how the system will perform in production. There may also be legal or contractual requirements for acceptance of the system.

7.13.4 Operational acceptance testing

Operational Acceptance Testing (OAT) is used to conduct operational readiness (pre-release) of a product, service or system as part of a **quality management system**. OAT is a common type of non-functional **software testing**, used mainly in **software development** and **software maintenance** projects. This type of testing focuses on the **operational readiness** of the system to be supported, and/or to become part of the production environment.

7.13.5 Acceptance testing in extreme programming

Acceptance testing is a term used in agile software development methodologies, particularly **extreme programming**, referring to the functional testing of a **user story** by

the software development team during the implementation phase.^[11]

The customer specifies scenarios to test when a user story has been correctly implemented. A story can have one or many acceptance tests, whatever it takes to ensure the functionality works. Acceptance tests are **black-box system tests**. Each acceptance test represents some expected result from the system. Customers are responsible for verifying the correctness of the acceptance tests and reviewing test scores to decide which failed tests are of highest priority. Acceptance tests are also used as regression tests prior to a production release. A user story is not considered complete until it has passed its acceptance tests. This means that new acceptance tests must be created for each iteration or the development team will report zero progress.^[12]

7.13.6 Types of acceptance testing

Typical types of acceptance testing include the following

User acceptance testing

This may include factory acceptance testing, i.e. the testing done by factory users before the product or system is moved to its destination site, after which site acceptance testing may be performed by the users at the site.

Operational acceptance testing Also known as operational readiness testing, this refers to the checking done to a system to ensure that processes and procedures are in place to allow the system to be used and maintained. This may include checks done to back-up facilities, procedures for disaster recovery, training for end users, maintenance procedures, and security procedures.

Contract and regulation acceptance testing In contract acceptance testing, a system is tested against acceptance criteria as documented in a contract, before the system is accepted. In regulation acceptance testing, a system is tested to ensure it meets governmental, legal and safety standards.

Alpha and beta testing Alpha testing takes place at developers’ sites, and involves testing of the operational system by internal staff, before it is released to external customers. Beta testing takes place at customers’ sites, and involves testing by a group of customers who use the system at their own locations and provide feedback, before the system is released to other customers. The latter is often called “field testing”.

7.13.7 List of acceptance-testing frameworks

- **Concordion, Specification by Example (SbE) framework**
 - Concordion.NET, acceptance testing in .NET
- **Cucumber, a behavior-driven development (BDD) acceptance test framework**
 - Capybara, Acceptance test framework for Ruby web applications
 - Behat, BDD acceptance framework for PHP
 - Lettuce, BDD acceptance framework for Python
- **Fabasoftware app.test for automated acceptance tests**
- **Framework for Integrated Test (Fit)**
 - FitNesse, a fork of Fit
- **iMacros**
- **ItsNat Java Ajax web framework with built-in, server based, functional web testing capabilities.**
- **Mocha, a popular web acceptance test framework based on Javascript and Node.js**
- **Ranorex**
- **Robot Framework**
- **Selenium**
- **Specification by example (Specs2)**
- **Watir**

7.13.8 See also

- Acceptance sampling
- Black-box testing
- Conference room pilot
- Development stage
- Dynamic testing
- Grey box testing
- Software testing
- System testing
- Test-driven development
- Unit testing
- White box testing

7.13.9 References

- [1] Black, Rex (August 2009). *Managing the Testing Process: Practical Tools and Techniques for Managing Hardware and Software Testing*. Hoboken, NJ: Wiley. ISBN 0-470-40415-9.
- [2] *Standard glossary of terms used in Software Testing, Version 2.1*. ISTQB. 2010.
- [3] *ISO/IEC/IEEE 29119-1-2013 Software and Systems Engineering - Software Testing - Part 1- Concepts and Definitions*. ISO. 2013. Retrieved 2014-10-14.
- [4] *ISO/IEC/IEEE DIS 29119-4 Software and Systems Engineering - Software Testing - Part 4- Test Techniques*. ISO. 2013. Retrieved 2014-10-14.
- [5] *ISO/IEC/IEEE 29119-2-2013 Software and Systems Engineering - Software Testing - Part 2- Test Processes*. ISO. 2013. Retrieved 2014-05-21.
- [6] Cimperman, Rob (2006). *UAT Defined: A Guide to Practical User Acceptance Testing*. Pearson Education. pp. Chapter 2. ISBN 9780132702621.
- [7] Goethem, Brian Hambling, Pauline van (2013). *User acceptance testing : a step-by-step guide*. BCS Learning & Development Limited. ISBN 9781780171678.
- [8] Pusuluri, Nageshwar Rao (2006). *Software Testing Concepts And Tools*. Dreamtech Press. p. 62. ISBN 9788177227123.
- [9] "Factory Acceptance Test (FAT)". Tuv.com. Retrieved September 18, 2012.
- [10] "Factory Acceptance Test". Inspection-for-industry.com. Retrieved September 18, 2012.
- [11] "Introduction to Acceptance/Customer Tests as Requirements Artifacts". *agilemodeling.com*. Agile Modeling. Retrieved 9 December 2013.
- [12] Don Wells. "Acceptance Tests". *Extremeprogramming.org*. Retrieved September 20, 2011.

7.13.10 Further reading

- Hambling, Brian; van Goethem, Pauline (2013). *User Acceptance Testing: A Step by Step Guide*. Swindon: BCS Learning and Development Ltd. ISBN 978-1-78017-167-8.

7.13.11 External links

- Acceptance Test Engineering Guide by Microsoft patterns & practices
- Article Using Customer Tests to Drive Development from Methods & Tools
- Article Acceptance TDD Explained from Methods & Tools
- Article User Acceptance Testing Challenges from Software Testing Help

7.14 Risk-based testing

Risk-based testing (RBT) is a type of **software testing** that functions as an organizational principle used to prioritize the tests of features and functions in software, based on the risk of failure, the function of their importance and likelihood or impact of failure.^{[1][2][3][4]} In theory, there are an infinite number of possible tests. Risk-based testing is a ranking of tests, and subtests, for functionality; test techniques such as **boundary-value analysis**, **all-pairs testing** and **state transition tables** aim to find the areas most likely to be defective.

7.14.1 Assessing risks

Comparing the changes between two releases or versions is key in order to assess risk. Evaluating critical business modules is a first step in prioritizing tests, but it does not include the notion of evolutionary risk. This is then expanded using two methods: **change-based testing** and **regression testing**.

- **Change-based testing** allows test teams to assess changes made in a release and then prioritize tests towards modified modules.
- **Regression testing** ensures that a change, such as a bug fix, did not introduce new faults into the software under test. One of the main reasons for regression testing is to determine whether a change in one part of the software has any effect on other parts of the software.

These two methods permit test teams to prioritize tests based on risk, change, and criticality of business modules. Certain technologies can make this kind of test strategy very easy to set up and to maintain with software changes.

7.14.2 Types of Risks

Risk can be identified as the probability that an undetected **software bug** may have a negative impact on the user of a system.^[5]

The methods assess risks along a variety of dimensions:

Business or Operational

- High use of a subsystem, function or feature
- Criticality of a subsystem, function or feature, including the cost of failure

Technical

- Geographic distribution of development team
- Complexity of a subsystem or function

External

- Sponsor or executive preference
- Regulatory requirements

E-business failure-mode related

- Static content defects
- Web page integration defects
- Functional behavior-related failure
- Service (Availability and Performance) related failure
- Usability and Accessibility-related failure
- Security vulnerability
- Large scale integration failure

[6]

7.14.3 References

- [1] Gerrard, Paul; Thompson, Neil (2002). *Risk Based E-Business Testing*. Artech House Publishers. ISBN 1-58053-314-0.
- [2] Bach, J. *The Challenge of Good Enough Software* (1995)
- [3] Bach, J. and Kaner, C. *Exploratory and Risk Based Testing* (2004)
- [4] Mika Lehto (October 25, 2011). "The concept of risk-based testing and its advantages and disadvantages". Ict-standard.org. Retrieved 2012-03-01.
- [5] Stephane Besson (2012-01-03). "Article info : A Strategy for Risk-Based Testing". Stickyminds.com. Retrieved 2012-03-01.
- [6] Gerrard, Paul and Thompson, Neil *Risk-Based Testing E-Business* (2002)

7.15 Software testing outsourcing

Software Testing Outsourcing is software testing carried out by an independent company or a group of people not directly involved in the process of **software development**.

Software testing is an essential phase of software development, however it is often viewed as a non-core activity for most organisations. Outsourcing enables an organisation to concentrate on its core development activities while external software testing experts handle the independent validation work. This offers many business benefits which include independent assessment leading

to enhanced delivery confidence, reduced time to market, lower infrastructure investment, predictable software quality, de-risking of deadlines and increased time to focus on development.

Software Testing Outsourcing can come in different forms:

- Full outsourcing of the entire test process (strategy, planning, execution and closure), often referred to as a Managed Testing Service
- Provision of additional resources for major projects
- One-off test often related to load, stress or performance testing
- Beta User Acceptance Testing. Utilising specialist focus groups coordinated by an external organisation

7.15.1 Top established global outsourcing cities

According to Tholons Global Services - Top 50,^[1] in 2009, Top Established and Emerging Global Outsourcing Cities in Testing function were:

1. Chennai, India
2. Cebu City, Philippines
3. Shanghai, China
4. Beijing, China
5. Kraków, Poland
6. Ho Chi Minh City, Vietnam

7.15.2 Top Emerging Global Outsourcing Cities

1. Chennai
2. Bucharest
3. São Paulo
4. Cairo

Cities were benchmark against six categories included: skills and scalability, savings, business environment, operational environment, business risk and non-business environment.

7.15.3 Vietnam outsourcing

Vietnam has become a major player in software outsourcing. Ho Chi Minh City's ability to meet clients' needs in scale and capacity, its maturing business environment, the country's stability in political and labor conditions, its increasing number of English speakers and its high service-level maturity make it attractive to foreign interests.^[2]

Vietnam's software industry has maintained annual growth rate of 30-50% during the past 10 years. From 2002 to 2013 revenue of the software industry increased to nearly 3 US\$ billion and the hardware industry increased to 36.8 US\$ billion. Many Vietnamese enterprises have been granted international certificates (CMM) for their software development.^[3]

7.15.4 Argentina outsourcing

Argentina's software industry has experienced an exponential growth in the last decade, positioning itself as one of the strategic economic activities in the country. As Argentina is just one hour ahead of North America's east coast, communication takes place in real time. Argentina's internet culture and industry is one the best, Facebook penetration in Argentina ranks 3rd worldwide and the country has the highest penetration of smart phones in Latin America (24%).^[4] Perhaps one of the most surprising facts is that the percentage that internet contributes to Argentina's Gross National Product (2.2%) ranks 10th in the world.^[5]

7.15.5 References

- [1] Tholons Global Services report 2009 Top Established and Emerging Global Outsourcing
- [2] LogiGear, PC World Viet Nam, Jan 2011
- [3] <http://www.forbes.com/sites/teconomy/2014/12/09/vietnam-it-services-climb-the-value-chain/> , vietnam it services climb the value chain
- [4] New Media Trend Watch: <http://www.newmediatrendwatch.com/markets-by-country/11-long-haul/35-argentina>
- [5] Infobae.com: <http://www.infobae.com/notas/645695-Internet-aportara-us24700-millones-al-PBI-de-la-Argentina-en-2011.html>

7.16 Tester driven development

Tester-driven development, or bug-driven development is an *anti-pattern* where the requirements are determined by bug reports or test results rather than for example the value or cost of a feature.

It is a tongue-in-cheek reference to **Test-driven development**, a widely used methodology in Agile software practices. In test driven development tests are used to drive the implementation towards fulfilling the requirements. Tester-driven development instead shortcuts the process by removing the determination of requirements and letting the testers (or “QA”) drive what they think the software should be through the QA process.

7.17 Test effort

In **software development**, **test effort** refers to the expenses for (still to come) tests. There is a relation with test costs and failure costs (direct, indirect, costs for fault correction). Some factors which influence test effort are: maturity of the **software development process**, **quality** and **testability** of the testobject, test infrastructure, **skills** of staff members, quality goals and **test strategy**.

7.17.1 Methods for estimation of the test effort

To analyse all factors is difficult, because most of the factors influence each other. Following approaches can be used for the estimation: top-down estimation and bottom-up estimation. The top-down techniques are formula based and they are relative to the expenses for development: **Function Point Analysis (FPA)** and **Test Point Analysis (TPA)** amongst others. Bottom-up techniques are based on detailed information and involve often experts. The following techniques belong here: **Work Breakdown Structure (WBS)** and **Wide Band Delphi (WBD)**.

We can also use the following techniques for estimating the test effort:

- Conversion of software size into person hours of effort directly using a conversion factor. For example, we assign 2 person hours of testing effort per one Function Point of software size or 4 person hours of testing effort per one use case point or 3 person hours of testing effort per one Software Size Unit
- Conversion of software size into testing project size such as Test Points or Software Test Units using a conversion factor and then convert testing project size into effort
- Compute testing project size using Test Points of Software Test Units. Methodology for deriving the testing project size in Test Points is not well documented. However, methodology for deriving Software Test Units is defined in a paper by Murali
- We can also derive software testing project size and effort using Delphi Technique or Analogy Based Estimation technique.

7.17.2 Test efforts from literature

In literature test efforts relative to total costs are between 20% and 70%. These values are amongst others dependent from the project specific conditions. When looking for the test effort in the single phases of the **test process**, these are diversely distributed: with about 40% for **test specification** and **test execution** each.

7.17.3 References

- Andreas Spillner, Tilo Linz, Hans Schäfer. (2006). *Software Testing Foundations - A Study Guide for the Certified Tester Exam - Foundation Level - ISTQB compliant*, 1st print. dpunkt.verlag GmbH, Heidelberg, Germany. ISBN 3-89864-363-8.
- Erik van Veenendaal (Hrsg. und Mitautor): *The Testing Practitioner*. 3. Auflage. UTN Publishers, CN Den Bosch, Niederlande 2005, ISBN 90-72194-65-9.
- Thomas Müller (chair), Rex Black, Sigrid Eldh, Dorothy Graham, Klaus Olsen, Maaret Pyhäjärvi, Geoff Thompson and Erik van Veendendal. (2005). *Certified Tester - Foundation Level Syllabus - Version 2005*, International Software Testing Qualifications Board (ISTQB), Möhrendorf, Germany. (PDF; 0,424 MB).
- Andreas Spillner, Tilo Linz, Thomas Roßner, Mario Winter: *Praxiswissen Softwaretest - Testmanagement: Aus- und Weiterbildung zum Certified Tester: Advanced Level nach ISTQB-Standard*. 1. Auflage. dpunkt.verlag GmbH, Heidelberg 2006, ISBN 3-89864-275-5.

7.17.4 External links

- **Wide Band Delphi**
- **Test Effort Estimation**

Chapter 8

Testing artefacts

8.1 IEEE 829

IEEE 829-2008, also known as the **829 Standard for Software and System Test Documentation**, is an **IEEE** standard that specifies the form of a set of documents for use in eight defined stages of **software testing** and **system testing**, each stage potentially producing its own separate type of document. The standard specifies the format of these documents, but does not stipulate whether they must all be produced, nor does it include any criteria regarding adequate content for these documents. These are a matter of judgment outside the purview of the standard.

The documents are:

- **Master Test Plan (MTP)**: The purpose of the Master Test Plan (MTP) is to provide an overall test planning and test management document for multiple levels of test (either within one project or across multiple projects).
- **Level Test Plan (LTP)**: For each LTP the scope, approach, resources, and schedule of the testing activities for its specified level of testing need to be described. The items being tested, the features to be tested, the testing tasks to be performed, the personnel responsible for each task, and the associated risk(s) need to be identified.
- **Level Test Design (LTD)**: Detailing test cases and the expected results as well as test pass criteria.
- **Level Test Case (LTC)**: Specifying the test data for use in running the test cases identified in the Level Test Design.
- **Level Test Procedure (LTPr)**: Detailing how to run each test, including any set-up preconditions and the steps that need to be followed.
- **Level Test Log (LTL)**: To provide a chronological record of relevant details about the execution of tests, e.g. recording which tests cases were run, who ran them, in what order, and whether each test passed or failed.
- **Anomaly Report (AR)**: To document any event that occurs during the testing process that requires investigation. This may be called a problem, test incident, defect, trouble, issue, anomaly, or error report. This document is deliberately named as an anomaly report, and not a fault report. The reason is that a discrepancy between expected and actual results can occur for a number of reasons other than a fault in the system. These include the expected results being wrong, the test being run incorrectly, or inconsistency in the requirements meaning that more than one interpretation could be made. The report consists of all details of the incident such as actual and expected results, when it failed, and any supporting evidence that will help in its resolution. The report will also include, if possible, an assessment of the impact of an incident upon testing.
- **Level Interim Test Status Report (LITSR)**: To summarize the interim results of the designated testing activities and optionally to provide evaluations and recommendations based on the results for the specific test level.
- **Level Test Report (LTR)**: To summarize the results of the designated testing activities and to provide evaluations and recommendations based on the results after test execution has finished for the specific test level.
- **Master Test Report (MTR)**: To summarize the results of the levels of the designated testing activities and to provide evaluations based on these results. This report may be used by any organization using the MTP. A management report providing any important information uncovered by the tests accomplished, and including assessments of the quality of the testing effort, the quality of the software system under test, and statistics derived from Anomaly Reports. The report also records what testing was done and how long it took, in order to improve any future test planning. This final document is used to indicate whether the software system under test is fit for purpose according to whether or not it has met acceptance criteria defined by project stakeholders.

8.1.1 Use of IEEE 829

The standard forms part of the training syllabus of the ISEB Foundation and Practitioner Certificates in Software Testing promoted by the British Computer Society. ISTQB, following the formation of its own syllabus based on ISEB's and Germany's ASQF syllabi, also adopted IEEE 829 as the reference standard for software and system test documentation.

8.1.2 External links

- *BS7925-2, Standard for Software Component Testing*

8.2 Test strategy

Compare with Test plan.

A **test strategy** is an outline that describes the testing approach of the software development cycle. It is created to inform project managers, testers, and developers about some key issues of the testing process. This includes the testing objective, methods of testing new functions, total time and resources required for the project, and the testing environment.

Test strategies describe how the product risks of the stakeholders are mitigated at the test-level, which types of test are to be performed, and which entry and exit criteria apply. They are created based on development design documents. System design documents are primarily used and occasionally, conceptual design documents may be referred to. Design documents describe the functionality of the software to be enabled in the upcoming release. For every stage of development design, a corresponding test strategy should be created to test the new feature sets.

8.2.1 Test Levels

The test strategy describes the test level to be performed. There are primarily three levels of testing: **unit testing**, **integration testing**, and **system testing**. In most software development organizations, the developers are responsible for unit testing. Individual testers or test teams are responsible for integration and system testing.

8.2.2 Roles and Responsibilities

The roles and responsibilities of test leader, individual testers, project manager are to be clearly defined at a project level in this section. This may not have names associated: but the role has to be very clearly defined.

Testing strategies should be reviewed by the developers. They should also be reviewed by test leads for all levels of

testing to make sure the coverage is complete yet not overlapping. Both the testing manager and the development managers should approve the test strategy before testing can begin.

8.2.3 Environment Requirements

Environment requirements are an important part of the test strategy. It describes what operating systems are used for testing. It also clearly informs the necessary OS patch levels and security updates required. For example, a certain test plan may require Windows XP Service Pack 3 to be installed as a prerequisite for testing.

8.2.4 Testing Tools

There are two methods used in executing test cases: manual and automated. Depending on the nature of the testing, it is usually the case that a combination of manual and automated testing is the best testing method.

8.2.5 Risks and Mitigation

Any risks that will affect the testing process must be listed along with the mitigation. By documenting a risk, its occurrence can be anticipated well ahead of time. Proactive action may be taken to prevent it from occurring, or to mitigate its damage. Sample risks are dependency of completion of coding done by sub-contractors, or capability of testing tools.

8.2.6 Test Schedule

A *test plan* should make an estimation of how long it will take to complete the testing phase. There are many requirements to complete testing phases. First, testers have to execute all test cases at least once. Furthermore, if a defect was found, the developers will need to fix the problem. The testers should then re-test the failed test case until it is functioning correctly. Last but not the least, the tester need to conduct regression testing towards the end of the cycle to make sure the developers did not accidentally break parts of the software while fixing another part. This can occur on test cases that were previously functioning properly.

The test schedule should also document the number of testers available for testing. If possible, assign test cases to each tester.

It is often difficult to make an accurate estimate of the test schedule since the testing phase involves many uncertainties. Planners should take into account the extra time needed to accommodate contingent issues. One way to make this approximation is to look at the time needed by the previous releases of the software. If the software is

new, multiplying the initial testing schedule approximation by two is a good way to start.

8.2.7 Regression test approach

When a particular problem is identified, the programs will be debugged and the fix will be done to the program. To make sure that the fix works, the program will be tested again for that criterion. Regression tests will make sure that one fix does not create some other problems in that program or in any other interface. So, a set of related test cases may have to be repeated again, to make sure that nothing else is affected by a particular fix. How this is going to be carried out must be elaborated in this section. In some companies, whenever there is a fix in one unit, all unit test cases for that unit will be repeated, to achieve a higher level of quality.

8.2.8 Test Groups

From the list of requirements, we can identify related areas, whose functionality is similar. These areas are the test groups. For example, in a railway reservation system, anything related to ticket booking is a functional group; anything related with report generation is a functional group. Same way, we have to identify the test groups based on the functionality aspect.

8.2.9 Test Priorities

Among test cases, we need to establish priorities. While testing software projects, certain test cases will be treated as the most important ones and if they fail, the product cannot be released. Some other test cases may be treated like cosmetic and if they fail, we can release the product without much compromise on the functionality. This priority levels must be clearly stated. These may be mapped to the test groups also.

8.2.10 Test Status Collections and Reporting

When test cases are executed, the test leader and the project manager must know, where exactly the project stands in terms of testing activities. To know where the project stands, the inputs from the individual testers must come to the test leader. This will include, what test cases are executed, how long it took, how many test cases passed, how many failed, and how many are not executable. Also, how often the project collects the status is to be clearly stated. Some projects will have a practice of collecting the status on a daily basis or weekly basis.

8.2.11 Test Records Maintenance

When the test cases are executed, we need to keep track of the execution details like when it is executed, who did it, how long it took, what is the result etc. This data must be available to the test leader and the project manager, along with all the team members, in a central location. This may be stored in a specific directory in a central server and the document must say clearly about the locations and the directories. The naming convention for the documents and files must also be mentioned.

8.2.12 Requirements traceability matrix

Main article: [Traceability matrix](#)

Ideally, the software must completely satisfy the set of requirements. From design, each requirement must be addressed in every single document in the software process. The documents include the HLD, LLD, source codes, unit test cases, integration test cases and the system test cases. In a requirements traceability matrix, the rows will have the requirements. The columns represent each document. Intersecting cells are marked when a document addresses a particular requirement with information related to the requirement ID in the document. Ideally, if every requirement is addressed in every single document, all the individual cells have valid section ids or names filled in. Then we know that every requirement is addressed. If any cells are empty, it represents that a requirement has not been correctly addressed.

8.2.13 Test Summary

The senior management may like to have test summary on a weekly or monthly basis. If the project is very critical, they may need it even on daily basis. This section must address what kind of test summary reports will be produced for the senior management along with the frequency.

The test strategy must give a clear vision of what the testing team will do for the whole project for the entire duration. This document can be presented to the client, if needed. The person, who prepares this document, must be functionally strong in the product domain, with very good experience, as this is the document that is going to drive the entire team for the testing activities. Test strategy must be clearly explained to the testing team members right at the beginning of the project.

8.2.14 See also

- [Software testing](#)
- [Test case](#)

- Risk-based testing

8.2.15 References

- Ammann, Paul and Offutt, Jeff. *Introduction to software testing*. New York: Cambridge University Press, 2008
- Bach, James (1999). “Test Strategy” (PDF). Retrieved October 31, 2011.
- Dasso, Aristides. *Verification, validation and testing in software engineering*. Hershey, PA: Idea Group Pub., 2007

8.3 Test plan

A **test plan** is a document detailing the objectives, target market, internal beta team, and processes for a specific beta test for a **software** or hardware product. The plan typically contains a detailed understanding of the eventual **workflow**.

8.3.1 Test plans

A test plan documents the strategy that will be used to verify and ensure that a product or system meets its design specifications and other requirements. A test plan is usually prepared by or with significant input from **test engineers**.

Depending on the product and the responsibility of the organization to which the test plan applies, a test plan may include a strategy for one or more of the following:

- *Design Verification or Compliance test* - to be performed during the development or approval stages of the product, typically on a small sample of units.
- *Manufacturing or Production test* - to be performed during preparation or assembly of the product in an ongoing manner for purposes of performance verification and quality control.
- *Acceptance or Commissioning test* - to be performed at the time of delivery or installation of the product.
- *Service and Repair test* - to be performed as required over the service life of the product.
- *Regression test* - to be performed on an existing operational product, to verify that existing functionality didn't get broken when other aspects of the environment are changed (e.g., upgrading the platform on which an existing application runs).

A complex system may have a high level test plan to address the overall requirements and supporting test plans to address the design details of subsystems and components.

Test plan document formats can be as varied as the products and organizations to which they apply. There are three major elements that should be described in the test plan: Test Coverage, Test Methods, and Test Responsibilities. These are also used in a formal **test strategy**.

Test coverage

Test coverage in the test plan states what requirements will be verified during what stages of the product life. Test Coverage is derived from design specifications and other requirements, such as safety standards or regulatory codes, where each requirement or specification of the design ideally will have one or more corresponding means of verification. Test coverage for different product life stages may overlap, but will not necessarily be exactly the same for all stages. For example, some requirements may be verified during Design Verification test, but not repeated during Acceptance test. Test coverage also feeds back into the design process, since the product may have to be designed to allow test access.

Test methods

Test methods in the test plan state how test coverage will be implemented. Test methods may be determined by standards, regulatory agencies, or contractual agreement, or may have to be created new. Test methods also specify test equipment to be used in the performance of the tests and establish pass/fail criteria. Test methods used to verify hardware design requirements can range from very simple steps, such as visual inspection, to elaborate test procedures that are documented separately.

Test responsibilities

Test responsibilities include what organizations will perform the test methods and at each stage of the product life. This allows test organizations to plan, acquire or develop test equipment and other resources necessary to implement the test methods for which they are responsible. Test responsibilities also includes, what data will be collected, and how that data will be stored and reported (often referred to as “deliverables”). One outcome of a successful test plan should be a record or report of the verification of all design specifications and requirements as agreed upon by all parties.

8.3.2 IEEE 829 test plan structure

IEEE 829-2008, also known as the 829 Standard for Software Test Documentation, is an **IEEE** standard that spec-

ifies the form of a set of documents for use in defined stages of software testing, each stage potentially producing its own separate type of document.^[1] These stages are:

- Test plan identifier
- Introduction
- Test items
- Features to be tested
- Features not to be tested
- Approach
- Item pass/fail criteria
- Suspension criteria and resumption requirements
- Test deliverables
- Testing tasks
- Environmental needs
- Responsibilities
- Staffing and training needs
- Schedule
- Risks and contingencies
- Approvals

The IEEE documents that suggest what should be contained in a test plan are:

- 829-2008 *IEEE Standard for Software and System Test Documentation*^[1]
 - 829-1998 *IEEE Standard for Software Test Documentation* (superseded by 829-2008)^[2]
 - 829-1983 *IEEE Standard for Software Test Documentation* (superseded by 829-1998)^[3]
- 1008-1987 *IEEE Standard for Software Unit Testing*^[4]
- 1012-2004 *IEEE Standard for Software Verification and Validation*^[5]
 - 1012-1998 *IEEE Standard for Software Verification and Validation* (superseded by 1012-2004)^[6]
 - 1012-1986 *IEEE Standard for Software Verification and Validation Plans* (superseded by 1012-1998)^[7]
- 1059-1993 *IEEE Guide for Software Verification & Validation Plans* (withdrawn)^[8]

8.3.3 See also

- Software testing
- Test suite
- Test case
- Test script
- Scenario testing
- Session-based testing
- IEEE 829
- Ad hoc testing

8.3.4 References

- [1] 829-2008 — *IEEE Standard for Software and System Test Documentation*. 2008. doi:10.1109/IEEESTD.2008.4578383. ISBN 978-0-7381-5747-4.
- [2] 829-1998 — *IEEE Standard for Software Test Documentation*. 1998. doi:10.1109/IEEESTD.1998.88820. ISBN 0-7381-1443-X.
- [3] 829-1983 — *IEEE Standard for Software Test Documentation*. 1983. doi:10.1109/IEEESTD.1983.81615. ISBN 0-7381-1444-8.
- [4] 1008-1987 - *IEEE Standard for Software Unit Testing*. 1986. doi:10.1109/IEEESTD.1986.81001. ISBN 0-7381-0400-0.
- [5] 1012-2004 - *IEEE Standard for Software Verification and Validation*. 2005. doi:10.1109/IEEESTD.2005.96278. ISBN 978-0-7381-4642-3.
- [6] 1012-1998 - *IEEE Standard for Software Verification and Validation*. 1998. doi:10.1109/IEEESTD.1998.87820. ISBN 0-7381-0196-6.
- [7] 1012-1986 - *IEEE Standard for Software Verification and Validation Plans*. 1986. doi:10.1109/IEEESTD.1986.79647. ISBN 0-7381-0401-9.
- [8] 1059-1993 - *IEEE Guide for Software Verification and Validation Plans*. 1994. doi:10.1109/IEEESTD.1994.121430. ISBN 0-7381-2379-X.

8.3.5 External links

- Public domain RUP test plan template at Sourceforge (templates are currently inaccessible but sample documents can be seen here: DBV Samples)
- Test plans and test cases

8.4 Traceability matrix

A **traceability matrix** is a document, usually in the form of a table, that correlates any two **baselined documents** that require a many-to-many relationship to determine the completeness of the relationship. It is often used with high-level **requirements** (these often consist of marketing requirements) and detailed requirements of the product to the matching parts of **high-level design**, detailed design, **test plan**, and **test cases**.

A requirements traceability matrix may be used to check to see if the current project requirements are being met, and to help in the creation of a **request for proposal**,^[1] **software requirements specification**,^[2] various deliverable documents, and project plan tasks.^[3]

Common usage is to take the identifier for each of the items of one document and place them in the left column. The identifiers for the other document are placed across the top row. When an item in the left column is related to an item across the top, a mark is placed in the intersecting cell. The number of relationships are added up for each row and each column. This value indicates the mapping of the two items. Zero values indicate that no relationship exists. It must be determined if a relationship must be made. Large values imply that the relationship is too complex and should be simplified.

To ease the creation of traceability matrices, it is advisable to add the relationships to the source documents for both backward traceability and forward traceability. That way, when an item is changed in one baselined document, it's easy to see what needs to be changed in the other.

8.4.1 Sample traceability matrix

8.4.2 See also

- **Requirements traceability**
- **Software engineering**

8.4.3 References

- [1] Egeland, Brad (April 25, 2009). "Requirements Traceability Matrix". pmtips.net. Retrieved April 4, 2013.
- [2] "DI-IPSC-81433A, DATA ITEM DESCRIPTION SOFTWARE REQUIREMENTS SPECIFICATION (SRS)". everyspec.com. December 15, 1999. Retrieved April 4, 2013.
- [3] Carlos, Tom (October 21, 2008). Requirements Traceability Matrix - RTM. PM Hut, October 21, 2008. Retrieved October 17, 2009 from <http://www.pmhut.com/requirements-traceability-matrix-rtm>.

8.4.4 External links

- **Bidirectional Requirements Traceability** by Linda Westfall
- **StickyMinds** article: **Traceability Matrix** by Karthikeyan V
- **Why Software Requirements Traceability Remains a Challenge** by Andrew Kannenberg and Dr. Hossein Saiedian

8.5 Test case

This article is about the term in software engineering. For the use of the term in law, see **Test case (law)**.

A **test case**, in **software engineering**, is a set of conditions under which a tester will determine whether an **application**, **software system** or one of its features is working as it was originally established for it to do. The mechanism for determining whether a software program or system has passed or failed such a test is known as a **test oracle**. In some settings, an oracle could be a **requirement** or **use case**, while in others it could be a **heuristic**. It may take many test cases to determine that a software program or system is considered sufficiently scrutinized to be released. Test cases are often referred to as **test scripts**, particularly when written - when they are usually collected into **test suites**.

8.5.1 Formal test cases

In order to fully test that all the requirements of an application are met, there must be at least two test cases for each requirement: one positive test and one negative test. If a requirement has sub-requirements, each sub-requirement must have at least two test cases. Keeping track of the link between the requirement and the test is frequently done using a **traceability matrix**. Written test cases should include a description of the functionality to be tested, and the preparation required to ensure that the test can be conducted.

A formal written test-case is characterized by a known input and by an expected output, which is worked out before the test is executed. The known input should test a **precondition** and the expected output should test a **postcondition**.

8.5.2 Informal test cases

For applications or systems without formal requirements, test cases can be written based on the accepted normal operation of programs of a similar class. In some schools

of testing, test cases are not written at all but the activities and results are reported after the tests have been run.

In **scenario testing**, hypothetical stories are used to help the tester think through a complex problem or system. These scenarios are usually not written down in any detail. They can be as simple as a diagram for a testing environment or they could be a description written in prose. The ideal scenario test is a story that is motivating, credible, complex, and easy to evaluate. They are usually different from test cases in that test cases are single steps while scenarios cover a number of steps of the key.

8.5.3 Typical written test case format

A test case is usually a single step, or occasionally a sequence of steps, to test the correct behaviour/functionality, features of an application. An expected result or expected outcome is usually given.

Additional information that may be included:

- test case ID
- test case description
- test step or order of execution number
- related requirement(s)
- depth
- test category
- author
- check boxes for whether the test can be or has been automated
- pass/fail
- remarks

Larger test cases may also contain prerequisite states or steps, and descriptions.

A written test case should also contain a place for the actual result.

These steps can be stored in a word processor document, spreadsheet, database or other common repository.

In a database system, you may also be able to see past test results and who generated the results and the system configuration used to generate those results. These past results would usually be stored in a separate table.

Test suites often also contain

- Test summary
- Configuration

Besides a description of the functionality to be tested, and the preparation required to ensure that the test can be conducted, the most time consuming part in the test case is creating the tests and modifying them when the system changes.

Under special circumstances, there could be a need to run the test, produce results, and then a team of experts would evaluate if the results can be considered as a pass. This happens often on new products' performance number determination. The first test is taken as the base line for subsequent test / product release cycles.

Acceptance tests, which use a variation of a written test case, are commonly performed by a group of **end-users** or clients of the system to ensure the developed system meets the requirements specified or the contract. User acceptance tests are differentiated by the inclusion of **happy path** or positive test cases to the almost complete exclusion of negative test cases.

8.5.4 See also

- [Classification Tree Method](#)

8.5.5 References

8.5.6 External links

- [Writing Software Security Test Cases - Putting security test cases into your test plan](#) by Robert Auger
- [Software Test Case Engineering](#) By Ajay Bhagwat

8.6 Test data

Test data is data which has been specifically identified for use in tests, typically of a computer program.

Some data may be used in a confirmatory way, typically to verify that a given set of input to a given function produces some expected result. Other data may be used in order to challenge the ability of the program to respond to unusual, extreme, exceptional, or unexpected input.

Test data may be produced in a focused or systematic way (as is typically the case in domain testing), or by using other, less-focused approaches (as is typically the case in high-volume randomized automated tests). Test data may be produced by the tester, or by a program or function that aids the tester. Test data may be recorded for re-use, or used once and then forgotten.

8.6.1 Limitations

It is not always possible to produce enough data for testing. The amount of data to be tested is determined or

limited by considerations such as time, cost and quality. Time to produce, cost to produce and quality of the test data, and efficiency

8.6.2 Domain testing

Domain testing is a family of test techniques that focus on the test data. This might include identifying common or critical inputs, representatives of a particular equivalence class model, values that might appear at the boundaries between one equivalence class and another, outrageous values that should be rejected by the program, combinations of inputs, or inputs that might drive the product towards a particular set of outputs.

8.6.3 Test data generation

Software testing is an important part of the Software Development Life Cycle today. It is a labor-intensive and also accounts for nearly half of the cost of the system development. Hence, it is desired that parts of testing should be automated. An important problem in testing is that of generating quality test data and is seen as an important step in reducing the cost of software testing. Hence, **test data generation** is an important part of software testing.

8.6.4 See also

- Software testing
- Test data generation
- Unit test
- Test plan
- Test suite
- Scenario test
- Session-based test

8.6.5 References

- “The evaluation of program-based software test data adequacy criteria”, E. J. Weyuker, Communications of the ACM (abstract and references)

8.7 Test suite

In software development, a **test suite**, less commonly known as a *validation suite*, is a collection of **test cases** that are intended to be used to test a software program to show that it has some specified set of behaviours. A test suite often contains detailed instructions or goals for

each collection of test cases and information on the system configuration to be used during testing. A group of test cases may also contain prerequisite states or steps, and descriptions of the following tests.

Collections of test cases are sometimes incorrectly termed a **test plan**, a **test script**, or even a **test scenario**.

8.7.1 Types

Occasionally, test suites are used to group similar test cases together. A system might have a smoke test suite that consists only of **smoke tests** or a test suite for some specific functionality in the system. It may also contain all tests and signify if a test should be used as a smoke test or for some specific functionality.

In **Model-based testing**, one distinguishes between *abstract test suites*, which are collections of abstract test cases derived from a high-level model of the **system under test** and *executable test suites*, which are derived from abstract test suites by providing the concrete, lower-level details needed execute this suite by a program.^[1] An abstract test suite cannot be directly used on the actual **system under test** (SUT) because abstract test cases remain at a high abstraction level and lack concrete details about the SUT and its environment. An executable test suite works on a sufficiently detailed level to correctly communicate with the SUT and a **test harness** is usually present to interface the executable test suite with the SUT.

A test suite for a **primality testing subroutine** might consist of a list of numbers and their primality (prime or composite), along with a testing subroutine. The testing subroutine would supply each number in the list to the primality tester, and verify that the result of each test is correct.

8.7.2 See also

- Scenario test
- Software testing
- Test case

8.7.3 References

- [1] Hakim Kahlouche, César Viho, and Massimo Zendri, “An Industrial Experiment in Automatic Generation of Executable Test Suites for a Cache Coherency Protocol”, Proc. International Workshop on Testing of Communicating Systems (IWTCS'98), Tomsk, Russia, September 1998.

8.8 Test script

A **test script** in **software testing** is a set of instructions that will be performed on the **system under test** to test

that the system functions as expected.

There are various means for executing test scripts.

- **Manual testing.** These are more commonly called test cases.
- **Automated testing**
 - Short program written in a programming language used to test part of the functionality of a software system. Test scripts written as a short program can either be written using a special automated functional GUI test tool (such as HP QuickTest Professional, Borland Silk-Test, and Rational Robot) or in a well-known programming language (such as C++, C#, Tcl, Expect, Java, PHP, Perl, Powershell, Python, or Ruby).
 - Extensively parameterized short programs a.k.a. **Data-driven testing**
 - Reusable steps created in a table a.k.a. **keyword-driven** or **table-driven testing**.

These last two types are also done in manual testing.

Automated testing is advantageous for a number of reasons: tests may be executed continuously without the need for human intervention, they are easily repeatable, and often faster. Automated tests are useful in situations where the test is to be executed several times, for example as part of **regression testing**. Automated tests can be disadvantageous when poorly written, leading to incorrect testing or broken tests being carried out.

Disadvantages of automated testing are that automated tests can — like any piece of software — be poorly written or simply break during playback. They also can only examine what they have been programmed to examine. Since most systems are designed with human interaction in mind, it is good practice that a human tests the system at some point. A trained manual tester can notice that the system under test is misbehaving without being prompted or directed; automated tests can only examine what they have been programmed to examine. When used in regression testing, manual testers can find new bugs while ensuring that old bugs do not reappear while an automated test can only ensure the latter. Mixed testing, with automated and manual testing, is often used; automating what needs to be tested often and can be easily checked by a machine, and using manual testing to do **test design** and **exploratory testing**.

One shouldn't fall into the trap of spending more time automating a test than it would take to simply execute it manually, unless it is planned to be executed several times.

8.8.1 See also

- **Software testing**

- **Unit test**
- **Test plan**
- **Test suite**
- **Test case**
- **Scenario testing**
- **Session-based testing**

8.9 Test harness

In software testing, a **test harness** or **automated test framework** is a collection of software and test data configured to test a program unit by running it under varying conditions and monitoring its behavior and outputs. It has two main parts: the **test execution engine** and the **test script repository**.

Test harnesses allow for the **automation of tests**. They can call functions with supplied parameters and print out and compare the results to the desired value. The test harness is a hook to the developed code, which can be tested using an **automation framework**.

A test harness should allow specific tests to run (this helps in optimising), orchestrate a runtime environment, and provide a capability to analyse results.

The typical objectives of a test harness are to:

- Automate the testing process.
- Execute test suites of test cases.
- Generate associated test reports.

A test harness **may** provide some of the following benefits:

- Increased productivity due to automation of the testing process.
- Increased probability that **regression testing** will occur.
- Increased quality of software components and application.
- Ensure that subsequent test runs are exact duplicates of previous ones.
- Testing can occur at times that the office is not staffed (e.g. at night)
- A test script may include conditions and/or uses that are otherwise difficult to simulate (load, for example)

An alternative definition of a test harness is software constructed to facilitate integration testing. Where **test stubs** are typically components of the application under development and are replaced by working component as the application is developed (top-down design), test harnesses are external to the application being tested and simulate services or functionality not available in a test environment. For example, if you're building an application that needs to interface with an application on a mainframe computer but none is available during development, a test harness may be built to use as a substitute. A test harness may be part of a project deliverable. It's kept outside of the application source code and may be reused on multiple projects. Because a test harness simulates application functionality — it has no knowledge of test suites, test cases or test reports. Those things are provided by a testing framework and associated automated testing tools.

8.9.1 Notes

- Agile Processes in Software Engineering and Extreme Programming, Pekka Abrahamsson, Michele Marchesi, Frank Maurer, Springer, Jan 1, 2009

Chapter 9

Static testing

9.1 Static code analysis

Static program analysis is the analysis of computer software that is performed without actually executing programs (analysis performed on executing programs is known as **dynamic analysis**).^[1] In most cases the analysis is performed on some version of the **source code**, and in the other cases, some form of the **object code**.

The term is usually applied to the analysis performed by an **automated tool**, with human analysis being called program understanding, **program comprehension**, or **code review**. **Software inspections** and **Software walkthroughs** are also used in the latter case.

9.1.1 Rationale

The sophistication of the analysis performed by tools varies from those that only consider the behavior of individual statements and declarations, to those that include the complete **source code** of a program in their analysis. The uses of the information obtained from the analysis vary from highlighting possible coding errors (e.g., the **lint** tool) to **formal methods** that mathematically prove properties about a given program (e.g., its behavior matches that of its specification).

Software metrics and **reverse engineering** can be described as forms of static analysis. Deriving software metrics and static analysis are increasingly deployed together, especially in creation of embedded systems, by defining so-called *software quality objectives*.^[2]

A growing commercial use of static analysis is in the verification of properties of software used in **safety-critical** computer systems and locating potentially **vulnerable** code.^[3] For example the following industries have identified the use of static code analysis as a means of improving the quality of increasingly sophisticated and complex software:

1. **Medical software**: The U.S. **Food and Drug Administration** (FDA) has identified the use of static analysis for medical devices.^[4]
2. **Nuclear software**: In the UK the **Health and Safety**

Executive recommends the use of static analysis on **Reactor Protection Systems**.^[5]

3. **Aviation software** (in combination with dynamic analysis)^[6]

A study in 2012 by VDC Research reports that 28.7% of the embedded software engineers surveyed currently use static analysis tools and 39.7% expect to use them within 2 years.^[7] A study from 2010 found that 60% of the interviewed developers in European research projects made at least use of their basic IDE built-in static analyzers. However, only about 10% employed an additional other (and perhaps more advanced) analysis tool.^[8]

In the application security industry the name *Static Application Security Testing* (SAST) is also used. Actually, SAST is an important part of Security Development Life-cycles (SDLs) such as the SDL defined by Microsoft^[9] and a common practice in software companies.^[10]

9.1.2 Tool types

The **OMG** (**Object Management Group**) published a study regarding the types of software analysis required for **software quality** measurement and assessment. This document on “How to Deliver Resilient, Secure, Efficient, and Easily Changed IT Systems in Line with CISQ Recommendations” describes three levels of software analysis.^[11]

Unit Level Analysis that takes place within a specific program or subroutine, without connecting to the context of that program.

Technology Level Analysis that takes into account interactions between unit programs to get a more holistic and semantic view of the overall program in order to find issues and avoid obvious false positives.

System Level Analysis that takes into account the interactions between unit programs, but without being limited to one specific technology or programming language.

A further level of software analysis can be defined.

Mission/Business Level Analysis that takes into account the business/mission layer terms, rules and processes that are implemented within the software system for its operation as part of enterprise or program/mission layer activities. These elements are implemented without being limited to one specific technology or programming language and in many cases are distributed across multiple languages but are statically extracted and analyzed for system understanding for mission assurance.

9.1.3 Formal methods

Formal methods is the term applied to the analysis of software (and computer hardware) whose results are obtained purely through the use of rigorous mathematical methods. The mathematical techniques used include denotational semantics, axiomatic semantics, operational semantics, and abstract interpretation.

By a straightforward reduction to the **halting problem**, it is possible to prove that (for any **Turing complete** language), finding all possible run-time errors in an arbitrary program (or more generally any kind of violation of a specification on the final result of a program) is **undecidable**: there is no mechanical method that can always answer truthfully whether an arbitrary program may or may not exhibit runtime errors. This result dates from the works of **Church**, **Gödel** and **Turing** in the 1930s (see: **Halting problem** and **Rice's theorem**). As with many undecidable questions, one can still attempt to give useful approximate solutions.

Some of the implementation techniques of formal static analysis include:^[12]

- **Model checking**, considers systems that have **finite state** or may be reduced to finite state by **abstraction**;
- **Data-flow analysis**, a lattice-based technique for gathering information about the possible set of values;
- **Abstract interpretation**, to model the effect that every statement has on the state of an abstract machine (i.e., it 'executes' the software based on the mathematical properties of each statement and declaration). This abstract machine over-approximates the behaviours of the system: the abstract system is thus made simpler to analyze, at the expense of *incompleteness* (not every property true of the original system is true of the abstract system). If properly done, though, abstract interpretation is *sound* (every property true of the abstract system can be mapped to a true property of the original system).^[13] The **Frama-c** value analysis plugin and **Polyspace** heavily rely on abstract interpretation.
- **Hoare logic**, a formal system with a set of logical rules for reasoning rigorously about the **correctness**

of computer programs. There is tool support for some programming languages (e.g., the **SPARK** programming language (a subset of **Ada**) and the **Java Modeling Language** — **JML** — using **ESC/Java** and **ESC/Java2**, **Frama-c WP** (weakest precondition) plugin for the C language extended with **ACSL** (**ANSI/ISO C Specification Language**)).

- **Symbolic execution**, as used to derive mathematical expressions representing the value of mutated variables at particular points in the code.

9.1.4 See also

- **Shape analysis** (software)
- **Formal semantics of programming languages**
- **Formal verification**
- **Code audit**
- **Documentation generator**
- **List of tools for static code analysis**

9.1.5 References

- [1] Wichmann, B. A.; Canning, A. A.; Clutterbuck, D. L.; Winsbarrow, L. A.; Ward, N. J.; Marsh, D. W. R. (Mar 1995). "Industrial Perspective on Static Analysis." (PDF). *Software Engineering Journal*: 69–75. Archived from the original (PDF) on 2011-09-27.
- [2] "Software Quality Objectives for Source Code" (PDF). *Proceedings: Embedded Real Time Software and Systems 2010 Conference*, ERTS2010.org, Toulouse, France: Patrick Briand, Martin Brochet, Thierry Cambois, Emmanuel Coutenceau, Olivier Guetta, Daniel Mainberte, Frederic Mondot, Patrick Munier, Loic Noury, Philippe Spozio, Frederic Retailleau.
- [3] *Improving Software Security with Precise Static and Runtime Analysis* (PDF), Benjamin Livshits, section 7.3 "Static Techniques for Security". Stanford doctoral thesis, 2006.
- [4] FDA (2010-09-08). "Infusion Pump Software Safety Research at FDA". Food and Drug Administration. Retrieved 2010-09-09.
- [5] Computer based safety systems - technical guidance for assessing software aspects of digital computer based protection systems, http://www.hse.gov.uk/nuclear/operational/tech_asst_guides/tast046.pdf
- [6] Position Paper CAST-9. Considerations for Evaluating Safety Engineering Approaches to Software Assurance // FAA, Certification Authorities Software Team (CAST), January, 2002: "Verification. A combination of both static and dynamic analyses should be specified by the applicant/developer and applied to the software."

- [7] VDC Research (2012-02-01). “Automated Defect Prevention for Embedded Software Quality”. VDC Research. Retrieved 2012-04-10.
- [8] Prause, Christian R., René Reiners, and Silviya Dencheva. “Empirical study of tool support in highly distributed research projects.” Global Software Engineering (ICGSE), 2010 5th IEEE International Conference on. IEEE, 2010 <http://ieeexplore.ieee.org/ielx5/5581168/5581493/05581551.pdf>
- [9] M. Howard and S. Lipner. The Security Development Lifecycle: SDL: A Process for Developing Demonstrably More Secure Software. Microsoft Press, 2006. ISBN 978-0735622142 I
- [10] Achim D. Brucker and Uwe Sodan. Deploying Static Application Security Testing on a Large Scale. In GI Sicherheit 2014. Lecture Notes in Informatics, 228, pages 91-101, GI, 2014. <https://www.brucker.ch/bibliography/download/2014/brucker.ea-sast-experiences-2014.pdf>
- [11] http://www.omg.org/CISQ_compliant_IT_Systemsv.4-3.pdf
- [12] Vijay D'Silva et al. (2008). “A Survey of Automated Techniques for Formal Software Verification” (PDF). Transactions On CAD. Retrieved 2015-05-11.
- [13] Jones, Paul (2010-02-09). “A Formal Methods-based verification approach to medical device software analysis”. Embedded Systems Design. Retrieved 2010-09-09.

9.1.6 Bibliography

- Syllabus and readings for Alex Aiken’s Stanford CS295 course.
- Ayewah, Nathaniel; Hovemeyer, David; Morgenthaler, J. David; Penix, John; Pugh, William (2008). “Using Static Analysis to Find Bugs”. *IEEE Software* **25** (5): 22–29. doi:10.1109/MS.2008.130.
- Brian Chess, Jacob West (Fortify Software) (2007). *Secure Programming with Static Analysis*. Addison-Wesley. ISBN 978-0-321-42477-8.
- Flemming Nielson, Hanne R. Nielson, Chris Hankin (1999, corrected 2004). *Principles of Program Analysis*. Springer. ISBN 978-3-540-65410-0.
- “Abstract interpretation and static analysis,” International Winter School on Semantics and Applications 2003, by David A. Schmidt

9.1.7 Sources

- Kaner, Cem; Nguyen, Hung Q; Falk, Jack (1988). *Testing Computer Software* (Second ed.). Boston: Thomson Computer Press. ISBN 0-47135-846-0.
- Static Testing C++ Code: A utility to check library usability

9.1.8 External links

- Code Quality Improvement - Coding standards conformance checking (DDJ)
- Competition on Software Verification (SV-COMP)
- Episode 59: Static Code Analysis Interview (Podcast) at *Software Engineering Radio*
- Implementing Automated Governance for Coding Standards Explains why and how to integrate static code analysis into the build process
- Integrate static analysis into a software development process
- .NET Static Analysis (InfoQ)
- Static Code Analysis - Polyspace
- The SAMATE Project, a resource for Automated Static Analysis tools

9.2 Software review

A **software review** is “A process or meeting during which a software product is examined by a project personnel, managers, users, customers, user representatives, or other interested parties for comment or approval”.^[1]

In this context, the term “software product” means “any technical document or partial document, produced as a deliverable of a software development activity”, and may include documents such as contracts, project plans and budgets, requirements documents, specifications, designs, source code, user documentation, support and maintenance documentation, test plans, test specifications, standards, and any other type of specialist work product.

9.2.1 Varieties of software review

Software reviews may be divided into three categories:

- **Software peer reviews** are conducted by the author of the work product, or by one or more colleagues of the author, to evaluate the technical content and/or quality of the work.^[2]
- **Software management reviews** are conducted by management representatives to evaluate the status of work done and to make decisions regarding downstream activities.
- **Software audit reviews** are conducted by personnel external to the software project, to evaluate compliance with specifications, standards, contractual agreements, or other criteria.

9.2.2 Different types of Peer reviews

- **Code review** is systematic examination (often as peer review) of computer source code.
- **Pair programming** is a type of code review where two persons develop code together at the same workstation.
- **Inspection** is a very formal type of peer review where the reviewers are following a well-defined process to find defects.
- **Walkthrough** is a form of peer review where the author leads members of the development team and other interested parties through a software product and the participants ask questions and make comments about defects.
- **Technical review** is a form of peer review in which a team of qualified personnel examines the suitability of the software product for its intended use and identifies discrepancies from specifications and standards.

9.2.3 Formal versus informal reviews

“Formality” identifies the degree to which an activity is governed by agreed (written) rules. Software review processes exist across a spectrum of formality, with relatively unstructured activities such as “buddy checking” towards one end of the spectrum, and more formal approaches such as walkthroughs, technical reviews, and software inspections, at the other. IEEE Std. 1028-1997 defines formal structures, roles, and processes for each of the last three (“formal peer reviews”), together with **software audits**.^[1]

Research studies tend to support the conclusion that formal reviews greatly outperform informal reviews in cost-effectiveness. Informal reviews may often be unnecessarily expensive (because of time-wasting through lack of focus), and frequently provide a sense of security which is quite unjustified by the relatively small number of real defects found and repaired.

9.2.4 IEEE 1028 generic process for formal reviews

IEEE Std 1028 defines a common set of activities for “formal” reviews (with some variations, especially for software audit). The sequence of activities is largely based on the **software inspection** process originally developed at IBM by Michael Fagan.^[3] Differing types of review may apply this structure with varying degrees of rigour, but all activities are mandatory for inspection:

- **0. [Entry evaluation]:** The Review Leader uses a standard checklist of entry criteria to ensure that optimum conditions exist for a successful review.
- **1. Management preparation:** Responsible management ensure that the review will be appropriately resourced with staff, time, materials, and tools, and will be conducted according to policies, standards, or other relevant criteria.
- **2. Planning the review:** The Review Leader identifies or confirms the objectives of the review, organises a team of Reviewers, and ensures that the team is equipped with all necessary resources for conducting the review.
- **3. Overview of review procedures:** The Review Leader, or some other qualified person, ensures (at a meeting if necessary) that all Reviewers understand the review goals, the review procedures, the materials available to them, and the procedures for conducting the review.
- **4. [Individual] Preparation:** The Reviewers individually prepare for group examination of the work under review, by examining it carefully for *anomalies* (potential defects), the nature of which will vary with the type of review and its goals.
- **5. [Group] Examination:** The Reviewers meet at a planned time to pool the results of their preparation activity and arrive at a consensus regarding the status of the document (or activity) being reviewed.
- **6. Rework/follow-up:** The Author of the work product (or other assigned person) undertakes whatever actions are necessary to repair defects or otherwise satisfy the requirements agreed to at the Examination meeting. The Review Leader verifies that all action items are closed.
- **7. [Exit evaluation]:** The Review Leader verifies that all activities necessary for successful review have been accomplished, and that all outputs appropriate to the type of review have been finalised.

9.2.5 Value of reviews

The most obvious value of software reviews (especially formal reviews) is that they can identify issues earlier and more cheaply than they would be identified by testing or by field use (the **defect detection process**). The cost to find and fix a defect by a well-conducted review may be one or two orders of magnitude less than when the same defect is found by test execution or in the field.

A second, but ultimately more important, value of software reviews is that they can be used to train technical

authors in the development of extremely low-defect documents, and also to identify and remove process inadequacies that encourage defects (the **defect prevention process**).

This is particularly the case for **peer reviews** if they are conducted early and often, on samples of work, rather than waiting until the work has been completed. Early and frequent reviews of small work samples can identify systematic errors in the Author's work processes, which can be corrected before further faulty work is done. This improvement in Author skills can dramatically reduce the time it takes to develop a high-quality technical document, and dramatically decrease the error-rate in using the document in downstream processes.

As a general principle, the earlier a technical document is produced, the greater will be the impact of its defects on any downstream activities and their work products. Accordingly, greatest value will accrue from early reviews of documents such as marketing plans, contracts, project plans and schedules, and requirements specifications. Researchers and practitioners have shown the effectiveness of reviewing process in finding bugs and security issues.^[4]

9.2.6 See also

- Egoless programming
- Introduced error

9.2.7 References

- [1] IEEE Std . 1028-1997, "IEEE Standard for Software Reviews", clause 3.5
- [2] Wieggers, Karl E. (2001). *Peer Reviews in Software: A Practical Guide*. Addison-Wesley. p. 14. ISBN 0201734850.
- [3] Fagan, Michael E: "Design and Code Inspections to Reduce Errors in Program Development", *IBM Systems Journal*, Vol. 15, No. 3, 1976; "Inspecting Software Designs and Code", *Datamation*, October 1977; "Advances In Software Inspections", *IEEE Transactions in Software Engineering*, Vol. 12, No. 7, July 1986
- [4] Charles P.Pfleeger, Shari Lawrence Pfleeger. Security in Computing. Fourth edition. ISBN 0-13-239077-9

9.3 Software peer review

In software development, **peer review** is a type of **software review** in which a work product (document, code, or other) is examined by its author and one or more colleagues, in order to evaluate its technical content and quality.

9.3.1 Purpose

The purpose of a peer review is to provide "a disciplined engineering practice for detecting and correcting defects in software artifacts, and preventing their leakage into field operations" according to the **Capability Maturity Model**.

When performed as part of each **Software development process** activity, peer reviews identify problems that can be fixed early in the lifecycle.^[1] That is to say, a peer review that identifies a requirements problem during the **Requirements analysis** activity is cheaper and easier to fix than during the **Software architecture** or **Software testing** activities.

The National Software Quality Experiment,^[2] evaluating the effectiveness of peer reviews, finds, "a favorable return on investment for software inspections; savings exceeds costs by 4 to 1". To state it another way, it is four times more costly, on average, to identify and fix a software problem later.

9.3.2 Distinction from other types of software review

Peer reviews are distinct from **management reviews**, which are conducted by management representatives rather than by colleagues, and for management and control purposes rather than for technical evaluation. They are also distinct from **software audit reviews**, which are conducted by personnel external to the project, to evaluate compliance with specifications, standards, contractual agreements, or other criteria.

9.3.3 Review processes

Main article: **Software review**

Peer review processes exist across a spectrum of formality, with relatively unstructured activities such as "buddy checking" towards one end of the spectrum, and more formal approaches such as **walkthroughs**, **technical peer reviews**, and **software inspections**, at the other. The IEEE defines formal structures, roles, and processes for each of the last three.^[3]

Management representatives are typically not involved in the conduct of a peer review except when included because of specific technical expertise or when the work product under review is a management-level document. This is especially true of line managers of other participants in the review.

Processes for formal peer reviews, such as **software inspections**, define specific roles for each participant, quantify stages with entry/exit criteria, capture **software metrics** on the peer review process.

9.3.4 “Open source” reviews

In the free / open source community, something like peer review has taken place in the engineering and evaluation of computer software. In this context, the rationale for peer review has its equivalent in Linus’s law, often phrased: “Given enough eyeballs, all bugs are shallow”, meaning “If there are enough reviewers, all problems are easy to solve.” Eric S. Raymond has written influentially about peer review in software development.^[4]

9.3.5 References

- [1] Kolawa, Adam; Huizinga, Dorota (2007). *Automated Defect Prevention: Best Practices in Software Management*. Wiley-IEEE Computer Society Press. p. 261. ISBN 0-470-04212-5.
- [2] National Software Quality Experiment Resources and Results
- [3] IEEE Std. 1028-2008, “IEEE Standard for Software Reviews and Audits”
- [4] Eric S. Raymond. "The Cathedral and the Bazaar".

9.4 Software audit review

A **software audit review**, or software audit, is a type of software review in which one or more auditors who are not members of the software development organization conduct “An independent examination of a software product, software process, or set of software processes to assess compliance with specifications, standards, contractual agreements, or other criteria”.^[1]

“Software product” mostly, but not exclusively, refers to some kind of technical document. IEEE Std. 1028 offers a list of 32 “examples of software products subject to audit”, including documentary products such as various sorts of plan, contracts, specifications, designs, procedures, standards, and reports, but also non-documentary products such as data, test data, and deliverable media.

Software audits are distinct from software peer reviews and software management reviews in that they are conducted by personnel external to, and independent of, the software development organization, and are concerned with compliance of products or processes, rather than with their technical content, technical quality, or managerial implications.

The term “software audit review” is adopted here to designate the form of software audit described in IEEE Std. 1028.

9.4.1 Objectives and participants

“The purpose of a software audit is to provide an independent evaluation of conformance of software products and processes to applicable regulations, standards, guidelines, plans, and procedures”.^[2] The following roles are recommended:

- The *Initiator* (who might be a manager in the audited organization, a customer or user representative of the audited organization, or a third party), decides upon the need for an audit, establishes its purpose and scope, specifies the evaluation criteria, identifies the audit personnel, decides what follow-up actions will be required, and distributes the audit report.
- The *Lead Auditor* (who must be someone “free from bias and influence that could reduce his ability to make independent, objective evaluations”) is responsible for administrative tasks such as preparing the audit plan and assembling and managing the audit team, and for ensuring that the audit meets its objectives.
- The *Recorder* documents anomalies, action items, decisions, and recommendations made by the audit team.
- The *Auditors* (who must be, like the Lead Auditor, free from bias) examine products defined in the audit plan, document their observations, and recommend corrective actions. (There may be only a single auditor.)
- The *Audited Organization* provides a liaison to the auditors, and provides all information requested by the auditors. When the audit is completed, the audited organization should implement corrective actions and recommendations.

9.4.2 Tools

Parts of Software audit could be done using static analysis tools that analyze application code and score its conformance with standards, guidelines, best practices. From the List of tools for static code analysis some are covering a very large spectrum from code to architecture review, and could be use for benchmarking.

9.4.3 References

- [1] IEEE Std. 1028-1997, *IEEE Standard for Software Reviews*, clause 3.2
- [2] IEEE Std. 10281997, clause 8.1

9.5 Software technical review

A **software technical review** is a form of **peer review** in which “a team of qualified personnel ... examines the suitability of the software product for its intended use and identifies discrepancies from specifications and standards. Technical reviews may also provide recommendations of alternatives and examination of various alternatives” (IEEE Std. 1028-1997, *IEEE Standard for Software Reviews*, clause 3.7).^[1]

“Software product” normally refers to some kind of technical document. This might be a **software design document** or **program source code**, but **use cases**, **business process definitions**, **test case specifications**, and a variety of other technical documentation, may also be subject to technical review.

Technical review differs from **software walkthroughs** in its specific focus on the technical quality of the product reviewed. It differs from **software inspection** in its ability to suggest direct alterations to the product reviewed, and its lack of a direct focus on training and process improvement.

The term **formal technical review** is sometimes used to mean a software inspection. A 'Technical Review' may also refer to an acquisition lifecycle event or **Design review**.

9.5.1 Objectives and participants

The purpose of a technical review is to arrive at a technically superior version of the work product reviewed, whether by correction of defects or by recommendation or introduction of alternative approaches. While the latter aspect may offer facilities that software inspection lacks, there may be a penalty in time lost to technical discussions or disputes which may be beyond the capacity of some participants.

IEEE 1028 recommends the inclusion of participants to fill the following roles:

The **Decision Maker** (the person for whom the technical review is conducted) determines if the review objectives have been met.

The **Review Leader** is responsible for performing administrative tasks relative to the review, ensuring orderly conduct, and ensuring that the review meets its objectives.

The **Recorder** documents anomalies, action items, decisions, and recommendations made by the review team.

Technical staff are active participants in the review and evaluation of the software product.

Management staff may participate for the purpose of identifying issues that require management resolution.

Customer or user representatives may fill roles determined by the Review Leader prior to the review.

A single participant may fill more than one role, as appropriate.

9.5.2 Process

A formal technical review will follow a series of activities similar to that specified in clause 5 of IEEE 1028, essentially summarised in the article on **software review**.

9.5.3 References

- [1] “The Software Technical Review Process”.

9.6 Management review

- **Management review**, a magazine from **American Management Association**
- **Software management review**

Management Review: A cross functional review by an organization's top management with a goal of assessing the organizations success at achieving objectives established for the business system thus ensuring its continued suitability, adequacy and effectiveness. Management review typically includes analysis of: Customer satisfaction / customer feedback Cost of poor quality Performance trends within the business Achievement of objectives defined in the business plan Results of internal audits Status of corrective and preventative actions Follow up from previous reviews

9.7 Software inspection

Inspection in **software engineering**, refers to **peer review** of any work product by trained individuals who look for defects using a well defined process. An inspection might also be referred to as a **Fagan inspection** after Michael Fagan, the creator of a very popular software inspection process.

9.7.1 Introduction

An inspection is one of the most common sorts of review practices found in software projects. The goal of the inspection is for all of the inspectors to reach consensus on a work product and approve it for use in the project. Commonly inspected work products include **software requirements specifications** and **test plans**. In an inspection, a work product is selected for review and a team is gathered for an inspection meeting to review the work product. A moderator is chosen to moderate the meeting. Each inspector prepares for the meeting by reading

the work product and noting each defect. **The goal of the inspection is to identify defects.** In an inspection, a defect is any part of the work product that will keep an inspector from approving it. For example, if the team is inspecting a software requirements specification, each defect will be text in the document which an inspector disagrees with.

9.7.2 The Inspection process

The inspection process was developed^[1] in the mid-1970s and it has later been extended and modified.

The process should have entry criteria that determine if the inspection process is ready to begin. This prevents unfinished work products from entering the inspection process. The entry criteria might be a checklist including items such as “The document has been spell-checked”.

The stages in the inspections process are: Planning, Overview meeting, Preparation, Inspection meeting, Rework and Follow-up. The Preparation, Inspection meeting and Rework stages might be iterated.

- **Planning:** The inspection is planned by the moderator.
- **Overview meeting:** The author describes the background of the work product.
- **Preparation:** Each inspector examines the work product to identify possible defects.
- **Inspection meeting:** During this meeting the reader reads through the work product, part by part and the inspectors point out the defects for every part.
- **Rework:** The author makes changes to the work product according to the action plans from the inspection meeting.
- **Follow-up:** The changes by the author are checked to make sure everything is correct.

The process is ended by the moderator when it satisfies some predefined exit criteria. The term inspection refers to one of the most important elements of the entire process that surrounds the execution and successful completion of a software engineering project.

9.7.3 Inspection roles

During an inspection the following roles are used.

- **Author:** The person who created the work product being inspected.

- **Moderator:** This is the leader of the inspection. The moderator plans the inspection and coordinates it.
- **Reader:** The person reading through the documents, one item at a time. The other inspectors then point out defects.
- **Recorder/Scribe:** The person that documents the defects that are found during the inspection.
- **Inspector:** The person that examines the work product to identify possible defects.

9.7.4 Related inspection types

Code review

A **code review** can be done as a special kind of inspection in which the team examines a sample of code and fixes any defects in it. In a code review, a defect is a block of code which does not properly implement its requirements, which does not function as the programmer intended, or which is not incorrect but could be improved (for example, it could be made more readable or its performance could be improved). In addition to helping teams find and fix bugs, code reviews are useful for both cross-training programmers on the code being reviewed and for helping junior developers learn new programming techniques.

Peer Reviews

Peer reviews are considered an industry best-practice for detecting software defects early and learning about software artifacts. Peer Reviews are composed of **software walkthroughs** and software inspections and are integral to software product engineering activities. A collection of coordinated knowledge, skills, and behaviors facilitates the best possible practice of Peer Reviews. The elements of Peer Reviews include the structured review process, standard of excellence product checklists, defined roles of participants, and the forms and reports.

Software inspections are the most rigorous form of Peer Reviews and fully utilize these elements in detecting defects. Software walkthroughs draw selectively upon the elements in assisting the producer to obtain the deepest understanding of an artifact and reaching a consensus among participants. Measured results reveal that Peer Reviews produce an attractive return on investment obtained through accelerated learning and early defect detection. For best results, Peer Reviews are rolled out within an organization through a defined program of preparing a policy and procedure, training practitioners and managers, defining measurements and populating a database structure, and sustaining the roll out infrastructure.

9.7.5 See also

- Software engineering
- List of software engineering topics
- Capability Maturity Model (CMM)

9.7.6 References

- [1] IBM Technical Report RC 21457 Log 96856 April 26, 1999.

9.7.7 External links

- Review and inspection practices
- Article Software Inspections by Ron Radice
- Comparison of different inspection and review techniques

9.8 Fagan inspection

A **Fagan inspection** is a structured process of trying to find defects in development documents such as programming code, specifications, designs and others during various phases of the software development process. It is named after Michael Fagan who is credited with being the inventor of formal software inspections.

Fagan Inspection defines a process as a certain activity with a pre-specified entry and exit criteria. In every activity or operation for which entry and exit criteria are specified Fagan Inspections can be used to validate if the output of the process complies with the exit criteria specified for the process. Fagan Inspection uses a group review method used to evaluate output of a given process.

9.8.1 Examples

Examples of activities for which Fagan Inspection can be used are:

- Requirement specification
- Software/Information System architecture (for example DYA)
- Programming (for example for iterations in XP or DSDM)
- Software testing (for example when creating test scripts)

9.8.2 Usage

The software development process is a typical application of Fagan Inspection; software development process is a series of operations which will deliver a certain end product and consists of operations like requirements definition, design, coding up to testing and maintenance. As the costs to remedy a defect are up to 10-100 times less in the early operations compared to fixing a defect in the maintenance phase it is essential to find defects as close to the point of insertion as possible. This is done by inspecting the output of each operation and comparing that to the output requirements, or exit-criteria of that operation.

Criteria

Entry criteria are the criteria or requirements which must be met to enter a specific process.^[1] For example for Fagan inspections the high- and low-level documents must comply with specific entry-criteria before they can be used for a formal inspection process.

Exit criteria are the criteria or requirements which must be met to complete a specific process. For example for Fagan inspections the low-level document must comply with specific exit-criteria (as specified in the high-level document) before the development process can be taken to the next phase.

The exit-criteria are specified in a high-level document, which is then used as the standard to compare the operation result (low-level document) to during the inspections. Deviations of the low-level document from the requirements specified in the high-level document are called defects and can be categorized in Major Defects and Minor Defects.

Defects

According to M.E. Fagan, "A defect is an instance in which a requirement is not satisfied."^[1]

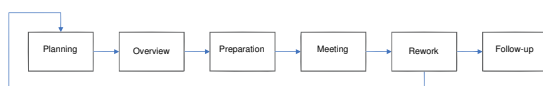
In the process of software inspection the defects which are found are categorized in two categories: major and minor defects (often many more categories are used). The defects which are statements or declarations that are incorrect, or even missing information can be classified as major defects: the software will not function correctly when these defects are not being solved.

In contrast to major defects, minor defects do not threaten the correct functioning of the software, but are mostly small errors like spelling mistakes in documents or optical issues like incorrect positioning of controls in a program interface.

Typical operations

In a typical Fagan inspection the inspection process consists of the following operations:^[1]

- Planning
 - Preparation of materials
 - Arranging of participants
 - Arranging of meeting place
- Overview
 - Group education of participants on the materials under review
 - Assignment of roles
- Preparation
 - The participants review the item to be inspected and supporting material to prepare for the meeting noting any questions or possible defects
 - The participants prepare their roles
- Inspection meeting
 - Actual finding of defect
- Rework
 - Rework is the step in software inspection in which the defects found during the inspection meeting are resolved by the author, designer or programmer. On the basis of the list of defects the low-level document is corrected until the requirements in the high-level document are met.
- Follow-up
 - In the follow-up phase of software inspections all defects found in the inspection meeting should be corrected (as they have been fixed in the rework phase). The moderator is responsible for verifying that this is indeed the case. He should verify if all defects are fixed and no new defects are inserted while trying to fix the initial defects. It is crucial that all defects are corrected as the costs of fixing them in a later phase of the project will be 10 to 100 times higher compared to the current costs.



Fagan inspection basic model

Follow-up

In the follow-up phase of a Fagan Inspection, defects fixed in the rework phase should be verified. The moderator is usually responsible for verifying rework. Sometimes fixed work can be accepted without being verified, such as when the defect was trivial. In non-trivial cases, a full re-inspection is performed by the inspection team (not only the moderator).

If verification fails, go back to the rework process.

9.8.3 Roles

The participants of the inspection process are normally just members of the team that is performing the project. The participants fulfill different roles within the inspection process:^{[2][3]}

- Author/Designer/Coder: the person who wrote the low-level document
- Reader: paraphrases the document
- Reviewers: reviews the document from a testing standpoint
- Moderator: responsible for the inspection session, functions as a coach

9.8.4 Benefits and results

By using inspections the number of errors in the final product can significantly decrease, creating a higher quality product. In the future the team will even be able to avoid errors as the inspection sessions give them insight in the most frequently made errors in both design and coding providing avoidance of error at the root of their occurrence. By continuously improving the inspection process these insights can even further be used ^[1][Fagan, 1986].

Together with the qualitative benefits mentioned above major “cost improvements” can be reached as the avoidance and earlier detection of errors will reduce the amount of resources needed for debugging in later phases of the project.

In practice very positive results have been reported by large corporations like IBM indicating that 80-90% of defects can be found and savings in resources up to 25% can be reached ^[1][Fagan, 1986].

9.8.5 Improvements

Although the Fagan Inspection method has proved to be very effective, improvements have been suggested by multiple researchers. Genuchten for example has been researching the usage of an **Electronic Meeting System**

(EMS) to improve the productivity of the meetings with positive results ^[4] [Genuchten, 1997].

Other researchers propose the usage of software that keeps a database of detected errors and automatically scans program code for these common errors ^[5] [Doolan, 1992]. This again should result in improved productivity.

9.8.6 Example

In the diagram a very simple example is given of an inspection process in which a two-line piece of code is inspected on the basis on a high-level document with a single requirement.

As can be seen in the high-level document for this project is specified that in all software code produced variables should be declared 'strong typed'. On the basis of this requirement the low-level document is checked for defects. Unfortunately a defect is found on line 1, as a variable is not declared 'strong typed'. The defect found is then reported in the list of defects found and categorized according to the categorizations specified in the high-level document.

9.8.7 References

- [1] Fagan, M.E., Advances in Software Inspections, July 1986, IEEE Transactions on Software Engineering, Vol. SE-12, No. 7, Page 744-751
- [2] M.E., Fagan (1976). "Design and Code inspections to reduce errors in program development". *IBM Systems Journal* **15** (3): pp. 182–211. doi:10.1147/sj.153.0182.
- [3] Eickelmann, Nancy S, Ruffolo, Francesca, Baik, Jongmoon, Anant, A, 2003 An Empirical Study of Modifying the Fagan Inspection Process and the Resulting Main Effects and Interaction Effects Among Defects Found, Effort Required, Rate of Preparation and Inspection, Number of Team Members and Product 1st Pass Quality, Proceedings of the 27th Annual NASA Goddard/IEEE Software Engineering Workshop
- [4] Genuchten, M; Cornelissen, W; Van Dijk, C (Winter 1997–1998). "Supporting Inspections with an Electronic Meeting System". *Journal of Management Information Systems* **14** (3): 165–179.
- [5] Doolan, E.P. (February 1992). "Experience with Fagan's Inspection Method". *Software—Practice And Experience* **22** (2): 173–182. doi:10.1002/spe.4380220205.
- [So, 1995] So, S, Lim, Y, Cha, S.D., Kwon, Y.J, 1995 An Empirical Study on Software Error Detection: Voting, Instrumentation, and Fagan Inspection *, Proceedings of the 1995 Asia Pacific Software Engineering Conference (APSEC '95), Page 345-351

Other Useful References not called out in the text

- [Laitenberger, 1999] Laitenberger, O, DeBaud, J.M, 1999 An encompassing life cycle centric survey of software inspection, *Journal of Systems and Software* 50 (2000), Page 5-31
- The **Author**, who presents the software product in step-by-step manner at the walk-through meeting, and is probably responsible for completing most action items;

9.9 Software walkthrough

In software engineering, a **walkthrough** or **walk-through** is a form of **software peer review** "in which a designer or programmer leads members of the development team and other interested parties through a software product, and the participants ask questions and make comments about possible errors, violation of development standards, and other problems".^[1]

"Software product" normally refers to some kind of technical document. As indicated by the IEEE definition, this might be a **software design document** or **program source code**, but **use cases**, **business process definitions**, **test case specifications**, and a variety of other technical documentation may also be walked through.

A walkthrough differs from **software technical reviews** in its openness of structure and its objective of familiarization. It differs from **software inspection** in its ability to suggest direct alterations to the product reviewed, its lack of a direct focus on training and process improvement, and its omission of process and product measurement.

9.9.1 Process

A walkthrough may be quite informal, or may follow the process detailed in IEEE 1028 and outlined in the article on **software reviews**.

9.9.2 Objectives and participants

In general, a walkthrough has one or two broad objectives: to gain feedback about the technical quality or content of the document; and/or to familiarize the audience with the content.

A walkthrough is normally organized and directed by the author of the technical document. Any combination of interested or technically qualified personnel (from within or outside the project) may be included as seems appropriate.

IEEE 1028^[1] recommends three specialist roles in a walkthrough:

- The **Walkthrough Leader**, who conducts the walkthrough, handles administrative tasks, and ensures orderly conduct (and who is often the Author); and
- The **Recorder**, who notes all anomalies (potential defects), decisions, and action items identified during the walkthrough meetings.

9.9.3 See also

- Cognitive walkthrough
- Reverse walkthrough

9.9.4 References

- [1] IEEE Std. 1028-1997, *IEEE Standard for Software Reviews*, clause 3.8

9.10 Code review

Code review is systematic examination (often known as **peer review**) of computer **source code**. It is intended to find and fix mistakes overlooked in the **initial development phase**, improving both the overall **quality of software** and the developers' skills. Reviews are done in various forms such as **pair programming**, informal walkthroughs, and formal **inspections**.^[1]

9.10.1 Introduction

Code reviews can often find and remove common **vulnerabilities** such as **format string exploits**, **race conditions**, **memory leaks** and **buffer overflows**, thereby improving software security. Online software repositories based on **Subversion** (with **Redmine** or **Trac**), **Mercurial**, **Git** or others allow groups of individuals to collaboratively review code. Additionally, specific **tools** for collaborative code review can facilitate the code review process.

Automated **code reviewing software** lessens the task of reviewing large chunks of code on the **developer** by systematically checking source code for known vulnerabilities. A 2012 study by VDC Research reports that 17.6% of the embedded software engineers surveyed currently use automated tools for peer code review and 23.7% expect to use them within 2 years.^[2]

Capers Jones' ongoing analysis of over 12,000 software development projects showed that the latent defect discovery rate of formal inspection is in the 60-65% range. For informal inspection, the figure is less than 50%. The latent defect discovery rate for most forms of testing is about 30%.^[3]

Typical code review rates are about 150 lines of code per hour. Inspecting and reviewing more than a few hundred lines of code per hour for critical software (such as safety critical **embedded software**) may be too fast to find errors.^{[4][5]} Industry data indicates that code reviews can accomplish at most an 85% defect removal rate with an average rate of about 65%.^[6]

The types of defects detected in code reviews have also been studied. Based on empirical evidence, it seems that up to 75% of code review defects affect software evolvability rather than functionality, making code reviews an excellent tool for software companies with long product or system life cycles.^{[7][8]}

9.10.2 Types

Code review practices fall into two main categories: formal code review and lightweight code review.^[1]

Formal code review, such as a **Fagan inspection**, involves a careful and detailed process with multiple participants and multiple phases. Formal code reviews are the traditional method of review, in which **software developers** attend a series of meetings and review code line by line, usually using printed copies of the material. Formal inspections are extremely thorough and have been proven effective at finding defects in the code under review.

Lightweight code review typically requires less overhead than formal code inspections, though it can be equally effective when done properly. Lightweight reviews are often conducted as part of the normal development process:

- Over-the-shoulder – one developer looks over the author's shoulder as the latter walks through the code.
- Email pass-around – source code management system emails code to reviewers automatically after checkin is made.
- **Pair programming** – two authors develop code together at the same workstation, as it is common in **Extreme Programming**.
- Tool-assisted code review – authors and reviewers use software tools, informal ones such as **pastebins** and **IRC**, or specialized tools designed for peer code review.

Some of these are also known as **walkthrough** (informal) or "critique" (fast and informal) code review types.

Many teams that eschew traditional, formal code review use one of the above forms of lightweight review as part of their normal development process. A code review case study published in the book *Best Kept Secrets of Peer Code Review* found that lightweight reviews uncovered as many bugs as formal reviews, but were faster and more cost-effective.

9.10.3 Criticism

Historically, formal code reviews have required a considerable investment in preparation for the review event and execution time.

Use of code analysis tools can support this activity. Especially tools that work in the IDE as they provide direct feedback to developers of coding standard compliance.

9.10.4 See also

- Software review
- Software inspection
- Debugging
- Software testing
- Static code analysis
- Performance analysis
- Automated code review
- List of tools for code review
- Pair Programming

9.10.5 References

- [1] Kolawa, Adam; Huizinga, Dorota (2007). *Automated Defect Prevention: Best Practices in Software Management*. Wiley-IEEE Computer Society Press. p. 260. ISBN 0-470-04212-5.
- [2] VDC Research (2012-02-01). "Automated Defect Prevention for Embedded Software Quality". VDC Research. Retrieved 2012-04-10.
- [3] Jones, Capers; Ebert, Christof (April 2009). "Embedded Software: Facts, Figures, and Future". IEEE Computer Society. Retrieved 2010-10-05.
- [4] Ganssle, Jack (February 2010). "A Guide to Code Inspections" (PDF). The Ganssle Group. Retrieved 2010-10-05.
- [5] Kemerer, C.F.; Paulk, M.C. (July–Aug 2009). "The Impact of Design and Code Reviews on Software Quality: An Empirical Study Based on PSP Data". IEEE Transactions on Software Engineering. Retrieved 2012-03-21. Check date values in: |date= (help)
- [6] Jones, Capers (June 2008). "Measuring Defect Potentials and Defect Removal Efficiency" (PDF). Crosstalk, The Journal of Defense Software Engineering. Retrieved 2010-10-05.
- [7] Mantyla, M.V.; Lassenius, C (May–June 2009). "What Types of Defects Are Really Discovered in Code Reviews?" (PDF). IEEE Transactions on Software Engineering. Retrieved 2012-03-21.
- [8] Siy, Harvey; Votta, Lawrence (2004-12-01). "Does the Modern Code Inspection Have Value?" (PDF). *unomaha.edu*. Retrieved 2015-02-17.

9.10.6 Further reading

- Jason Cohen (2006). *Best Kept Secrets of Peer Code Review (Modern Approach. Practical Advice.)*. Smartbearsoftware.com. ISBN 1-59916-067-6.

9.10.7 External links

- "A Guide to Code Inspections" (Jack G. Ganssle)
- Article Four Ways to a Practical Code Review
- Security code review guidelines
- What is Code Review? by Tom Huston
- Code Review - Write your code right

9.11 Automated code review

Automated code review software checks source code for compliance with a predefined set of rules or best practices. The use of analytical methods to inspect and review source code to detect bugs has been a standard development practice. This process can be accomplished both manually and in an automated fashion.^[1] With automation, software tools provide assistance with the code review and inspection process. The review program or tool typically displays a list of warnings (violations of programming standards). A review program can also provide an automated or a programmer-assisted way to correct the issues found.

Some **static code analysis** tools can be used to assist with automated code review. They do not compare favorably to manual reviews, however they can be done faster and more efficiently. These tools also encapsulate deep knowledge of underlying rules and semantics required to perform this type analysis such that it does not require the human code reviewer to have the same level of expertise as an expert human auditor.^[1] Many **Integrated Development Environments** also provide basic automated code review functionality. For example the **Eclipse**^[2] and **Microsoft Visual Studio**^[3] IDEs support a variety of plugins that facilitate code review.

Next to **static code analysis** tools, there are also tools that analyze and **visualize software structures** and help humans to better understand these. Such systems are geared more to analysis because they typically do not contain a predefined set of rules to check software against. Some of these tools (e.g. **Imagix 4D**, **Resharper**, **SonarJ**, **Sotoarc**, **Structure101**, **ACTool**^[4]) allow one to define target architectures and enforce that target architecture constraints are not violated by the actual software implementation.

9.11.1 Automated code review tools

Main article: List of tools for static code analysis

9.11.2 See also

- Program analysis (computer science)
- Automated code analysis levels and requirements

9.11.3 References

- [1] Gomes, Ivo; Morgado, Pedro; Gomes, Tiago; Moreira, Rodrigo (2009). "An overview of the Static Code Analysis approach in Software Development" (PDF). Unversidade do Porto. Retrieved 2010-10-03.
- [2] "Collaborative Code Review Tool Development". www.eclipse.org. Retrieved 2010-10-13.
- [3] "Code Review Plug-in for Visual Studio 2008, Review-Pal". www.codeproject.com. Retrieved 2010-10-13.
- [4] Architecture Consistency plugin for Eclipse

9.12 Code reviewing software

Code reviewing software is computer software that helps humans find flaws in program source code. It can be divided into two categories:

- Automated code review software checks source code against a predefined set of rules and produces reports.
 - Different types of browsers visualise software structure and help humans better understand its structure. Such systems are geared more to analysis because they typically do not contain a predefined set of rules to check software against.
- Manual code review tools allow people to collaboratively inspect and discuss changes, storing the history of the process for future reference.

9.13 Static code analysis

Static program analysis is the analysis of computer software that is performed without actually executing programs (analysis performed on executing programs is known as *dynamic analysis*).^[1] In most cases the analysis is performed on some version of the source code, and in the other cases, some form of the object code.

The term is usually applied to the analysis performed by an *automated tool*, with human analysis being called program understanding, *program comprehension*, or *code review*. Software inspections and Software walkthroughs are also used in the latter case.

9.13.1 Rationale

The sophistication of the analysis performed by tools varies from those that only consider the behavior of individual statements and declarations, to those that include the complete *source code* of a program in their analysis. The uses of the information obtained from the analysis vary from highlighting possible coding errors (e.g., the lint tool) to *formal methods* that mathematically prove properties about a given program (e.g., its behavior matches that of its specification).

Software metrics and reverse engineering can be described as forms of static analysis. Deriving software metrics and static analysis are increasingly deployed together, especially in creation of embedded systems, by defining so-called *software quality objectives*.^[2]

A growing commercial use of static analysis is in the verification of properties of software used in safety-critical computer systems and locating potentially vulnerable code.^[3] For example the following industries have identified the use of static code analysis as a means of improving the quality of increasingly sophisticated and complex software:

1. **Medical software:** The U.S. Food and Drug Administration (FDA) has identified the use of static analysis for medical devices.^[4]
2. **Nuclear software:** In the UK the **Health and Safety Executive** recommends the use of static analysis on **Reactor Protection Systems**.^[5]
3. **Aviation software** (in combination with dynamic analysis)^[6]

A study in 2012 by VDC Research reports that 28.7% of the embedded software engineers surveyed currently use static analysis tools and 39.7% expect to use them within 2 years.^[7] A study from 2010 found that 60% of the interviewed developers in European research projects made at least use of their basic IDE built-in static analyzers. However, only about 10% employed an additional other (and perhaps more advanced) analysis tool.^[8]

In the application security industry the name *Static Application Security Testing* (SAST) is also used. Actually, SAST is an important part of Security Development Life-cycles (SDLs) such as the SDL defined by Microsoft^[9] and a common practice in software companies.^[10]

9.13.2 Tool types

The OMG (Object Management Group) published a study regarding the types of software analysis required for software quality measurement and assessment. This document on “How to Deliver Resilient, Secure, Efficient, and Easily Changed IT Systems in Line with CISQ Recommendations” describes three levels of software analysis.^[11]

Unit Level Analysis that takes place within a specific program or subroutine, without connecting to the context of that program.

Technology Level Analysis that takes into account interactions between unit programs to get a more holistic and semantic view of the overall program in order to find issues and avoid obvious false positives.

System Level Analysis that takes into account the interactions between unit programs, but without being limited to one specific technology or programming language.

A further level of software analysis can be defined.

Mission/Business Level Analysis that takes into account the business/mission layer terms, rules and processes that are implemented within the software system for its operation as part of enterprise or program/mission layer activities. These elements are implemented without being limited to one specific technology or programming language and in many cases are distributed across multiple languages but are statically extracted and analyzed for system understanding for mission assurance.

9.13.3 Formal methods

Formal methods is the term applied to the analysis of software (and computer hardware) whose results are obtained purely through the use of rigorous mathematical methods. The mathematical techniques used include denotational semantics, axiomatic semantics, operational semantics, and abstract interpretation.

By a straightforward reduction to the halting problem, it is possible to prove that (for any Turing complete language), finding all possible run-time errors in an arbitrary program (or more generally any kind of violation of a specification on the final result of a program) is undecidable: there is no mechanical method that can always answer truthfully whether an arbitrary program may or may not exhibit runtime errors. This result dates from the works of Church, Gödel and Turing in the 1930s (see: Halting problem and Rice’s theorem). As with many undecidable questions, one can still attempt to give useful approximate solutions.

Some of the implementation techniques of formal static analysis include:^[12]

- **Model checking**, considers systems that have finite state or may be reduced to finite state by abstraction;
- **Data-flow analysis**, a lattice-based technique for gathering information about the possible set of values;
- **Abstract interpretation**, to model the effect that every statement has on the state of an abstract machine (i.e., it ‘executes’ the software based on the mathematical properties of each statement and declaration). This abstract machine over-approximates the behaviours of the system: the abstract system is thus made simpler to analyze, at the expense of *incompleteness* (not every property true of the original system is true of the abstract system). If properly done, though, abstract interpretation is *sound* (every property true of the abstract system can be mapped to a true property of the original system).^[13] The **Frama-c** value analysis plugin and **Polyspace** heavily rely on abstract interpretation.
- **Hoare logic**, a formal system with a set of logical rules for reasoning rigorously about the correctness of computer programs. There is tool support for some programming languages (e.g., the **SPARK** programming language (a subset of Ada) and the **Java Modeling Language — JML** — using **ESC/Java** and **ESC/Java2**, **Frama-c WP** (weakest precondition) plugin for the C language extended with **ACSL** (ANSI/ISO C Specification Language)).
- **Symbolic execution**, as used to derive mathematical expressions representing the value of mutated variables at particular points in the code.

9.13.4 See also

- Shape analysis (software)
- Formal semantics of programming languages
- Formal verification
- Code audit
- Documentation generator
- List of tools for static code analysis

9.13.5 References

- [1] Wichmann, B. A.; Canning, A. A.; Clutterbuck, D. L.; Winsbarrow, L. A.; Ward, N. J.; Marsh, D. W. R. (Mar 1995). “Industrial Perspective on Static Analysis.” (PDF). *Software Engineering Journal*: 69–75. Archived from the original (PDF) on 2011-09-27.

- [2] “Software Quality Objectives for Source Code” (PDF). *Proceedings: Embedded Real Time Software and Systems 2010 Conference*, ERTS2010.org, Toulouse, France: Patrick Briand, Martin Brochet, Thierry Cambois, Emmanuel Coutenceau, Olivier Guetta, Daniel Mainberte, Frederic Mondot, Patrick Munier, Loic Noury, Philippe Spozio, Frederic Retailleau.
- [3] *Improving Software Security with Precise Static and Runtime Analysis* (PDF), Benjamin Livshits, section 7.3 “Static Techniques for Security”. Stanford doctoral thesis, 2006.
- [4] FDA (2010-09-08). “Infusion Pump Software Safety Research at FDA”. Food and Drug Administration. Retrieved 2010-09-09.
- [5] Computer based safety systems - technical guidance for assessing software aspects of digital computer based protection systems, http://www.hse.gov.uk/nuclear/operational/tech_asst_guides/tast046.pdf
- [6] Position Paper CAST-9. Considerations for Evaluating Safety Engineering Approaches to Software Assurance // FAA, Certification Authorities Software Team (CAST), January, 2002: “Verification. A combination of both static and dynamic analyses should be specified by the applicant/developer and applied to the software.”
- [7] VDC Research (2012-02-01). “Automated Defect Prevention for Embedded Software Quality”. VDC Research. Retrieved 2012-04-10.
- [8] Prause, Christian R., René Reiners, and Silviya Dencheva. “Empirical study of tool support in highly distributed research projects.” *Global Software Engineering (ICGSE)*, 2010 5th IEEE International Conference on. IEEE, 2010 <http://ieeexplore.ieee.org/ielx5/5581168/5581493/05581551.pdf>
- [9] M. Howard and S. Lipner. *The Security Development Lifecycle: SDL: A Process for Developing Demonstrably More Secure Software*. Microsoft Press, 2006. ISBN 978-0735622142 I
- [10] Achim D. Brucker and Uwe Sodan. *Deploying Static Application Security Testing on a Large Scale*. In *GI Sicherheit 2014. Lecture Notes in Informatics*, 228, pages 91-101, GI, 2014. <https://www.brucker.ch/bibliography/download/2014/brucker.ea-sast-experiences-2014.pdf>
- [11] http://www.omg.org/CISQ_compliant_IT_Systemsv.4-3.pdf
- [12] Vijay D’Silva et al. (2008). “A Survey of Automated Techniques for Formal Software Verification” (PDF). *Transactions On CAD*. Retrieved 2015-05-11.
- [13] Jones, Paul (2010-02-09). “A Formal Methods-based verification approach to medical device software analysis”. *Embedded Systems Design*. Retrieved 2010-09-09.
- Ayewah, Nathaniel; Hovemeyer, David; Morgenthaler, J. David; Penix, John; Pugh, William (2008). “Using Static Analysis to Find Bugs”. *IEEE Software* **25** (5): 22–29. doi:10.1109/MS.2008.130.
- Brian Chess, Jacob West (Fortify Software) (2007). *Secure Programming with Static Analysis*. Addison-Wesley. ISBN 978-0-321-42477-8.
- Flemming Nielson, Hanne R. Nielson, Chris Hankin (1999, corrected 2004). *Principles of Program Analysis*. Springer. ISBN 978-3-540-65410-0.
- “Abstract interpretation and static analysis,” International Winter School on Semantics and Applications 2003, by David A. Schmidt

9.13.7 Sources

- Kaner, Cem; Nguyen, Hung Q; Falk, Jack (1988). *Testing Computer Software* (Second ed.). Boston: Thomson Computer Press. ISBN 0-47135-846-0.
- Static Testing C++ Code: A utility to check library usability

9.13.8 External links

- Code Quality Improvement - Coding standards conformance checking (DDJ)
- Competition on Software Verification (SV-COMP)
- Episode 59: Static Code Analysis Interview (Podcast) at *Software Engineering Radio*
- Implementing Automated Governance for Coding Standards Explains why and how to integrate static code analysis into the build process
- Integrate static analysis into a software development process
- .NET Static Analysis (InfoQ)
- Static Code Analysis - Polyspace
- The SAMATE Project, a resource for Automated Static Analysis tools

9.13.6 Bibliography

- Syllabus and readings for Alex Aiken’s Stanford CS295 course.

9.14 List of tools for static code analysis

This is a list of tools for static code analysis.

9.14.1 By language

Multi-language

- **Axivion Bauhaus Suite** – A tool for Ada, C, C++, C#, and Java code that performs various analyses such as architecture checking, interface analyses, and clone detection.
- **Black Duck Suite** – Analyzes the composition of software source code and binary files, searches for reusable code, manages open source and third-party code approval, honors the legal obligations associated with mixed-origin code, and monitors related security vulnerabilities.
- **CAST Application Intelligence Platform** – Detailed, audience-specific dashboards to measure quality and productivity. 30+ languages, C, C++, Java, .NET, Oracle, PeopleSoft, SAP, Siebel, Spring, Struts, Hibernate and all major databases.
- **Cigital SecureAssist** - A lightweight IDE plugin that points out common security vulnerabilities in real time as the developer is coding. Supports Java, .NET, and PHP.
- **ConQAT** – Continuous quality assessment toolkit that allows flexible configuration of quality analyses (architecture conformance, clone detection, quality metrics, etc.) and dashboards. Supports Java, C#, C++, JavaScript, ABAP, Ada and many other languages.
- **Coverity SAVE** – A static code analysis tool for C, C++, C# and Java source code. Coverity commercialized a research tool for finding bugs through static analysis, the Stanford Checker. Scans using Coverity are available free of charge for open-source projects.^[1]
- **DMS Software Reengineering Toolkit** – Supports custom analysis of C, C++, C#, Java, COBOL, PHP, Visual Basic and many other languages. Also COTS tools for clone analysis, dead code analysis, and style checking.
- **HP Fortify Static Code Analyzer** – Helps developers identify software security vulnerabilities in C/C++, Java, JSP, .NET, ASP.NET, classic ASP, ColdFusion, PHP, Visual Basic 6, VBScript, JavaScript, PL/SQL, T-SQL, Python, Objective-C and COBOL and configuration files.
- **GrammarTech CodeSonar** – Defect detection (buffer overruns, memory leaks, etc.), concurrency and security checks, architecture visualization and software metrics for C, C++, and Java source code.
- **IBM Rational AppScan Source Edition** – Analyzes source code to identify security vulnerabilities while integrating security testing with software development processes and systems. Supports C/C++, .NET, Java, JSP, JavaScript, ColdFusion, Classic ASP, PHP, Perl, Visual Basic 6, PL/SQL, T-SQL, and COBOL.
- **Imagix 4D** – Identifies problems in variable use, task interaction and concurrency, especially in embedded applications, as part of an overall system for understanding, improving and documenting C, C++ and Java code.
- **Kiuwan** – supports Objective-C, Java, JSP, Javascript, PHP, C, C++, ABAP, COBOL, JCL, C#, PL/SQL, Transact-SQL, SQL, Visual Basic, VB.NET, Android, and Hibernate code.
- **LDRA Testbed** – A software analysis and testing tool suite for C, C++, Ada83, Ada95 and Assembler (Intel, Freescale, Texas Instruments).
- **MALPAS** – A software static analysis toolset for a variety of languages including Ada, C, Pascal and Assembler (Intel, PowerPC and Motorola). Used primarily for safety critical applications in Nuclear and Aerospace industries.
- **Moose** – Moose started as a software analysis platform with many tools to manipulate, assess or visualize software. It can evolve to a more generic data analysis platform. Supported languages are C/C++, Java, Smalltalk, .NET, more may be added.
- **Parasoft** – Provides static analysis (pattern-based, flow-based, in-line, metrics) for C, C++, Java, .NET (C#, VB.NET, etc.), JSP, JavaScript, XML, and other languages. Through a Development Testing Platform, static code analysis functionality is integrated with unit testing, peer code review, runtime error detection and traceability.
- **Copy/Paste Detector (CPD)** – PMDs duplicate code detection for (e.g.) Java, JSP, C, C++, ColdFusion, PHP and JavaScript^[2] code.
- **Polyspace** – Uses abstract interpretation to detect and prove the absence of certain run time errors in source code for C, C++, and Ada
- **Pretty Diff** - A language-specific code comparison tool that features language-specific analysis reporting in addition to language-specific minification and beautification algorithms.
- **Protecode** – Analyzes the composition of software source code and binary files, searches for open source and third party code and their associated licensing obligations. Can also detect security vulnerabilities.
- **Klocwork** – Provides security vulnerability, standards compliance (MISRA, ISO 26262 and others),

defect detection and build-over-build trend analysis for C, C++, C# and Java.

- **Rogue Wave Software OpenLogic** – Scans source code and binaries to identify open source code and licenses, manages open source policies and approvals, reports security vulnerabilities, and provides open source technical support.
- **Semmlle** – Supports C, C++, C#, Java, JavaScript, Objective-C, Python and Scala.
- **SofCheck Inspector** – Static detection of logic errors, race conditions, and redundant code for Ada and Java; automatically extracts pre/postconditions from code.
- **SonarQube** – A continuous inspection engine to manage the technical debt: unit tests, complexity, duplication, design, comments, coding standards and potential problems. Supports languages: ABAP, C, C++, CSS, Objective-C, COBOL, C#, Flex, Forms, Groovy, Java, JavaScript, Natural, PHP, PL/SQL, Visual Basic 6, Web, XML, Python.
- **Sotoarc/Sotograph** – Architecture and quality in-depth analysis and monitoring for C, C++, C#, Java, ABAP.
- **SQuORE** is a multi-purpose and multi-language monitoring tool^[3] for software projects.
- **SourceMeter** - A platform-independent, command-line static source code analyzer for Java, C/C++, RPG IV (AS/400) and Python.
- **Veracode** – Finds security flaws in application binaries and bytecode without requiring source. Supported languages include C, C++, .NET (C#, C++/CLI, VB.NET, ASP.NET), Java, JSP, ColdFusion, PHP, Ruby on Rails, JavaScript (including Node.js), Objective-C, Classic ASP, Visual Basic 6, and COBOL, including mobile applications on the Windows Mobile, BlackBerry, Android, and iOS platforms and written in JavaScript cross platform frameworks.^[4]
- **Yasca** – Yet Another Source Code Analyzer, a plugin-based framework to scan arbitrary file types, with plugins for C/C++, Java, JavaScript, ASP, PHP, HTML/CSS, ColdFusion, COBOL, and other file types. It integrates with other scanners, including FindBugs, PMD, and Pixy.

.NET

- **.NET Compiler Platform** (Codename "Roslyn") - Open-source compiler framework for C# and Visual Basic .NET developed by Microsoft .NET. Provides an API for analyzing and manipulating syntax.

- **CodeIt.Right** – Combines static code analysis and automatic refactoring to best practices which allows automatic correction of code errors and violations; supports C# and VB.NET.
- **CodeRush** – A plugin for Visual Studio which alerts users to violations of best practices.
- **FxCop** – Free static analysis for Microsoft .NET programs that compiles to CIL. Standalone and integrated in some Microsoft Visual Studio editions; by Microsoft.
- **NDepend** – Simplifies managing a complex .NET code base by analyzing and visualizing code dependencies, by defining design rules, by doing impact analysis, and by comparing different versions of the code. Integrates into Visual Studio.
- **Parasoft dotTEST** – A static analysis, unit testing, and code review plugin for Visual Studio; works with languages for Microsoft .NET Framework and .NET Compact Framework, including C#, VB.NET, ASP.NET and Managed C++.
- **StyleCop** – Analyzes C# source code to enforce a set of style and consistency rules. It can be run from inside of Microsoft Visual Studio or integrated into an MSBuild project.

Ada

- **AdaControl** – A tool to control occurrences of various entities or programming patterns in Ada code, used for checking coding standards, enforcement of safety related rules, and support for various manual inspections.
- **CodePeer** – An advanced static analysis tool that detects potential run-time logic errors in Ada programs.
- **Fluctuat** – Abstract interpreter for the validation of numerical properties of programs.
- **LDRA Testbed** – A software analysis and testing tool suite for Ada83/95.
- **Polyspace** – Uses abstract interpretation to detect and prove the absence of certain run time errors in source code.
- **SofCheck Inspector** – (Bought by AdaCore) Static detection of logic errors, race conditions, and redundant code for Ada; automatically extracts pre/postconditions from code.

C/C++

- **Astrée** – finds all potential runtime errors by abstract interpretation, can prove the absence of runtime errors and can prove functional assertions; tailored towards safety-critical C code (e.g. avionics).
- **BLAST** – (Berkeley Lazy Abstraction Software verification Tool) – An open-source software model checker for C programs based on lazy abstraction (follow-on project is CPAchecker.^[5]).
- **Cppcheck** – Open-source tool that checks for several types of errors, including use of STL.
- **cpplint** – An open-source tool that checks for compliance with Google’s style guide for C++ coding.
- **Clang** – An open-source compiler that includes a static analyzer (Clang Static Analyzer).
- **Coccinelle** – An open-source source code pattern matching and transformation.
- **Cppdepend** – Simplifies managing a complex C/C++ code base by analyzing and visualizing code dependencies, by defining design rules, by doing impact analysis, and comparing different versions of the code.
- **ECLAIR** – A platform for the automatic analysis, verification, testing and transformation of C and C++ programs.
- **Eclipse (software)** – An open-source IDE that includes a static code analyzer (CODAN).
- **Fluctuat** – Abstract interpreter for the validation of numerical properties of programs.
- **Frama-C** – An open-source static analysis framework for C.
- **Goanna** – A software analysis tool for C/C++.
- **Klocwork Insight** – A static analysis tool for C/C++.
- **Lint** – The original static code analyzer for C.
- **LDRA Testbed** – A software analysis and testing tool suite for C/C++.
- **Parasoft C/C++test** – A C/C++ tool that does static analysis, unit testing, code review, and runtime error detection; plugins available for Visual Studio and Eclipse-based IDEs.
- **PC-Lint** – A software analysis tool for C/C++.
- **Polyspace** – Uses abstract interpretation to detect and prove the absence of run time errors, Dead Code in source code as well as used to check all MISRA (2004, 2012) rules (directives, non directives).

- **PVS-Studio** – A software analysis tool for C, C++, C++11, C++/CX (Component Extensions).
- **PRQA QA-C and QA-C++** – Deep static analysis of C/C++ for quality assurance and guideline/coding standard enforcement with MISRA support.
- **SLAM project** – a project of Microsoft Research for checking that software satisfies critical behavioral properties of the interfaces it uses.
- **Sparse** – An open-source tool designed to find faults in the Linux kernel.
- **Splint** – An open-source evolved version of Lint, for C.

Java

- **Checkstyle** – Besides some static code analysis, it can be used to show violations of a configured coding standard.
- **FindBugs** – An open-source static bytecode analyzer for Java (based on Jakarta BCEL) from the University of Maryland.
- **IntelliJ IDEA** – Cross-platform Java IDE with own set of several hundred code inspections available for analyzing code on-the-fly in the editor and bulk analysis of the whole project.
- **JArchitect** – Simplifies managing a complex Java code base by analyzing and visualizing code dependencies, by defining design rules, by doing impact analysis, and by comparing different versions of the code.
- **Jtest** – Testing and static code analysis product by Parasoft.
- **LDRA Testbed** – A software analysis and testing tool suite for Java.
- **PMD** – A static ruleset based Java source code analyzer that identifies potential problems.
- **SemmlerCode** – Object oriented code queries for static program analysis.
- **Sonargraph (formerly SonarJ)** – Monitors conformance of code to intended architecture, also computes a wide range of software metrics.
- **Soot** – A language manipulation and optimization framework consisting of intermediate languages for Java.
- **Squale** – A platform to manage software quality (also available for other languages, using commercial analysis tools though).

- **SonarQube** – is an open source platform for Continuous Inspection of code quality.
- **SourceMeter** - A platform-independent, command-line static source code analyzer for Java, C/C++, RPG IV (AS/400) and Python.
- **ThreadSafe** – A static analysis tool for Java focused on finding concurrency bugs.

JavaScript

- **Google's Closure Compiler** – JavaScript optimizer that rewrites code to be faster and smaller, and checks use of native JavaScript functions.
- **JSLint** – JavaScript **syntax checker** and validator.
- **JSHint** – A community driven fork of JSLint.

Objective-C/Objective-C++

- **Clang** – The free Clang project includes a static analyzer. As of version 3.2, this analyzer is included in **Xcode**.^[6]

Opa

- **Opa** includes its own static analyzer. As the language is intended for web application development, the strongly statically typed compiler checks the validity of high-level types for web data, and prevents by default many vulnerabilities such as **XSS** attacks and database code injections.

Packaging

- **Lintian** – Checks Debian software packages for common inconsistencies and errors.
- **Rpmlint** – Checks for common problems in rpm packages.

Perl

- **Perl::Critic** – A tool to help enforce common Perl best practices. Most best practices are based on Damian Conway's Perl Best Practices book.
- **PerlTidy** – Program that acts as a **syntax checker** and tester/enforcer for coding practices in Perl.
- **Padre** – An IDE for Perl that also provides static code analysis to check for common beginner errors.

PHP

- **RIPS** – A static code analyzer and audit framework for vulnerabilities in PHP applications.

PL/SQL

- **TOAD** - A PL/SQL development environment with a Code xPert component that reports on general code efficiency as well as specific programming issues.

Python

- **Pylint** – Static code analyzer. Quite stringent; includes many stylistic warnings as well.
- **PyCharm** – Cross-platform Python IDE with code inspections available for analyzing code on-the-fly in the editor and bulk analysis of the whole project.

9.14.2 Formal methods tools

Tools that use **sound**, i.e. no false negatives, **formal methods** approach to static analysis (e.g., using static **program assertions**):

- **Astrée** – finds all potential **runtime errors** by **abstract interpretation**, can prove the absence of runtime errors and can prove functional assertions; tailored towards safety-critical C code (e.g. avionics).
- **CodePeer** – Statically determines and documents **pre-** and **post-conditions** for Ada subprograms; statically checks **preconditions** at all call sites.
- **ECLAIR** – Uses formal methods-based static code analysis techniques such as **abstract interpretation** and **model checking** combined with **constraint satisfaction** techniques to detect or prove the absence of certain **run time errors** in source code.
- **ESC/Java and ESC/Java2** – Based on **Java Modeling Language**, an enriched version of Java.
- **Frama-C** – An open-source static analysis framework for C.
- **MALPAS** – A formal methods tool that uses **directed graphs** and **regular algebra** to prove that software under analysis correctly meets its mathematical specification.
- **Polyspace** – Uses **abstract interpretation**, a formal methods based technique,^[7] to detect and prove the absence of certain **run time errors** in source code for C/C++, and Ada
- **SPARK Toolset** including the **SPARK Examiner** – Based on the **SPARK** language, a subset of Ada.

9.14.3 See also

- Automated code review
- Best Coding Practices
- Dynamic code analysis
- Software metrics
- Integrated development environment (IDE) and Comparison of integrated development environments. IDEs will usually come with built-in support for static code analysis, or with an option to integrate such support. Eclipse offers such integration mechanism for most different types of extensions (plugins).

9.14.4 References

- [1] “Coverity Scan - Static Analysis”. *scan.coverity.com*. Retrieved 2015-06-17.
- [2] “PMD - Browse /pmd/5.0.0 at SourceForge.net”. Retrieved Dec 9, 2012.
- [3] Baldassari, Boris (2012). “SQuORE: a new approach to software project assessment”, International Conference on Software and Systems Engineering and their Applications, Nov. 2012, Paris, France.
- [4] “White Box Testing/Binary Static Analysis (SAST)”. *Veracode.com*. Retrieved 2015-04-01.
- [5] “CPAchecker”. 2015-02-08.
- [6] “Static Analysis in Xcode”. Apple. Retrieved 2009-09-03.
- [7] Cousot, Patrick (2007). “The Role of Abstract Interpretation in Formal Methods” (PDF). IEEE International Conference on Software Engineering and Formal Methods. Retrieved 2010-11-08.

9.14.5 External links

- The Web Application Security Consortium’s Static Code Analysis Tool List
- Java Static Checkers at DMOZ
- List of Java static code analysis plugins for Eclipse
- List of static source code analysis tools for C
- SAMATE-Source Code Security Analyzers
- SATE – Static Analysis Tool Exposition
- “A Comparison of Bug Finding Tools for Java”, by Nick Rutar, Christian Almazan, and Jeff Foster, University of Maryland. Compares Bandera, ESC/Java 2, FindBugs, JLint, and PMD.
- “Mini-review of Java Bug Finders”, by Rick Jelliffe, O'Reilly Media.

Chapter 10

GUI testing and review

10.1 GUI software testing

In software engineering, **graphical user interface testing** is the process of testing a product's graphical user interface to ensure it meets its specifications. This is normally done through the use of a variety of test cases.

10.1.1 Test Case Generation

To generate a set of test cases, test designers attempt to cover all the functionality of the system and fully exercise the GUI itself. The difficulty in accomplishing this task is twofold: to deal with domain size and with sequences. In addition, the tester faces more difficulty when they have to do regression testing.

Unlike a CLI (command line interface) system, a GUI has many operations that need to be tested. A relatively small program such as Microsoft WordPad has 325 possible GUI operations.^[1] In a large program, the number of operations can easily be an order of magnitude larger.

The second problem is the sequencing problem. Some functionality of the system may only be accomplished with a sequence of GUI events. For example, to open a file a user may have to first click on the File Menu, then select the Open operation, use a dialog box to specify the file name, and focus the application on the newly opened window. Increasing the number of possible operations increases the sequencing problem exponentially. This can become a serious issue when the tester is creating test cases manually.

Regression testing becomes a problem with GUIs as well. A GUI may change significantly, even though the underlying application does not. A test designed to follow a certain path through the GUI may then fail since a button, menu item, or dialog may have changed location or appearance.

These issues have driven the GUI testing problem domain towards automation. Many different techniques have been proposed to automatically generate test suites that are complete and that simulate user behavior.

Most of the testing techniques attempt to build on those previously used to test CLI (Command Line Interface)

programs, but these can have scaling problems when applied to GUI's. For example, Finite State Machine-based modeling^{[2][3]} — where a system is modeled as a finite state machine and a program is used to generate test cases that exercise all states — can work well on a system that has a limited number of states but may become overly complex and unwieldy for a GUI (see also model-based testing).

10.1.2 Planning and artificial intelligence

A novel approach to test suite generation, adapted from a CLI technique^[4] involves using a planning system.^[5] Planning is a well-studied technique from the artificial intelligence (AI) domain that attempts to solve problems that involve four parameters:

- an initial state,
- a goal state,
- a set of operators, and
- a set of objects to operate on.

Planning systems determine a path from the initial state to the goal state by using the operators. As a simple example of a planning problem, given two words and a single operation which replaces a single letter in a word with another, the goal might be to change one word into another.

In ^[1] the authors used the planner IPP^[6] to demonstrate this technique. The system's UI is first analyzed to determine the possible operations. These become the operators used in the planning problem. Next an initial system state is determined, and a goal state is specified that the tester feels would allow exercising of the system. The planning system determines a path from the initial state to the goal state, which becomes the test plan.

Using a planner to generate the test cases has some specific advantages over manual generation. A planning system, by its very nature, generates solutions to planning problems in a way that is very beneficial to the tester:

1. The plans are always valid. The output of the system is either a valid and correct plan that uses the operators to attain the goal state or no plan at all. This is beneficial because much time can be wasted when manually creating a test suite due to invalid test cases that the tester thought would work but didn't.
2. A planning system pays attention to order. Often to test a certain function, the test case must be complex and follow a path through the GUI where the operations are performed in a specific order. When done manually, this can lead to errors and also can be quite difficult and time consuming to do.
3. Finally, and most importantly, a planning system is goal oriented. The tester is focusing test suite generation on what is most important, testing the functionality of the system.

When manually creating a test suite, the tester is more focused on how to test a function (i. e. the specific path through the GUI). By using a planning system, the path is taken care of and the tester can focus on what function to test. An additional benefit of this is that a planning system is not restricted in any way when generating the path and may often find a path that was never anticipated by the tester. This problem is a very important one to combat.^[7]

Another method of generating GUI test cases simulates a novice user. An expert user of a system tends to follow a direct and predictable path through a GUI, whereas a novice user would follow a more random path. A novice user is then likely to explore more possible states of the GUI than an expert.

The difficulty lies in generating test suites that simulate 'novice' system usage. Using **Genetic algorithms** have been proposed to solve this problem.^[7] Novice paths through the system are not random paths. First, a novice user will learn over time and generally won't make the same mistakes repeatedly, and, secondly, a novice user is following a plan and probably has some domain or system knowledge.

Genetic algorithms work as follows: a set of 'genes' are created randomly and then are subjected to some task. The genes that complete the task best are kept and the ones that don't are discarded. The process is again repeated with the surviving genes being replicated and the rest of the set filled in with more random genes. Eventually one gene (or a small set of genes if there is some threshold set) will be the only gene in the set and is naturally the best fit for the given problem.

In the case of GUI testing, the method works as follows. Each gene is essentially a list of random integer values of some fixed length. Each of these genes represents a path through the GUI. For example, for a given tree of widgets, the first value in the gene (each value is called an allele) would select the widget to operate on, the following

alleles would then fill in input to the widget depending on the number of possible inputs to the widget (for example a pull down list box would have one input...the selected list value). The success of the genes are scored by a criterion that rewards the best 'novice' behavior.

A system to do this testing for the X window system, but extensible to any windowing system is described in.^[7] The **X Window** system provides functionality (via **XServer** and the editors' protocol) to dynamically send GUI input to and get GUI output from the program without directly using the GUI. For example, one can call `XSendEvent()` to simulate a click on a pull-down menu, and so forth. This system allows researchers to automate the gene creation and testing so for any given application under test, a set of novice user test cases can be created.

10.1.3 Running the test cases

At first the strategies were migrated and adapted from the CLI testing strategies.

Mouse position capture

A popular method used in the CLI environment is capture/playback. Capture playback is a system where the system screen is "captured" as a bitmapped graphic at various times during system testing. This capturing allowed the tester to "play back" the testing process and compare the screens at the output phase of the test with expected screens. This validation could be automated since the screens would be identical if the case passed and different if the case failed.

Using capture/playback worked quite well in the CLI world but there are significant problems when one tries to implement it on a GUI-based system.^[8] The most obvious problem one finds is that the screen in a GUI system may look different while the state of the underlying system is the same, making automated validation extremely difficult. This is because a GUI allows graphical objects to vary in appearance and placement on the screen. Fonts may be different, window colors or sizes may vary but the system output is basically the same. This would be obvious to a user, but not obvious to an automated validation system.

Event capture

To combat this and other problems, testers have gone 'under the hood' and collected GUI interaction data from the underlying windowing system.^[9] By capturing the window 'events' into logs the interactions with the system are now in a format that is decoupled from the appearance of the GUI. Now, only the event streams are captured. There is some filtering of the event streams necessary since the streams of events are usually very detailed and

most events aren't directly relevant to the problem. This approach can be made easier by using an **MVC** architecture for example and making the view (i. e. the GUI here) as simple as possible while the model and the controller hold all the logic. Another approach is to use the software's **built-in assistive technology**, to use an **HTML interface** or a **three-tier architecture** that makes it also possible to better separate the user interface from the rest of the application.

Another way to run tests on a GUI is to build a driver into the GUI so that commands or events can be sent to the software from another program.^[7] This method of directly sending events to and receiving events from a system is highly desirable when testing, since the input and output testing can be fully automated and user error is eliminated.

10.1.4 See also

- **List of GUI testing tools**

10.1.5 References

- [1] Atif M. Memon, **M.E. Pollack** and M.L. Soffa. Using a Goal-driven Approach to Generate Test Cases for GUIs. ICSE '99 Proceedings of the 21st international conference on Software engineering.
- [2] J.M. Clarke. Automated test generation from a Behavioral Model. In Proceedings of Pacific Northwest Software Quality Conference. IEEE Press, May 1998.
- [3] S. Esmelioglu and L. Apfelbaum. Automated Test generation, execution and reporting. In Proceedings of Pacific Northwest Software Quality Conference. IEEE Press, October 1997.
- [4] A. Howe, A. von Mayrhauser and R.T. Mraz. Test case generation as an AI planning problem. *Automated Software Engineering*, 4:77-106, 1997.
- [5] "Hierarchical GUI Test Case Generation Using Automated Planning" by Atif M. Memon, **Martha E. Pollack**, and Mary Lou Soffa. *IEEE Trans. Softw. Eng.*, vol. 27, no. 2, 2001, pp. 144-155, IEEE Press.
- [6] J. Koehler, B. Nebel, J. Hoffman and Y. Dimopoulos. Extending planning graphs to an ADL subset. *Lecture Notes in Computer Science*, 1348:273, 1997.
- [7] D.J. Kasik and H.G. George. Toward automatic generation of novice user test scripts. In M.J. Tauber, V. Bellotti, R. Jeffries, J.D. Mackinlay, and J. Nielsen, editors, *Proceedings of the Conference on Human Factors in Computing Systems : Common Ground*, pages 244-251, New York, 13-18 April 1996, ACM Press.
- [8] L.R. Kepple. The black art of GUI testing. *Dr. Dobb's Journal of Software Tools*, 19(2):40, Feb. 1994.
- [9] M.L. Hammontree, J.J. Hendrickson and B.W. Hensley. Integrated data capture and analysis tools for research and testing on graphical user interfaces. In P. Bauersfeld, J. Bennett and G. Lynch, editors, *Proceedings of the Conference on Human Factors in Computing System*, pages 431-432, New York, NY, USA, May 1992. ACM Press.

10.2 Usability testing

Usability testing is a technique used in **user-centered interaction design** to evaluate a product by testing it on users. This can be seen as an irreplaceable **usability** practice, since it gives direct input on how real users use the system.^[1] This is in contrast with **usability inspection** methods where experts use different methods to evaluate a user interface without involving users.

Usability testing focuses on measuring a human-made product's capacity to meet its intended purpose. Examples of products that commonly benefit from usability testing are **foods**, consumer products, **web sites** or web applications, **computer interfaces**, documents, and devices. Usability testing measures the usability, or ease of use, of a specific object or set of objects, whereas general **human-computer interaction** studies attempt to formulate universal principles.

10.2.1 What usability testing is not

Simply gathering opinions on an object or document is **market research** or **qualitative research** rather than usability testing. Usability testing usually involves systematic observation under controlled conditions to determine how well people can use the product.^[2] However, often both qualitative and usability testing are used in combination, to better understand users' motivations/perceptions, in addition to their actions.

Rather than showing users a rough draft and asking, "Do you understand this?", usability testing involves watching people trying to *use* something for its intended purpose. For example, when testing instructions for assembling a toy, the test subjects should be given the instructions and a box of parts and, rather than being asked to comment on the parts and materials, they are asked to put the toy together. Instruction phrasing, illustration quality, and the toy's design all affect the assembly process.

10.2.2 Methods

Setting up a usability test involves carefully creating a **scenario**, or realistic situation, wherein the person performs a list of tasks using the product being tested while observers watch and take notes. Several other test instruments such as scripted instructions, **paper prototypes**, and pre- and post-test questionnaires are also used to gather feedback on the product being tested. For example, to

test the attachment function of an e-mail program, a scenario would describe a situation where a person needs to send an e-mail attachment, and ask him or her to undertake this task. The aim is to observe how people function in a realistic manner, so that developers can see problem areas, and what people like. Techniques popularly used to gather data during a usability test include **think aloud protocol**, **co-discovery learning** and **eye tracking**.

Hallway testing

Hallway testing is a quick, cheap method of usability testing in which **randomly**-selected people — e.g., those passing by in the hallway — are asked to try using the product or service. This can help designers identify “brick walls,” problems so serious that users simply cannot advance, in the early stages of a new design. Anyone but project designers and engineers can be used (they tend to act as “expert reviewers” because they are too close to the project).

Remote usability testing

In a scenario where usability evaluators, developers and prospective users are located in different countries and time zones, conducting a traditional lab usability evaluation creates challenges both from the cost and logistical perspectives. These concerns led to research on remote usability evaluation, with the user and the evaluators separated over space and time. Remote testing, which facilitates evaluations being done in the context of the user’s other tasks and technology can be either synchronous or asynchronous. Synchronous usability testing methodologies involve video conferencing or employ remote application sharing tools such as WebEx. The former involves real time one-on-one communication between the evaluator and the user, while the latter involves the evaluator and user working separately.^[3]

Asynchronous methodologies include automatic collection of user’s click streams, user logs of critical incidents that occur while interacting with the application and subjective feedback on the interface by users.^[4] Similar to an in-lab study, an asynchronous remote usability test is task-based and the platforms allow you to capture clicks and task times. Hence, for many large companies this allows you to understand the **WHY** behind the visitors’ intents when visiting a website or mobile site. Additionally, this style of user testing also provides an opportunity to segment feedback by demographic, attitudinal and behavioral type. The tests are carried out in the user’s own environment (rather than labs) helping further simulate real-life scenario testing. This approach also provides a vehicle to easily solicit feedback from users in remote areas quickly and with lower organizational overheads.

Numerous tools are available to address the needs of both these approaches. WebEx and Go-to-meeting are

the most commonly used technologies to conduct a synchronous remote usability test.^[5] However, synchronous remote testing may lack the immediacy and sense of “presence” desired to support a collaborative testing process. Moreover, managing inter-personal dynamics across cultural and linguistic barriers may require approaches sensitive to the cultures involved. Other disadvantages include having reduced control over the testing environment and the distractions and interruptions experienced by the participants’ in their native environment.^[6] One of the newer methods developed for conducting a synchronous remote usability test is by using virtual worlds.^[7] In recent years, conducting usability testing asynchronously has also become prevalent and allows testers to provide their feedback at their free time and in their own comfort at home.

Expert review

Expert review is another general method of usability testing. As the name suggests, this method relies on bringing in experts with experience in the field (possibly from companies that specialize in usability testing) to evaluate the usability of a product.

A **Heuristic evaluation** or **Usability Audit** is an evaluation of an interface by one or more Human Factors experts. Evaluators measure the usability, efficiency, and effectiveness of the interface based on usability principles, such as the 10 usability heuristics originally defined by Jakob Nielsen in 1994.^[8]

Nielsen’s Usability Heuristics, which have continued to evolve in response to user research and new devices, include:

- Visibility of System Status
- Match Between System and the Real World
- User Control and Freedom
- Consistency and Standards
- Error Prevention
- Recognition Rather Than Recall
- Flexibility and Efficiency of Use
- Aesthetic and Minimalist Design
- Help Users Recognize, Diagnose, and Recover from Errors
- Help and Documentation

Automated expert review

Similar to expert reviews, **automated expert reviews** provide usability testing but through the use of programs

given rules for good design and heuristics. Though an automated review might not provide as much detail and insight as reviews from people, they can be finished more quickly and consistently. The idea of creating surrogate users for usability testing is an ambitious direction for the Artificial Intelligence community.

A/B testing

Main article: [A/B testing](#)

In web development and marketing, A/B testing or split testing is an experimental approach to web design (especially user experience design), which aims to identify changes to web pages that increase or maximize an outcome of interest (e.g., click-through rate for a banner advertisement). As the name implies, two versions (A and B) are compared, which are identical except for one variation that might impact a user's behavior. Version A might be the one currently used, while version B is modified in some respect. For instance, on an e-commerce website the purchase funnel is typically a good candidate for A/B testing, as even marginal improvements in drop-off rates can represent a significant gain in sales. Significant improvements can be seen through testing elements like copy text, layouts, images and colors.

Multivariate testing or bucket testing is similar to A/B testing but tests more than two versions at the same time.

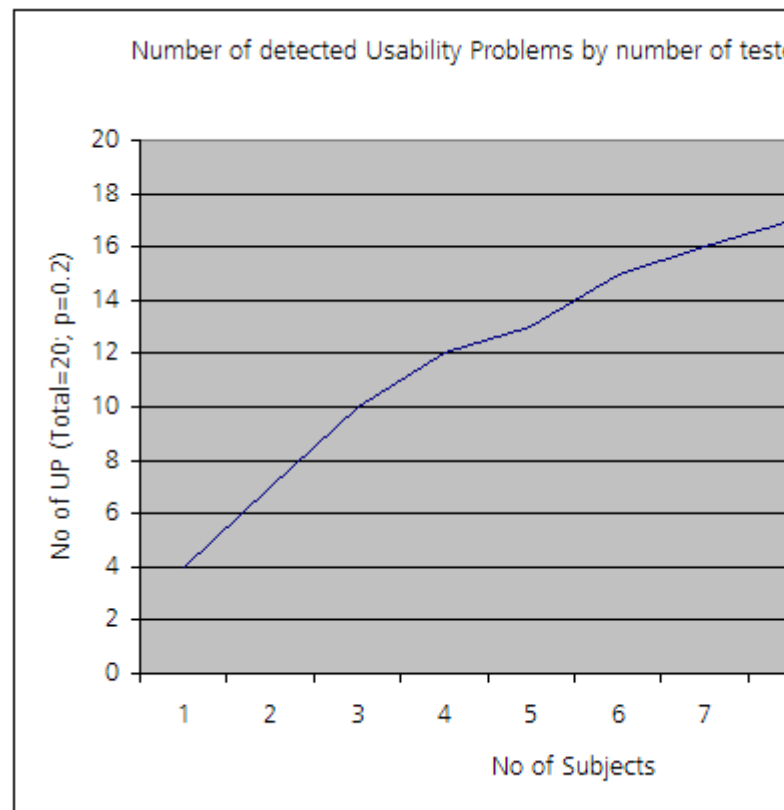
10.2.3 How many users to test?

In the early 1990s, Jakob Nielsen, at that time a researcher at Sun Microsystems, popularized the concept of using numerous small usability tests—typically with only five test subjects each—at various stages of the development process. His argument is that, once it is found that two or three people are totally confused by the home page, little is gained by watching more people suffer through the same flawed design. “Elaborate usability tests are a waste of resources. The best results come from testing no more than five users and running as many small tests as you can afford.”^[9] Nielsen subsequently published his research and coined the term *heuristic evaluation*.

The claim of “Five users is enough” was later described by a mathematical model^[10] which states for the proportion of uncovered problems U

$$U = 1 - (1 - p)^n$$

where p is the probability of one subject identifying a specific problem and n the number of subjects (or test sessions). This model shows up as an asymptotic graph towards the number of real existing problems (see figure below).



In later research Nielsen's claim has eagerly been questioned with both *empirical evidence*^[11] and more advanced *mathematical models*.^[12] Two key challenges to this assertion are:

1. Since usability is related to the specific set of users, such a small sample size is unlikely to be representative of the total population so the data from such a small sample is more likely to reflect the sample group than the population they may represent
2. Not every usability problem is equally easy-to-detect. Intractable problems happen to decelerate the overall process. Under these circumstances the progress of the process is much shallower than predicted by the Nielsen/Landauer formula.^[13]

It is worth noting that Nielsen does not advocate stopping after a single test with five users; his point is that testing with five users, fixing the problems they uncover, and then testing the revised site with five different users is a better use of limited resources than running a single usability test with 10 users. In practice, the tests are run once or twice per week during the entire development cycle, using three to five test subjects per round, and with the results delivered within 24 hours to the designers. The number of users actually tested over the course of the project can thus easily reach 50 to 100 people.

In the early stage, when users are most likely to immediately encounter problems that stop them in their tracks, almost anyone of normal intelligence can be used as a test subject. In stage two, testers will recruit test subjects

across a broad spectrum of abilities. For example, in one study, experienced users showed no problem using any design, from the first to the last, while naive user and self-identified power users both failed repeatedly.^[14] Later on, as the design smooths out, users should be recruited from the target population.

When the method is applied to a sufficient number of people over the course of a project, the objections raised above become addressed: The sample size ceases to be small and usability problems that arise with only occasional users are found. The value of the method lies in the fact that specific design problems, once encountered, are never seen again because they are immediately eliminated, while the parts that appear successful are tested over and over. While it's true that the initial problems in the design may be tested by only five users, when the method is properly applied, the parts of the design that worked in that initial test will go on to be tested by 50 to 100 people.

10.2.4 Example

A 1982 **Apple Computer** manual for developers advised on usability testing:^[15]

1. "Select the target audience. Begin your human interface design by identifying your target audience. Are you writing for businesspeople or children?"
2. Determine how much target users know about Apple computers, and the subject matter of the software.
3. Steps 1 and 2 permit designing the user interface to suit the target audience's needs. Tax-preparation software written for accountants might assume that its users know nothing about computers but are expert on the tax code, while such software written for consumers might assume that its users know nothing about taxes but are familiar with the basics of Apple computers.

Apple advised developers, "You should begin testing as soon as possible, using drafted friends, relatives, and new employees":^[15]

Our testing method is as follows. We set up a room with five to six computer systems. We schedule two to three groups of five to six users at a time to try out the systems (often without their knowing that it is the software rather than the system that we are testing). We have two of the designers in the room. Any fewer, and they miss a lot of what is going on. Any more and the users feel as though there is always someone breathing down their necks.

Designers must watch people use the program in person, because^[15]

Ninety-five percent of the stumbling blocks are found by watching the body language of the users. Watch for squinting eyes, hunched shoulders, shaking heads, and deep, heart-felt sighs. When a user hits a snag, he will assume it is "on account of he is not too bright": he will not report it; he will hide it ... Do not make assumptions about why a user became confused. Ask him. You will often be surprised to learn what the user thought the program was doing at the time he got lost.

10.2.5 Usability Testing Education

Usability testing has been a formal subject of academic instruction in different disciplines.^[16]

10.2.6 See also

- ISO 9241
- Software testing
- Educational technology
- Universal usability
- Commercial eye tracking
- Don't Make Me Think
- Software performance testing
- System Usability Scale (SUS)
- Test method
- Tree testing
- RITE Method
- Component-Based Usability Testing
- Crowdsourced testing
- Usability goals
- Heuristic evaluation
- Diary studies

10.2.7 References

- [1] Nielsen, J. (1994). Usability Engineering, Academic Press Inc, p 165
- [2] <http://jerz.setonhill.edu/design/usability/intro.htm>
- [3] Andreasen, Morten Sieker; Nielsen, Henrik Villemann; Schröder, Simon Ormholt; Stage, Jan (2007). "Proceedings of the SIGCHI conference on Human factors in computing systems - CHI '07". p. 1405. doi:10.1145/1240624.1240838. ISBN 9781595935939. lchapter= ignored (help)

- [4] Dray, Susan; Siegel, David (2004). "Remote possibilities?". *Interactions* **11** (2): 10. doi:10.1145/971258.971264.
- [5] <http://www.techved.com/blog/remote-usability>
- [6] Dray, Susan; Siegel, David (March 2004). "Remote possibilities?: international usability testing at a distance". *Interactions* **11** (2): 10–17. doi:10.1145/971258.971264.
- [7] Chalil Madathil, Kapil; Joel S. Greenstein (May 2011). "Synchronous remote usability testing: a new approach facilitated by virtual worlds". *Proceedings of the 2011 annual conference on Human factors in computing systems*. CHI '11: 2225–2234. doi:10.1145/1978942.1979267. ISBN 9781450302289.
- [8] "Heuristic Evaluation". Usability First. Retrieved April 9, 2013.
- [9] "Usability Testing with 5 Users (Jakob Nielsen's Alert-box)". useit.com. 2000-03-13.; references Jakob Nielsen, Thomas K. Landauer (April 1993). "A mathematical model of the finding of usability problems". *Proceedings of ACM INTERCHI'93 Conference (Amsterdam, The Netherlands, 24–29 April 1993)*.
- [10] Virzi, R. A. (1992). "Refining the Test Phase of Usability Evaluation: How Many Subjects is Enough?". *Human Factors* **34** (4): 457–468. doi:10.1177/001872089203400407.
- [11] <http://citeseer.ist.psu.edu/spool01testing.html>
- [12] Caulton, D. A. (2001). "Relaxing the homogeneity assumption in usability testing". *Behaviour & Information Technology* **20** (1): 1–7. doi:10.1080/01449290010020648.
- [13] Schmettow, Heterogeneity in the Usability Evaluation Process. In: M. England, D. & Beale, R. (ed.), *Proceedings of the HCI 2008*, British Computing Society, 2008, 1, 89-98
- [14] Bruce Tognazzini. "Maximizing Windows".
- [15] Meyers, Joe; Tognazzini, Bruce (1982). *Apple IIe Design Guidelines* (PDF). Apple Computer. pp. 11–13, 15.
- [16] Breuch, Lee-Ann; Mark Zachry; Clay Spinuzzi (April 2001). "Usability Instruction in Technical Communication Programs" (PDF). *Journal of Business and Technical Communication* **15** (2): 223–240. doi:10.1177/105065190101500204. Retrieved 3 March 2014.

10.2.8 External links

- Usability.gov

10.3 Think aloud protocol

Think-aloud protocol (or think-aloud protocols; also **talk-aloud protocol**) is a used to gather data in **usability testing** in product design and development, in **psychology** and a range of social sciences (e.g., reading, writing, translation research, decision making, and **process tracing**). The think-aloud method was introduced in the usability field by Clayton Lewis ^[1] while he was at IBM, and is explained in *Task-Centered User Interface Design: A Practical Introduction* by C. Lewis and J. Rieman.^[2] The method was developed based on the techniques of **protocol analysis** by Ericsson and Simon.^{[3][4][5]}

Think-aloud protocols involve participants thinking aloud as they are performing a set of specified tasks. Users are asked to say whatever they are looking at, thinking, doing, and feeling as they go about their task. This enables observers to see first-hand the process of task completion (rather than only its final product). Observers at such a test are asked to objectively take notes of everything that users say, without attempting to interpret their actions and words. Test sessions are often audio- and video-recorded so that developers can go back and refer to what participants did and how they reacted. The purpose of this method is to make explicit what is implicitly present in subjects who are able to perform a specific task.

A related but slightly different data-gathering method is the **talk-aloud protocol**. This involves participants only describing their action but not giving explanations. This method is thought to be more objective in that participants merely report how they go about completing a task rather than interpreting or justifying their actions (see the standard works by Ericsson & Simon).

As Kuusela and Paul ^[6] state the think-aloud protocol can be divided into two different experimental procedures. The first one is the concurrent think-aloud protocol, collected during the decision task. The second procedure is the retrospective think-aloud protocol, gathered after the decision task.

10.3.1 See also

- Comparison of usability evaluation methods
- Protocol analysis
- Partial concurrent thinking aloud
- Retrospective Think Aloud

10.3.2 References

- [1] Lewis, C. H. (1982). *Using the "Thinking Aloud" Method In Cognitive Interface Design* (Technical report). IBM. RC-9265.

- [2] http://groupplab.cpsc.ucalgary.ca/saul/hci_topics/tcsd-book/chap-1_v-1.html *Task-Centered User Interface Design: A Practical Introduction*, by Clayton Lewis and John Rieman.
- [3] Ericsson, K., & Simon, H. (May 1980). "Verbal reports as data". *Psychological Review* **87** (3): 215–251. doi:10.1037/0033-295X.87.3.215.
- [4] Ericsson, K., & Simon, H. (1987). "Verbal reports on thinking". In C. Faerch & G. Kasper (eds.). *Introspection in Second Language Research*. Clevedon, Avon: Multilingual Matters. pp. 24–54.
- [5] Ericsson, K., & Simon, H. (1993). *Protocol Analysis: Verbal Reports as Data* (2nd ed.). Boston: MIT Press. ISBN 0-262-05029-3.
- [6] Kuusela, H., & Paul, P. (2000). "A comparison of concurrent and retrospective verbal protocol analysis". *American Journal of Psychology* (University of Illinois Press) **113** (3): 387–404. doi:10.2307/1423365. JSTOR 1423365. PMID 10997234.

10.4 Usability inspection

Usability inspection is the name for a set of methods where an evaluator inspects a user interface. This is in contrast to **usability testing** where the usability of the interface is evaluated by testing it on real users. Usability inspections can generally be used early in the development process by evaluating prototypes or specifications for the system that can't be tested on users. Usability inspection methods are generally considered to be cheaper to implement than testing on users.^[1]

Usability inspection methods include:

- **Cognitive walkthrough** (task-specific)
- **Heuristic evaluation** (holistic)
- **Pluralistic walkthrough**

10.4.1 References

- [1] Nielsen, Jakob. *Usability Inspection Methods*. New York, NY: John Wiley and Sons, 1994

10.4.2 External links

- [Summary of Usability Inspection Methods](#)

10.4.3 See also

- **Heuristic evaluation**
- **Comparison of usability evaluation methods**

10.5 Cognitive walkthrough

The **cognitive walkthrough** method is a **usability inspection method** used to identify **usability** issues in interactive systems, focusing on how easy it is for new users to accomplish tasks with the system. **Cognitive walkthrough** is task-specific, whereas **heuristic evaluation** takes a holistic view to catch problems not caught by this and other **usability inspection methods**. The method is rooted in the notion that users typically prefer to learn a system by using it to accomplish tasks, rather than, for example, studying a manual. The method is prized for its ability to generate results quickly with low cost, especially when compared to **usability testing**, as well as the ability to apply the method early in the design phases, before coding even begins.

10.5.1 Introduction

A cognitive walkthrough starts with a **task analysis** that specifies the sequence of steps or actions required by a user to accomplish a task, and the system responses to those actions. The designers and developers of the software then walk through the steps as a group, asking themselves a set of questions at each step. Data is gathered during the walkthrough, and afterwards a report of potential issues is compiled. Finally the software is redesigned to address the issues identified.

The effectiveness of methods such as cognitive walkthroughs is hard to measure in applied settings, as there is very limited opportunity for controlled experiments while developing software. Typically measurements involve comparing the number of usability problems found by applying different methods. However, Gray and Salzman called into question the validity of those studies in their dramatic 1998 paper "Damaged Merchandise", demonstrating how very difficult it is to measure the effectiveness of **usability inspection methods**. The consensus in the usability community is that the cognitive walkthrough method works well in a variety of settings and applications.

10.5.2 Walking through the tasks

After the task analysis has been made the participants perform the walkthrough by asking themselves a set of questions for each subtask. Typically four questions are asked:^[1]

- **Will the user try to achieve the effect that the subtask has?** Does the user understand that this subtask is needed to reach the user's goal?
- **Will the user notice that the correct action is available?** E.g. is the button visible?

- **Will the user understand that the wanted sub-task can be achieved by the action?** E.g. the right button is visible but the user does not understand the text and will therefore not click on it.
- **Does the user get appropriate feedback?** Will the user know that they have done the right thing after performing the action?

By answering the questions for each subtask usability problems will be noticed.

10.5.3 Common mistakes

In teaching people to use the walkthrough method, Lewis & Rieman have found that there are two common misunderstandings:^[2]

1. The evaluator doesn't know how to perform the task themselves, so they stumble through the interface trying to discover the correct sequence of actions—and then they evaluate the stumbling process. (The user should identify and perform the **optimal** action sequence.)
2. The walkthrough does not test real users on the system. The walkthrough will often identify many more problems than you would find with a single, unique user in a single test session.

10.5.4 History

The method was developed in the early nineties by Wharton, et al., and reached a large usability audience when it was published as a chapter in Jakob Nielsen's seminal book on usability, "Usability Inspection Methods." The Wharton, et al. method required asking four questions at each step, along with extensive documentation of the analysis. In 2000 there was a resurgence in interest in the method in response to a CHI paper by Spencer who described modifications to the method to make it effective in a real software development setting. Spencer's streamlined method required asking only two questions at each step, and involved creating less documentation. Spencer's paper followed the example set by Rowley, et al. who described the modifications to the method that they made based on their experience applying the methods in their 1992 CHI paper "The Cognitive Jogthrough".

10.5.5 References

- [1] C. Wharton et al. "The cognitive walkthrough method: a practitioner's guide" in J. Nielsen & R. Mack "Usability Inspection Methods" pp. 105-140.
- [2] <http://hcibib.org/tcuid/chap-4.html#4-1>

10.5.6 Further reading

- Blackmon, M. H. Polson, P.G. Muneo, K & Lewis, C. (2002) Cognitive Walkthrough for the Web CHI 2002 vol.4 No.1 pp463–470
- Blackmon, M. H. Polson, Kitajima, M. (2003) Repairing Usability Problems Identified by the Cognitive Walkthrough for the Web CHI 2003 pp497–504.
- Dix, A., Finlay, J., Abowd, G., D., & Beale, R. (2004). Human-computer interaction (3rd ed.). Harlow, England: Pearson Education Limited. p321.
- Gabrielli, S. Mirabella, V. Kimani, S. Catarci, T. (2005) Supporting Cognitive Walkthrough with Video Data: A Mobile Learning Evaluation Study MobileHCI '05 pp77–82.
- Goillau, P., Woodward, V., Kelly, C. & Banks, G. (1998) Evaluation of virtual prototypes for air traffic control - the MACAW technique. In, M. Hanson (Ed.) Contemporary Ergonomics 1998.
- Good, N. S. & Krekelberg, A. (2003) Usability and Privacy: a study of KaZaA P2P file-sharing CHI 2003 Vol.5 no.1 pp137–144.
- Gray, W. & Salzman, M. (1998). Damaged merchandise? A review of experiments that compare usability evaluation methods, *Human-Computer Interaction vol.13 no.3*, 203-61.
- Gray, W.D. & Salzman, M.C. (1998) Repairing Damaged Merchandise: A rejoinder. *Human-Computer Interaction vol.13 no.3* pp325–335.
- Hornbaek, K. & Frokjaer, E. (2005) Comparing Usability Problems and Redesign Proposal as Input to Practical Systems Development CHI 2005 391-400.
- Jeffries, R. Miller, J. R. Wharton, C. Uyeda, K. M. (1991) User Interface Evaluation in the Real World: A comparison of Four Techniques Conference on Human Factors in Computing Systems pp 119 – 124
- Lewis, C. Polson, P, Wharton, C. & Rieman, J. (1990) Testing a Walkthrough Methodology for Theory-Based Design of Walk-Up-and-Use Interfaces Chi '90 Proceedings pp235–242.
- Mahatody, Thomas / Sagar, Mouldi / Kolski, Christophe (2010). State of the Art on the Cognitive Walkthrough Method, Its Variants and Evolutions, *International Journal of Human-Computer Interaction*, 2, 8 741-785.
- Rowley, David E., and Rhoades, David G (1992). The Cognitive Jogthrough: A Fast-Paced User Interface Evaluation Procedure. *Proceedings of CHI '92*, 389-395.

- **Sears, A.** (1998) The Effect of Task Description Detail on Evaluator Performance with Cognitive Walkthroughs CHI 1998 pp259–260.
- **Spencer, R.** (2000) The Streamlined Cognitive Walkthrough Method, Working Around Social Constraints Encountered in a Software Development Company CHI 2000 vol.2 issue 1 pp353–359.
- **Wharton, C. Bradford, J. Jeffries, J. Franzke, M.** Applying Cognitive Walkthroughs to more Complex User Interfaces: Experiences, Issues and Recommendations CHI '92 pp381–388.

10.5.7 External links

- [Cognitive Walkthrough](#)

10.5.8 See also

- [Cognitive dimensions](#), a framework for identifying and evaluating elements that affect the usability of an interface.
- [Comparison of usability evaluation methods](#)

10.6 Heuristic evaluation

This article is about usability evaluation. For a list of Heuristic analysis topics ranging from application of heuristics to antivirus software, see [Heuristic analysis](#).

A **heuristic evaluation** is a [usability inspection](#) method for computer software that helps to identify [usability problems](#) in the [user interface \(UI\) design](#). It specifically involves evaluators examining the interface and judging its compliance with recognized usability principles (the “heuristics”). These evaluation methods are now widely taught and practiced in the [new media](#) sector, where UIs are often designed in a short space of time on a budget that may restrict the amount of money available to provide for other types of interface testing.

10.6.1 Introduction

The main goal of heuristic evaluations is to identify any problems associated with the design of user interfaces. Usability consultant **Jakob Nielsen** developed this method on the basis of several years of experience in teaching and consulting about [usability engineering](#).

Heuristic evaluations are one of the most informal methods^[1] of usability inspection in the field of [human-computer interaction](#). There are many sets of usability design heuristics; they are not mutually exclusive and cover many of the same aspects of user interface design.

Quite often, usability problems that are discovered are categorized—often on a numeric scale—according to their estimated impact on user performance or acceptance. Often the heuristic evaluation is conducted in the context of [use cases](#) (typical user tasks), to provide [feedback](#) to the developers on the extent to which the interface is likely to be compatible with the intended users’ needs and preferences.

The simplicity of heuristic evaluation is beneficial at the early stages of design. This usability inspection method does not require user testing which can be burdensome due to the need for users, a place to test them and a payment for their time. Heuristic evaluation requires only one expert, reducing the complexity and expended time for evaluation. Most heuristic evaluations can be accomplished in a matter of days. The time required varies with the size of the artifact, its complexity, the purpose of the review, the nature of the usability issues that arise in the review, and the competence of the reviewers. Using heuristic evaluation prior to user testing will reduce the number and severity of design errors discovered by users. Although heuristic evaluation can uncover many major usability issues in a short period of time, a criticism that is often leveled is that results are highly influenced by the knowledge of the expert reviewer(s). This “one-sided” review repeatedly has different results than [software performance testing](#), each type of testing uncovering a different set of problems.

10.6.2 Nielsen’s heuristics

Jakob Nielsen’s heuristics are probably the most-used usability heuristics for user interface design. Nielsen developed the heuristics based on work together with **Rolf Molich** in 1990.^{[1][2]} The final set of heuristics that are still used today were released by Nielsen in 1994.^[3] The heuristics as published in Nielsen’s book *Usability Engineering* are as follows^[4]

-

Visibility of system status:

The system should always keep users informed about what is going on, through appropriate feedback within reasonable time.

-

Match between system and the real world:

The system should speak the user’s language, with words, phrases and concepts familiar to the user, rather than system-oriented terms. Follow real-world conventions, making information appear in a natural and logical order.

-

User control and freedom:

Users often choose system functions by mistake and will need a clearly marked “emergency exit” to leave the unwanted state without having to go through an extended dialogue. Support undo and redo.

-

Consistency and standards:

Users should not have to wonder whether different words, situations, or actions mean the same thing. Follow platform conventions.

-

Error prevention:

Even better than good error messages is a careful design which prevents a problem from occurring in the first place. Either eliminate error-prone conditions or check for them and present users with a confirmation option before they commit to the action.

-

Recognition rather than recall:

Minimize the user’s memory load by making objects, actions, and options visible. The user should not have to remember information from one part of the dialogue to another. Instructions for use of the system should be visible or easily retrievable whenever appropriate.

-

Flexibility and efficiency of use:

Accelerators—unseen by the novice user—may often speed up the interaction for the expert user such that the system can cater to both inexperienced and experienced users. Allow users to tailor frequent actions.

-

Aesthetic and minimalist design:

Dialogues should not contain information which is irrelevant or rarely needed. Every extra unit of information in a dialogue competes with the relevant units of information and diminishes their relative visibility.

-

Help users recognize, diagnose, and recover from errors:

Error messages should be expressed in plain language (no codes), precisely indicate the problem, and constructively suggest a solution.

-

Help and documentation:

Even though it is better if the system can be used without documentation, it may be necessary to provide help and documentation. Any such information should be easy to search, focused on the user’s task, list concrete steps to be carried out, and not be too large.

10.6.3 Gerhardt-Powals’ cognitive engineering principles

Although Nielsen is considered the expert and field leader in heuristics, Jill Gerhardt-Powals also developed a set of **cognitive** principles for enhancing computer performance.^[5] These heuristics, or principles, are similar to Nielsen’s heuristics but take a more holistic approach to evaluation. Gerhardt Powals’ principles^[6] are listed below.

- **Automate unwanted workload:**

- free cognitive resources for high-level tasks.
- eliminate mental calculations, estimations, comparisons, and unnecessary thinking.

- **Reduce uncertainty:**

- display data in a manner that is clear and obvious.

- **Fuse data:**

- reduce **cognitive load** by bringing together lower level data into a higher-level summation.

- **Present new information with meaningful aids to interpretation:**

- use a familiar framework, making it easier to absorb.
- use everyday terms, metaphors, etc.

- **Use names that are conceptually related to function:**

- Context-dependent.
- Attempt to improve recall and recognition.
- Group data in consistently meaningful ways to decrease search time.

- **Limit data-driven tasks:**

- Reduce the time spent assimilating raw data.
- Make appropriate use of color and graphics.

- **Include in the displays only that information needed by the user at a given time.**

- **Provide multiple coding of data when appropriate.**

- **Practice judicious redundancy.**

10.6.4 Weinschenk and Barker classification

Susan Weinschenk and Dean Barker created a categorization of heuristics and guidelines by several major providers into the following twenty types:^[7]

- 1. User Control:** heuristics that check whether the user has enough control of the interface.
- 2. Human Limitations:** the design takes into account human limitations, cognitive and sensorial, to avoid overloading them.
- 3. Modal Integrity:** the interface uses the most suitable *modality* for each task: auditory, visual, or motor/kinesthetic.
- 4. Accommodation:** the design is adequate to fulfill the needs and behaviour of each targeted user group.
- 5. Linguistic Clarity:** the language used to communicate is efficient and adequate to the audience.
- 6. Aesthetic Integrity:** the design is visually attractive and tailored to appeal to the target population.
- 7. Simplicity:** the design will not use unnecessary complexity.
- 8. Predictability:** users will be able to form a *mental model* of how the system will behave in response to actions.
- 9. Interpretation:** there are codified rules that try to guess the user *intentions* and anticipate the actions needed.
- 10. Accuracy:** There are no errors, i.e. the result of user actions correspond to their goals.
- 11. Technical Clarity:** the concepts represented in the interface have the highest possible correspondence to the *domain* they are modeling.
- 12. Flexibility:** the design can be adjusted to the needs and behaviour of each particular user.
- 13. Fulfillment:** the *user experience* is adequate.
- 14. Cultural Propriety:** user's cultural and social expectations are met.
- 15. Suitable Tempo:** the pace at which users works with the system is adequate.
- 16. Consistency:** different parts of the system have the same style, so that there are no different ways to represent the same information or behavior.
- 17. User Support:** the design will support learning and provide the required assistance to usage.
- 18. Precision:** the steps and results of a task will be what the user wants.
- 19. Forgiveness:** the user will be able to recover to an adequate state after an error.
- 20. Responsiveness:** the interface provides enough feed-

back information about the system status and the task completion.

10.6.5 See also

- *Usability inspection*
- *Progressive disclosure*
- *Cognitive bias*
- *Cognitive dimensions*, a framework for evaluating the design of notations, user interfaces and programming languages

10.6.6 References

- [1] Nielsen, J., and Molich, R. (1990). Heuristic evaluation of user interfaces, Proc. ACM CHI'90 Conf. (Seattle, WA, 1–5 April), 249-256
- [2] Molich, R., and Nielsen, J. (1990). Improving a human-computer dialogue, Communications of the ACM 33, 3 (March), 338-348
- [3] Nielsen, J. (1994). Heuristic evaluation. In Nielsen, J., and Mack, R.L. (Eds.), *Usability Inspection Methods*, John Wiley & Sons, New York, NY
- [4] Nielsen, Jakob (1994). *Usability Engineering*. San Diego: Academic Press. pp. 115–148. ISBN 0-12-518406-9.
- [5] Gerhardt-Powals, Jill (1996). "Cognitive engineering principles for enhancing human - computer performance". *International Journal of Human-Computer Interaction* 8 (2): 189–211. doi:10.1080/10447319609526147.
- [6] Heuristic Evaluation - Usability Methods – What is a heuristic evaluation? Usability.gov
- [7] Jeff Sauro. "What's the difference between a Heuristic Evaluation and a Cognitive Walkthrough?". MeasuringUsability.com.

10.6.7 Further reading

- Dix, A., Finlay, J., Abowd, G., D., & Beale, R. (2004). *Human-computer interaction* (3rd ed.). Harlow, England: Pearson Education Limited. p324
- Gerhardt-Powals, Jill (1996). *Cognitive Engineering Principles for Enhancing Human-Computer Performance*. "International Journal of Human-Computer Interaction", 8(2), 189-21
- Hvannberg, E., Law, E., & Lárusdóttir, M. (2007) "Heuristic Evaluation: Comparing Ways of Finding and Reporting Usability Problems", *Interacting with Computers*, 19 (2), 225-240
- Nielsen, J. and Mack, R.L. (eds) (1994). *Usability Inspection Methods*, John Wiley & Sons Inc

10.6.8 External links

- Jakob Nielsen's introduction to Heuristic Evaluation - Including fundamental points, methodologies and benefits.
- Alternate First Principles (Tognazzini) - Including Jakob Nielsen's ten rules of thumb
- Heuristic Evaluation at Usability.gov
- Heuristic Evaluation in the RKBExplorer
- Remote (online) Heuristic Evaluation Tool at usabilityTEST.com.

10.7 Pluralistic walkthrough

The **Pluralistic Walkthrough** (also called a **Participatory Design Review**, **User-Centered Walkthrough**, **Storyboarding**, **Table-Topping**, or **Group Walkthrough**) is a usability inspection method used to identify usability issues in a piece of software or website in an effort to create a maximally usable **human-computer interface**. The method centers on using a group of users, developers and usability professionals to step through a task scenario, discussing usability issues associated with dialog elements involved in the scenario steps. The group of experts used is asked to assume the role of typical users in the testing. The method is prized for its ability to be utilized at the earliest design stages, enabling the resolution of usability issues quickly and early in the design process. The method also allows for the detection of a greater number of usability problems to be found at one time due to the interaction of multiple types of participants (users, developers and usability professionals). This type of usability inspection method has the additional objective of increasing developers' sensitivity to users' concerns about the product design.

10.7.1 Procedure

Walkthrough Team

A walkthrough team must be assembled prior to the pluralistic walkthrough. Three types of participants are included in the walkthrough: representative users, product developers and **human factors** (usability) engineers/professionals. Users should be representative of the target audience, and are considered the primary participants in the usability evaluation. Product developers answer questions about design and suggest solutions to interface problems users have encountered. Human factors professionals usually serve as the facilitators and are also there to provide feedback on the design as well as recommend design improvements. The role of the facilitator is to guide users through tasks and facilitate collaboration

between users and developers. It is best to avoid having a product developer assume the role of facilitator, as they can get defensive to criticism of their product.

Materials

The following materials are needed to conduct a pluralistic walkthrough:

- Room large enough to accommodate approximately 6-10 users, 6-10 developers and 2-3 usability engineers
- Printed screen-shots (paper prototypes) put together in packets in the same order that the screens would be displayed when users were carrying out the specific tasks. This includes hard copy panels of screens, dialog boxes, menus, etc. presented in order.
- Hard copy of the task scenario for each participant. There are several scenarios defined in this document complete with the data to be manipulated for the task. Each participant receives a package that enables him or her to write a response (i.e. the action to take on that panel) directly onto the page. The task descriptions for the participant are short direct statements.
- Writing utensils for marking up screen shots and filling out documentation and questionnaires.

Participants are given written instructions and rules at the beginning of the walkthrough session. The rules indicate to all participants (users, designers, usability engineers) to:

- Assume the role of the user
- To write on the panels the actions they would take in pursuing the task at hand
- To write any additional comments about the task
- Not flip ahead to other panels until they are told to
- To hold discussion on each panel until the facilitator decides to move on

Tasks

Pluralistic walkthroughs are group activities that require the following steps be followed:

1. Participants are presented with the instructions and the ground rules mentioned above. The task description and scenario package are also distributed.

2. Next, a product expert (usually a product developer) gives a brief overview of key product concepts and interface features. This overview serves the purpose of stimulating the participants to envision the ultimate final product (software or website), so that the participants would gain the same knowledge and expectations of the ultimate product that product end users are assumed to have.
3. The usability testing then begins. The scenarios are presented to the panel of participants and they are asked to write down the sequence of actions they would take in attempting to complete the specified task (i.e. moving from one screen to another). They do this individually without conferring amongst each other.
4. Once everyone has written down their actions independently, the participants discuss the actions that they suggested for that task. They also discuss potential usability problems. The order of communication is usually such that the representative users go first so that they are not influenced by the other panel members and are not deterred from speaking.
5. After the users have finished, the usability experts present their findings to the group. The developers often explain their rationale behind their design. It is imperative that the developers assume an attitude of welcoming comments that are intended to improve the usability of their product.
6. The walkthrough facilitator presents the correct answer if the discussion is off course and clarifies any unclear situations.
7. After each task, the participants are given a brief questionnaire regarding the usability of the interface they have just evaluated.
8. Then the panel moves on to the next task and round of screens. This process continues until all the scenarios have been evaluated.

Throughout this process, usability problems are identified and classified for future action. The presence of the various types of participants in the group allows for a potential synergy to develop that often leads to creative and collaborative solutions. This allows for a focus on user-centered perspective while also considering the engineering constraints of practical system design.

10.7.2 Characteristics of Pluralistic Walkthrough

Other types of usability inspection methods include: Cognitive Walkthroughs, Interviews, Focus Groups, Remote Testing and Think Aloud Protocol. Pluralistic Walkthroughs share some of the same characteristics

with these other traditional walkthroughs, especially with cognitive walkthroughs, but there are some defining characteristics (Nielsen, 1994):

- The main modification, with respect to usability walkthroughs, was to include three types of participants: representative users, product developers, and human factors (usability) professionals.
- Hard-copy screens (panels) are presented in the same order in which they would appear online. A task scenario is defined, and participants confront the screens in a linear path, through a series of user interface panels, just as they would during the successful conduct of the specified task online, as the site/software is currently designed.
- Participants are all asked to assume the role of the user for whatever user population is being tested. Thus, the developers and the usability professionals are supposed to try to put themselves in the place of the users when making written responses.
- The participants write down the action they would take in pursuing the designated task online, before any further discussion is made. Participants are asked to write their responses in as much detail as possible down to the keystroke or other input action level. These written responses allow for some production of quantitative data on user actions that can be of value.
- It is only after all participants have written the actions they would take that discussion would begin. The representative users offer their discussion first and discuss each scenario step. Only after the users have exhausted their suggestions do the usability experts and product developers offer their opinions.

10.7.3 Benefits and Limitations

Benefits

There are several benefits that make the pluralistic usability walkthrough a valuable tool.

- Early systematic look at a new product, gaining early performance and satisfaction data from users about a product. Can provide early performance and satisfaction data before costly design strategies have been implemented.
- Strong focus on user centered design in task analysis, leading to more problems identified at an earlier point in development. This reduces the iterative test-redesign cycle by utilizing immediate feedback and discussion of design problems and possible solutions while users are present.

- Synergistic redesign because of the group process involving users, developers and usability engineers. The discussion of the identified problems in a multidisciplinary team will spawn creative, usable and quick solutions.
- Valuable quantitative and qualitative data is generated through users' actions documented by written responses.
- Product developers at the session gain appreciation for common user problems, frustrations or concerns regarding the product design. Developers become more sensitive to users' concerns.

Limitations

There are several limitations to the pluralistic usability walkthrough that affect its usage.

- The walkthrough can only progress as quickly as the slowest person on each panel. The walkthrough is a group exercise and, therefore, in order to discuss a task/screen as a group, we must wait for all participants to have written down their responses to the scenario. The session can feel laborious if too slow.
- A fairly large group of users, developers and usability experts has to be assembled at the same time. Scheduling could be a problem.
- All the possible actions can't be simulated on hard copy. Only one viable path of interest is selected per scenario. This precludes participants from browsing and exploring, behaviors that often lead to additional learning about the user interface.
- Product developers might not feel comfortable hearing criticism about their designs.
- Only a limited number of scenarios (i.e. paths through the interface) can be explored due to time constraints.
- Only a limited amount of recommendations can be discussed due to time constraints.

10.7.4 Further reading

- Dix, A., Finlay, J., Abowd, G., D., and Beale, R. Human-computer interaction (3rd ed.). Harlow, England: Pearson Education Limited, 2004.
- Nielsen, Jakob. Usability Inspection Methods. New York, NY: John Wiley and Sons, 1994.
- Preece, J., Rogers, Y., and Sharp, H. Interaction Design. New York, NY: John Wiley and Sons, 2002.

- Bias, Randolph G., "The Pluralistic Usability Walkthrough: Coordinated Emphathies," in Nielsen, Jakob, and Mack, R. eds, Usability Inspection Methods. New York, NY: John Wiley and Sons, 1994.

10.7.5 External links

- [List of Usability Evaluation Methods and Techniques](#)
- [Pluralistic Usability Walkthrough](#)

10.7.6 See also

- [Comparison of usability evaluation methods](#)

10.8 Comparison of usability evaluation methods

Source: Genise, Pauline. "Usability Evaluation: Methods and Techniques: Version 2.0" August 28, 2002. University of Texas.

10.8.1 See also

- [Usability inspection](#)
- [Exploring two methods of usability testing: concurrent versus retrospective think-aloud protocols](#)
- [Partial concurrent thinking aloud](#)

Chapter 11

Text and image sources, contributors, and licenses

11.1 Text

- **Software testing** Source: https://en.wikipedia.org/wiki/Software_testing?oldid=677114935 Contributors: Lee Daniel Crocker, Brion VIBBER, Bryan Derksen, Robert Merkel, The Anome, Stephen Gilbert, Ed Poor, Verloren, Andre Engels, ChrisSteinbach, Infrogmation, GABaker, Willsmith, Rp, Kku, Phoe6, Pcb21, Ahoerstemeier, Ronz, Nanobug, Andres, Mxn, Smack, JASpencer, Selket, Furrykef, Miterdale, Robbot, MrJones, Fredrik, RedWolf, Lowellian, Yosri, Ashdurbat, Academic Challenger, Auric, Faught, Wlievens, Hadal, Tobias Bergemann, ReneS~enwiki, Matthew Stannard, Thv, Tprosser, Levin, Msm, Brequinda, Pashute, Jjamison, CyborgTosser, Craigwb, JimD, FrYGuY, Khalid hassani, Alvestrand, Cptchipjew, Chowbok, Utcursch, SURIV, Beland, Sam Hovevar, Srittau, Joyous!, Andreas Kaufmann, Abdull, Canterbury Tail, AliveFreeHappy, Imroy, Felix Wiemann, Discospinster, Rich Farmbrough, Rhobite, Sylvainmarquis, Michal Jurosz, Mazi, Notinasnaid, Paul August, ESKog, Moa3333, FrankCostanza, S.K., JoeSmack, Blake8086, CanisRufus, Edward Z. Yang, Chairboy, PhilHibbs, Shanes, ChrisB, Bobo192, Harald Hansen, Smalljim, Rmattson, MaxHund, Danimal~enwiki, Rje, MPerel, Helix84, JesseHogan, Hooperbloom, Jakew, Mdd, Storm Rider, Gary, Richard Harvey, Walter Görlitz, Halsteadk, Conan, Andrew Gray, Shadowcheets, Calton, Goldom, Snowwolf, Wtmitchell, Gdavidp, Yuckfoo, Danhash, 2mcm, Nimowy, Nibblus, Versageek, Bsdlogical, Shimeru, Nuno Tavares, RHaworth, Pmberry, Uncle G, MattGiuca, SP-KP, GregorB, Sega381, Kanenas, MassGalactusUniversum, Graham87, Bilbo1507, FreplySpang, ThomasOwens, Dvyost, Rjwilmsi, Jsled, Jake Wartenberg, Ian Lancaster, Halovivek, Amire80, MZM~cBride, Jehochman, Yamamoto Ichiro, A Man In Black, Lancerkind, Mpradeep, DallyingLlama, Kri, Chobot, Bornhj, DVdm, Roboto de Ajvol, Pinecar, YurikBot, Albanaco, Wavelength, ChiLiBeserker, Anonymous editor, ChristianEdwardGruber, Epolk, Chaser, Flavioxavier, Akamad, Stephenb, Rsrikanth05, Bovineone, Wiki alf, AlMac, Epim~enwiki, Tejas81, Retired username, Kingpomba, PhilipO, Rjllabs, Paul.h, Bkil, Misza13, Lomn, Ttam, Ospalh, Zephyrjs, Gorson78, Tonym88, Closedmouth, Claygate, GraemeL, Smurrayinchester, Kevin, Poulpy, Allens, Eptin, Rwww, Kgf0, Psychade, Sardanaphalus, SmackBot, Haza-w, Incnis Mersi, KnowledgeOfSelf, CopperMurdoch, Gary Kirk, Davewild, Pieleric, Dazzla, Puraniksameer, Bpluss, Unforgettableid, Gilliam, Ohnoitsjamie, Andy M. Wang, Anwar saadat, Bluebot, QTCaptain, Huge Bananas, Gil mo, Mikethegreen, EncMstr, Roscelese, Jenny MacKinnon, Mfactor, M Johnson, DHN-bot~enwiki, Konstable, Jmax-, MaxSem, Munaz, VMS Mosaic, Allan McInnes, Radagast83, Cybercobra, Valenciano, Mr Minchin, Dacoutts, Richard001, A.R., DMacks, Michael Bernstein, Ihatetoregister, A5b, Mailtoramkumar, Guehene, ThurnerRupert, Dcarrion, Krashlondon, Derek farn, Kuru, Ocee, Agopinath, Breno, Shadowlynk, Chaiths, Minna Sora no Shita, IronGargoyle, Neokamek, Shepmaster, Kompere, Techsmith, Noah Salzman, Sankshah, Dicklyon, Anonymous anonymous, Barunbiswas, David.alex.lamb, Hu12, Mike Doughney, Rschwieb, Esoltas, Dakart, RekishiEJ, AGK, Tawkerbot2, Anthonares, Plainplow, AbsolutDan, Ahy1, Randhirreddy, Stansult, Hsingh77, Leujohn, Michael B. Trausch, Mblumber, Ravialluru, Havlatm, DryCleanOnly, Gogo Dodo, Bazzargh, Jmckey, Pascal.Tesson, DumbBOT, Omicronpersei8, Wikid77, Qwyrxian, Bigwyrn, JacobBramley, Headbomb, Lumpish Scholar, Alphius, Vaniac, Peashy, Mentifisto, Ebde, The prophet wizard of the crayon cake, Seaphoto, Sasquatch525, Mhaitham.shammaa, Miker@sundialservices.com, Feyenatic london, Tusharpandya, Ellenaz, Ad88110, Bavinothkumar, Gdo01, Oddity-, Alphachimpbot, Rex black, Blair Bonnett, Dougher, Kdakin, JAnDbot, Serge Toper, Nick Hickman, ThomasO1989, MER-C, XeNoyZ~enwiki, Michig, Akhiladi007, Andonic, Josheisenberg, TAnthony, Kitdaddio, Shoejar, VoABot II, Digitalfunda, CattleGirl, Tedickey, ELinguist, Indon, Sxm20, Giggy, Qazwsxedcrfvtg-byhn~enwiki, Rajesh mathur, Wwmbes, Allstarecho, SunSw0rd, Cpl Syx, ArmadilloFromHell, DerHexer, Bobisthebest, Oicumayberight, 0612, DRogers, Jackson Peebles, Electiontechnology, Hdt83, A R King, NAHID, Pysuresh, Johnny.cache, MichaelBolton, ScaledLizard, AlexiusHoratius, Tulkolahten, Ash, Uktim63, Erkan Yilmaz, Paranomia, J.delanoy, Trusilver, Rgoodermote, Rmstein, Tippers, Rlsheehan, Nigholith, IceManBrazil, Venkatreddyc, SpigotMap, Gurchzilla, Aphstein, Staceyeschneider, SJP, Vijaythormothe, PhilippeAntras, Juliancolton, Entropy, Comestyles, Raspalchima, Andrewcmcardle, Bobdanny, Jowler, Bonadea, Ja 62, Chris Pickett, Inwind, Useight, CardinalDan, Venu6132000, Remi0o, Sooner Dave, Wikieditor06, 28bytes, Alappuzhakaran, VolkovBot, Certellus, Mrh30, Cvcby, Jeff G., Strmore, Shze, Philip Trueman, JuneGloom07, TXiKiBoT, Oshwah, JPFitzmaurice, Robinson weijman, Zurishaddai, Anonymous Dissident, Vishwas008, Someguy1221, Lradrama, Tdjones74021, LeaveSleaves, Amty4all, ^demonBot2, BotKung, Inray, Softtest123, ZhonghuaDragon2, Madhero88, Forlornturtle, Meters, Synthebot, Yesyoubee, Falcon8765, Enviroboy, Sapphic, Priya4212, Alhenry2006, Shindevijaykr, Winchelsea, Joshymit, John S Eden, Caltas, Softwaretest1, SatishKumarB, WikiWilliamP, Ankurj, Bentogoa, Happy-sailor, Toddst1, Oysterguitarist, Chrzastek, Samansouri, Mad Bunny, Rockynook, Testinggeek, Lagrange613, Tmaufer, Slowbro, AlanUS, Dravecky, StaticGull, Superbeecat, JonJosephA, Burakseren, Denisarona, Sitush, Ruptan, ClueBot, The Thing That Should Not Be, RitigalaJayasena, Robenel, Sachxn, Buxbaum666, Gar3t, Ea8f93wala, Jm266, GururajOaksys, Nambika.marian, Joneskoo, Pointillist, Shishirhegde, Losaltosboy, Drewster1829, Excirial, Jusdafax, Robbie098, Scottri, Canterj, Swtechwr, DeltaQuad, Thehelpfulone, Aravindan Shanmugasundaram, Tagro82, Dalric, Aitias, Declan Kavanagh, Certes, Promoal~enwiki, IJA, Mpilaeten, Johnuniqu, Apparition11, Skyqa, N8mills, Steveozone, Honey88foru, XLinkBot, Spitfire, Dndnd80, SwirlBoy39, Stickee, Jstastny, Rror, Little Mountain

- 5, Avoided, Srikant.sharma, Rowlye, Mitch Ames, WikHead, ErkinBatu, PL290, Dekart, ZooFari, Johndci, Addbot, Tipeli, Grayfell, Mabdul, Betterusername, Kelstrup, Metagraph, Hubschrauber729, Ronhjones, TutterMouse, OBloodyHell, Anorthup, Leszek Jańczuk, Wombat77, NjardarBot, MrOllie, Download, Ryoga Godai, Favonian, Annepetersen, JosephDonahue, SamatBot, Otis80hobson, Terrillja, Tassedethe, CemKaner, TCL India, Softwaretesting101, Lightbot, Madvin, Nksp07, Gail, Jarble, Yngupta, Margin1522, Legobot, Thread-union, PlankBot, Luckas-bot, Ag2402, Yobot, 2D, Fraggel81, Legobot II, Bdog9121, Amirobot, Adam Hauner, Georgie Canadian, AnomieBOT, Noq, ThaddeusB, NoBot42, Jim1138, Kalkundri, Piano non troppo, Bindu Laxminarayan, Ericholmstrom, Kingpin13, Solde, Softwaretesting1001, Silverbullet234, Flewis, Blueasberry, Pepsi12, Materialschemist, Slsh, Anubhavbansal, Citation bot, E2eamon, Eumolpo, ArthurBot, Gsmgm, Testingexpert, Obersachsebot, Xqbot, Qatutor, Bigtwilkins, Atester, Addihockey10, Anna Frodesiak, Raynald, Corruptcopper, T4tarzan, Mathonius, Der Falke, Dvansant, Sergey1984, Joaquin008, SD5, Pomoxis, ImALion, Prari, FrescoBot, FalconL, Hemnath18, Mark Renier, Downsize43, Javier.eguiluz, Cgvak, GeoTe, Wifione, Oashi, Enumera, ZenerV, Jluedem, HamburgerRadio, Citation bot 1, Guybrush1979, Boxplot, Shubo mu, Pinethicket, I dream of horses, AliaksandrAA, Rahuljaitley82, W2qasource, Cjhawk22, Consummate virtuoso, Vasywriter, Contributor124, Jschnur, RedBot, Oliver1234-enwiki, SpaceFlight89, MertyWiki, Mike-Dogma, Hutch1989r15, Riagu, Sachipra, Trappist the monk, SchreyP, Newbie59, Lotje, Baxtersmalls, Skalar7, Drxim, Paudelp, Gonchibols012, Vsoid, Minimac, Spadoink, DARTH SIDIOUS 2, Mean as custard, RjwilmsiBot, DaisyMLL, Brunodeschenes.qc, VernoWhitney, EmausBot, Orphan Wiki, Acather96, Diego.pamio, Menzogna, Albertnetymk, Deogratias5, Walthouser, RA0808, Solarra, Tommy2010, K6ka, Dana4ka, Ppolpp, Ilarihenrik, Dbelhumeur02, Listmeister, Andygreeny, Mburdis, Cymru.lass, Bex84, Anna88banana, QEDK, Tolly4bolly, Testmaster2010, Senatum, Praveen.karri, ManojPhilipMathen, Qiassist, Donner60, Orange Suede Sofa, ElfriedeDustin, Perlundholm, Somdeb Chakraborty, TYelliot, Rocketrod1960, Geosak, Will Beback Auto, ClueBot NG, Jack Greenmaven, Uzma Gamal, CocuBot, MelbourneStar, This lousy T-shirt, Satellizer, Pias93, Millermk, BruceRuxton, Mtoxcv, Cntras, ScottSteiner, Widr, Rame-shaLB, G0gogcsc300, Anon5791, Henri662, Helpful Pixie Bot, Filadifei, Dev1240, Wbm1058, Vijay.ram.pm, Ignasiokambale, Mm-greiner, Lowercase sigmabot, PauloEduardo, Pine, Softwrite, Manekari, TheyCallMeHeartbreaker, Jobin RV, Okal Otieno, Netra Naha, Chamolinaresh, MrBill3, Jasonvaiddya123, Cangoroo11, Mayast, Klilidiplomus, Shiv sangwan, BattyBot, Pratyya Ghosh, Hghyux, Softwareqa, W.D., Leomcbride, Ronwarshawsky, Kothiwai, Cyberbot II, Padenton, Carlos.l.sanchez, Puzzlefand123asdfs, Testersupdate, Michecsz, Testingfan, Codename Lisa, Arno La Murette, Faye dimarco, KellyHass, Drivermadness, Shahidna23, Cheetal heyk, Nine smith, Aleek vivk, Frosty, James12345, Keithklain, Copyry, Dekanherald, 069952497a, LaurentBossavit, Mahbubur-r-aaman, Faizan, Epicgenius, Kuldeepsheoran1, Rootswings, Pradeep Ligan, I am One of Many, Eyesnore, Lsteinb, Lewissall1, Jesa934, Zhenya000, Blashser, Babitaarora, Durgatome, Ugog Nizdast, Zenibus, Stevetalk, Quenhitran, Jkannry, Tapas.23571113, IrfanSha, Coreymotela, Hakiowiki, Ownyourstuff, Monkbob, Vieque, Fyddlestix, Arpit Bajpai(Abhimanyu), Sanchezluis2020, Pol29-enwiki, Poudelksu, Vetripedia, Mrdev9, Prnbtr, Frawr, RationalBlasphemist, Jenny Evans 34, Nickeeroom, EXPTIME-complete, TristramShandy13, ExploringU, Rajeevfl, Contributorauthor, Ishita14, Some Gadget Geek, AkuaRegina, Mountainelephant, Softwaretestingclass, GeneAmbeau, Ellenka 18, KasparBot, Bakosjen, Bartlettra, Credib7, Pedrocaleia, C a swtest, Anne viswanath and Anonymous: 1866
- **Black-box testing** *Source:* https://en.wikipedia.org/wiki/Black-box_testing?oldid=676071182 *Contributors:* Deb, Michael Hardy, Poor Yorick, Radiojon, Khym Chanur, Robbot, Jmabel, Jondel, Asparagus, Tobias Bergemann, Geeoharee, Mark.murphy, Rstens, Karl Naylor, Canterbury Tail, Discospinster, Rich Farmbrough, Notinasnail, Fluzwup, S.K., Lambchop, AKGhetto, Mathieu, Hooperbloom, Clement-Seveillac, Liao, Walter Görlitz, Andrewpmk, Caesura, Wtmitchell, Docboat, Daveydwieb, LOL, Isnow, Chrys, Ian Pitchford, Pinecar, YurikBot, NawlinWiki, Epim-enwiki, Zephyrjs, Benito78, Rwww, Kgf0, A bit iffy, Otheus, AndreniW, Haymaker, Xaosflux, DividedByNegativeZero, GoneAwayNowAndRetired, Bluebot, Thumperward, Frap, Mr Minchin, Blake-, DylanW, DMacks, PAS, Kuru, Shijaz, Hu12, Courcelles, Lahiru k, Colinky, Picaroon, CWY2190, NickW557, SuperMidget, Rsutherland, Thijs!bot, Ebde, AntiVandalBot, Michig, Hugh.glasier, Jay Gatsby, Tedickey, 28421u2232nfencenc, DRogers, Electiontechnology, Ash, Erkan Yilmaz, DanDoughty, PerformanceTester, SteveChervitzTrutane, Aervanath, WJBscribe, Chris Pickett, Retiono Virginian, UnitedStatesian, Kbrose, SieBot, Toddst1, NEUrOO, Nschoot, ClueBot, Mpilaeten, XLinkBot, Sietec, ErkinBatu, Subversive.sound, Addbot, Nitingai, Betterusername, Sergei, MrOllie, OIEnglish, Jarble, Luckas-bot, Ag2402, TaBOT-zerem, AnomieBOT, Rubinbot, Solde, Xqbot, JimVC3, RibotBOT, Pradameinhoff, Shadowjams, Cnwiliams, Clarkcj12, WikitanvirBot, RA0808, Donner60, Ileshko, ClueBot NG, Jack Greenmaven, Widr, Solar Police, Gayathri nambiar, TheyCallMeHeartbreaker, Avi260192, A'bad group, James12345, Ekips39, PupidoggCS, Haminoon, Incognito668, Ginsuloft, Bluebloodpole, Happy Attack Dog, Sadnanit and Anonymous: 195
 - **Exploratory testing** *Source:* https://en.wikipedia.org/wiki/Exploratory_testing?oldid=663008784 *Contributors:* VilleAine, Bender235, Sole Soul, TheParanoidOne, Walter Görlitz, Alai, Vegaswikian, Pinecar, Epim-enwiki, Kgf0, SmackBot, Bluebot, Decltype, BUPHA-GUS55, Imageforward, Dougher, Morrillonline, Elopio, DRogers, Erkan Yilmaz, Chris Pickett, SiriusDG, Softtest123, Doab, Toddst1, Jeff.fry, Quercus basaseachicensis, Mpilaeten, IQDave, Lakeworks, XLinkBot, Addbot, Lightbot, Fiftyquid, DRogers, Shadowjams, Oashi, I dream of horses, Trappist the monk, Aoidh, JnRouvignac, Whylom, GoingBatty, EdoBot, Widr, Helpful Pixie Bot, Leomcbride, Testingfan, ET STC2013 and Anonymous: 47
 - **Session-based testing** *Source:* https://en.wikipedia.org/wiki/Session-based_testing?oldid=671732695 *Contributors:* Kku, Walter Görlitz, Alai, Pinecar, JulesH, Bluebot, Waggars, JenKilmer, DRogers, CmcMahon, Chris Pickett, DavidMJam, Jeff.fry, WikHead, Mortense, ScenarioScientist, Bjosman, Srinivasskc, Engpharmer, ChrisGualtieri, Mkltesthead and Anonymous: 20
 - **Scenarios testing** *Source:* https://en.wikipedia.org/wiki/Scenarios_testing?oldid=620374360 *Contributors:* Rp, Kku, Ronz, Abdull, Bobo192, Walter Görlitz, Alai, Karbinski, Pinecar, Epim-enwiki, Brandon, Shepard, SmackBot, Bluebot, Kuru, Hu12, JaGa, Tikiwont, Chris Pickett, Cindamuse, Yintan, Addbot, AnomieBOT, Kingpin13, Cekli829, RjwilmsiBot, EmausBot, ClueBot NG, Smtchahal, Muon, Helpful Pixie Bot, தனெகாசி சும்பிரமணியன், Sainianu088, Pas007, Nimmalik77, Surfer43, Monkbob and Anonymous: 31
 - **Equivalence partitioning** *Source:* https://en.wikipedia.org/wiki/Equivalence_partitioning?oldid=641535532 *Contributors:* Enric Naval, Walter Görlitz, Stephan Leeds, SCEhardt, Zoz, Pinecar, Nmondal, Retired username, Wisgary, Attilios, SmackBot, Mirokado, JennyRad, CmdrObot, Harej bot, Blaisorblade, Ebde, Frank1101, Erechtheus, Jj137, Dougher, Michig, Tedickey, DRogers, Jtowler, Robinson weijman, Ianr44, Justus87, Kjtobo, PipepBot, Addbot, LucienBOT, Sunithasiri, Throw it in the Fire, Ingenhut, Vasinov, Rakesh82, GoingBatty, Jerry4100, AvicAWB, HossMo, Martinkeesen, Mbrann747, OkieCoder, HobbyWriter, Shikharsingh01, Jautran and Anonymous: 32
 - **Boundary-value analysis** *Source:* https://en.wikipedia.org/wiki/Boundary-value_analysis?oldid=651926219 *Contributors:* Ahoerste-meier, Radiojon, Ceady, Chadernook, Andreas Kaufmann, Walter Görlitz, Velella, Sesh, Stemonitis, Zoz, Pinecar, Nmondal, Retired username, Wisgary, Benito78, Attilios, AndreniW, Gilliam, Psiphorg, Mirokado, Bluebot, Freck Verkerk, CmdrObot, Harej bot, Ebde, AntiVandalBot, DRogers, Linuxbabu-enwiki, IceManBrazil, Jtowler, Robinson weijman, Rei-bot, Ianr44, LetMeLookItUp, XLinkBot, Addbot, Stemburn, Eumolpo, Sophus Bie, Duggpm, Sunithasiri, ZéroBot, EdoBot, ClueBot NG, Ruchir1102, Micrypt, Michaeldunn123, Krishjugal, Mojdaydyr, Kephir, Matheus Faria, TranquilHope and Anonymous: 59
 - **All-pairs testing** *Source:* https://en.wikipedia.org/wiki/All-pairs_testing?oldid=666855845 *Contributors:* Rstens, Stesmo, Cmdrjameson, RussBlau, Walter Görlitz, Pinecar, Nmondal, RussBot, SteveLoughran, Brandon, Addshore, Garganti, Cydebot, MER-C, Ash, Erkan Yilmaz, Chris Pickett, Ashwin palaparthi, Jeremy Reeder, Finnirind, Kjtobo, Melcombe, Chris4uk, Qwfp, Addbot, MrOllie, Tassedethe,

Yobot, Bookworm271, AnomieBOT, Citation bot, Rajushalem, Raghu1234, Capricorn42, Rexrange, LuisCavalheiro, Regancy42, WikitanvirBot, GGink, Faye dimarco, Drivermadness, Gjmurphy564, Shearflyer, Monkbob, Ericsh and Anonymous: 43

- Fuzz testing** *Source:* https://en.wikipedia.org/wiki/Fuzz_testing?oldid=665005213 *Contributors:* The Cunctator, The Anome, Dwheeler, Zippy, Edward, Kku, Haakon, Ronz, Dcoetzee, Doradus, Furrykef, Blashyrk, HaeB, David Gerard, Dratman, Leonard G., Bowlb, Mckaysalisbury, Neale Monks, ChrisRuvolo, Rich Farmbrough, Nandhp, Smalljim, Enric Naval, Mpeisenbr, Hooperbloob, Walter Görlitz, Guy Harris, Deacon of Pndapetzim, Marudubshinki, GregAsche, Pinecar, YurikBot, RussBot, Irishguy, Malaiya, Victor Stinner, SmackBot, Martinmeyer, McGeddon, Autarch, Thumperward, Letdorf, Emurphy42, JonHarder, Zirconscoot, Derek farn, Sadeq, Minna Sora no Shita, User At Work, Hu12, CmdrObot, FlyingToaster, Neelix, Marqueeed, A876, ErrantX, Povman, Siggimund, Malvineous, Tremilux, Kgfleischmann, Gwern, Jim.henderson, Leyo, Stephanakib, Aphstein, VolkovBot, Mezzaluna, Softtest123, Dirkbb, Monty845, Andypdavis, Stevehughes, Tmaufer, Jruderman, Ari.takanen, Manuel.oriol, Zarkthehackerralliance, Starofale, PixelBot, Posix memalign, DumZ-iBoT, XLinkBot, Addbot, Fluffernutter, MrOllie, Yobot, AnomieBOT, Materials scientist, LilHelpa, MikeEddington, Xqbot, Yurymik, SwissPokey, FrescoBot, T0pgear09, Informationh0b0, Niri.M, Lionaneesh, Dinamik-bot, Rmahfoud, ZéroBot, H3ilBot, F.duchene, Rc-sprinter123, ClueBot NG, Helpful Pixie Bot, Jvase, Pedro Victor Alves Silvestre, BattyBot, Midael75, SoledadKabocha, Amitkankar, There is a T101 in your kitchen and Anonymous: 112
- Cause-effect graph** *Source:* https://en.wikipedia.org/wiki/Cause%E2%80%93effect_graph?oldid=606271859 *Contributors:* The Anome, Michael Hardy, Andreas Kaufmann, Rich Farmbrough, Bilbo1507, Rjwilmsi, Tony1, Nbarth, Wleizero, Pgr94, DRogers, Yobot, OllieFury, Helpful Pixie Bot, TheTrishaChatterjee and Anonymous: 5
- Model-based testing** *Source:* https://en.wikipedia.org/wiki/Model-based_testing?oldid=668246481 *Contributors:* Michael Hardy, Kku, Thv, S.K., CanisRufus, Bobo192, Hooperbloob, Mdd, TheParanoidOne, Bluemoose, Vonkje, Pinecar, Wavelength, Gaius Cornelius, Test-tools-enwiki, Mjchonoles, That Guy, From That Show!, SmackBot, FlashSheridan, Antti.huima, Suka, Yan Kuligin, Ehheh, Garganti, CmdrObot, Sdorance, MDE, Click23, Mattisse, Thijs!bot, Tedickey, Jowler, MarkUtting, Mirko.conrad, Adivalea, Tatzelworm, Arjayay, MystBot, Addbot, MrOllie, LaaknorBot, Williamglasby, Richard R White, Yobot, Solde, Atester, Drilnoth, Alvin Seville, Anthony.faucogney, Mark Renier, Jluudem, Smartesting, Vrenator, Micskeiz, Eldad.palachi, EmausBot, John of Reading, ClueBot NG, Widr, Jzander, Helpful Pixie Bot, BG19bot, Yx101, CitationCleanerBot, Daveed84x, Eslamimehr, Stephanepechard, JeffHaldeman, Dahlweid, Monkbob, Cornutum, CornutumProject, Nathala.naresh and Anonymous: 88
- Web testing** *Source:* https://en.wikipedia.org/wiki/Web_testing?oldid=666079231 *Contributors:* JASpencer, SEWilco, Rchandra, Andreas Kaufmann, Walter Görlitz, MassGalactusUniversum, Pinecar, Jangid, SmackBot, Darth Panda, P199, Chuckley, Thadius856, MER-C, JamesBWatson, Gherget, Narayanraman, Softtest123, Andy Dingley, TubularWorld, AWiersch, Swtechwr, XLinkBot, Addbot, Doug-sTech, Yobot, Jetfreeman, Snizza, Macrofiend, Hedge777, Thehelpfulbot, Runnerweb, Danielcornell, KarlDubost, Dhiraj1984, Testgeek, EmausBot, Abdul sma, DthomasJL, AAriel42, Helpful Pixie Bot, In.Che., Harshadsamant, Tawaregs08.it, Erwin33, Woella, Emumt, Nara Sangaa, Ctdciddy, JimHolmesOH, Komper-enwiki, Rgraf, DanielaSzt1, Sanju.toyou, Rybec, Joebarh, Shailesh.shivakumar and Anonymous: 64
- Installation testing** *Source:* https://en.wikipedia.org/wiki/Installation_testing?oldid=667311105 *Contributors:* Matthew Stannard, April kathleen, Thardas, Aranel, Hooperbloob, TheParanoidOne, Pinecar, SmackBot, Telestylo, WhatamIdoing, Mr.sqa, MichaelDeady, Paulbulman, Catrope, CultureDrone, Erik9bot, Lotje and Anonymous: 13
- White-box testing** *Source:* https://en.wikipedia.org/wiki/White-box_testing?oldid=676949378 *Contributors:* Deb, Ixf64, Greenrd, Radion, Furrykef, Fought, Tobias Bergemann, DavidCary, Mark.murphy, Andreas Kaufmann, Noisy, Pluke, S.K., Mathieu, Giraffedata, Hooperbloob, JYolkowski, Walter Görlitz, Arthena, Yady, Caesura, Vella, Culix, Jhntex, Daranz, Isnow, Chrys, Old Moonraker, Chobot, The Rambling Man, Pinecar, ErrOneous, Hyad, DeadEyeArrow, Closedmouth, Ffangs, Dupz, SmackBot, Moeron, CSZero, Mscuthbert, AnOddName, PankajPeriwal, Bluebot, Thumperward, Tscabot, Mr Minchin, Kuru, Hyenaste, Hu12, Jacksprat, JStewart, Juanmamb, Ravialluru, Rsutherland, Thijs!bot, Mentifisto, Ebde, Dougher, Lfstevens, Michig, Tedickey, DRogers, Erkan Yilmaz, Dan-Doughty, Chris Pickett, Kyle the bot, Philip Trueman, DoorsAjar, TXiKiBoT, Qxz, Yilloslime, Jpalm 98, Yintan, Aillema, Happysailor, Toddst1, Svick, Denisarona, Nvrijn, Mpilaeten, Johnuniq, XLinkBot, Menthaxpiperita, Addbot, MrOllie, Bartledan, Luckas-bot, Ag2402, Pbtogourou, Kasukurthi.vrc, Pikachu-enwiki, Rubinbot, Solde, Materials scientist, Danno uk, Pradameinhoff, Sushiflinger, Prari, Mezod, Pinethicket, RedBot, MaxDel, Suffusion of Yellow, K6ka, Tolly4bolly, Bobogoo, Sven Manguard, ClueBot NG, Waterski24, Noot alghoubain, Antiqueight, Kanigan, HMISSolent, Michaeldunn123, AdventurousSquirrel, Gaur1982, BattyBot, Pushparaj k, Vnishaat, Azure dude, Ash890, Tentinator, JeffHaldeman, Monkbob, ChamithN, Bharath9676, BU Rob13 and Anonymous: 146
- Code coverage** *Source:* https://en.wikipedia.org/wiki/Code_coverage?oldid=656064908 *Contributors:* Damian Yerrick, Robert Merkel, Jdpipe, Dwheeler, Kku, Snoyes, JASpencer, Quux, RedWolf, Altenmann, Centic, Wlievens, HaeB, BenFrantzDale, Prosilaeas, Matt Crypto, Picapica, JavaTenor, Andreas Kaufmann, Abdull, Smharr4, AliveFreeHappy, Ebelular, Nigelj, Janna Isabot, Hob Gadling, Hooperbloob, Walter Görlitz, BlackMamba-enwiki, Suruena, Blaxthos, Penumbra2000, Allen Moore, Pinecar, YurikBot, Nawlins-Wiki, Test-tools-enwiki, Patlecat-enwiki, Rwww, Attilios, SmackBot, Ianb1469, Alksb, NickHodges, Kurykh, Thumperward, Nix-eagle, LouScheffer, JustAnotherJoe, A5b, Derek farn, JorisvS, Gibber blot, Beetstra, DagErlingSmørgrav, Auteurs-enwiki, CmdrObot, Hertzprung, Abhinavvaid, Ken Gallagher, Phatom87, Cydebot, SimonKagstrom, Jkeen, Julius.shaw, Ad88110, Kdakin, MER-C, Greens-burger, Johannes Simon, Tiagofassoni, Abednigo, Gwern, Erkan Yilmaz, Ntalamai, LDRA, AntiSpamBot, RenniePet, Mati22081979, Jtheires, Ixat totep, Aivosto, Bingbangbong, Hqb, Sebastian.Dietrich, Jamelan, Billinghurst, Andy Dingley, Cindamuse, Jerryobject, Mj1000, Wimdevalk, Digantorama, M4gnum0n, Aitias, U2perkunas, XLinkBot, Sferik, Quinntaylor, Ghettoaster, TutterMouse, Anorthup, MrOllie, LaaknorBot, Technoparkcorp, Legobot, Luckas-bot, Yobot, TaBOT-zerem, X746e, AnomieBOT, MehrdadAfshari, Materials scientist, JGMalcolm, Xqbot, Agasta, Miracleworker5263, Parasoft-pl, Wmwmurray, FrescoBot, Andresmlinar, Gaudol, Vasy-writer, Roadbiker53, Aislingdonnelly, Nat hilly, Veralift, MywikiaccountSA, Blacklily, Dr ecksk, Coveragemeter, Argonesce, Miller-lyte87, Witten rules, Stoilkov, EmausBot, John of Reading, JJMax, FredCassidy, ZéroBot, Thargor Orlando, Faulknerck2, Didgee-doo, Rpapo, Mittgaurav, Nintendo64, Ptrb, Chester Markel, Testcocoan, RuggeroB, Nin1975, Henri662, Helpful Pixie Bot, Scubamunki, Taibah U, Quamrana, BG19bot, Infofred, CitationCleanerBot, Sdesalas, Billie usagi, Hunghuuhoang, Walterkelly-dms, BattyBot, Snow78124, Pratyga Ghosh, QARon, Coombes358, Alonergan76, Rob amos, Mhaghghat, Ethically Yours, Flipperville, Monkbob and Anonymous: 194
- Modified Condition/Decision Coverage** *Source:* https://en.wikipedia.org/wiki/Modified_condition/decision_coverage?oldid=672000309 *Contributors:* Andreas Kaufmann, Suruena, Tony1, SmackBot, Vardhanw, Freek Verkerk, Pindakaas, Thijs!bot, Sigmundur, Crazytete101, Alexbot, Addbot, Yobot, Xqbot, FrescoBot, Tsunhimtse, ZéroBot, Markiewp, Jabraham mw, Štefica Horvat, There is a T101 in your kitchen, Flipperville, Monkbob, TGGarner and Anonymous: 18

- **Fault injection** *Source:* https://en.wikipedia.org/wiki/Fault_injection?oldid=650150792 *Contributors:* CyborgTosser, Chowbok, Andreas Kaufmann, Suruena, Joriki, RHaworth, DaGizza, SteveLoughran, Tony1, CapitalR, Foobiker, WillDo, Firealwayssworks, DatabACE, Jeff G., Tmaufer, Ari.takanen, Auntof6, Dboehmer, Addbot, LaaknorBot, Luckas-bot, Yobot, Piano non troppo, Paff1, GoingBatty, Paul.Dan.Marinescu, ClueBot NG, HMSSolent, BrianPatBeyond, BlevintronBot, Lugia2453, Martinschneider, Pkreiner and Anonymous: 31
- **Bebugging** *Source:* <https://en.wikipedia.org/wiki/Bebugging?oldid=514346301> *Contributors:* Kaihsu, Andreas Kaufmann, SmackBot, Okeyes, Alaibot, Foobiker, Jchaw, Erkan Yilmaz, Dawynn, Yobot and Anonymous: 5
- **Mutation testing** *Source:* https://en.wikipedia.org/wiki/Mutation_testing?oldid=675053932 *Contributors:* Mrwojo, Ustrnme h8er, Andreas Kaufmann, Martpol, Jarfil, Walter Görlitz, LFaraone, Nihiltres, Quuxplusone, Pinecar, Bhny, Pieleric, Htmllapps, JonHarder, Fuhghettaboutit, Derek farn, Antonielly, Mycroft.Holmes, Wikid77, Dogaroon, Magioladitis, Jeffoffutt, ObjectivismLover, GiuseppeDiGuglielmo, El Pantera, Brilesbp, Ari.takanen, JoeHillen, Rohansahgal, XLinkBot, Addbot, Mickaël Delahaye, Davidmus, Yobot, Sae1962, Felixwikihudson, Yuejia, ClueBot NG, BG19bot, IluvatarBot, Epicgenius, JeffHaldeman, Marcinkaw, Monkbot, Tumeropadre, Oo d0l0b oo and Anonymous: 76
- **Non-functional testing** *Source:* https://en.wikipedia.org/wiki/Non-functional_testing?oldid=652092899 *Contributors:* Walter Görlitz, Andrewpmk, Pinecar, Open2universe, SmackBot, Gilliam, Mikethegreen, Alaibot, Dima1, JaGa, Addere, Kumar74, Burakseren, P.srikanta, Erik9bot, Ontist, Samgoulding1 and Anonymous: 14
- **Software performance testing** *Source:* https://en.wikipedia.org/wiki/Software_performance_testing?oldid=674340711 *Contributors:* Robert Merkel, SimonP, Ronz, Ghewgill, Alex Vinokur-enwiki, Matthew Stannard, David Johnson, Rstens, Matt Crypto, Jewbacca, Andreas Kaufmann, D6, Oliver Lineham, Notinasnaid, Janna Isabot, Smalljim, Hooperbloom, Walter Görlitz, Versageek, Woohookitty, Palica, BD2412, Rjwilmsi, Koenigsberg, Intgr, Gwernol, Pinecar, YurikBot, Aeusoes1, Topperfalkon, Gururajs, Wizzard, Rwalker, Jeremy Visser, AMbroodEY, Veinor, SmackBot, KATremer, KnowledgeOfSelf, Wilsonmar, Argyriou, Softlogica, Freek Verkerk, Weregibil, Brian.a.wilson, Optakeover, Hu12, Shoeofdeath, Igoldste, Bourgeoispy, Msadler, AbsolutDan, CmdrObot, ShelfSkewed, Wselph, Cydebot, Ravialluru, AntiVandalBot, MER-C, Michig, SunSw0rd, Ronbarak, JaGa, MartinBot, R'n'B, Nono64, J.delanoy, Trusilver, Rsbarber, Iulus Ascanius, Ken g6, Philip Trueman, Davidschmelzer, Sebastian.Dietrich, Grotendeels Onschadelijk, Andy Dingley, Coroberti, Timurto, Burakseren, Sfan00 IMG, Wahab80, GururajOaksym, M4gnum0n, Muhandes, Swtechwr, SchreiberBike, M.boli, Apodelko, Mywikicontribs, Raysecurity, XLinkBot, Gnowor, Bbryson, Maimai009, Addbot, Jncraton, Pratheepraj, Shirtwaist, MrOllie, Yobot, Deicool, Jim1138, Materialsscientist, Anubhavbansal, Edepriest, Wktsugue, Shimser, Vrenator, Stroppolo, Kbustin00, Dhiraj1984, Ianmolynz, Armadillo-eleven, Dwvisser, Pnieloud, Mrmatiko, Ocaasi, Cit helper, Donner60, Jdlow1, TYelliot, Petr, ClueBot NG, MelbourneStar, CaroleHenson, Widr, Hagoth, Filadifei, BG19bot, Aisteco, HenryJames141, APerson, Abhasingh.02, Eitanklein75, Solstan, Noveltywh, Sfgiants1995, Dzmzh, Makesalive, Keepwish, Delete12, Jvkiet, AKS.9955, Lauramocanita, Andrew pfeiffer, Crystallizedcarbon, Kuldeep-rana1989, Shailesh.shivakumar and Anonymous: 266
- **Stress testing (software)** *Source:* [https://en.wikipedia.org/wiki/Stress_testing_\(software\)?oldid=631480139](https://en.wikipedia.org/wiki/Stress_testing_(software)?oldid=631480139) *Contributors:* Awaterl, Tobias Bergemann, CyborgTosser, Trevj, Walter Görlitz, Pinecar, RussBot, Rjllabs, SmackBot, Hu12, Philofred, Aednichols, Brian R Hunter, Niceguyedc, Addbot, Yobot, AnomieBOT, Con-struct, Shadowjams, LucienBOT, Ndanielm and Anonymous: 15
- **Load testing** *Source:* https://en.wikipedia.org/wiki/Load_testing?oldid=676031048 *Contributors:* Nurg, Faught, Jpo, Rstens, Beland, Icaims, Jpg, Wrp103, S.K., Hooperbloom, Walter Görlitz, Nimowy, Gene Nygaard, Woohookitty, ArrowmanCoder, BD2412, Rjwilmsi, Scoops, Bgwhite, Pinecar, Gaius Cornelius, Gururajs, Whitejay251, Shinhan, Arthur Rubin, Veinor, SmackBot, Wilsonmar, Jpvinall, Jruuska, Gilliam, LinguistAtLarge, Radagast83, Misterlump, Rklawton, JHunterJ, Hu12, AbsolutDan, Ravialluru, Tusharpandya, MER-C, Michig, Magioladitis, Ff1959, JaGa, Rlsheehan, PerformanceTester, SpigotMap, Crossdader, Ken g6, Adscherer, Jo.witte, Merrill77, Czei, Archdog99, Wahab80, M4gnum0n, Swtechwr, Photodeus, XLinkBot, Bbryson, Addbot, Bernard2, Bkranson, CanadianLinuxUser, Belmond, Gail, Ettrig, Yobot, AnomieBOT, Rubinbot, Snizza, Shadowjams, FrescoBot, Informationh0b0, Lotje, BluePyth, Manzee, Mean as custard, NameIsRon, VernoWhitney, Dhiraj1984, El Tonerino, Testgeek, ScottMasonPrice, Yossin-enwiki, Robert.maclean, Ronnn, Derbyridgeback, Daonguyen95, Pushtotest, Shilpagpt, Joe knepley, Gadaloo, ClueBot NG, AAriel42, Gordon McKeown, Gbegic, SireenO-Mari, Theopolisme, Shadriner, Itsyousuf, In.Che., Philip2001, Shashi1212, Frontaal, Neoevans, Ronwarshawsky, Emumt, AnonymousD-DoS, DanielaSZTBM, Ctdiddi, Nettiehu, Rgraf, Zje80, Christian Paulsen-enwiki, AreYouFreakingKidding, DanielaSzt1, MarijnN72, Mikerowan007, Loadtracer, Loadfocus, Sharmaprakher, Abarkth99, BobVermont, Smith02885, Danykurian, Pureload, Greitz876, Laraaz-zam, Laurenfo and Anonymous: 136
- **Volume testing** *Source:* https://en.wikipedia.org/wiki/Volume_testing?oldid=544672643 *Contributors:* Faught, Walter Görlitz, Pinecar, Closedmouth, SmackBot, Terry1944, Octahedron80, Alaibot, Thru the night, EagleFan, Kumar74, BotKung, Thingg, Addbot and Anonymous: 9
- **Scalability testing** *Source:* https://en.wikipedia.org/wiki/Scalability_testing?oldid=592405851 *Contributors:* Edward, Beland, Velella, GregorB, Pinecar, Malcolm, SmackBot, CmdrObot, Alaibot, JaGa, Methylgrace, Kumar74, M4gnum0n, DumZiBoT, Avoided, Addbot, Yobot, AnomieBOT, Mo ainm, ChrisGualtieri, Sharmaprakher and Anonymous: 11
- **Compatibility testing** *Source:* https://en.wikipedia.org/wiki/Compatibility_testing?oldid=642987980 *Contributors:* Bearcat, Alison9, Pinecar, Rwww, SmackBot, Arkitus, RekishiEJ, Alaibot, Jimj wpg, Neelov, Kumar74, Iain99, Addbot, LucienBOT, Jesse V., Mean as custard, DexDor, Thine Antique Pen, ClueBot NG, BPositive, Suvana 25, Gmporr, Gowdhaman3390 and Anonymous: 14
- **Portability testing** *Source:* https://en.wikipedia.org/wiki/Portability_testing?oldid=673682276 *Contributors:* Andreas Kaufmann, Cmdr-jameson, Pharos, Nibblus, Bgwhite, SmackBot, OSborn, Tapir Terrific, Magioladitis, Andrezein, Biscuitin, The Public Voice, Addbot, Erik9bot, DertyMunke and Anonymous: 3
- **Security testing** *Source:* https://en.wikipedia.org/wiki/Security_testing?oldid=667440367 *Contributors:* Andreas Kaufmann, Walter Görlitz, Brookie, Kinu, Pinecar, SmackBot, Gardener60, Gilliam, Bluebot, JonHarder, MichaelBillington, Aaravind, Bwpach, Stenaught, Dxwell, Epr123, ThisIsAce, Bigtimepeace, Ravi.alluru@applabs.com, MER-C, JA.Davidson, VolkovBot, Someguy1221, WereSpielChequers, Softwaretest1, Uncle Milty, Gavenko a, Joneskoo, DanielPharos, Spitfire, Addbot, ConCompS, Glane23, ImperatorExercitus, Shadowjams, Erik9bot, Pinethicket, Lotje, Ecran, David Stuble, ClueBot NG, MerllwBot, Ixim dschaefer and Anonymous: 117
- **Attack patterns** *Source:* https://en.wikipedia.org/wiki/Attack_patterns?oldid=605589590 *Contributors:* Falcon Kirtaran, Bender235, Hooperbloom, FrankTobia, Friedfish, Bachrach44, DouglasHeld, Retired username, Jkelly, SmackBot, Od Mishehu, Dudecon, RomanSpa, Alaibot, Natalie Erin, Manionc, Rich257, Nono64, Smokizzy, RockyH, JabbaTheBot, R00m c, Addbot, Bobbyquine, Enauspeaker, Helpful Pixie Bot and Anonymous: 3

- **Pseudolocalization** *Source:* <https://en.wikipedia.org/wiki/Pseudolocalization?oldid=640241302> *Contributors:* Pnm, CyborgTosser, Mboverload, Kutulu, ArthurDenture, Kznf, Josh Parris, Pinecar, SmackBot, Thumperward, Günter Lissner, Autoterm, Khazar, Bzadik, Thijs!bot, MarshBot, Miker@sundialservices.com, Gavrant, Jerschneid, Vipinhari, Jeremy Reeder, Andy Dingley, Dawn Bard, Traveler78, Svick, JL-Bot, Arithmandar, SchreiberBike, Addbot, Bdjcomic, A:-)Brunuś, Yobot, Nlhenk, ClueBot NG, Ryo567, Risalmin, BennyZ, Î¿½ and Anonymous: 13
- **Recovery testing** *Source:* https://en.wikipedia.org/wiki/Recovery_testing?oldid=556251070 *Contributors:* Elipongo, Alansohn, Rjwilmsi, .digamma, Pinecar, SmackBot, Habam, Rich257, DH85868993, Leandromartinez, Vikramsharma13, Addbot, LAAFan, Erik9bot, Nikolay Shtabel and Anonymous: 10
- **Soak testing** *Source:* https://en.wikipedia.org/wiki/Soak_testing?oldid=671356539 *Contributors:* A1r, Walter Görlitz, MZMcBride, Pinecar, Jams Watton, Mdd4696, Reedy Bot, JPFitzmaurice, Midlandstoday, P mohanavan, DanielPharos, AnomieBOT, Zero Thrust, Vasywriter, JnRouvignac, ClueBot NG, BG19bot and Anonymous: 18
- **Characterization test** *Source:* https://en.wikipedia.org/wiki/Characterization_test?oldid=607155034 *Contributors:* David Edgar, Dbenbenn, Jjamison, Andreas Kaufmann, Jkl, GabrielSjoberg, Mathiastck, Pinecar, JLaTondre, SmackBot, Colonies Chris, Ulner, Robofish, Alaibot, MarshBot, Dougher, BrianOfRugby, PhilippeAntras, Alberto Savoia, Swtechwr, Locobot, Rockin291 and Anonymous: 8
- **Unit testing** *Source:* https://en.wikipedia.org/wiki/Unit_testing?oldid=676515051 *Contributors:* Zundark, Timo Honkasalo, Nate Silva, Hfastedge, Ubiquity, Kku, GTBacchus, Pcb21, Looxix~enwiki, Ahoerstemeier, Haakon, Clausen, Edaelon, Hashar, Joggleran, Furrykef, Saltine, Veghead, Robbot, Fredrik, Tobias Bergemann, Thv, BenFrantzDale, Jjamison, Ssd, Rookkey, Craigwb, Wmahan, Neile, Hayne, Beland, Karl Dickman, Andreas Kaufmann, Canterbury Tail, AliveFreeHappy, Discospinster, Rich Farmbrough, Ardonik, Notinasnoid, Paul August, S.K., CanisRufus, Edward Z. Yang, Irrbloss, Smalljim, MaxHund, Ahc, Hooperbloop, Walter Görlitz, Sligocki, Themillofkeytone, Pantosys, Derbeth, RainbowOfLight, Joeggi, Mcsee, Spamguy, Tiroche, Rjwilmsi, .digamma, Boccobrock, OmriSegal, Vlad Patryshev, Allen Moore, Guille.hoardings~enwiki, Margosbot~enwiki, Winhunter, Chobot, Bgwhite, Tony Morris, FrankTobia, Pinecar, YurikBot, ChristianEdwardGruber, DanMS, Stephenb, SteveLoughran, Rsrikanth05, Richardkmiller, SAE1962, ScottyWZ, Sozin, El T, Stumps, Matt Heard, Attilios, Tyler Oderkirk, SmackBot, FlashSheridan, Gilliam, Ohnoitsjamie, NickGarvey, KaragouniS, Autarch, Mheusser, Nbarth, Colonies Chris, MaxSem, Tscabot, VMS Mosaic, Dmulter, Allan McInnes, Jonhanson, Kuru, Beetstra, Martinig, Kvng, Corvi, JoeBot, Nbryant, Sketch051, Eewild, Hsingh77, Rogerborg, Mtomczak, Bakersg13, Pmerson, Hari Surendran, Ryans.ryu, Cydebot, Ravialluru, Lo2u, Thijs!bot, TFriesen, Ultimus, Dflam, Bjzaba, AntiVandalBot, Konman72, Miker@sundialservices.com, Dougher, Nearyan, JAnDbot, ThomasO1989, Michig, Elilo, MickeyWiki, Magioladitis, JamesBWatson, Rjnienaber, DRogers, S3000, J.delanoy, Mr. Disguise, RenniePet, Sybersnake, Hanacy, Chris Pickett, User77764, VolkovBot, Anderbubble, Ravindrat, Philip Trueman, DoorsAjar, Gggggdxdn, Zed toocool, Longhorn72, Andy Dingley, AlleborgoBot, PGWg, Radagast3, Pablasse, Jpalm 98, Asavoia, Toddst1, Mhhanley, Mhenke, SimonTrew, Dillard421, Svick, AlanUS, Denisarona, Brian Geppert, Mild Bill Hiccup, Shyam 48, Sspiro, Excirial, Swtechwr, Ottawa4ever, Kamots, Happypoesms, RoyOshero, Skunkboy74, XLinkBot, ChuckEsterbrook, Addbot, Mortense, RPHv, Anorthup, Vishnava, MrOllie, Influent1, SpBot, Paling Alchemist, Paulocheque, Jarble, Luckas~bot, Yobot, Fragg81, Denispur, Nallimbot, AnomieBOT, Democrati-cLuntz, Verec, Solde, The Evil IP address, Earlypsychosis, RibotBOT, Tstroege, Martin Majlis, Goswamivijay, Target drone, Hyper-sonic12, I dream of horses, Robvanvee, Nat hillary, Vrenator, Onel5969, RjwilmsiBot, Bdiijkstra, Unittest123, Ibbn, K6ka, Lotosotol, ChuispastonBot, Xanchester, ClueBot NG, Willem-Paul, Gareth Griffith-Jones, Saalam123, Widr, Helpful Pixie Bot, Sujith.srao, Nick Lewis CNH, Garionh, Rbrunner7, Angadn, Chmarkine, Mark.summerfield, Anbu121, BattyBot, Ciaran.lyons, MahdiBot, Leomcbride, Ojan53, Alumd, Mahbubur-r-aaman, Burton247, Jianhui67, NateBourgoin, Monkbot, Elilopian, Alakzi, TomCab81, Pluspad, Trasd and Anonymous: 441
- **Self-testing code** *Source:* https://en.wikipedia.org/wiki/Self-testing_code?oldid=661307669 *Contributors:* Ed Poor, Andreas Kaufmann, Rich Farmbrough, Spoon!, GregorB, Malcolma, Alaibot, UnCatBot, Addbot, Yobot, Erik9bot and Anonymous: 3
- **Test fixture** *Source:* https://en.wikipedia.org/wiki/Test_fixture?oldid=649846287 *Contributors:* Rp, Andreas Kaufmann, Jeodesic, Walter Görlitz, Tabletop, BD2412, Eubot, ZacParkplatz, Frazz, Ripounet, Heathd, Wernight, Silencer1981, CommonsDelinker, Rlsheehan, WHonekamp, Pkgx, Martarius, PixelBot, Patricio Paez, Addbot, Joribatet, Rohieb, FrescoBot, LucienBOT, RCHenningsgard, Humanoc, Brambleclawx, Ingeniero-aleman, ClueBot NG, Khazar2, Monkbot, Filedelinkerbot and Anonymous: 20
- **Method stub** *Source:* https://en.wikipedia.org/wiki/Method_stub?oldid=599134183 *Contributors:* Kku, Ggodard, Itai, Sj, Joaopaulo1511, Andreas Kaufmann, Perey, Rich Farmbrough, S.K., MBisanz, Walter Görlitz, Drbreznjev, Ceyockey, IguanaScales, Vary, Bhadani, Bratch, FlaBot, Pinecar, YurikBot, Segv11, SmackBot, Bluebot, Can't sleep, clown will eat me, Rrburke, Radagast83, Antonielli, Dicklyon, Courcelles, CmdrObot, Alaibot, AntiVandalBot, Husond, Michig, Magioladitis, VoABot II, Robotman1974, Cander0000, STBot, Mange01, Deep Alexander, ClueBot, RitigalaJayasena, Hollih, MystBot, Addbot, Mityaha, LaaknorBot, Extvia, Quebec99, Mark Renier, Sae1962, DrillBot, Ermey~enwiki, RjwilmsiBot, Thisarticleisastub, Dasoman, Dariusz wozniak and Anonymous: 31
- **Mock object** *Source:* https://en.wikipedia.org/wiki/Mock_object?oldid=675794637 *Contributors:* Kku, Charles Matthews, Robbot, Tobias Bergemann, HangingCurve, Khalid hassani, Andreas Kaufmann, Marx Gomes, Edward Z. Yang, Nigelj, Babomb, Hooperbloop, Stephan Leeds, Derbeth, AN(Ger), Blaxthos, Ataru, BenWilliamson, Rstandefer, Allen Moore, Grlea, Pinecar, JamesShore, SteveLoughran, TEB728, SmackBot, NickHodges, Bluebot, Autarch, Jprg1966, Thumperward, A. B., MaxSem, Frap, Author, Spurrmoses, Cybercobra, Dcamp314, Antonielli, Martinig, Redeagle688, DI2000, Hu12, Paul Foxworthy, SkyWalker, Eric Le Bigot, Cydebot, Marchaos, ClinkingDog, Kc8tpz, Nrabnowitz, Thijs!bot, Simonwacker, Ellissound, Elilo, Allanlewis, Whitehawk julie, R'n'B, Yjwong, Mange01, Ice-ManBrazil, Warlordwolf, Mkarlesky, VolkovBot, ABF, Philip Trueman, Lmajano, Andy Dingley, Le-sens-commun, AlanUS, Martarius, DHGarrette, Dhoerl, Colcas, CodeCaster, Rahaeli, RoyOshero, Dekart, SlubGlub, Tomrbj, Addbot, Ennu93, Ghettoblaster, Cst17, Yobot, Pecaperopeli, KamikazeBot, Ciphers, MaterialsScientist, Citation bot, 16x9, Krackekumar, Rodrigez~enwiki, Lotje, Acather96, Hanavy, Sceerj, WikiPuppies, MerllwBot, Helpful Pixie Bot, Repentsinner, Dexbot, Baltostar, CheViana and Anonymous: 120
- **Lazy systematic unit testing** *Source:* https://en.wikipedia.org/wiki/Lazy_systematic_unit_testing?oldid=601791661 *Contributors:* Andreas Kaufmann, RHaworth, AJHSimons and Yobot
- **Test Anything Protocol** *Source:* https://en.wikipedia.org/wiki/Test_Anything_Protocol?oldid=662580210 *Contributors:* Gaurav, Two Bananas, Andreas Kaufmann, RossPatterson, Shlomif, Petdance, Mindmatrix, Schwern, Dbagnall, Pinecar, SmackBot, Frap, Shunpiker, AndyArmstrong, Cepim, Justatheory, MichaelRWolf, Cydebot, Alaibot, BrotherE, Kzafer, Thr4wn, Ysth, Tarchannen, Renormalist, Yintan, Saint Aardvark, RjHerrick, Jarble, Yobot, Wrelwser43, Primaler, John of Reading, H3llBot, Myfreeweb, Brunodepaulak, Chmarkine, Catwell2, Millerhbm, Calvin-j-taylor, Ranfordb, Briandela and Anonymous: 34
- **XUnit** *Source:* <https://en.wikipedia.org/wiki/XUnit?oldid=675550240> *Contributors:* Damian Yerrick, Nate Silva, Kku, Ahoerstemeier, Furrykef, RedWolf, Pengo, Uzume, Srittau, Andreas Kaufmann, Qef, MBisanz, RudaMoura, Caesura, Kenyon, Woohookitty, Lucienne,

Tloche, Lasombra, Schwern, Pinecar, YurikBot, Adam1213, Pagrashtak, Ori Peleg, FlashSheridan, BurntSky, Bluebot, Jerome Charles Potts, MaxSem, Addshore, Slakr, Chuckley, Patrikj, RhpPhillips, Green caterpillar, Khatru2, Thijs!bot, Kleb~enwiki, Simonwacker, SebastianBergmann, Magioladitis, Hroðulf, PhilippeAntras, Chris Pickett, VolkovBot, Jpalm 98, OsamaBinLogin, Mat i, Carriearchdale, Addbot, Mortense, MrOllie, Download, AnomieBOT, Gowr, LilHelpa, Dvib, EmausBot, Kranix, MindSpringer, Filadifei, Kamorrissey, C.horsdal, ShimmeringHorizons, François Robere and Anonymous: 59

- **List of unit testing frameworks** *Source:* https://en.wikipedia.org/wiki/List_of_unit_testing_frameworks?oldid=677107957 *Contributors:* Brandf, Jdpipe, Edward, Kku, Gaurav, Phoe6, Markvp, Darac, Furrykef, Northgrove, MikeSchinkel, David Gerard, Thv, Akadruid, Grincho, Uzume, Alexf, Torsten Will, Simoneau, Burschik, Fuzlyssa, Andreas Kaufmann, Abdull, Damieng, RandalSchwartz, MM-Sequeira, AliveFreeHappy, Bender235, Papeschr, Walter Görlitz, Roguer, Nereocystis, Diego Moya, Crimson117, Yipdw, Nimowy, Vassilvk, Zootm, Weitzman, Mindmatrix, Tabletop, Ravidgemole, Calréfa Wéná, Mandarax, Yurik, Rjwilmsi, Cxbrx, BDerrly, Jevon, Horvathbalazs, Schwern, Bgwhite, Virtualblackfox, Pinecar, SteveLoughran, LesmanaZimmer, Legalize, Stassats, Alan0098, Pagrashtak, Praseodymium, Sylvestre~enwiki, Ospalh, Nlu, Jvoegele, Kenguest, JLaTondre, Mengmeng, Jeremy.collins, Banus, Eoinwoods, SmackBot, Imz, KAtremer, JoshDuffMan, Senfo, Chris the speller, Bluebot, Autarch, Vcmpk, Metalim, Vid, Frap, KevM, Clements, Ritchie333, Paddy3118, BTiffin, Loopology, Harryboyles, Beetstra, BP, Huntc, Hu12, Justatheory, Travisj, Donald Hosek, Stenyak, RhpPhillips, Jokes Free4Me, Pmoura, Pgr94, MeekMark, D3j409, Harrigan, Sgould, TempestSA, Mblumber, Yukoba~enwiki, Zanhshieh, ThevikasIN, Hlopetz, Pesto, Wernight, DSLeB, DrMiller, JustAGal, J.e, Nick Number, Philipcraig, Kleb~enwiki, Guy Macon, Billyoneal, CompSciStud4U, Davidcl, Ellisound, MebSter, Rob Kam, BrotherE, MiguelMunoz, TimSSG, EagleFan, Jetxee, Eeera, Rob Hinks, Gwern, STBot, Wdevauld, Philippe.beaudoin, R'n'B, Erkan Yilmaz, Tadpole9, IceManBrazil, Asimjalis, Iceaturtles, LDRA, Grshiplett, Lunakid, Pentapus, Chris Pickett, Squares, Tarvaina~enwiki, User77764, C1vineoflife, Mklarinsky, X!, Sutirthadatta, DaoKaiohsin, Jwgrenning, Grimley517, Simonscarfe, Andy Dingley, Mikofski, SirGeek CSP, RalfHandl, Dlindequi, Mj1000, OsamaBinLogin, Geldenhuys, Svick, Prekeageo, Tognopop, FredericTorres, Skiwi~enwiki, Ates Goral, PuercoPop, Jerrico Gamis, RJanicek, Ropata, SummerWithMorons, James Hugard, Ilya78, Martin Moene, Ryadav, Rmkeebble, Boemmels, Jim Kring, Joelittlejohn, TobyFernsler, Angoca, M4gnum0n, Shabbychef, Ebar7207, PensiveCoder, ThomasAagaardJensen, Arjayay, Swtechwr, AndreasBWagner, Basvodde, Uniwalk, Johnuniq, SF007, Arjenmarkus, XLinkBot, Holger.krekel, Mdkorhon, Mifter, AJHSimons, MystBot, Duffbeerforme, Siffert, Addbot, Mortense, Anorthup, Sydevelopments, Asashour, Ckrahe, JTR5121819, Codefly, Tassedethe, Figureoutyourself, Flip, Yobot, Torsknot, Marcllevel3, JavaCS, AnomieBOT, Wickorama, Decatur-en, LilHelpa, Chompx, Maine3002, Fltoledo, DataWraith, Morder, Avi.kaye, Cybjit, Miguemunoz, Gpremer, Norrby, FrescoBot, Mark Renier, Rjollo, Shlynyu, SHIMODA Hiroshi, Artem M. Pelenitsyn, Antonylees, Juedem, Kwiki, A-Evgeniy, Berny68, David smallfield, Sellerbracke, Tim András, Winterst, Ian-blumel, Kiranthorath, Oestape, Generalov.sergey, Rcunit, Jrosdahl, Olaf Dietsche, Lotje, Gurdiga, Bdicroce, Dalepres, ChronoKinetic, Adardesign, Bdcon, Updatehelper, GabiS, Rsiman, Andrey86, Hboutemy, John of Reading, Jens Lüdemann, Bdiijkstra, Александр Чуранов, Kristofer Karlsson, Niocr, NagyLoutre, Jeffrey Ratcliffe~enwiki, Iekmuf, GregoryCrosswhite, Cruftcraft, Mitmacher313, Daruini, Sarviline, ClueBot NG, ObjexxWiki, Ptrb, Ten0s, Simeonfs, Magesteve, Yince, Saalam123, Vibhuti.amit, Shadriner, Strike Eagle, Avantika789, BG19bot, Benelot, Cpunit root, Ptrexford, Atconway, Mark Arsten, Bigwhite.cn, Rawoke, Tobias.trelle, Chmarkine, Madgarm, Lcorneliussen, Bvenners, Dennisloydjr, Aisteco, Mlasaj, BattyBot, Neilvandyke, Whart222, Imsky, Leomcbride, Haprog, Rnagrodzki, Cromlech666, Alumd, Doggum, Lriffel00, QARon, Duthen, Janschaefer79, AndreasMangold, Mr.onefifth, Alexpodlesny, Fireman lh, Andrewmarlow, Mruegg, Fedell, Daniel Zhang~enwiki, Gvauvert, Bowersenior, Andhos, Htejera, Jubianchi, GravRidr, Dmt-123, Olly The Happy, Seddryck, Monkbot, Khouston1, Shadowfen, Breezywoody, Akhabibullina, ZZromanZZ, Modocache, Rafrancoso, Elilopian, Swirlywonder, Grigutis, Ccremarencos, Rohan.khanna and Anonymous: 516
- **SUnit** *Source:* <https://en.wikipedia.org/wiki/SUnit?oldid=629665079> *Contributors:* Frank Shearar, Andreas Kaufmann, D6, Hooperbloob, TheParanoidOne, Mcsee, Diegof79, Nigosh, Bluebot, Nbarth, Olekva, Cydebot, Chris Pickett, Djmckee1, Jerryobject, HenryHayes, Helpful Pixie Bot, Epigenius, Burrburr and Anonymous: 4
- **JUnit** *Source:* <https://en.wikipedia.org/wiki/JUnit?oldid=672951038> *Contributors:* Nate Silva, Frecklefoot, TakuyaMurata, Furrykef, Grendelkhan, RedWolf, Iosif~enwiki, KellyCoinGuy, Ancheta Wis, WiseWoman, Ausir, Matt Crypto, Vina, Tumberumba, Andreas Kaufmann, AliveFreeHappy, RossPatterson, Rich Farmbrough, Abelson, TerraFrost, Nigelj, Cmdrjameson, Hooperbloob, Walter Görlitz, Yamla, Dsaff, Ilya, Tloche, Rastus, Sylvestre Zabala, FlaBot, UkPaolo, YurikBot, Pseudomonas, Byj2000, Vlad, Darc, Kenguest, Lt-wiki-bot, Paulsharpe, LeonardoRob0t, JLaTondre, Poulpy, Eptin, Harrisony, Kenji Toyama, SmackBot, Pbb, Faisalakeel, Ohnoitsjamie, Bluebot, Thumperward, Darth Panda, Gracenotes, MaxSem, Frap, Doug Bell, Cat Parade, PaulHurleyuk, Antonielli, Green caterpillar, Cydebot, DONOVAN, Torc2, Andmatt, Biyer, Thijs!bot, Eprb123, Hervegirod, Kleb~enwiki, Gioto, Dougher, JAnDbot, MER-C, KuwarOnline, East718, Plasmfire, Ftiercel, Gwern, R'n'B, Artaxiad, Ntalamai, Tikiwont, Anomen, Tweisbach, Randomalious, VolkovBot, Science4sail, Mdediana, DaoKaiohsin, Softtest123, Andy Dingley, Eye of slink, Resurgent insurgent, SirGeek CSP, Jpalm 98, Duplicity, Jerryobject, Free Software Knight, Kent Beck, Manish85dave, Ashwinikvp, Esmimis, VOGELLA, M4gnum0n, Stypex, SF007, Mahmudludag, Neilireson, Sandipk singh, Quinntaylor, MrOllie, MrVanBot, JTR5121819, Jarble, Legobot, Yobot, Pcap, Wickorama, Blueasberry, MaterialsScientist, Schlauer Gerd, BeauMartinez, POajdbhf, Popoxee, Softwaresavant, FrescoBot, Mark Renier, D'ohBot, Sae1962, Salvan, NamshubWriter, B3t, Ghostkadost, Txt.file, KillerGardevoir, JnRouvignac, RjwilmsiBot, Ljr1981, ZéroBot, Bulversator, TropicalFishes, Kuoja, J0506, Tobias.trelle, Frogging101, Funkymanas, Doggum, Gildor478, Rubygnome, Ilias19760, Sohashaik, Viam Ferream, Nick-PhillipsRDF and Anonymous: 127
- **CppUnit** *Source:* <https://en.wikipedia.org/wiki/CppUnit?oldid=664774033> *Contributors:* Tobias Bergemann, David Gerard, Andreas Kaufmann, Mecanismo, TheParanoidOne, Anthony Appleyard, Rjwilmsi, SmackBot, Thumperward, Frap, Cydebot, Lews Therin, Ikebana, ColdShine, DrMiller, Martin Rizzo, Yanxiaowen, Idioma-bot, DSParillo, WereSpielChequers, Jayelston, Sysuphos, Rhododendrites, Addbot, GoldenMedian, Mgfz, Yobot, Amenel, Conrad Braam, DatabaseBot, JnRouvignac, Oliver H, BG19bot, Arranna, Dexbot, Rezonansowy and Anonymous: 17
- **Test::More** *Source:* <https://en.wikipedia.org/wiki/Test%3A%3AMore?oldid=673804246> *Contributors:* Scott, Pjf, Mindmatrix, Schwern, RussBot, Unforgiven24, SmackBot, Magioladitis, Addbot, Dawynn, Tassedethe, Wickorama and Anonymous: 3
- **NUnit** *Source:* <https://en.wikipedia.org/wiki/NUnit?oldid=675551088> *Contributors:* RedWolf, Hadal, Mattflaschen, Tobias Bergemann, Thv, Sj, XtinaS, Cwbrandsma, Andreas Kaufmann, Abelson, S.K., Hooperbloob, Reidhoch, RHaworth, CodeWonk, Rastus, Nigosh, Pinecar, RodasSmith, B0sh, Bluebot, MaxSem, Zsinj, Whpq, Cydebot, Valodzka, PaddyMcDonald, Ike-bana, MicahElliott, Thijs!bot, Pnewhook, Hosamaly, Magioladitis, StefanPapp, JaGa, Gwern, Largoplazo, VolkovBot, Djmckee1, Jerryobject, ImageRemovalBot, SamuelTheGhost, Gfinzer, Brianpeiris, XLinkBot, Addbot, Mattousai, Sydevelopments, Jarble, Ben Ben, Ulrich.b, Jacosi, NinjaCross, Gypwage, Toomuchsalt, RedBot, NiccicN, Kellyselden, Titodutta, Softzen, Mnk92, Rprouse, Lflanagan and Anonymous: 49
- **NUnitAsp** *Source:* <https://en.wikipedia.org/wiki/NUnitAsp?oldid=578259547> *Contributors:* Edward, Andreas Kaufmann, Mormegil, Root4(one), Hooperbloob, Cydebot, GatoRaider, Djmckee1, SummerWithMorons and AnomieBOT

- **CsUnit** *Source:* <https://en.wikipedia.org/wiki/CsUnit?oldid=641381310> *Contributors:* Andreas Kaufmann, Stuartyeates, Mengmeng, SmackBot, MaxSem, Cydebot, Djmckee1, Jerryobject, Free Software Knight, Addbot, Yobot, AvicBot, BattyBot and Anonymous: 2
- **HtmlUnit** *Source:* <https://en.wikipedia.org/wiki/HtmlUnit?oldid=648285543> *Contributors:* Edward, Lkesteloot, Tobias Bergemann, Andreas Kaufmann, Nigelj, Diego Moya, SmackBot, KAtremer, Zanetu, Frap, Cydebot, Jj137, Tedickey, Djordje1979, Agentq314, Addbot, Mabdul, Asashour, AnomieBOT, BulldogBeing, LucienBOT, DARTH SIDIOUS 2, Bishop2067, Mguillem-enwiki, Sdesalas, BattyBot, Michaelgang and Anonymous: 27
- **Test automation** *Source:* https://en.wikipedia.org/wiki/Test_automation?oldid=672709089 *Contributors:* Deb, Edward, Kku, Ixf64, JASpencer, Ancheta Wis, Thv, Beland, Abdull, AliveFreeHappy, Jpg, Rich Farmbrough, Wrp103, Notinasnoid, Shlomif, Kbh3rd, Elipongo, Helix84, Hooperbloob, Octoferret, Goutham, Walter Görlitz, Carioca, Nimowy, Versageek, Marasmusine, RHaworth, Rickjpelleg, Radiant!, Marudubshinki, Rjwilmsi, Jake Wartenberg, CodeWonk, Florian Huber, Crazycomputers, Chills42, Hatch68, SteveLoughran, Robertvan1, Veledan, Grafen, Sundaramkumar, SmackBot, FlashSheridan, Gilliam, Ohnoitsjamie, Chris the speller, Anupam naik, Radagast83, RolandR, Michael Bernstein, Kuru, Yan Kuligin, MTSbot-enwiki, Hu12, Dreftymac, StephaneKlein, CmdrObot, EdwardMiller, Hesa, Mark Kilby, Cydebot, MC10, Enoch the red, Ryepie, Thijs!bot, Qwyrxian, JJ.Lin, Seaphoto, Dougher, WordSurd, MER-C, Nthep, Morrillonline, Magioladitis, Benjamin Geiger, JamesBWatson, DRogers, Vadimka-enwiki, Gherget, R'n'B, Ash, Tushar291081, Maurice Carbonaro, Raghublr, Ferpectionist, Ldimaggi, MendipBlue, STBotD, Chrisbepest, SoCalSuperEagle, VolkovBot, Eawens, Gaggargwal2000, Jackfork, Andy Dingley, AlleborgoBot, Kumarsameer, Prakash Nadkarni, VVVBot, Caltas, Softwaretest1, Ankurj, Faris747, Matthewedwards, SSmithNY, Ttrevers, Sbono, Ryadav, Zulfikaralib, Auntof6, Excirial, OracleDBGuru, M4gnum0n, Swtechwr, Aleksd, Mr.scavenger-enwiki, Gmacgregor, Egivoni, Webbbbbbber, Johnuniq, Apparition11, XLinkBot, Bbryson, Addbot, Pfhjvb0, Mortense, MrOllie, LaaknorBot, Asashour, RichardHoulitz, 83nj1, Amitkaria2k, Luckas-bot, Yobot, Shijuraj, Checkshirt, AnomieBOT, Winmacro, Bhagat.Abhijeet, Jim1138, Akr7577, Flopsy Mopsy and Cottonmouth, 5nizza, Hswiki, Materials scientist, Robinson Weijman, Zorgon7, Qatutor, Bigtwilkins, सरोज कुमार ढकाल, Heydaysoft, Capricorn42, Bihco, Gibs2001, Qlabs impetus, Pomoxis, FrescoBot, Jluedem, DivineAlpha, Fumitol, Xadhix, Qtpautomation, Ameya barve, Radiostationary, Ssingaraju, Srideep TestPlant, DARTH SIDIOUS 2, Mean as custard, DRAGON BOOSTER, EmausBot, PeterBizz, Tumaka, Dbelhumour02, John Cline, Christina thi, AndrewN, Mdanrel, Jay-Sebastos, ProfessionalTST, Megaride, ADobey, ElfriedeDustin, ChuispastonBot, RockMagnetist, ZachGT, Sonicyouth86, Testautomator, ClueBot NG, Nima.shahhosini, Shankar.sathiamurthi, O.Koslowski, ScottSteiner, Alec-Loxberg, G0gogcsc300, Gbegic, Jkoprax, Ststeinbauer, Helpful Pixie Bot, Filadifei, Waikh, BG19bot, Alaattinoz, Northamerica1000, Vogelt, Raymondlafourchette, In.Che., Mark Arsten, Johndunham, CitationCleanerBot, Worksoft-wayne, Jaxtester, Woella, HenryJames141, BattyBot, Krishnaegs, Leomcbride, Gtucker78, Palmirotheking, Nara Sangaa, MikaelFries, Michekszs, Edustin, Mr. Wikipediania, Faye dimarco, Drivermadness, Suna bocha, CindyJokinen, Frosty, Jamesx12345, 9th3mpt, Rapd56, Namingbot, Cvarada, Blesuer, Mikkorpela, Creftos, ScriptRockSam, Crestech1, Saffram, Abhikansh.jain, Jianhui67, Donaldanderson47, M.aiello00, Rubensmits, Praveen pinnela, Swaroop 9, Ayushyogi, Monkbot, Ishita Arora, Barcvem, Nilesh1806 and Anonymous: 338
- **Test bench** *Source:* https://en.wikipedia.org/wiki/Test_bench?oldid=664170720 *Contributors:* Abdull, Rich Farmbrough, Cbdorsett, FreplySpang, Joe Decker, Pinocar, SmackBot, Bluebot, PrimeHunter, OrphanBot, Sidearm, Singamayya, Arch dude, Magioladitis, J. Sparrow, Amitgusain, Ktr101, Tgruwell, Addbot, Testbench, AnomieBOT, E2eamon, Ali65, Erik9bot, Remotelysensed, Dorecchio, Dinamik-bot, AndyHe829, Racexr11, Dolovis, Staszek Lem, Compfreak7, Jihadcola, Briancarlton, Kcnirmiti, Mdepti.wiki and Anonymous: 17
- **Test execution engine** *Source:* https://en.wikipedia.org/wiki/Test_execution_engine?oldid=668288310 *Contributors:* Andreas Kaufmann, Abdull, Walter Görlitz, BD2412, Grafen, Fabrictramp, Cander0000, ChildofMidnight, Ali65, FrescoBot, Rontaih, Northamerica1000, Roshan220195 and Anonymous: 8
- **Test stubs** *Source:* https://en.wikipedia.org/wiki/Test_stub?oldid=671253434 *Contributors:* Deb, Andreas Kaufmann, Christianvinter, John Broughton, EncMstr, Courcelles, Lark ascending, Dougher, Tomrbj, Addbot, Chiefhuggypbear, Yobot, FrescoBot, Meridith K, Thisarticleisastub, ClueBot NG, Nishsvn, Papapasan, Kedaarrao1993 and Anonymous: 7
- **Testware** *Source:* <https://en.wikipedia.org/wiki/Testware?oldid=641147345> *Contributors:* Andreas Kaufmann, SteveLoughran, Avalon, SmackBot, Robofish, Wikid77, Nick Number, Gzkn, ZhonghuaDragon, Assadmalik, Wireless friend, Citation bot, Citation bot 1, Northamerica1000 and Anonymous: 3
- **Test automation framework** *Source:* https://en.wikipedia.org/wiki/Test_automation?oldid=672709089 *Contributors:* Deb, Edward, Kku, Ixf64, JASpencer, Ancheta Wis, Thv, Beland, Abdull, AliveFreeHappy, Jpg, Rich Farmbrough, Wrp103, Notinasnoid, Shlomif, Kbh3rd, Elipongo, Helix84, Hooperbloob, Octoferret, Goutham, Walter Görlitz, Carioca, Nimowy, Versageek, Marasmusine, RHaworth, Rickjpelleg, Radiant!, Marudubshinki, Rjwilmsi, Jake Wartenberg, CodeWonk, Florian Huber, Crazycomputers, Chills42, Hatch68, SteveLoughran, Robertvan1, Veledan, Grafen, Sundaramkumar, SmackBot, FlashSheridan, Gilliam, Ohnoitsjamie, Chris the speller, Anupam naik, Radagast83, RolandR, Michael Bernstein, Kuru, Yan Kuligin, MTSbot-enwiki, Hu12, Dreftymac, StephaneKlein, CmdrObot, EdwardMiller, Hesa, Mark Kilby, Cydebot, MC10, Enoch the red, Ryepie, Thijs!bot, Qwyrxian, JJ.Lin, Seaphoto, Dougher, WordSurd, MER-C, Nthep, Morrillonline, Magioladitis, Benjamin Geiger, JamesBWatson, DRogers, Vadimka-enwiki, Gherget, R'n'B, Ash, Tushar291081, Maurice Carbonaro, Raghublr, Ferpectionist, Ldimaggi, MendipBlue, STBotD, Chrisbepest, SoCalSuperEagle, VolkovBot, Eawens, Gaggargwal2000, Jackfork, Andy Dingley, AlleborgoBot, Kumarsameer, Prakash Nadkarni, VVVBot, Caltas, Softwaretest1, Ankurj, Faris747, Matthewedwards, SSmithNY, Ttrevers, Sbono, Ryadav, Zulfikaralib, Auntof6, Excirial, OracleDBGuru, M4gnum0n, Swtechwr, Aleksd, Mr.scavenger-enwiki, Gmacgregor, Egivoni, Webbbbbbber, Johnuniq, Apparition11, XLinkBot, Bbryson, Addbot, Pfhjvb0, Mortense, MrOllie, LaaknorBot, Asashour, RichardHoulitz, 83nj1, Amitkaria2k, Luckas-bot, Yobot, Shijuraj, Checkshirt, AnomieBOT, Winmacro, Bhagat.Abhijeet, Jim1138, Akr7577, Flopsy Mopsy and Cottonmouth, 5nizza, Hswiki, Materials scientist, Robinson Weijman, Zorgon7, Qatutor, Bigtwilkins, सरोज कुमार ढकाल, Heydaysoft, Capricorn42, Bihco, Gibs2001, Qlabs impetus, Pomoxis, FrescoBot, Jluedem, DivineAlpha, Fumitol, Xadhix, Qtpautomation, Ameya barve, Radiostationary, Ssingaraju, Srideep TestPlant, DARTH SIDIOUS 2, Mean as custard, DRAGON BOOSTER, EmausBot, PeterBizz, Tumaka, Dbelhumour02, John Cline, Christina thi, AndrewN, Mdanrel, Jay-Sebastos, ProfessionalTST, Megaride, ADobey, ElfriedeDustin, ChuispastonBot, RockMagnetist, ZachGT, Sonicyouth86, Testautomator, ClueBot NG, Nima.shahhosini, Shankar.sathiamurthi, O.Koslowski, ScottSteiner, Alec-Loxberg, G0gogcsc300, Gbegic, Jkoprax, Ststeinbauer, Helpful Pixie Bot, Filadifei, Waikh, BG19bot, Alaattinoz, Northamerica1000, Vogelt, Raymondlafourchette, In.Che., Mark Arsten, Johndunham, CitationCleanerBot, Worksoft-wayne, Jaxtester, Woella, HenryJames141, BattyBot, Krishnaegs, Leomcbride, Gtucker78, Palmirotheking, Nara Sangaa, MikaelFries, Michekszs, Edustin, Mr. Wikipediania, Faye dimarco, Drivermadness, Suna bocha, CindyJokinen, Frosty, Jamesx12345, 9th3mpt, Rapd56, Namingbot, Cvarada, Blesuer, Mikkorpela, Creftos, ScriptRockSam, Crestech1, Saffram, Abhikansh.jain, Jianhui67, Donaldanderson47, M.aiello00, Rubensmits, Praveen pinnela, Swaroop 9, Ayushyogi, Monkbot, Ishita Arora, Barcvem, Nilesh1806 and Anonymous: 338
- **Data-driven testing** *Source:* https://en.wikipedia.org/wiki/Data-driven_testing?oldid=675576471 *Contributors:* Andreas Kaufmann, Amorymeltzer, Rjwilmsi, Lockley, Cornellrockey, Pinocar, SAE1962, Rwww, SmackBot, EdGl, AlaiBot, Fabrictramp, Rajwiki, Phanis-

- rikar, Sbono, Sean.co.za, XLinkBot, Addbot, MrOllie, Zaphodikus, Mrinmayee.p, Cbojar, 2Alen, Justincheng12345-bot, ChrisGualtieri, Byteslayer7 and Anonymous: 30
- **Modularity-driven testing** *Source:* https://en.wikipedia.org/wiki/Modularity-driven_testing?oldid=578161829 *Contributors:* Rich Farmbrough, Walter Görlitz, Ron Ritzman, Pinecar, Avalon, SmackBot, AlaiBot, Minnaert, Phanisrikar, Yobot, Erik9bot, BG19bot, Fedelis4198 and Anonymous: 5
 - **Keyword-driven testing** *Source:* https://en.wikipedia.org/wiki/Keyword-driven_testing?oldid=656678700 *Contributors:* RossPatterson, Lowmagnet, Hooperbloob, Walter Görlitz, Rjwilmsi, Pinecar, RussBot, Jonathan Webley, SAE1962, Rwww, SmackBot, Bluebot, Conortodd, Ultimus, MarshBot, Maguschen, Zoobeerhall, Culudamar, Scraimer, Erkan Yilmaz, Ken g6, Jtowler, Squids and Chips, Technopat, Phanisrikar, AlleborgoBot, Sparrowman980, JL-Bot, Sean.co.za, Yun-Yuuzhan (lost password), Swteterinca, XLinkBot, Addbot, MrOllie, Download, SpBot, 5nizza, MaterialsScientist, Jeff seattle, Heydaysoft, GrouchoBot, Jonathon Wright, Eagle250, Ukkuru, Jessewigbbs, Tobias.trelle, MarkCTest, Justincheng12345-bot, Anish10110, Chris Schotanus-enwiki, Kem254, Monkbob and Anonymous: 63
 - **Hybrid testing** *Source:* https://en.wikipedia.org/wiki/Hybrid_testing?oldid=662487042 *Contributors:* Bgwhite, Horologium, Vishwas008, MrOllie, Bunnyhop11, AmeliorationBot, AnomieBOT, Jonathon Wright, ThePurpleHelmet, Dwelch67 and Anonymous: 7
 - **Lightweight software test automation** *Source:* https://en.wikipedia.org/wiki/Lightweight_software_test_automation?oldid=592746348 *Contributors:* Pnm, Greenrd, CanisRufus, John Vandenberg, BD2412, Rjwilmsi, Bluebot, Colonies Chris, Torc2, JamesDmccaffrey, OracleDBGuru, Verbal, Tutterz, Helpful Pixie Bot, ChrisGualtieri and Anonymous: 6
 - **Software testing controversies** *Source:* https://en.wikipedia.org/wiki/Software_testing_controversies?oldid=674783669 *Contributors:* JASpencer, Centrx, Andreas Kaufmann, Walter Görlitz, RHaworth, Pinecar, SmackBot, Wikiisawesome, Softtest123, Lightbot, Yobot, PigFlu Oink, DrilBot, Derelictfrog, BattyBot, Testingfan, Monkbob and Anonymous: 6
 - **Test-driven development** *Source:* https://en.wikipedia.org/wiki/Test-driven_development?oldid=676237022 *Contributors:* Damian Yerrick, Ed Poor, SimonP, Eurlief, TakuyaMurata, Edaelon, Nohat, Furrykef, Gakrivas, RickBeton, Craig Stuntz, Sverdrup, KellyCoin-Guy, Faught, Hadal, Astaines, Jleedev, Pengo, Tobias Bergemann, Enochlau, DavidCary, Mboverload, Khalid hassani, AnthonySteele, Mberteig, Beland, SethTisue, Heirpixel, Sam Hocesvar, Kevin Rector, Abdull, Canterbury Tail, AliveFreeHappy, Madduck, Mathias126, Parklandspanaway, Asgeirn, Nigelj, Shenme, R. S. Shaw, Mr2001, Notnoisy, Mdd, Larham, Gary, Walter Görlitz, Droob, Topping, Nugget-boy, Daira Hopwood, Mckoss, Teemu Leisti, Calréfa Wéná, Kbdank71, Dougluce, Kristjan Wager, Bcwhite, Pinecar, PhilipR, YurikBot, SteveLoughran, Blutfink, Ojcit, Długosz, SAE1962, Mosquitopsu, Stemcd, Deuxpi, Closedmouth, JLaTondre, Attilios, Jonkpa, SmackBot, Radak, Kellen, AutumnSnow, Patrickdepinguin, Gmcrews, Autarch, Thumperward, Nbarth, Emurphy42, MaxSem, Waratah-enwiki, Evolve2k, Daniel.Cardenas, Kpugh, Franyhi, PradeepArya1109, Jrzv, Antonielly, Michael miceli, Dally Horton, Ehheh, Martinig, Achorny, Dtmilano, Galatoni, Micah hainline, Rulesdoc, Shoez, Cydebot, CFMWiki1, Gogo Dodo, On5deu, Underpants, Ebrahim, Wikid77, FreOn, Dougher, Krzyk2, Sanchom, Michig, Magioladitis, VoABot II, Tedickey, Jonb ee, SharShar, Phlip2005, Lenin1991, WLU, Sullivan.t, Dhdblues, Kabir1976, Kvdveer, Chris Pickett, Martial75, Mkarkesky, VolkovBot, Sporti, Mkksingha, LeaveSleaves, Swadsen, Andy Dingley, Mossd, Jpalm 98, Mhhanley, JDBravo, Svick, Themacboy, Hzhbcl, ClueBot, Alksentrs, Grantbow, DHGarrette, Shyam 48, Excirial, Alexbot, SchreiberBike, Hariharan wiki, Samwashburn3, RoyOsherove, XLinkBot, Xagronaut, Lumberjake, SilvenonBot, JacobProffitt, Addbot, Mortense, Anorthup, Raghunathan.george, Virgiltrasca, NjardarBot, MrOllie, Download, Geometry.steve, Zorrobot, Middayexpress, Luckas-bot, Yobot, AnomieBOT, St.General, MaterialsScientist, TwilightSpirit, ArthurBot, MauritsBot, Xqbot, Gigi fire, V6Zi34, Gishu Pillai, Олександр Кравчук, Shadowjams, Mark Renier, Downsize43, Szwejk, SaltOfTheFlame, CraigTreptow, D'ohBot, Hagai Cibulski, Supreme Deliciousness, AmphBot, Oligomous, MeUser42, Jglynn43, Sideways713, Valyt, EmausBot, BillyPre-set, Trum123-enwiki, GoingBatty, Mnorbury, ZéroBot, Fbeppler, Israghavan, Arminru, San chako, TYelliot, ClueBot NG, MelbourneS-star, Adair2324, O.Koslowski, Widr, Electriccatfish2, Rbrunner7, Chmarkine, Falcn42, Ogennadi, Lugia2453, Stephaniefontana, Choriem, Johnnybifter, Softzen, Whapp, Timofiev, Marcinkaw, Monkbob, Trogodyte, Khouston1, Sanchezluis2020, Ryancook2002, ScottAnthonyRoss, Udit.1990 and Anonymous: 357
 - **Agile testing** *Source:* https://en.wikipedia.org/wiki/Agile_testing?oldid=666627343 *Contributors:* Pnm, Chowbok, Mdd, Walter Görlitz, Gurch, Pinecar, Luiscolorado, Sardanaphalus, Icaruspassion, ScottWambler, AGK, Manistar, Eewild, Random name, Although, Alanbly, Vertium, Kosmocentric, Patrickegan, Weimont, Webrew, Podge82, M2Ys4U, Denisarona, The Thing That Should Not Be, Vaibhav.nimbalkar, Johnuniq, XLinkBot, MrOllie, AnomieBOT, Ericholmstrom, LilHelpa, Lisacrispin, FrescoBot, Hemnath18, Zonafan39, Agilista, Janetgregoryca, GoingBatty, MathMaven, Agilettesting, Ehendrickson, 28bot, ClueBot NG, Henri662, Helpful Pixie Bot, ParaTom, Okevin, Who.was.phone, MarkCTest, Mpkhosla, Softzen, Badbud65, Baumgartnerm, Mastermb and Anonymous: 71
 - **Bug bash** *Source:* https://en.wikipedia.org/wiki/Bug_bash?oldid=662893354 *Contributors:* DragonflySixtyseven, Andreas Kaufmann, Rich Farmbrough, BD2412, Pinecar, ENeville, Retired username, Thumperward, Archippus, MisterHand, Freek Verkerk, Cander0000, Traveler100, Bonams, Yobot, AnomieBOT, Citation bot, Helpful Pixie Bot, Filadifei and Anonymous: 4
 - **Pair Testing** *Source:* https://en.wikipedia.org/wiki/Pair_testing?oldid=676241058 *Contributors:* Andreas Kaufmann, Walter Görlitz, Woohookitty, Tabletop, Josh Parris, Tony1, SmackBot, Neoneif, Universal Cereal Bus, Cmr08, Jafeluv, MrOllie, LilHelpa, Prasantam, Bjosman, ClueBot NG, Lewissall1, Jimbou-enwiki, Juhuyuta and Anonymous: 8
 - **Manual testing** *Source:* https://en.wikipedia.org/wiki/Manual_testing?oldid=671243906 *Contributors:* Walter Görlitz, Woohookitty, Josh Parris, Pinecar, Rwxrwxrwx, ArielGold, SmackBot, Gilliam, IronGargoyle, Iridescent, Eewild, JohnCD, Cybock911, AlaiBot, Morrilonline, Donperk, Ashish.aggrawal17, Meetusingh, Saurabha5, Denisarona, JL-Bot, SuperHamster, Predatoraction, Nath1991, OIEnglish, SwisterTwister, Hairhorn, AdjustShift, MaterialsScientist, Pinethicket, Orenburg1, Trappist the monk, DARTH SIDIOUS 2, RjwilmsiBot, Tumaka, L Kensington, Kgarima, Somdeb Chakraborty, ClueBot NG, Wikishahill, Helpful Pixie Bot, Softwrite, MusikAnimal, Pratyaya Ghosh, Mogism, Lavados, Monkbob, Maddinenid09, Bikash ranjan swain and Anonymous: 86
 - **Regression testing** *Source:* https://en.wikipedia.org/wiki/Regression_testing?oldid=669634511 *Contributors:* Tobias Hoevekamp, Robert Merkel, Deb, Marijn, Cabalamat, Vsync, Wlievens, Hadal, Tobias Bergemann, Matthew Stannard, Thv, Neile, Antandrus, Jacob grace, Srittau, Urhixidur, Abdull, Mike Rosoft, AliveFreeHappy, Janna Isabot, Hooperbloob, Walter Görlitz, HongPong, Marudubshinki, Kesla, MassGalactusUniversum, SqueakBox, Strait, Amire80, Andrew Eisenberg, Chobot, Scoops, Pinecar, Snarius, Lt-wiki-bot, SmackBot, Brenda Kenyon, Yanyoyega, Emj, Chris the speller, Estyler, Antonielly, Dee Jay Randall, Maxwellb, LandruBek, CmdrObot, Eewild, Abhinavvaid, Ryans.ryu, Gregbard, Cydebot, Krauss, Ravialluru, Michaelas10, Bazzargh, Christian75, AntiVandalBot, Designatevoid, MikeLynch, Cdunn2001, MER-C, Michig, MickeyWiki, Baccyak4H, DRogers, S3000, Toon05, STBotD, Chris Pickett, Labalius, Boon-goman, Zhenqinli, Forlornturtle, Enti342, Svick, Benefactor123, Doug.hoffman, Spock of Vulcan, Swtechr, 7, XLinkBot, Addbot, Elsendero, Anorthup, Jarble, Ptbogourou, Nallimbot, Noq, MaterialsScientist, Neurolysis, Qatutor, Iiren, A.amitkumar, Qfissler, BenzolBot, Mariotto2009, Cnwiliams, SchreyP, Throwaway85, Zvn, Rsavenkov, Kamarou, RjwilmsiBot, NameIsRon, Msillil, Menzogna,

- Ahsan.nabi.khan, Alan ffm, Dacian.epure, L Kensington, Luckydrink1, Petr, Will Beback Auto, ClueBot NG, Gareth Griffith-Jones, This lousy T-shirt, G0gogsc300, Henri662, Helpful Pixie Bot, Philipchiappini, Pacerier, Kmincey, Parvuselephantus, Herve272, Hector224, EricEnfermero, Carlos.l.sanchez, Softzen, Monkbob, Abarkth99, Mjandrewsnet, Dheeraj.005gupta and Anonymous: 192
- **Ad hoc testing** *Source:* https://en.wikipedia.org/wiki/Ad_hoc_testing?oldid=675746543 *Contributors:* Faught, Walter Görlitz, Josh Parris, Sjö, Pinecar, Epim-enwiki, DRogers, Erkan Yilmaz, Robinson weijman, Yintan, Ottawa4ever, IQDave, Addbot, Pmod, Yobot, Solde, Yunshui, Pankajkittu, Lhb1239, Sharkanana, James12345, Eyesnore, Drakecb and Anonymous: 24
 - **Sanity testing** *Source:* https://en.wikipedia.org/wiki/Sanity_check?oldid=673609780 *Contributors:* Lee Daniel Crocker, Verloren, Pierre-Abbat, Karada, Dysprosia, Itai, Auric, Martinwguy, Nunh-huh, BenFrantzDale, Andycjp, Histrion, Fittysix, Sietse Snel, Viriditas, Polluks, Walter Görlitz, Oboler, Qwertyus, Strait, Pinecar, RussBot, Pyroclastic, Saberwyn, Closedmouth, SmackBot, Melchoir, McGeddon, Mike-walk, Kaimiddleton, Rrburke, Fullstop, NeilFraser, Stratadrake, Haus, JForget, Wafulz, Ricardol, Wikid77, D4g0thur, AntiVandalBot, Alphachimpbot, BrotherE, R'n'B, Chris Pickett, Steel1943, Lechatjaune, Gorank4, SimonTrew, Chillum, Mild Bill Hiccup, Arjayay, Lucky Bottlecap, UlrichAAB, LeaW, Matma Rex, Favonian, Legobot, Yobot, Kingpin13, Pinethicket, Consummate virtuoso, Banej, TobeBot, Andrey86, Donner60, ClueBot NG, Accelerometer, Webinfoonline, Mmckmg, Andyhowlett, Monkbob and Anonymous: 82
 - **Integration testing** *Source:* https://en.wikipedia.org/wiki/Integration_testing?oldid=664137098 *Contributors:* Deb, Jiang, Furrykef, Michael Rawdon, Onebyone, DataSurfer, GreatWhiteNortherner, Thv, Jewbacca, Abdull, Discospinster, Notinasnoid, Paul August, Hooperbloop, Walter Görlitz, Lordfaust, Qaddosh, Halovivek, Amire80, Arzach, Banaticus, Pinecar, ChristianEdwardGruber, Ravedave, Pegship, Tom Morris, SmackBot, Mauls, Gilliam, Mheusser, Arunka-enwiki, Addshore, ThurnerRupert, Krashlondon, Michael miceli, SkyWalker, Marek69, Ehabmehedi, Michig, Cbenedetto, TheRanger, DRogers, J.delanoy, Yonidebot, Jtowler, Ravindrat, SRCHFD, Wyldtwyst, Zhenqinli, Synthebot, VVVBot, Flyer22, Faradayplank, Steven Crossin, Svick, Cellovergara, Spokeninsanskrit, ClueBot, Avoided, Myhister, Cmungall, Gggh, Addbot, Luckas-bot, Kmerenkov, Solde, Materialschemist, RibotBOT, Sergey1984, Ryanboyle2009, DrillBot, I dream of horses, Savh, ZéroBot, ClueBot NG, Asukite, Widr, HMSolent, Softwareqa, Kimriatray and Anonymous: 140
 - **System testing** *Source:* https://en.wikipedia.org/wiki/System_testing?oldid=676685869 *Contributors:* Ronz, Thv, Beland, Jewbacca, Abdull, AliveFreeHappy, Bobo192, Hooperbloop, Walter Görlitz, GeorgeStepanek, RainbowOfLight, Woohookitty, SusanLarson, Chobot, Roboto de Ajvol, Pinecar, ChristianEdwardGruber, NickBush24, Ccompton, Closedmouth, A bit iffy, SmackBot, BiT, Gilliam, Skizzik, DHN-bot-enwiki, Freek Verkerk, Valenciano, Ssweeting, Ian Dalziel, Argon233, Wchkwok, Ravialluru, Mojo Hand, Tmopkiss, Michig, DRogers, Ash, Anant vyas2002, STBotD, Vmahi9, Harveysburger, Philip Trueman, Vishwas008, Zhenqinli, Techman224, Manway, AndreChou, 7, Mpilaeten, DumZiBoT, Lauwerens, Myhister, Addbot, Morning277, Lightbot, AnomieBOT, Kingpin13, Solde, USConsLib, Omnipaedista, Bftsg, Downsize43, Cnwiliams, TobeBot, RCHenningsgard, Suffusion of Yellow, Bex84, ClueBot NG, Creeper jack1, Aman sn17, TI, Gracchus, Tentinator, Lars.Krienke and Anonymous: 117
 - **System integration testing** *Source:* https://en.wikipedia.org/wiki/System_integration_testing?oldid=672400149 *Contributors:* Kku, Bearcat, Andreas Kaufmann, Rich Farmbrough, Walter Görlitz, Fat pig73, Pinecar, Gaius Cornelius, Jpbowen, Flup, Rwww, Bluebot, Mikethegreen, Radagast83, Panchitaville, CmdrObot, Myasuda, Kubanczyk, James086, Alphachimpbot, Magioladitis, VoABot II, DRogers, JeromeJerome, Anna Lincoln, Barbie, Aliasgarshakir, Zachary Murray, AnomieBOT, FrescoBot, Mawcs, SchreyP, Carminowe of Hendra, AvicAWB, Charithk, Andrewmillen, ChrisGualtieri, TheFrog001 and Anonymous: 36
 - **Acceptance testing** *Source:* https://en.wikipedia.org/wiki/Acceptance_testing?oldid=673637033 *Contributors:* Eloquence, Timo Honkasalo, Deb, William Avery, SimonP, Michael Hardy, GTBacchus, PeterBrooks, Xanzzibar, Enochlau, Mjemmeson, Jpp, Panzi, Mike Rosoft, Ascánder, Pearle, Hooperbloop, Walter Görlitz, Caesura, Ksnow, CloudNine, Woohookitty, RHaworth, Liftoph, Halovivek, Amire80, FlaBot, Old Moonraker, Riki, Intr, Gwernol, Pinecar, YurikBot, Hyad, Jgladding, Rodasmith, Dhollm, GraemeL, Fram, Whaa?, Ffangs, DVD R W, Myroslav, SmackBot, Phylburn, Jemtreadwell, Bournejc, DHN-bot-enwiki, Midnightcomm, Alphajuliet, Normxxx, Hu12, CapitalR, Ibadibam, Shirulashem, Viridae, PKT, BetacommandBot, Pajz, Divyadeepsharma, Seaphoto, RJFerret, MartinDK, Swpb, Qem, Granburguesa, Olson.sr, DRogers, Timmy12, Rlsheehan, Chris Pickett, Carse, VolkovBot, Dahcalan, TXiKiBoT, ^demonBot2, Djmckee1, AlleborgoBot, Caltas, Toddst1, Jojalozzo, ClueBot, Hutcher, Emilybache, Melizg, Alexbot, JimJavascript, Muhandes, Rhododendrites, Jmarranz, Jamestochter, Mpilaeten, SoxBot III, Apparition11, Well-rested, Mifter, Myhister, Meise, Mortense, MeijdenB, Davidbatet, Margin1522, Legobot, Yobot, Milk's Favorite Bot II, Xqbot, TheAMmollusc, DSisypBot, Claudio figueiredo, Wikipetan, Winterst, I dream of horses, Cnwiliams, Newbie59, Lotje, Eco30, Phamti, RjwilmsiBot, EmausBot, WikitanvirBot, TuHan-Bot, Fæ, Kaitanen, Daniel.r.bell, ClueBot NG, Amitg47, Dlevy-telerik, Infrablue, Pine, HadanMarv, BattyBot, Bouxetuv, Tcxspears, ChrisGualtieri, Salimchami, Kekir, Vanamonde93, Emilesilvis, Simplewhite12, Michaonwiki, Andre Piantino, Usa63woods, Sslavov, Marcgrub and Anonymous: 163
 - **Risk-based testing** *Source:* https://en.wikipedia.org/wiki/Risk-based_testing?oldid=675543733 *Contributors:* Deb, Ronz, MSGJ, Andreas Kaufmann, Walter Görlitz, Chobot, Gilliam, Chris the speller, Lorezsky, Hu12, Paulgerrard, DRogers, Tdjones74021, IQDave, Addbot, Ronhjones, Lightbot, Yobot, AnomieBOT, Noq, Jim1138, VestaLabs, Henri662, Helpful Pixie Bot, Herve272, Belgarath7000, Monkbob, JulianneChladny, Keithrhill5848 and Anonymous: 19
 - **Software testing outsourcing** *Source:* https://en.wikipedia.org/wiki/Software_testing_outsourcing?oldid=652044250 *Contributors:* Discospinster, Woohookitty, Algebraist, Pinecar, Bhny, SmackBot, Elagatis, JesseRafe, Robofish, TastyPoutine, Hu12, Kirk Hilliard, BetacommandBot, Magioladitis, Tedickey, Dawn Bard, Promoa1-enwiki, Addbot, Pratheepraj, Tesstty, AnomieBOT, Piano non troppo, Mean as custard, Jenks24, NewbieIT, MelbourneStar, Lolawrites, BG19bot, BattyBot, Anujgupta2 979, Tom1492, ChrisGualtieri, JaneStewart123, Gonarg90, Lmcdmag, Reattesting, Vitalywiki, Trungvn87 and Anonymous: 10
 - **Tester driven development** *Source:* https://en.wikipedia.org/wiki/Tester_Driven_Development?oldid=594076985 *Contributors:* Bearcat, Malcolm, Fram, BOTijo, EmausBot, AvicBot, Johanlundberg2 and Anonymous: 3
 - **Test effort** *Source:* https://en.wikipedia.org/wiki/Test_effort?oldid=544576801 *Contributors:* Ronz, Furrykef, Notinasnoid, Lockley, Pinecar, SmackBot, DCDuring, Chris the speller, Alaibot, Mr pand, AntiVandalBot, Erkan Yilmaz, Chemuturi, Lakeworks, Addbot, Downsize43, Contributor124, Helodia and Anonymous: 6
 - **IEEE 829** *Source:* https://en.wikipedia.org/wiki/Software_test_documentation?oldid=643777803 *Contributors:* Damian Yerrick, GABaker, Kku, CesarB, Haakon, Grendelkhan, Shizhao, Fredrik, Korath, Matthew Stannard, Walter Görlitz, Pmberly, Utuado, FlaBot, Pinecar, Robertvan1, A.R., Firefox13, Hu12, Inukjuak, Grey Goshawk, Donmillion, Methygrace, Paulgerrard, J.delanoy, STBotD, VladV, Addbot, Iexec1, Antariksawan, Nasa-verve, RedBot, Das.steichen, ChuispastonBot, Ghalloun, RapPayne, Malindrom, Hebriden and Anonymous: 41
 - **Test strategy** *Source:* https://en.wikipedia.org/wiki/Test_strategy?oldid=672277820 *Contributors:* Ronz, Michael Devore, Rpyle731, Mboverload, D6, Christopher Lamothe, Alansohn, Walter Görlitz, RHaworth, Pinecar, Malcolm, Avalon, Shepard, SmackBot, Freek

- Verkerk, Alaibot, Fabrictramp, Dirkbb, Denisarona, Mild Bill Hiccup, M4gnum0n, Mandarhambir, HarlandQPitt, Addbot, BartJan-deLeuw, LogoX, Jayaramg, Liheng300, Downsize43, Santhoshmars, John of Reading, AlexWolfx, Autoerrant, ClueBot NG, Henri662, Altair, Ankitamor, Minhaj21, DoctorKubla and Anonymous: 83
- **Test plan** *Source:* https://en.wikipedia.org/wiki/Test_plan?oldid=677114561 *Contributors:* SimonP, Ronz, Charles Matthews, Dave6, Matthew Stannard, Thv, Craigwb, Jason Quinn, SWAdair, MarkSweep, Aecis, Aaronbrick, Foobaz, Walter Görlitz, RJFJR, Wacko, Jeff3000, -Ril-, Ketiltrout, NSR, Pinecar, RussBot, Stephenb, Alynna Kasmira, RL0919, Zwobot, Scope creep, E Wing, NHSavage, Drable, SmackBot, Commander Keane bot, Schmiteye, Jla04, Hongooi, KaiserbBot, Freek Verkerk, AndrewStelman, Jgorse, Wagggers, Kindx, Randhirreddy, Gogo Dodo, Omicronpersei8, Thijs!bot, Padma vgp, Mk*, Oriwall, Canadian-Bacon, JAnDbot, MER-C, Michig, Kitdadio, Pedro, VoABot II, AuburnPilot, Icbkr, Yparedes-enwiki, Tgeairn, Rlsheehan, Uncle Dick, Hennessey, Patrick, Mellissa.mccconnell, Moonbeachx, Roshanoinam, Thunderwing, Jaganathcfs, ClueBot, The Thing That Should Not Be, Niceguyedc, Ken tabor, M4gnum0n, Rror, Addbot, Luckas-bot, OllieFury, LogoX, Grantmidnight, Ismarc, Shadowjams, Downsize43, Orphan Wiki, WikitanvirBot, Bashnya25, Rcsprinter123, ClueBot NG, MelbourneStar, Widr, Theopolisme, OndraK, Pine, Epicgenius, Kbpkumar, Bakosjen, Dishank3 and Anonymous: 269
 - **Traceability matrix** *Source:* https://en.wikipedia.org/wiki/Traceability_matrix?oldid=671263622 *Contributors:* Deb, Ahoerstermeier, Ronz, Yvesb, Fry-kun, Charles Matthews, Furrykef, Andreas Kaufmann, Discospinster, Pamar, Mdd, Walter Görlitz, Marudubshinki, Graham87, Mathbot, Gurch, Pinecar, Sardanaphalus, Gilliam, Timneue22, Kuru, AGK, Markbassett, Dgw, Donmillion, DRogers, Rette-tast, Mariolina, IPSOS, Craigwbrown, Pravinparmarce, Billinghamurst, ClueBot, Excirial, XLinkBot, Addbot, MrOllie, AnomieBOT, FrescoBot, WikiTome, Thebluemanager, Shambhaviroy, Solarra, ZéroBot, Herp Derp, தனெ்காசி சும்பிரமணியன், ChrisGualtieri, SFK2 and Anonymous: 108
 - **Test case** *Source:* https://en.wikipedia.org/wiki/Test_case?oldid=671388358 *Contributors:* Furrykef, Pilaf-enwiki, Thv, Iondiode, AliveFreeHappy, ColBatGuano, MaxHund, Hooperbloob, Mdd, Walter Görlitz, Mr Adequate, Velella, Suruena, RJFJR, RainbowOfLight, Sciurinae, Nibblus, Dovid, MassGalactusUniversum, Nmthompson, Shervinafshar, Pinecar, Flavioxavier, Sardanaphalus, Gilliam, RayAYang, Darth Panda, Freek Verkerk, Gothmog.es, Gobonobo, Lenoxus, AGK, Eastlaw, Torc421, Travelbird, Merutak, Thijs!bot, Eprb123, Wernight, AntiVandalBot, Magioladitis, VoABot II, Kevinmon, Allstarecho, Pavel Zubkov, DarkFalls, Yennth, Jwh335, Jtowler, Chris Pickett, DarkBlueSeid, Sean D Martin, LeaveSleaves, Thejesh.cg, Tomaxer, System21, Yintan, Peter7723, JL-Bot, Thorncrag, ClueBot, Zack wadghiri, BOTarate, SoxBot III, Addbot, Cst17, MrOllie, LaaknorBot, Fraggie81, Amirobot, MaterialsScientist, Locobot, PrimeObjects, Renu gautam, Pinethicket, Momergil, Unikaman, Niri.M, Maniacs29, Vikasbucha, Vrenator, Cowpig, EmausBot, WikitanvirBot, Mo ainm, ZéroBot, John Cline, Ebrambot, ClueBot NG, Srikaa123, MadGuy7023, The Anonymouse, Shaileshsingh5555, Abhinav Yd and Anonymous: 171
 - **Test data** *Source:* https://en.wikipedia.org/wiki/Test_data?oldid=666572779 *Contributors:* JASpencer, Craigwb, Alvestrand, Fg2, Zntrip, Uncle G, Pinecar, Stephenb, SmackBot, Onorem, Nnesbit, Qwfp, AlexandrDmitri, MaterialsScientist, I dream of horses, SentinelAlpha, ClueBot NG, Snotbot, Gakiwate and Anonymous: 17
 - **Test suite** *Source:* https://en.wikipedia.org/wiki/Test_suite?oldid=645239892 *Contributors:* Andreas Kaufmann, Abdull, Martpol, Liao, Walter Görlitz, Alai, A-hiro, FreplySpang, Pinecar, KGasso, Derek farn, JzG, CapitalR, Kenneth Burgener, Unixtastic, VasilievVV, Lake-works, Addbot, Luckas-bot, Denispir, Wonderfl, Newman.x, Vasywriter, Cnwiliams, ClueBot NG, BG19bot, Stephenwanjau, Abhirajan12 and Anonymous: 28
 - **Test script** *Source:* https://en.wikipedia.org/wiki/Test_script?oldid=600623870 *Contributors:* Thv, Rchandra, PaulMEDwards, Hooperbloob, Walter Görlitz, RJFJR, Alai, MassGalactusUniversum, Ub-enwiki, Pinecar, JLaTondre, SmackBot, Jruuska, Teiresias-enwiki, Bluebot, Freek Verkerk, Eewild, Michig, Gwern, Redrocket, Jtowler, Sujaikareik, Falterion, Sean.co.za, Addbot, Pfhjvb0, Xqbot, Erik9bot, JnRouvignac, ClueBot NG, Chris1991 and Anonymous: 25
 - **Test harness** *Source:* https://en.wikipedia.org/wiki/Test_harness?oldid=666336787 *Contributors:* Greenrd, Furrykef, Caknuck, Wlievens, Urhixidur, Abdull, AliveFreeHappy, Kgaughan, Caesura, Tony Sidaway, DenisYurkin, Mindmatrix, Calréfa Wéná, Allen Moore, Pinecar, Topperfalkon, Avalon, SmackBot, Dntown dan seattle, Dugrocker, Brainwavz, SQAT, Ktr101, Alexbot, Addbot, Ali65, ClueBot NG, ChrisGualtieri, Nishsvn and Anonymous: 32
 - **Static testing** *Source:* https://en.wikipedia.org/wiki/Static_program_analysis?oldid=668929812 *Contributors:* AlexWasFirst, Ted Longstaffe, Vkuncak, Ixfd64, Tregoweth, Ahoerstermeier, TUF-KAT, Julesd, Ed Brey, David.Monnaux, Psychonaut, Wlievens, Thv, Kravietz, Gadium, Vina, Rpm-enwiki, Andreas Kaufmann, AliveFreeHappy, Guanabot, Leibniz, Vp, Peter M Gerdes, Yonkie, Walter Görlitz, Diego Moya, Suruena, Kazvorpai, Ruud Koot, Marudubshinki, Graham87, Qwertys, Rjwilmsi, Ground Zero, Mike Van Emmerik, Chobot, Berrinam, Crowfeather, Pinecar, Renox, Jschlosser, Cryptic, Goffrie, Jpbowen, CaliforniaAliBaba, Creando, GraemeL, Rwww, SmackBot, FlashSheridan, Thumperward, Schwallex, A5b, Derek farn, Anujgoyal, Antonielly, JForget, Simeon, Wikid77, Ebde, RobotG, Obiwankenobi, Magioladitis, Cic, Lgirvin, JoelSherrill, Erkan Yilmaz, DatabACE, Andareed, StaticCast, Ferengi, Sashakir, SieBot, Sttaft, Toddst1, Ks0stm, Wolfch, JanInad, Mutilin, Swtechwr, Dekisugi, HarrivBOT, Hoco24, Tinus74, MrOllie, Lightbot, Legobot, Luckas-bot, Yobot, AnomieBOT, Kskjy, Villeez, Shadowjams, FrescoBot, Fderepas, Jisunjang, TjBot, Dbelhumeur02, ZéroBot, Jabraham mw, Ptrb, JohnGDrever, Helpful Pixie Bot, Wbm1058, BG19bot, JacobTrue, BattyBot, Ablighnicta, Jionpedia, Freddygauss, Fran buchmann, Paul2520, Knife-in-the-drawer and Anonymous: 108
 - **Software review** *Source:* https://en.wikipedia.org/wiki/Software_review?oldid=650417729 *Contributors:* Karada, William M. Connolly, Andreas Kaufmann, AliveFreeHappy, Woohookitty, XLerate, Bovineone, David Biddulph, SmackBot, Bluebot, Audriusa, Matchups, Colonel Warden, Donmillion, Madjidi, Dima I, A Nobody, XLinkBot, Tassedethe, Gail, Yobot, AnomieBOT, Danno uk, SassoBot, Jschnur, RjwilmsiBot, Irfibwp, Rcsprinter123, Rolf acker, Helpful Pixie Bot, Mitatur and Anonymous: 24
 - **Software peer review** *Source:* https://en.wikipedia.org/wiki/Software_peer_review?oldid=659297789 *Contributors:* Ed Poor, Michael Hardy, Karada, Ed Brey, Andreas Kaufmann, AliveFreeHappy, Gronky, Rjwilmsi, Sdorman, Kjenks, Bovineone, Bluebot, Donmillion, PKT, Zakahori, MarkKozel, Kezz90, Anonymous101, Danno uk, Lauri.pirttiaho, Helpful Pixie Bot, Monkbot, Miraclexix and Anonymous: 10
 - **Software audit review** *Source:* https://en.wikipedia.org/wiki/Software_audit_review?oldid=560402299 *Contributors:* Tregoweth, Andreas Kaufmann, Zro, Woohookitty, Kralizec!, SmackBot, Donmillion, JaGa, Katharineamy, Yobot, Romain Jouvét, Codename Lisa and Anonymous: 4
 - **Software technical review** *Source:* https://en.wikipedia.org/wiki/Software_technical_review?oldid=570437645 *Contributors:* Edward, Andreas Kaufmann, SmackBot, Markbassett, Donmillion, Gnewf, Sarahj2107, Anna Lincoln, Erik9bot, Thehelpfulbot, Helpful Pixie Bot and Anonymous: 5

- **Management review** *Source:* https://en.wikipedia.org/wiki/Management_review?oldid=599942391 *Contributors:* Karada, Andreas Kaufmann, Giraffedata, Ardrick47, Rintrah, Bovineone, Deckiller, SmackBot, André Koehne, Donmillion, Outlook, Octopus-Hands, Bagpip-ingScotsman, Galena11, JustinHagstrom, Anticipation of a New Lover's Arrival, The, Vasywriter, Gumhoefer and Anonymous: 4
- **Software inspection** *Source:* https://en.wikipedia.org/wiki/Software_inspection?oldid=668284237 *Contributors:* Kku, Fuzheado, Wik, Bovlb, Andreas Kaufmann, Arminius, Bgwhite, Stephenb, SteveLoughran, JohnDavidson, Occono, David Biddulph, SmackBot, Bigbluefish, AutumnSnow, PJTraill, AndrewStellman, A.R., Ft1~enwiki, Michaelbusch, Ivan Pozdeev, WeggeBot, Rmallins, Ebde, Seaphoto, Big-MikeW, Vivio Testarossa, PeterNuernberg, Addbot, Yobot, Amirobot, KamikazeBot, Seccio, Mtilli, EmausBot, ClueBot NG, ISTB351, Nmcou, Anujasp, Alvarogili, Pcellsworth and Anonymous: 36
- **Fagan inspection** *Source:* https://en.wikipedia.org/wiki/Fagan_inspection?oldid=663071346 *Contributors:* Zundark, ChrisG, Altenmann, Tagishsimon, MacGyverMagic, Arthena, Drbreznjev, JIP, Rjwilmsi, Okok, Bhny, Gaius Cornelius, BOT-Superzerocool, Zerodamage, Mjevans, Attilios, Bigbluefish, Gaff, PJTraill, Bluebot, Can't sleep, clown will eat me, Courcelles, The Letter J, The Font, Gimmetrow, Nick Number, Epeeefleche, Talkaboutquality, Ash, Kezz90, Pedro.haru, Iwearavolcomhat, Icarusgeek, SoxBot, Addbot, Tassedethe, Luckasbot, Yobot, Stebanoid, Trappist the monk, Hockeyc, RjwilmsiBot, BobK77, Slightsmile, Mkjadhav, BG19bot, BattyBot, Monkbob and Anonymous: 35
- **Software walkthrough** *Source:* https://en.wikipedia.org/wiki/Software_walkthrough?oldid=646456627 *Contributors:* Peter Kaminski, Andreas Kaufmann, Diego Moya, Zntrip, Stuartyeates, Reyk, SmackBot, Jherm, Karafias, Donmillion, Gnewf, Jocoder, Ken g6, SieBot, DanielPharos, Yobot, MaterialsScientist, MathsPoetry, OriolBonjochGassol, John Cline and Anonymous: 12
- **Code review** *Source:* https://en.wikipedia.org/wiki/Code_review?oldid=676797548 *Contributors:* Ed Poor, Ryguasu, Dwheeler, Flamurai, Pcb21, Ronz, Enigmasoldier, Furrykef, Bevo, Robbot, Sverdrup, Craigwb, Tom-, Khalid hassani, Stevietheman, Oneiros, MattOConnor, Andreas Kaufmann, Magicpop, AliveFreeHappy, Project2501a, CanisRufus, Lauciusa, BlueNovember, Hooperbloop, Tlaresch, Ynhockey, Mindmatrix, Rjwilmsi, Salix alba, FlaBot, Integr, Bgwhite, RussBot, Rajeshd, Stephenb, Brucevdk, Jpowersny2, LeonardoRob0t, SmackBot, KAtremer, Matchups, ThurnerRupert, Derek farn, StefanVanDerWalt, Msabramo, Martinig, Pvlavos, Madjidi, Gioto, Smartbear, Srice13, Jesselong, Cander0000, Talkaboutquality, STBot, Jdelanoy, DanielVale, Argaeen, Manassehkat, VolkovBot, Rrobason, Aivosto, Doctor-Caligari, Kirian~enwiki, Jamelan, Mratzloff, MattiasAndersson, Fnegroni, Wolfch, Nevware, Mutilin, Swtechwr, Alla tedesca, XLinkBot, Scottb1978, Dsmic, Addbot, ChipX86, MrOllie, Steleki, Legobot, Yobot, Themfromspace, Digsav, AnomieBOT, Snizza, Xqbot, Adange, Ksipa, Craig Pemberton, Bunyk, Gbolton, RedBot, EmausBot, WikitanvirBot, NateEag, ZéroBot, AlcherBlack, TyA, Jabraham mw, Ktnptkr, Helpful Pixie Bot, Sh41pedia, BattyBot, Pchap10k, Frosty, Mahbubur-r-aaman, Gorohoroh, Monkbob, Abarkth99, Vieque, OM-PIRE, Donnerpeter, Furion19 and Anonymous: 99
- **Automated code review** *Source:* https://en.wikipedia.org/wiki/Automated_code_review?oldid=661875100 *Contributors:* RedWolf, Andreas Kaufmann, AliveFreeHappy, Amooore, John Vandenberg, Wknight94, Closedmouth, JLaTondre, Rwww, SmackBot, Elliot Shank, HelloAnnyong, Pvlavos, Mellery, Pgr94, Cydebot, OtherMichael, Leolaursen, Cic, Aivosto, Swtechwr, Addbot, Download, Yobot, Amirbot, NathanoNL, ThaddeusB, Jxramos, FrescoBot, IO Device, Lmerwin, Gaudol, JnRouvignac, ZéroBot, Jabraham mw, Tracerbee~enwiki, Fehnker, Ptrb, Nacx08 and Anonymous: 22
- **Code reviewing software** *Source:* https://en.wikipedia.org/wiki/Code_reviewing_software?oldid=593596111 *Contributors:* Techtonik, Andreas Kaufmann, Woohookitty, LauriO~enwiki, SmackBot, Elonka, FlashSheridan, EdGI, Pvlavos, JamesBWatson, Cander0000, Windymilla, FrescoBot, Jabraham mw, Ptrb, Mogism and Anonymous: 8
- **Static code analysis** *Source:* https://en.wikipedia.org/wiki/Static_program_analysis?oldid=668929812 *Contributors:* AlexWasFirst, Ted Longstaffe, Vkcuncak, Ixf64, Tregoweth, Ahoerstemeier, TUF-KAT, Julesd, Ed Brey, David.Monniaux, Psychonaut, Wlievens, Thv, Kravietz, Gadium, Vina, Rpm~enwiki, Andreas Kaufmann, AliveFreeHappy, Guanabot, Leibniz, Vp, Peter M Gerdes, Yonkie, Walter Görlitz, Diego Moya, Suruena, Kazvorpai, Ruud Koot, Marudubshinki, Graham87, Qwertyus, Rjwilmsi, Ground Zero, Mike Van Emmerik, Chobot, Berrinam, Crowfeather, Pinocar, Renox, Jschlosser, Cryptic, Goffrie, Tjarrett, Jpbowen, CaliforniaAliBaba, Creando, GraemeL, Rwww, SmackBot, FlashSheridan, Thumperward, Schwallex, A5b, Derek farn, Anujgoyal, Antonielly, JForget, Simeon, Wikid77, Ebde, RobotG, Obiwankenobi, Magioladitis, Cic, Lgirvin, JoelSherrill, Erkan Yilmaz, DatabACE, Andareed, StaticCast, Ferengi, Sashakir, SieBot, Stafft, Toddst1, Ks0stm, Wolfch, JanInad, Mutilin, Swtechwr, Dekisugi, HarriBOT, Hoco24, Tinus74, MrOllie, Lightbot, Legobot, Luckasbot, Yobot, AnomieBOT, Kskyj, Vileez, Shadowjams, FrescoBot, Fdrepas, Jisunjang, TjBot, Dbelhumour02, ZéroBot, Jabraham mw, Ptrb, JohnGDrever, Helpful Pixie Bot, Wbm1058, BG19bot, JacobTrue, BattyBot, Ablighnicta, Jionpedia, Freddygauss, Fran buchmann, Paul2520, Knife-in-the-drawer and Anonymous: 108
- **List of tools for static code analysis** *Source:* https://en.wikipedia.org/wiki/List_of_tools_for_static_code_analysis?oldid=676343956 *Contributors:* AlexWasFirst, William Avery, Asim, Dwheeler, Mrwojo, Edward, Breakpoint, Tregoweth, Haakon, Ronz, Ed Brey, Traal, David.Monniaux, Northgrove, Sander123, Psychonaut, Bernhard.kaindl, Aetheling, David Gerard, Orangemike, Vaucouleur, Kravietz, Dash, Beland, Achituv~enwiki, Rosen, RickScott, Scovetta, Andreas Kaufmann, Rodolfo Borges, Pepsiman, Jayjg, AliveFreeHappy, Rich Farmbrough, Amooore, Vp, Pavel Vozenilek, CanisRufus, Diomidis Spinellis, Nickj, Bdoserror, Baijum81, Jeodesic, Jdabney, Hooperbloop, Petdance, Capi x, Dethtron5000, Diego Moya, Biofuel, Krishchik, Wdfarmer, Runtime, Bkuhn, Woohookitty, RHaworth, Jerseyko, Donaldsbell@yahoo.com, Pkuczynski, Ruud Koot, Tabletop, Tlroche, Angusmclellan, Amire80, Drpaule, Ysangkok, Perrella, Mike Van Emmerik, Czar, Atif.hussain, Dmooney, Bgwhite, Pinocar, RussBot, Xoloz, Jschlosser, Joebeone, Cate, Gaius Cornelius, Cryptic, Test-tools~enwiki, Chick Bowen, Jredwards, Catamorphism, Taed, Irishguy, Malcolm, Anetode, Tjarrett, Jpbowen, Mikeblas, Falcon9x5, Avraham, Lajmon, Kenguest, JLaTondre, Gesslein, SmackBot, Mmernex, FlashSheridan, Shabda, Rajah9, Yamaguchi, DomQ, Bluebot, Senarclens, Kengell, Ber, Schwallex, Nbougalis, Frap, Nixeagle, Dmulter, Weregibil, Elliot Shank, Derek farn, Paulwells, DHR, PSeibert~enwiki, Ariefwn, JzG, Fishoak, Disavian, Ralthor, Neerajsangal, Sundström, Gahs, Tasc, Parikshit Narkhede, Yoderj, Hu12, HelloAnnyong, Bensonwu, Rogério Brito, Ydegrow, Pvlavos, Sadvonnikov, Nharav, Wws, Imeshev, ShelfSkewed, Pmerson, Lentower, NewSkool, Phatom87, AndrewHowse, Cydebot, B, Notopia, Iceberg1414, NoahSussman, N5iln, Dtgriocom, Pokeypokes, Nick Number, SteloKim, Chrystalice, Bitner, Amette, Sffubs, Dmkean, Slacka123, Verilog, Toutoune25, Magioladitis, Rrtuckwell, Tedickey, Cic, Curdeius, Giggy, Pausch, Bchess, Stubb~enwiki, Sreich, Gwern, Grauenwolf, R'n'B, Verdatum, Pth81, DatabACE, Athaenara, Pmjtoca, Venkatreddyc, LDRA, Bknittel, Kent SofCheck, Collinpark, Tlegall, Tradsud, Qu3a, Shiva.rock, Monathan, BB-Froggy, Gbickford, Aivosto, Guillem.Bernat, StaticCast, Wegra, BlackVegetable, Felmon, FergusBolger, Esdev, Timekeeper77, McCulley, Rainco, Yansky, Sashakir, Benneman~enwiki, Pitkelevo, Future-Domain, Rdbuckle, G b hall, Rssh, Fewaffles, Stafft, Jerryobject, Ehajiyev, Vfeditor, Mj1000, Jehiah, Faganp, Douglaska, Vlseregy, ShadowPhox, Mjdjohns5, Cgisquet, Wesnerm, Pmollins, Henk Poley, Benrick, Martarius, Staniuk, Dpnew, Pfunk1410, Sourceanalysis, Jcuk 2007, Excirial, Orang, Solodon, Pauljansen42, Swtechwr, Dekisugi, StanContributor, Fowlay, Borishollas, Fwaldman, Hello484, Azrael Nightwalker, AlanM1, Velizar.vesselinov, Gwandoya, Linehanjt, Rpelisse, Alexius08, Sameer0s, Addbot, Freddy.mallet, Prasanna vps, PraveenNet, Jsub, Tomtheeditor, Pdohara, Bgi, PurpleAluminiumPoodle, Checkshirt, Siva77, Wakusei, Ronaldbradford, Dvice null,

- Bjcosta, Tkvavle, Epierrrel, Wikieditoroftoday, Hyd danmar, Wickorama, Piano non troppo, Kskyj, Istoyanov, LilHelpa, Skilner, Kfhief6, The.gaboo, Parasoft-pl, CxQL, Lalb, Flamingcyanide, Drdeee, Nandotamu, A.zitzewitz, Serge Baranovsky, Teknopup, Ettl.martin-enwiki, Bakotat, AlexeyT2, FrescoBot, Llib xoc, GarenParham, Demarant, Newtang, Uncopy, Lmerwin, Stephen.gorton, Minhuk.kwon, Apc-man, Gaudol, Albert688, Dukeofgaming, Jisunjang, Rhuuck, Alextelea, Tonygrout, Skrik69, Jamieayre, PSmacchia, Vor4, Gryllida, Fontignie, Zfalconz, Vrenator, Moonwolf14, Issam lahlali, Bellingard, Runchalfdan, Jayabra17, Adarw, JnRouvignac, Gotofritz, Jopa fan, Dinis.Cruz, Iulian.serbanoiu, Armadillo-eleven, Xodlop, Waeswaes, Ljr1981, John of Reading, Pkortve, Exatex-enwiki, Bantoo12, Cp-parchitect, Mrlongleg, Dnozey, Optimyth, Dbelhumeur02, Mandrikov, InaToncheva, 70x7plus1, Romgerale, AManWithNoPlan, O2user, Rpapo, Sachrist, Tsaavik, Jabraham mw, Richsz, Mentibot, Tracerbee-enwiki, Krlooney, Devpitcher, Wiki jmeno, InaTonchevaToncheva, 1polaco, Bnmike, MarkusLitz, Helpfulsome, ClueBot NG, Ptrb, Jeff Song, Tlownie, Libouban, PaulEremeeff, JohnGDrever, Caoilte.guiry, Wikimaf, Tddcodemaster, Gogege, Damorin, Nandorjosef, Alexcenthousiast, Mcandre, Matsgd, BG19bot, Klausjansen, Nico.anquetil, Northamerica1000, Camwik75, Khozman, Lgawowski, Hsardin, Javier.salado, Dclucas, Chmarkine, Kgnazdowsky, Jessethompson, David wild2, Claytoncarney, BattyBot, Mccabesoftware, Ablighncta, RMatthias, Imology, HillGyuri, Alumd, Pizzutillo, Msmithers6, Lixhunter, Heychoii, Daniel.kaestner, Loic.etienne, Roberto Bagnara, Oceanesa, DamienPo, Jjehannet, Cmminera, ScrumMan, Dmimat, Fran buchmann, Ocpjp7, Securechecker1, Omnext, Sedmedia, Ths11180, Серж Тихомиров, Fuduprinz, SJ Defender, Benjamin hummel, Sampson, Avkonst, Makstov, D60c4p, BevB2014, Halleck45, Jacoblarfors, ITP Panorama, TheodorHerzl, Hanzalot, Vereslajos, Edainwestoc, Simon S Jennings, JohnTerry21, Guruwoman, Luisdoreste, Miogab, Matthiaseinig, Jdahse, Bkjiuwan, Christophe Dujarric, Mbjimenez, Realvizu, Marcopasserini65, Racodond, El flaco ik, Tibor.bakota, ChristopheBallihaut and Anonymous: 612
- **GUI software testing** *Source:* https://en.wikipedia.org/wiki/Graphical_user_interface_testing?oldid=666952008 *Contributors:* Deb, Pnm, Kku, Ronz, Craigwb, Andreas Kaufmann, AliveFreeHappy, Imroy, Rich Farmbrough, Liberatus, Jhertel, Walter Görlitz, Holek, Mass-GalactusUniversum, Rjwilmsi, Hardburn, Pinecar, Chaser, SteveLoughran, Gururajs, SAE1962, Josephtate, SmackBot, Jruuska, Unforgettableid, Hu12, Dreftymac, CmdrObot, Hesa, Pgr94, Cydebot, Anupam, MER-C, David Eppstein, Staceyeschneider, Ken g6, Jeff G., SiriusDG, Cmbay, Steven Crossin, Mdjohns5, Wahab80, Mild Bill Hiccup, Rockfang, XLinkBot, Alexius08, Addbot, Paul6feet1, Yobot, Rdancer, Wakusei, Equatin, Mcristinel, 10metreh, JnRouvignac, Dru of Id, O.Koslowski, BG19bot, ChrisGualtieri and Anonymous: 52
 - **Usability testing** *Source:* https://en.wikipedia.org/wiki/Usability_testing?oldid=670447644 *Contributors:* Michael Hardy, Ronz, Rossami, Manika, Wwheeler, Omegatron, Pigsonthewing, Tobias Bergemann, Fredcondo, MichaelMcGuffin, Discospinster, Rich Farmbrough, Dobrien, Xezbeth, Pavel Vozenilek, Bender235, ZeroOne, Ylee, Spalding, Janna Isabot, MaxHund, Hooperbloom, Arthens, Diego Moya, Geoffsauer, ChrisJMoore, Woohookitty, LizardWizard, Mindmatrix, RHaworth, Tomhab, Schmettow, Sjö, Aapo Laitinen, Alvin-cs, Pinecar, YurikBot, Hede2000, Brandon, Wikinstone, GraemeL, Azrael81, SmackBot, Alan Pascoe, DXBari, Cjohansen, Deli nk, Christopher Agnew, Kuru, DrJohnBrooke, Kcatz, Dennis G. Jerz, Gubernet, Philipumd, CmdrObot, Ivan Pozdeev, Tamarkot, Gumoz, Ravialluru, Siddhi, Gokusandwich, Pindakaas, Jhouckwh, Headbomb, Yettie0711, Bkillam, Karl smith, Dvandersluis, Jmike80, Malross, EagleFan, JaGa, Rlsheehan, Farreaching, Naniwako, Vmahi9, Jeff G., Technopat, Pghimire, Crònica-enwiki, Jean-Frédéric, Gmarinp, Toghome, JDBravo, Denisarona, Wikitonic, ClueBot, Leonard^Bloom, Toomuchwork, Mandalaz, Lakeworks, Kolyma, Fgnievinski, Download, Zorobot, Legobot, Luckas-bot, Yobot, Fraggel81, TaBOT-zerem, AnomieBOT, MikeBlockQuickBooksCPA, Blueraspberry, Citation bot, Xqbot, Antariksawan, Bihco, Millahna, A Quest For Knowledge, Shadowjams, Al Tereego, Hstetter, Bretclement, EmausBot, WikitanvirBot, Miamichic, Akjar13, Researcher1999, Josve05a, Dickohead, ClueBot NG, Willem-Paul, Jetuusp, Mchali, Helpful Pixie Bot, Breakthru10technologies, Op47, QualMod, CitationCleanerBot, BattyBot, Jtedinburgh, UsabilityCDSS, TwoMartiniTuesday, Bkyzer, Uxmaster, Vijaylaxmi Sharma, Itsraininglaura, Taigear, UniDIMEG, Aconversationalone, Alhussaini h, Devens100, Monkbob, Rtz92, Harrison Mann, Milan.simeunovic, Nutshell9, Vin020, MikeCoble and Anonymous: 126
 - **Think aloud protocol** *Source:* https://en.wikipedia.org/wiki/Think_aloud_protocol?oldid=673728579 *Contributors:* Tillwe, Ronz, Angela, Wik, Manika, Khalid hassani, Icairns, Aranel, Shanes, Diego Moya, Suruena, Nuggetboy, Zunk-enwiki, PeregrineAY, Calebjc, Pinecar, Akamad, Schultem, Ms2ger, SmackBot, DXBari, Delldot, Ohnoitsjamie, Dragice, Hetar, Ofol, Cydebot, Magioladitis, Robin S, Robksw, Technopat, Crònica-enwiki, Jammycaketin, TIY, Addbot, DOI bot, Shevek57, Yobot, Legobot II, Citation bot, Zojiji, Sae1962, Citation bot 1, RjwilmsiBot, Simone.borsci, Helpful Pixie Bot, Monkbob, Gagira UCL and Anonymous: 20
 - **Usability inspection** *Source:* https://en.wikipedia.org/wiki/Usability_inspection?oldid=590146399 *Contributors:* Andreas Kaufmann, Diego Moya, Lakeworks, Fgnievinski, AnomieBOT, Op47 and Anonymous: 1
 - **Cognitive walkthrough** *Source:* https://en.wikipedia.org/wiki/Cognitive_walkthrough?oldid=655157012 *Contributors:* Karada, Rdroz, Cyrius, Beta m, Kevin B12, Andreas Kaufmann, Rich Farmbrough, Srbauer, Spalding, Diego Moya, Gene Nygaard, Firsfron, FrancoisJordaan, Quale, Wavelength, Masran Silvaris, Macdorman, SmackBot, DXBari, Bluebot, Can't sleep, clown will eat me, Moephon, Xionbox, CmdrObot, Avillia, David Eppstein, Elusive Pete, Vanished user ojewerjuijaks344d, Naerii, Lakeworks, SimonB1212, Addbot, American Eagle, Tassedethe, SupperTina, Yobot, Alexgeek, Ocaasi, ClueBot NG and Anonymous: 35
 - **Heuristic evaluation** *Source:* https://en.wikipedia.org/wiki/Heuristic_evaluation?oldid=661561290 *Contributors:* Edward, Karada, Ronz, Angela, Fredcondo, Andreas Kaufmann, Art LaPella, Fyhuang, Diego Moya, Woohookitty, PhilippWeissenbacher, Rjwilmsi, Subversive, Kri, Chobot, JulesH, SmackBot, DXBari, Verne Equinox, Delldot, Turadg, Bluebot, Jonmmorgan, Khazar, SMasters, Bigpinkthing, RichardF, Cydebot, Clayoquot, AntiVandalBot, Hugh.glaser, JamesBWatson, Catgut, Wikip rhyre, Kjtobo, Lakeworks, XLinkBot, Felix Folio Secundus, Addbot, Zeppomedia, Lightbot, Citation bot, DamienT, KatieUM, Jonesey95, 0403554d, RjwilmsiBot, LuisCarlosrubino, Mrmatiko, ClueBot NG and Anonymous: 45
 - **Pluralistic walkthrough** *Source:* https://en.wikipedia.org/wiki/Pluralistic_walkthrough?oldid=632220585 *Contributors:* Andreas Kaufmann, Jayjg, Diego Moya, RHaworth, CmdrObot, Alaibot, Minnaert, AlexNewArtBot, Team Estonia, Lakeworks, FrescoBot, ClueBot NG, ChrisGualtieri and Anonymous: 4
 - **Comparison of usability evaluation methods** *Source:* https://en.wikipedia.org/wiki/Comparison_of_usability_evaluation_methods?oldid=530519159 *Contributors:* Ronz, Andrewman327, Diego Moya, Andreala, RHaworth, SmackBot, Eastlaw, Cydebot, Lakeworks, Simone.borsci, Jtedinburgh and Anonymous: 4

11.2 Images

- **File:8bit-dynamiclist.gif** *Source:* <https://upload.wikimedia.org/wikipedia/commons/1/1d/8bit-dynamiclist.gif> *License:* CC-BY-SA-3.0 *Contributors:* Own work *Original artist:* Seahen
- **File:Ambox_important.svg** *Source:* https://upload.wikimedia.org/wikipedia/commons/b/b4/Ambox_important.svg *License:* Public domain *Contributors:* Own work, based off of Image:Ambox scales.svg *Original artist:* Dsmurat (talk · contribs)

- **File:Ambox_wikify.svg** *Source:* https://upload.wikimedia.org/wikipedia/commons/e/e1/Ambox_wikify.svg *License:* Public domain *Contributors:* Own work *Original artist:* penubag
- **File:Blackbox.svg** *Source:* <https://upload.wikimedia.org/wikipedia/commons/f/f6/Blackbox.svg> *License:* Public domain *Contributors:* Transferred from en.wikipedia *Original artist:* Original uploader was Frap at en.wikipedia
- **File:Commons-logo.svg** *Source:* <https://upload.wikimedia.org/wikipedia/en/4/4a/Commons-logo.svg> *License:* ? *Contributors:* ? *Original artist:* ?
- **File:Crystal_Clear_app_browser.png** *Source:* https://upload.wikimedia.org/wikipedia/commons/f/fe/Crystal_Clear_app_browser.png *License:* LGPL *Contributors:* All Crystal icons were posted by the author as LGPL on kde-look *Original artist:* Everaldo Coelho and YellowIcon
- **File:Crystal_Clear_device_cdrom_unmount.png** *Source:* https://upload.wikimedia.org/wikipedia/commons/1/10/Crystal_Clear_device_cdrom_unmount.png *License:* LGPL *Contributors:* All Crystal Clear icons were posted by the author as LGPL on kde-look; *Original artist:* Everaldo Coelho and YellowIcon;
- **File:CsUnit2.5Gui.png** *Source:* <https://upload.wikimedia.org/wikipedia/en/3/3c/CsUnit2.5Gui.png> *License:* CC-BY-SA-3.0 *Contributors:* self-made *Original artist:* Manfred Lange
- **File:Disambig_gray.svg** *Source:* https://upload.wikimedia.org/wikipedia/en/5/5f/Disambig_gray.svg *License:* Cc-by-sa-3.0 *Contributors:* ? *Original artist:* ?
- **File:ECP.png** *Source:* <https://upload.wikimedia.org/wikipedia/commons/3/36/ECP.png> *License:* CC BY-SA 3.0 *Contributors:* Own work *Original artist:* Nmondal
- **File:Edit-clear.svg** *Source:* <https://upload.wikimedia.org/wikipedia/en/f/f2/Edit-clear.svg> *License:* Public domain *Contributors:* The Tango! Desktop Project. *Original artist:* The people from the Tango! project. And according to the meta-data in the file, specifically: "Andreas Nilsson, and Jakub Steiner (although minimally)."
- **File:Electronics_Test_Fixture.jpg** *Source:* https://upload.wikimedia.org/wikipedia/commons/0/08/Electronics_Test_Fixture.jpg *License:* CC BY-SA 3.0 *Contributors:* Own work *Original artist:* Davidbatet
- **File:Fagan_Inspection_Simple_flow.svg** *Source:* https://upload.wikimedia.org/wikipedia/commons/8/85/Fagan_Inspection_Simple_flow.svg *License:* CC0 *Contributors:* Own work *Original artist:* Bignose
- **File:Folder_Hexagonal_Icon.svg** *Source:* https://upload.wikimedia.org/wikipedia/en/4/48/Folder_Hexagonal_Icon.svg *License:* Cc-by-sa-3.0 *Contributors:* ? *Original artist:* ?
- **File:Free_Software_Portal_Logo.svg** *Source:* https://upload.wikimedia.org/wikipedia/commons/6/67/Nuvola_apps_emacs_vector.svg *License:* LGPL *Contributors:* Nuvola_apps_emacs.png *Original artist:* Nuvola_apps_emacs.png; David Vignoni
- **File:Freedesktop-logo-for-template.svg** *Source:* <https://upload.wikimedia.org/wikipedia/commons/7/7b/Freedesktop-logo-for-template.svg> *License:* GPL *Contributors:* Can be found in the freedesktop.org GIT repositories, as well as e.g. at [1]. The contents of the GIT repositories are (mainly) GPL, thus this file is GPL. *Original artist:* ScotXW
- **File:Functional_Test_Fixture_for_electroncis.jpg** *Source:* https://upload.wikimedia.org/wikipedia/commons/3/32/Functional_Test_Fixture_for_electroncis.jpg *License:* CC BY-SA 3.0 *Contributors:* Own work *Original artist:* Davidbatet
- **File:Green_bug_and_broom.svg** *Source:* https://upload.wikimedia.org/wikipedia/commons/8/83/Green_bug_and_broom.svg *License:* LGPL *Contributors:* File:Broom icon.svg, file:Green_bug.svg *Original artist:* Poznaniak, pozostali autorzy w plikach źródłowych
- **File:Htmlunit_logo.png** *Source:* https://upload.wikimedia.org/wikipedia/en/e/e0/Htmlunit_logo.png *License:* Fair use *Contributors:* taken from HtmlUnit web site.[1] *Original artist:* ?
- **File:Internet_map_1024.jpg** *Source:* https://upload.wikimedia.org/wikipedia/commons/d/d2/Internet_map_1024.jpg *License:* CC BY 2.5 *Contributors:* Originally from the English Wikipedia; description page is/was here. *Original artist:* The Opte Project
- **File:James_Webb_Primary_Mirror.jpg** *Source:* https://upload.wikimedia.org/wikipedia/commons/1/10/James_Webb_Primary_Mirror.jpg *License:* Public domain *Contributors:* NASA Image of the Day *Original artist:* NASA/MSFC/David Higginbotham
- **File:LampFlowchart.svg** *Source:* <https://upload.wikimedia.org/wikipedia/commons/9/91/LampFlowchart.svg> *License:* CC-BY-SA-3.0 *Contributors:* vector version of Image:LampFlowchart.png *Original artist:* svg by Booyabazooka
- **File:LibreOffice_4.0_Main_Icon.svg** *Source:* https://upload.wikimedia.org/wikipedia/commons/5/5a/LibreOffice_4.0_Main_Icon.svg *License:* CC BY-SA 3.0 *Contributors:* LibreOffice *Original artist:* The Document Foundation
- **File:Mbt-overview.png** *Source:* <https://upload.wikimedia.org/wikipedia/en/3/36/Mbt-overview.png> *License:* PD *Contributors:* ? *Original artist:* ?
- **File:Mbt-process-example.png** *Source:* <https://upload.wikimedia.org/wikipedia/en/4/43/Mbt-process-example.png> *License:* PD *Contributors:* ? *Original artist:* ?
- **File:Merge-arrow.svg** *Source:* <https://upload.wikimedia.org/wikipedia/commons/a/aa/Merge-arrow.svg> *License:* Public domain *Contributors:* ? *Original artist:* ?
- **File:Merge-arrows.svg** *Source:* <https://upload.wikimedia.org/wikipedia/commons/5/52/Merge-arrows.svg> *License:* Public domain *Contributors:* ? *Original artist:* ?
- **File:Mergefrom.svg** *Source:* <https://upload.wikimedia.org/wikipedia/commons/0/0f/Mergefrom.svg> *License:* Public domain *Contributors:* ? *Original artist:* ?

- **File:NUnit_GUI.png** Source: https://upload.wikimedia.org/wikipedia/commons/3/36/NUnit_GUI.png License: ZLIB Contributors: Own work Original artist: ?
- **File:Office-book.svg** Source: <https://upload.wikimedia.org/wikipedia/commons/a/a8/Office-book.svg> License: Public domain Contributors: This and myself. Original artist: Chris Down/Tango project
- **File:Question_book-new.svg** Source: https://upload.wikimedia.org/wikipedia/en/9/99/Question_book-new.svg License: Cc-by-sa-3.0 Contributors: Created from scratch in Adobe Illustrator. Based on Image:Question book.png created by User:Equazcion Original artist: Tkgd2007
- **File:Software_spanner.png** Source: https://upload.wikimedia.org/wikipedia/commons/8/82/Software_spanner.png License: CC-BY-SA-3.0 Contributors: Transferred from en.wikipedia; transfer was stated to be made by User:Rockfang. Original artist: Original uploader was CharlesC at en.wikipedia
- **File:System-installer.svg** Source: <https://upload.wikimedia.org/wikipedia/commons/d/db/System-installer.svg> License: Public domain Contributors: The Tango! Desktop Project Original artist: The people from the Tango! project
- **File:Test-driven_development.PNG** Source: https://upload.wikimedia.org/wikipedia/commons/9/9c/Test-driven_development.PNG License: CC BY-SA 3.0 Contributors: Own work (Original text: *I created this work entirely by myself.*) Original artist: Excirial (Contact me, Contributions)
- **File:Test_Automation_Interface.png** Source: https://upload.wikimedia.org/wikipedia/commons/8/89/Test_Automation_Interface.png License: CC BY-SA 3.0 Contributors: Own work Original artist: Anand Gopalakrishnan
- **File:Text_document_with_red_question_mark.svg** Source: https://upload.wikimedia.org/wikipedia/commons/a/a4/Text_document_with_red_question_mark.svg License: Public domain Contributors: Created by bdesham with Inkscape; based upon Text-x-generic.svg from the Tango project. Original artist: Benjamin D. Esham (bdesham)
- **File:US_Navy_090407-N-4669J-042_Sailors_assigned_to_the_air_department_of_the_aircraft_carrier_USS_George_H.W._Bush_(CVN_77)_test_the_ship's_catapult_systems_during_acceptance_trials.jpg** Source: https://upload.wikimedia.org/wikipedia/commons/7/7d/US_Navy_090407-N-4669J-042_Sailors_assigned_to_the_air_department_of_the_aircraft_carrier_USS_George_H.W._Bush_%28CVN_77%29_test_the_ship%27s_catapult_systems_during_acceptance_trials.jpg License: Public domain Contributors: This Image was released by the United States Navy with the ID 090407-N-4669J-042 (next). This tag does not indicate the copyright status of the attached work. A normal copyright tag is still required. See Commons:Licensing for more information. Original artist: U.S. Navy Photo by Mass Communication Specialist 2nd Class Jennifer L. Jaqua
- **File:Unbalanced_scales.svg** Source: https://upload.wikimedia.org/wikipedia/commons/f/fe/Unbalanced_scales.svg License: Public domain Contributors: ? Original artist: ?
- **File:Virzis_Formula.PNG** Source: https://upload.wikimedia.org/wikipedia/commons/f/f8/Virzis_Formula.PNG License: Public domain Contributors: Transferred from en.wikipedia; transferred to Commons by User:Kelly using CommonsHelper. Original artist: Original uploader was Schmettow at en.wikipedia. Later version(s) were uploaded by NickVeys at en.wikipedia.
- **File:Wiki_letter_w.svg** Source: https://upload.wikimedia.org/wikipedia/en/6/6c/Wiki_letter_w.svg License: Cc-by-sa-3.0 Contributors: ? Original artist: ?
- **File:Wiki_letter_w_cropped.svg** Source: https://upload.wikimedia.org/wikipedia/commons/1/1c/Wiki_letter_w_cropped.svg License: CC-BY-SA-3.0 Contributors: Wiki_letter_w.svg Original artist: Wiki_letter_w.svg: Jarkko Piironen
- **File:Wikibooks-logo.svg** Source: <https://upload.wikimedia.org/wikipedia/commons/f/fa/Wikibooks-logo.svg> License: CC BY-SA 3.0 Contributors: Own work Original artist: User:Bastique, User:Ramac et al.
- **File:Wikiversity-logo.svg** Source: <https://upload.wikimedia.org/wikipedia/commons/9/91/Wikiversity-logo.svg> License: CC BY-SA 3.0 Contributors: Snorky (optimized and cleaned up by verdyp) Original artist: Snorky (optimized and cleaned up by verdyp)

11.3 Content license

- Creative Commons Attribution-Share Alike 3.0